Syntax highlighting has been disabled due to code size.

```
# ╔═══════════════════════════════════════════════════════════════╗
# ||         MICHIE STREAM PLATFORM (MSP) — FULL SOURCE EXPORT   ||
# ||  Generated: 2026-03-25  |  All source files included below  ||
# ╚═══════════════════════════════════════════════════════════════╝
```

PROJECT CONTEXT

This is the **Michie Stream Platform (MSP)** — a blockchain music & NFT streaming platform.

Stack:
- Backend: Node.js / Express (`server.cjs`) + modular route handlers
- Frontend: Vanilla JS (`main.js`), five HTML pages
- Blockchain: ethers.js v5, Solidity smart contracts, AWS KMS
- Storage: IPFS, Redis, FFmpeg
- Design: SIGNAL dark-luxury system (Cormorant / Syne / Space Mono)

Key business rule: Any completed playthrough of content originating from a Favorites list is fully

BACKEND — SERVER

`server.cjs` (91.2 KB)

```javascript
'use strict';
/**
 * Michie Stream Platform — main Express server (CommonJS)
 *
 * ENV:
 *  PORT=3001
 *  LOG_LEVEL=info
 *  JWT_SECRET=supersecret
 *  PLAY_TOKEN_SECRET=anothersecret
 *  FFMPEG_PATH=...
 *  REDIS_URL=redis://localhost:6379
 *  IPFS_ENDPOINT=http://127.0.0.1:5001
 *  IPFS_GATEWAY=https://ipfs.io/ipfs/        ← replaces dead infura gateway
 *  ETH_RPC=https://mainnet.infura.io/v3/<KEY>
 *  MSP_PRIVATE_KEY=0x...
 *  PLATFORM_NFT_ADDRESS=0x...                ← Platform NFT contract address
 *  TLS_KEY_PATH=./certs/privkey.pem
 *  TLS_CERT_PATH=./certs/fullchain.pem
 *  AWS_REGION=us-east-1
 *  KMS_KEY_ID=arn:aws:kms:...
 *  FORCE_BLAKE3=true
```

```
 *  ETH_USD_RATE=2500                        ← fallback ETH/USD rate
 */

// ──────────────────────────── Global error handling ────────────────────────────
process.on('uncaughtException',  (err) => { console.error('Uncaught Exception:',  err.stack || err); p
process.on('unhandledRejection', (reason) => { console.error('Unhandled Rejection:', reason?.stack ||

// ──────────────────────────── Core & env ───────────────────────────────────────
const path    = require('path');
const fs      = require('fs-extra');
const crypto  = require('crypto');
require('dotenv').config();

// ──────────────────────────── Web ──────────────────────────────────────────────
const express   = require('express');
const helmet    = require('helmet');
const compression = require('compression');
const rateLimit = require('express-rate-limit');
const multer    = require('multer');

// ──────────────────────────── Media ────────────────────────────────────────────
const ffmpeg        = require('fluent-ffmpeg');
const ffprobe       = require('ffprobe');
const ffprobeStatic = require('ffprobe-static');
const sharp         = require('sharp');

// ──────────────────────────── Crypto & data ────────────────────────────────────
const { v4: uuidv4 } = require('uuid');
let blake3;
try {
  blake3 = require('blake3');
  console.log('BLAKE3 loaded');
} catch (e) {
  console.warn('BLAKE3 unavailable; using SHA-256 fallback:', e.message);
  if (process.env.FORCE_BLAKE3 === 'true') throw new Error('BLAKE3 required but failed to load');
}
const jwt   = require('jsonwebtoken');
const pino  = require('pino');
const { ethers } = require('ethers');

// ──────────────────────────── Redis ────────────────────────────────────────────
const redis = require('redis');
let redisClient;

// ──────────────────────────── IPFS ─────────────────────────────────────────────
const { create: createIpfs } = require('ipfs-http-client');
```

```
// ──────────────────────────── Local modules ────────────────────────────────────
const { awsKmsSignEIP712 } = require('./module_aws');

// ──────────────────────────── Validators ───────────────────────────────────────
let validateMetadata, validateOwnership;
try {
  ({ validateMetadata, validateOwnership } = require('./validator'));
} catch (_) {
  validateMetadata = () => true;
  validateOwnership = () => {};
}

// ──────────────────────────── Logger ───────────────────────────────────────────
const logsDir = path.resolve(process.cwd(), 'logs');
fs.ensureDirSync(logsDir);
const logger = pino(
  { level: process.env.LOG_LEVEL || 'info' },
  pino.destination(path.join(logsDir, 'metrics.log'))
);
logger.info('MSP server starting');

// ──────────────────────────── Business constants ───────────────────────────────
const FEES = {
  PLATFORM_ROYALTY_STANDARD:  0.05,   // 5%  — standard creator royalty fee
  PLATFORM_ROYALTY_NFT:       0.015,  // 1.5% — Platform NFT holder royalty fee
  PLATFORM_TIP:               0.03,   // 3%  — platform cut of all tips
  PLATFORM_NFT_SALE:          0.025,  // 2.5% — platform cut of all NFT sales
  PLATFORM_NFT_PRICE_USD:     10000,  // $10,000 USD — Platform NFT price floor
};

const SUBSCRIPTION_PLANS = {
  listener_tier1_monthly: { price_usd: 10.99,  days: 30,  type: 'listener', tier: 1 },
  listener_tier1_annual:  { price_usd: 131.88, days: 365, type: 'listener', tier: 1 },
  listener_tier1_rolling: { price_usd: 10.99,  days: 3,   type: 'listener', tier: 1 },
  listener_tier2_monthly: { price_usd: 19.99,  days: 30,  type: 'listener', tier: 2 },
  listener_tier2_annual:  { price_usd: 239.88, days: 365, type: 'listener', tier: 2 },
  listener_tier2_rolling: { price_usd: 19.99,  days: 3,   type: 'listener', tier: 2 },
  listener_tier3_monthly: { price_usd: 34.99,  days: 30,  type: 'listener', tier: 3 },
  listener_tier3_annual:  { price_usd: 419.88, days: 365, type: 'listener', tier: 3 },
  listener_tier3_rolling: { price_usd: 34.99,  days: 3,   type: 'listener', tier: 3 },
  creator_monthly:        { price_usd: 29.99,  days: 30,  type: 'creator',  tier: null },
  creator_annual:         { price_usd: 299.88, days: 365, type: 'creator',  tier: null },
  nft_creator_monthly:    { price_usd: 14.99,  days: 30,  type: 'nft_creator', tier: null },
  nft_creator_annual:     { price_usd: 179.88, days: 365, type: 'nft_creator', tier: null },
};

// ──────────────────────────── Stream duration limits ───────────────────────────
```

```
const STREAM_DURATION_LIMITS = {
  creator:              3 * 60 * 60 * 1000,
  platform_nft_creator: Infinity,
};

// ──────────────────────────── Rate limiter ────────────────────────────────────
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 1000,
  standardHeaders: true,
  legacyHeaders: false,
  message: 'Too many requests; try again later',
});

// ──────────────────────────── FFmpeg ──────────────────────────────────────────
try {
  const ffmpegInstaller = require('@ffmpeg-installer/ffmpeg');
  ffmpeg.setFfmpegPath(process.env.FFMPEG_PATH || ffmpegInstaller.path);
} catch (_) {
  if (process.env.FFMPEG_PATH) ffmpeg.setFfmpegPath(process.env.FFMPEG_PATH);
}

// ──────────────────────────── Provider & wallet ───────────────────────────────
let provider     = null;
let mspWallet    = null;
const DEV_MODE   = !process.env.ETH_RPC || !process.env.MSP_PRIVATE_KEY;

// ── DEV WALLET — REMOVE BEFORE PRODUCTION ────────────────────────────────────
// Bypasses profile existence + capability checks for the test wallet.
// Set DEV_MODE to false (by providing ETH_RPC + MSP_PRIVATE_KEY in .env) to disable.
const DEV_WALLET = DEV_MODE ? '0xDe1f33Ce3E81bc54ebD0F4BC3Ce0a2F64d7Ef399' : null;
// ─────────────────────────────────────────────────────────────────────────────

if (DEV_MODE) {
  logger.warn('ETH_RPC or MSP_PRIVATE_KEY not set — running in DEV MODE (on-chain calls disabled)');
  if (DEV_WALLET) logger.warn({ DEV_WALLET }, 'DEV_WALLET active — all permission checks bypassed for
} else {
  try {
    provider  = new ethers.providers.JsonRpcProvider(process.env.ETH_RPC);
    mspWallet = new ethers.Wallet(process.env.MSP_PRIVATE_KEY, provider);
    logger.info('Ethereum provider and wallet initialized');
  } catch (err) {
    logger.warn({ err }, 'Ethereum provider failed to init — running in DEV MODE');
  }
}

// ──────────────────────────── Contract addresses ──────────────────────────────
```

```
const contentCAAddress         = process.env.CONTENT_CA_ADDRESS         || '0x0000000000000000000000000
const streamingRegistryAddress = process.env.STREAMING_REGISTRY_ADDRESS || '0x0000000000000000000000000
const royaltyPayoutAddress     = process.env.ROYALTY_PAYOUT_ADDRESS     || '0x0000000000000000000000000
const escrowAddress            = process.env.ESCROW_CONTRACT_ADDRESS     || '0x0000000000000000000000000
const platformNftAddress       = process.env.PLATFORM_NFT_ADDRESS        || '0x0000000000000000000000000

const contentCAABI          = global.CONTENT_CA_ABI          || [];
const streamingRegistryABI  = global.STREAMING_REGISTRY_ABI  || [];
const royaltyPayoutABI      = global.ROYALTY_PAYOUT_ABI      || [];
const escrowABI             = global.ESCROW_ABI              || [];

// Minimal ERC-721 ABI for Platform NFT ownership checks
const ERC721_ABI = [
  'function balanceOf(address owner) view returns (uint256)',
  'function ownerOf(uint256 tokenId) view returns (address)',
];

let streamingContract = null;
if (mspWallet) {
  try {
    streamingContract = new ethers.Contract(streamingRegistryAddress, streamingRegistryABI, mspWallet)
    logger.info('Contracts initialized');
  } catch (err) {
    logger.warn({ err }, 'Contract init failed — on-chain calls will be skipped');
  }
}

// ──────────────────────────── Redis init ──────────────────────────────────────
(async () => {
  try {
    redisClient = redis.createClient({ url: process.env.REDIS_URL || 'redis://127.0.0.1:6379' });
    redisClient.on('error', (err) => logger.error({ err }, 'Redis error'));
    await redisClient.connect();
    logger.info('Redis connected');
  } catch (err) {
    logger.warn({ err }, 'Redis not connected (continuing without cache)');
  }
})();

// ──────────────────────────── IPFS client ─────────────────────────────────────
let ipfs = null;
try {
  ipfs = createIpfs({ url: process.env.IPFS_ENDPOINT || 'http://127.0.0.1:5001' });
  logger.info('IPFS client initialized');
} catch (err) {
  logger.warn({ err }, 'IPFS client not configured');
}
```

```
const IPFS_GATEWAY = process.env.IPFS_GATEWAY || 'https://ipfs.io/ipfs/';

// ──────────────────────────── Express app ──────────────────────────────────
const app = express();
app.set('trust proxy', 1);
app.use(helmet({
  hsts: { maxAge: 31536000, includeSubDomains: true, preload: true },
  contentSecurityPolicy: {
    useDefaults: true,
    directives: {
      scriptSrc:  ["'self'", "'unsafe-inline'", "'unsafe-eval'"],
      imgSrc:     ["'self'", 'data:', 'https:'],
      connectSrc: ["'self'", 'wss:', 'https:'],
    },
  },
}));
app.use(compression());
app.use(limiter);
app.use(express.json({ limit: '10mb' }));
app.use(express.static(path.join(process.cwd(), 'public')));
app.use(express.static(process.cwd()));
app.get('/', (req, res) => res.sendFile(path.join(process.cwd(), 'index.html')));

// Normalize all wallet addresses to lowercase so MetaMask/Coinbase lowercase
// addresses always match profiles regardless of how they were submitted.
app.use((req, res, next) => {
  if (req.body && req.body.wallet)    req.body.wallet    = req.body.wallet.toLowerCase();
  if (req.body && req.body.listener)  req.body.listener  = req.body.listener.toLowerCase();
  if (req.body && req.body.from_wallet) req.body.from_wallet = req.body.from_wallet.toLowerCase();
  if (req.body && req.body.to_wallet)   req.body.to_wallet   = req.body.to_wallet.toLowerCase();
  if (req.params && req.params.wallet)  req.params.wallet    = req.params.wallet.toLowerCase();
  next();
});

// ──────────────────────────── Multer ───────────────────────────────────────
fs.ensureDirSync(path.join(__dirname, 'temp'));

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, path.join(__dirname, 'temp')),
  filename:    (req, file, cb) => cb(null, uuidv4() + path.extname(file.originalname)),
});

const ALLOWED_MEDIA_MIMES = new Set([
  'audio/mpeg', 'audio/mp3', 'audio/wav', 'audio/x-wav', 'audio/wave',
  'audio/ogg', 'audio/flac', 'audio/x-flac', 'audio/aac', 'audio/mp4',
  'audio/x-m4a',
```

```
  'video/mp4', 'video/quicktime', 'video/x-matroska', 'video/webm',
]);
const ALLOWED_MEDIA_EXTS  = /\.(mp3|wav|ogg|flac|aac|m4a|mp4|mov|mkv|webm)$/i;
const ALLOWED_IMAGE_MIMES = new Set(['image/png', 'image/jpeg', 'image/webp']);
const ALLOWED_IMAGE_EXTS  = /\.(png|jpg|jpeg|webp)$/i;

const upload = multer({
  storage,
  limits: { fileSize: 500 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    if (file.fieldname === 'audio-file') {
      if (ALLOWED_MEDIA_MIMES.has(file.mimetype) && ALLOWED_MEDIA_EXTS.test(file.originalname)) {
        return cb(null, true);
      }
      return cb(new Error(
        `Unsupported media type: ${file.mimetype}. ` +
        'Accepted: MP3, WAV, OGG, FLAC, AAC, M4A, MP4, MOV, MKV, WebM'
      ));
    }
    if (file.fieldname === 'cover-image') {
      if (ALLOWED_IMAGE_MIMES.has(file.mimetype) && ALLOWED_IMAGE_EXTS.test(file.originalname)) {
        return cb(null, true);
      }
      return cb(new Error('Cover image must be PNG, JPG, or WebP'));
    }
    cb(null, true);
  },
});

// ──────────────────────────── Profiles (JSON store) ────────────────────────────
const profilesPath = path.resolve(process.cwd(), 'profiles.json');
fs.ensureFileSync(profilesPath);

async function loadProfiles() {
  try {
    const raw = JSON.parse(await fs.readFile(profilesPath, 'utf8'));
    // Normalize all keys to lowercase so MetaMask/Coinbase lowercase addresses
    // always match profiles saved with mixed-case checksummed addresses.
    const normalized = {};
    for (const [k, v] of Object.entries(raw)) {
      normalized[k.toLowerCase()] = v;
    }
    return normalized;
  }
  catch { return {}; }
}
async function saveProfiles(profiles) {
```

```
  await fs.writeFile(profilesPath, JSON.stringify(profiles, null, 2));
}

/**
 * Case-insensitive profile lookup — kept as a utility but loadProfiles
 * now normalizes keys so direct bracket access always works.
 */
function findProfile(profiles, wallet) {
  if (!wallet) return { profile: null, key: null };
  const key = wallet.toLowerCase();
  return { profile: profiles[key] || null, key: profiles[key] ? key : null };
}

// ──────────────────────────── Profile helpers ────────────────────────────────

function isSubscriptionActive(profile) {
  if (!profile) return false;
  return profile.subscription_expiry && Date.now() < profile.subscription_expiry;
}

function getListenerTier(profile) {
  if (!profile || profile.account_type !== 'listener') return 0;
  if (!isSubscriptionActive(profile)) return 0;
  return profile.listener_tier || 0;
}

function getCapabilityLevel(profile) {
  if (!profile) return 'none';
  const type   = profile.account_type;
  const active = isSubscriptionActive(profile);

  if (type === 'platform_nft_creator') {
    return active ? 'nft_creator_active' : 'nft_creator_passive';
  }
  if (type === 'creator') {
    return active ? 'creator_active' : 'creator_inactive';
  }
  if (type === 'listener') {
    const tier = active ? (profile.listener_tier || 1) : 0;
    return tier > 0 ? `listener_${tier}` : 'none';
  }
  if (type === 'admin') return 'admin';
  return 'none';
}

async function hasPlatformNft(walletAddress) {
  if (!provider) return false;
```

```
  try {
    const nftContract = new ethers.Contract(platformNftAddress, ERC721_ABI, provider);
    const balance = await nftContract.balanceOf(walletAddress);
    return balance.gt(0);
  } catch (e) {
    logger.warn({ err: e }, 'Platform NFT check failed');
    return false;
  }
}

function usdToEth(usdAmount) {
  const rate = parseFloat(process.env.ETH_USD_RATE || '2500');
  return (usdAmount / rate).toFixed(8);
}

// ──────────────────────────── DEV helper ──────────────────────────────────────

/**
 * Returns true if this wallet is the DEV_WALLET and DEV_MODE is active.
 * Use at the top of any route that needs an access guard to short-circuit it.
 */
function isDevWallet(wallet) {
  return !!(DEV_WALLET && wallet && wallet.toLowerCase() === DEV_WALLET.toLowerCase());
}

// ──────────────────────────── IPFS helpers ────────────────────────────────────
async function addDirectoryToIpfs(ipfsClient, dir) {
  const entries = [];
  const walk = async (base) => {
    for (const name of await fs.readdir(base)) {
      const full = path.join(base, name);
      const stat = await fs.stat(full);
      if (stat.isDirectory()) { await walk(full); }
      else {
        const rel = path.relative(dir, full).split(path.sep).join('/');
        entries.push({ path: rel, content: fs.createReadStream(full) });
      }
    }
  };
  await walk(dir);
  const added = [];
  for await (const r of ipfsClient.addAll(entries, { wrapWithDirectory: true})) added.push(r);
  const dirEntry = added.find((r) => r.path === '');
  if (!dirEntry) throw new Error('IPFS folder CID not found');
  return { folderCid: dirEntry.cid.toString(), files: added };
}
```

```
// ──────────────────────────── KMS / EIP-712 helpers ────────────────────────────
function derSigToRSV(derHex, digestHex) {
  const buf = Buffer.from(derHex.replace(/^0x/, ''), 'hex');
  if (buf[0] !== 0x30) throw new Error('Unexpected DER prefix');
  let offset = 2;
  if (buf[offset] !== 0x02) throw new Error('DER: missing r marker');
  offset++;
  const rlen = buf[offset++]; const r = buf.slice(offset, offset + rlen); offset += rlen;
  if (buf[offset] !== 0x02) throw new Error('DER: missing s marker');
  offset++;
  const slen = buf[offset++]; const s = buf.slice(offset, offset + slen);
  const r32 = Buffer.alloc(32); r.copy(r32, 32 - r.length);
  const s32 = Buffer.alloc(32); s.copy(s32, 32 - s.length);
  const rHex = '0x' + r32.toString('hex');
  const sHex = '0x' + s32.toString('hex');
  const digest = ethers.utils.arrayify(digestHex);
  for (const v of [27, 28]) {
    try {
      const addr = ethers.utils.recoverAddress(digest, { r: rHex, s: sHex, v });
      if (addr && ethers.utils.isAddress(addr)) return ethers.utils.joinSignature({ r: rHex, s: sHex,
    } catch (_) { /* try next */ }
  }
  return ethers.utils.joinSignature({ r: rHex, s: sHex, v: 27 });
}

async function signEIP712(domain, types, value) {
  const maybeSig = await awsKmsSignEIP712(domain, types, value, mspWallet);
  const digest   = ethers.utils._TypedDataEncoder.hash(domain, types, value);
  return /^0x30/i.test(maybeSig) ? derSigToRSV(maybeSig, digest) : maybeSig;
}

// ──────────────────────────── Quality validators ──────────────────────────────
async function validateQuality(filePath, contentType, devMode = false) {
  const data = await new Promise((res, rej) =>
    ffprobe(filePath, { path: ffprobeStatic.path }, (err, info) => err ? rej(err) : res(info))
  );
  const audioStream = data?.streams?.find(s => s.codec_type === 'audio');
  const videoStream = data?.streams?.find(s => s.codec_type === 'video');

  if (contentType === 'music' || contentType === 'podcast') {
    if (!audioStream) throw new Error('No audio stream found in uploaded file');
    // Skip bitrate check in DEV_MODE — any quality accepted for testing
    if (!devMode) {
      const bitrate = parseInt(audioStream.bit_rate || '0', 10);
      if (bitrate && bitrate < 128000) throw new Error(`Audio bitrate too low (${bitrate}bps, min 128
    }
  }
```

```
  if (contentType === 'video') {
    if (!videoStream) throw new Error('No video stream found — upload an MP4, MOV, MKV, or WebM file')
    if (!audioStream) throw new Error('Video must include an audio track');
    if (!devMode) {
      const vBitrate = parseInt(videoStream.bit_rate || data?.format?.bit_rate || '0', 10);
      if (vBitrate && vBitrate < 500000) throw new Error(`Video bitrate too low (${vBitrate}bps, min 5
    }
  }
  if (contentType === 'art_animated') {
    if (!videoStream) throw new Error('Animated art must be a video file (MP4, WebM)');
  }
  return { audioStream, videoStream, format: data?.format };
}

async function validateImage(filePath, devMode = false) {
  const meta = await sharp(filePath).metadata();
  if (!devMode) {
    if (meta.width < 1000 || meta.height < 1000) throw new Error('Cover image must be at least 1000×10
    const aspectRatio = meta.width / meta.height;
    if (Math.abs(aspectRatio - 1) > 0.05) throw new Error('Cover image must be square (1:1 aspect rati
  }
  const stats = await fs.stat(filePath);
  if (stats.size > 10 * 1024 * 1024) throw new Error('Cover image must be under 10MB');
}

// ═══════════════════════════════════════════════════════════════════════════
//  ROUTES
// ═══════════════════════════════════════════════════════════════════════════

// ──────────────────────────── Profile routes ──────────────────────────────

app.get('/api/profile/:wallet', async (req, res) => {
  const profiles = await loadProfiles();
  const profile  = profiles[req.params.wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  if (profile.account_type === 'platform_nft_creator' || profile.platform_nft_address) {
    const stillHolds = await hasPlatformNft(req.params.wallet);
    if (!stillHolds && profile.account_type === 'platform_nft_creator') {
      profile.account_type     = 'creator';
      profile.royalty_fee_rate = FEES.PLATFORM_ROYALTY_STANDARD;
      profile.platform_nft_address = null;
      profiles[req.params.wallet] = profile;
      await saveProfiles(profiles);
      logger.info({ wallet: req.params.wallet }, 'Platform NFT no longer held — downgraded to creator'
    }
  }
```

```
  res.json(profile);
});

app.post('/api/create-profile', async (req, res) => {
  const { wallet: rawWallet, name, account_type = 'listener' } = req.body || {};
  if (!rawWallet || !name) return res.status(400).json({ error: 'Missing wallet or name' });
  if (!['listener', 'creator'].includes(account_type)) {
    return res.status(400).json({ error: 'account_type must be listener or creator' });
  }
  // Always store with lowercase key so lookups are case-insensitive
  const wallet = rawWallet.toLowerCase();

  const profiles = await loadProfiles();
  if (profiles[wallet]) return res.status(409).json({ error: 'Profile already exists' });

  const user_id = uuidv4();
  profiles[wallet] = {
    user_id,
    name,
    wallet_address: wallet,
    account_type,
    listener_tier:          account_type === 'listener' ? 1 : null,
    listener_plan:          null,
    subscription_start:     null,
    subscription_expiry:    null,
    platform_nft_address:   null,
    royalty_fee_rate:       account_type === 'creator' ? FEES.PLATFORM_ROYALTY_STANDARD : null,
    nft_contract_address:   null,
    playlist_cids:          [],
    favorites:              [],
    dj_settings: {
      tips_enabled_default: true,
    },
    supporter_subaccount: {
      enabled:               false,
      linked_creator_wallet: wallet,
      royalty_beneficiary_of: [],
    },
  };

  await saveProfiles(profiles);
  res.json(profiles[wallet]);
});

app.post('/api/update-profile', async (req, res) => {
  const { wallet, nftContractAddress, playlistCid, djTipsDefault, name } = req.body || {};
```

```
  const profiles = await loadProfiles();
  if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

  if (nftContractAddress) profiles[wallet].nft_contract_address = nftContractAddress;
  if (playlistCid) profiles[wallet].playlist_cids = (profiles[wallet].playlist_cids || []).concat(play
  if (typeof djTipsDefault === 'boolean') profiles[wallet].dj_settings.tips_enabled_default = djTipsDe
  if (name) profiles[wallet].name = name;

  await saveProfiles(profiles);
  res.json({ success: true });
});

// ──────────────────────────── Subscription routes ──────────────────────────────

app.post('/api/subscribe', async (req, res) => {
  const { wallet, plan } = req.body || {};
  if (!wallet || !plan) return res.status(400).json({ error: 'Missing wallet or plan' });

  const planDef = SUBSCRIPTION_PLANS[plan];
  if (!planDef) {
    return res.status(400).json({
      error: `Unknown plan. Valid plans: ${Object.keys(SUBSCRIPTION_PLANS).join(', ')}`
    });
  }

  const profiles = await loadProfiles();
  if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

  const profile  = profiles[wallet];
  const now      = Date.now();
  const msPerDay = 24 * 60 * 60 * 1000;

  if (planDef.type === 'listener' && profile.account_type !== 'listener') {
    return res.status(400).json({ error: 'Listener plans are only for listener accounts' });
  }
  if (planDef.type === 'creator' && profile.account_type !== 'creator') {
    return res.status(400).json({ error: 'Creator plans are only for creator accounts' });
  }
  if (planDef.type === 'nft_creator' && profile.account_type !== 'platform_nft_creator') {
    return res.status(400).json({ error: 'NFT creator plans require a Platform NFT' });
  }

  const baseTime = (plan.endsWith('_rolling') || !isSubscriptionActive(profile))
    ? now
    : profile.subscription_expiry;

  profile.listener_plan       = plan.endsWith('_annual')  ? 'annual'
```

```
                              : plan.endsWith('_rolling') ? 'rolling' : 'monthly';
  profile.subscription_start  = now;
  profile.subscription_expiry = baseTime + planDef.days * msPerDay;
  if (planDef.tier) profile.listener_tier = planDef.tier;
  profile.last_subscription_price_eth = usdToEth(planDef.price_usd);

  profiles[wallet] = profile;
  await saveProfiles(profiles);

  res.json({
    success:   true,
    plan,
    tier:      profile.listener_tier,
    expiry:    profile.subscription_expiry,
    price_usd: planDef.price_usd,
    price_eth: profile.last_subscription_price_eth,
  });
});

// ──────────────────────────── Platform NFT routes ──────────────────────────────

app.post('/api/claim-platform-nft', async (req, res) => {
  const { wallet, nft_token_id } = req.body || {};
  if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

  const profiles = await loadProfiles();
  if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

  const holdsNft = await hasPlatformNft(wallet);
  if (!holdsNft) {
    return res.status(403).json({ error: 'Platform NFT not detected in this wallet. Purchase it first.
  }

  const profile    = profiles[wallet];
  const wasCreator = profile.account_type === 'creator';

  profile.account_type         = 'platform_nft_creator';
  profile.platform_nft_address = platformNftAddress;
  profile.royalty_fee_rate     = FEES.PLATFORM_ROYALTY_NFT;

  if (wasCreator && isSubscriptionActive(profile)) {
    logger.info({ wallet }, 'Creator upgraded to Platform NFT — subscription continues at NFT rate');
  }

  if (!profile.supporter_subaccount) {
    profile.supporter_subaccount = {
      enabled: false,
```

```
      linked_creator_wallet: wallet,
      royalty_beneficiary_of: [],
    };
  }

  profiles[wallet] = profile;
  await saveProfiles(profiles);

  res.json({
    success:          true,
    account_type:     profile.account_type,
    royalty_fee_rate: profile.royalty_fee_rate,
    message:          'Platform NFT claimed. You now have creator capabilities with a 1.5% royalty fee
  });
});

app.post('/api/check-platform-nft', async (req, res) => {
  const { wallet } = req.body || {};
  if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

  const holdsNft = await hasPlatformNft(wallet);
  const profiles = await loadProfiles();
  const profile  = profiles[wallet];

  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  if (holdsNft && profile.account_type !== 'platform_nft_creator') {
    profile.account_type         = 'platform_nft_creator';
    profile.royalty_fee_rate     = FEES.PLATFORM_ROYALTY_NFT;
    profile.platform_nft_address = platformNftAddress;
    profiles[wallet] = profile;
    await saveProfiles(profiles);
  } else if (!holdsNft && profile.account_type === 'platform_nft_creator') {
    profile.account_type         = 'creator';
    profile.royalty_fee_rate     = FEES.PLATFORM_ROYALTY_STANDARD;
    profile.platform_nft_address = null;
    profiles[wallet] = profile;
    await saveProfiles(profiles);
  }

  res.json({ holds_nft: holdsNft, account_type: profile.account_type, royalty_fee_rate: profile.royalt
});

// ──────────────────────────── Supporter sub-account ────────────────────────────

app.post('/api/add-supporter-subaccount', async (req, res) => {
  const { wallet } = req.body || {};
```

```
  if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
    return res.status(403).json({ error: 'Only creator accounts can add a supporter sub-account' });
  }

  profile.supporter_subaccount = {
    enabled:               true,
    linked_creator_wallet: wallet,
    royalty_beneficiary_of: profile.supporter_subaccount?.royalty_beneficiary_of || [],
  };

  profiles[wallet] = profile;
  await saveProfiles(profiles);
  res.json({ success: true, supporter_subaccount: profile.supporter_subaccount });
});

app.post('/api/toggle-supporter-subaccount', async (req, res) => {
  const { wallet, enabled } = req.body || {};
  if (!wallet || typeof enabled !== 'boolean') return res.status(400).json({ error: 'Missing wallet or

  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  if (!profile.supporter_subaccount) {
    return res.status(400).json({ error: 'No supporter sub-account exists. Call /api/add-supporter-sub
  }

  profile.supporter_subaccount.enabled = enabled;
  profiles[wallet] = profile;
  await saveProfiles(profiles);
  res.json({ success: true, enabled });
});

// ──────────────────────────── Access check ──────────────────────────────────

app.get('/api/access/:wallet', async (req, res) => {
  const profiles = await loadProfiles();
  const profile  = profiles[req.params.wallet];
  const level    = getCapabilityLevel(profile);
  const tier     = getListenerTier(profile);
```

```
  res.json({
    level,
    tier,
    account_type:        profile?.account_type     || null,
    subscription_expiry: profile?.subscription_expiry || null,
    active:              isSubscriptionActive(profile),
    royalty_fee_rate:    profile?.royalty_fee_rate  || null,
    dj_tips_default:     profile?.dj_settings?.tips_enabled_default ?? true,
    supporter_enabled:   profile?.supporter_subaccount?.enabled || false,
  });
});

// ──────────────────────────── Royalty splits ──────────────────────────────────

app.post('/api/set-royalty-splits', async (req, res) => {
  const { wallet, cid, splits } = req.body || {};
  if (!wallet || !cid || !splits) return res.status(400).json({ error: 'Missing fields' });

  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
    return res.status(403).json({ error: 'Only creators can set royalty splits' });
  }

  const passive_total = (splits.passive || []).reduce((sum, p) => sum + (p.percent || 0), 0);
  const total = (splits.artist || 0) + (splits.nft_holders || 0) +
                (splits.activity_pool || 0) + passive_total;
  if (Math.abs(total - 100) > 0.01) {
    return res.status(400).json({ error: `Splits must sum to 100%. Got: ${total}%` });
  }

  for (const p of (splits.passive || [])) {
    const recipient = profiles[p.wallet];
    if (!recipient) continue;
    const level       = getCapabilityLevel(recipient);
    const isTier3     = level === 'listener_3';
    const isSupporter = recipient.supporter_subaccount?.enabled;
    if (!isTier3 && !isSupporter) {
      return res.status(400).json({
        error: `Passive split recipient ${p.wallet} must be a Tier 3 listener or active supporter.`
      });
    }
  }

  if (!profile.royalty_splits) profile.royalty_splits = {};
```

```
  profile.royalty_splits[cid] = splits;
  profiles[wallet] = profile;
  await saveProfiles(profiles);

  res.json({ success: true, splits });
});

// ──────────────────────────── Tips ──────────────────────────────────────────

app.post('/api/tip', async (req, res) => {
  const {
    from_wallet, to_wallet, tip_type, amount_eth,
    dj_set_id, artist_splits, dj_percent,
  } = req.body || {};

  if (!from_wallet || !tip_type || !amount_eth) {
    return res.status(400).json({ error: 'Missing from_wallet, tip_type, or amount_eth' });
  }

  const grossEth    = parseFloat(amount_eth);
  const platformCut = grossEth * FEES.PLATFORM_TIP;
  const remaining   = grossEth - platformCut;
  let distribution  = [];

  if (tip_type === 'artist') {
    if (!to_wallet) return res.status(400).json({ error: 'Missing to_wallet for artist tip' });
    distribution.push({ wallet: to_wallet, amount_eth: remaining, role: 'artist' });

  } else if (tip_type === 'dj') {
    if (!dj_set_id) return res.status(400).json({ error: 'Missing dj_set_id for DJ tip' });

    const djSets = await loadDjSets();
    const set    = djSets[dj_set_id];
    if (set && set.tips_enabled === false) {
      return res.status(403).json({ error: 'This DJ set has tips disabled.' });
    }

    const djCut         = remaining * ((dj_percent || 100) / 100);
    const artistPoolCut = remaining - djCut;

    distribution.push({ wallet: to_wallet || set?.dj_wallet, amount_eth: djCut, role: 'dj' });

    if (artistPoolCut > 0 && artist_splits?.length) {
      const totalArtistPct = artist_splits.reduce((s, a) => s + (a.percent || 0), 0);
      for (const a of artist_splits) {
        distribution.push({
          wallet:     a.wallet,
```

```
        amount_eth: artistPoolCut * ((a.percent || 0) / totalArtistPct),
        role:       'artist_from_dj_tip',
      });
    }
  }
} else {
  return res.status(400).json({ error: 'tip_type must be artist or dj' });
}

const profiles   = await loadProfiles();
const tipper     = profiles[from_wallet];
const recognized = tipper && tipper.account_type !== null && isSubscriptionActive(tipper);

logger.info({ tip_type, from_wallet, gross_eth: grossEth, platform_cut_eth: platformCut, distributio

res.json({
  success:      true,
  gross_eth:    grossEth,
  platform_cut: platformCut,
  distribution,
  recognized,
  message: recognized ? 'Tip sent — you will be credited as a supporter.' : 'Tip sent anonymously.',
});
});

// ──────────────────────────── NFT sale fee ──────────────────────────────────

app.post('/api/nft-sale-fee', async (req, res) => {
  const { sale_price_eth, nft_type, seller_wallet, is_primary } = req.body || {};
  if (!sale_price_eth || !nft_type || !seller_wallet) {
    return res.status(400).json({ error: 'Missing sale_price_eth, nft_type, or seller_wallet' });
  }

  const priceEth = parseFloat(sale_price_eth);

  if (nft_type === 'platform' && is_primary === true) {
    return res.json({ success: true, platform_fee: 0, seller_gets: priceEth, note: 'Platform NFT prima
  }

  if (nft_type === 'platform') {
    const floorEth = parseFloat(usdToEth(FEES.PLATFORM_NFT_PRICE_USD));
    if (priceEth < floorEth) {
      return res.status(400).json({
        error:     `Platform NFT cannot be sold below $${FEES.PLATFORM_NFT_PRICE_USD} USD (${floorEth}
        floor_eth: floorEth,
      });
    }
```

```
  }

  const platformFee = priceEth * FEES.PLATFORM_NFT_SALE;
  const sellerGets  = priceEth - platformFee;

  logger.info({ nft_type, seller_wallet, sale_price_eth, platform_fee: platformFee }, 'NFT sale fee ca

  res.json({
    success:          true,
    sale_price_eth:   priceEth,
    platform_fee_pct: FEES.PLATFORM_NFT_SALE * 100,
    platform_fee_eth: platformFee,
    seller_gets_eth:  sellerGets,
  });
});

// ──────────────────────────── Upload route ────────────────────────────────────

app.post(
  '/api/upload',
  upload.fields([{ name: 'audio-file', maxCount: 1 }, { name: 'cover-image', maxCount: 1 }]),
  async (req, res) => {
    // In DEV_MODE without IPFS, use local file storage under public/catalog/
const useLocalStorage = DEV_MODE;
console.log('[UPLOAD DIAG] DEV_MODE:', DEV_MODE, '| useLocalStorage:', useLocalStorage, '| ETH_RPC:',


    const tempFiles = [];
    const cleanup = async () => {
      for (const p of tempFiles) await fs.remove(p).catch(() => {});
    };

    try {
      const audioFile  = req.files?.['audio-file']?.[0];
      const coverImage = req.files?.['cover-image']?.[0];

      if (audioFile)  tempFiles.push(audioFile.path);
      if (coverImage) tempFiles.push(coverImage.path);

      const {
        contentType: rawContentType = 'music',
        songTitle, artistName, description = '',
        album = '', bpm = '', episodeNumber = '', seriesName = '',
        releaseDate, dateCreated, withdrawAddress,
        userId, wallet,
        tags, mlc_iswc, mlc_ipi_name_number, isrc,
        mintNft,
```

```
} = req.body || {};

const VALID_TYPES = ['music', 'podcast', 'video', 'art_still', 'art_animated'];
const contentType = VALID_TYPES.includes(rawContentType) ? rawContentType : 'music';
const isAudioOnly = contentType === 'music' || contentType === 'podcast';
const isVideoType = contentType === 'video' || contentType === 'art_animated';
const isArtStill  = contentType === 'art_still';

if (!audioFile)   return res.status(400).json({ error: 'Media file is required' });
if (!coverImage)  return res.status(400).json({ error: 'Cover image is required' });
if (!songTitle)   return res.status(400).json({ error: 'Title is required' });
if (!artistName)  return res.status(400).json({ error: 'Artist name is required' });
if (!userId || !wallet) return res.status(400).json({ error: 'Missing userId or wallet' });

// Tags are optional — used for discovery and activity pool routing.
// Minimum enforcement happens at the UI level only.
const parsedTags = String(tags || '').split(',').map(t => t.trim()).filter(Boolean);

// Access guard — DEV_WALLET bypasses profile check
if (!isDevWallet(wallet)) {
  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) return res.status(403).json({ error: 'Profile not found. Create a profile first.
  if (![creator', 'platform_nft_creator'].includes(profile.account_type)) {
    return res.status(403).json({ error: 'A creator account is required to upload content.' });
  }
}

if (/^(track\d+|song\d+|test\d*|untitled)\.(mp3|mp4|wav|webm)$/i.test(audioFile.originalname)) {
  return res.status(400).json({ error: 'Please rename your file before uploading.' });
}

const probeData = await validateQuality(audioFile.path, contentType, DEV_MODE).catch(err => {
  throw new Error('Quality check: ' + err.message);
});
await validateImage(coverImage.path, DEV_MODE).catch(err => {
  throw new Error('Cover image: ' + err.message);
});

const profiles = await loadProfiles();
const profile  = profiles[wallet] || { royalty_fee_rate: FEES.PLATFORM_ROYALTY_STANDARD, playlis

const contentId  = uuidv4();
const tempDir    = path.join(__dirname, 'temp', contentId);
const hlsDir     = path.join(tempDir, 'hls');
tempFiles.push(tempDir);
```

```
  await fs.ensureDir(isArtStill ? tempDir : hlsDir);

  const inputPath      = audioFile.path;
  const coverImagePath = coverImage.path;
  const previewPath    = path.join(tempDir, isVideoType ? 'preview.mp4' : 'preview.mp3');

  const audioData    = await fs.readFile(inputPath);
  const sha256Audio  = crypto.createHash('sha256').update(audioData).digest('hex');
  const blake3Audio  = blake3 ? blake3.hash(audioData).toString('hex') : null;

  const coverData    = await fs.readFile(coverImagePath);
  const sha256Cover  = crypto.createHash('sha256').update(coverData).digest('hex');
  const blake3Cover  = blake3 ? blake3.hash(coverData).toString('hex') : null;

  const contentMetadata = {
    id:            contentId,
    title:         songTitle,
    description,
    creator: {
      name:           artistName,
      user_id:        userId,
      wallet_address: ethers.utils.keccak256(ethers.utils.toUtf8Bytes(wallet)),
    },
    content_type:      contentType,
    availability_type: 'on_demand',
    release_date:      releaseDate || new Date().toISOString().split('T')[0],
    tags:              parsedTags,
    files:             {},
    royalty_fee_rate:  profile.royalty_fee_rate,
    integrityHashes: {
      sha256Audio, sha256Cover,
      ...(blake3Audio ? { blake3Audio } : {}),
      ...(blake3Cover ? { blake3Cover } : {}),
    },
    ...(isAudioOnly && {
      mlc_metadata: {
        work_title:      songTitle,
        iswc:            mlc_iswc            || '',
        isrc:            isrc                || '',
        ipi_name_number: mlc_ipi_name_number || '',
        writers: [{ name: artistName, role: 'artist', ipi_name_number: mlc_ipi_name_number || '',
        publishers: [],
      },
    }),
    ...(contentType === 'music' && {
      ...(album ? { album } : {}),
      ...(bpm   ? { bpm: parseInt(bpm, 10) || null } : {}),
```

```
  }),
  ...(contentType === 'podcast' && {
    ...(episodeNumber ? { episode_number: parseInt(episodeNumber, 10) || null } : {}),
    ...(seriesName    ? { series_name: seriesName } : {}),
  }),
  ...(isVideoType && {
    video: {
      width:    probeData?.videoStream?.width  || null,
      height:   probeData?.videoStream?.height || null,
      codec:    probeData?.videoStream?.codec_name || null,
      duration: parseFloat(probeData?.format?.duration || '0') || null,
    },
  }),
};

let folderCid = null;

// ── DEV_MODE: local catalog storage ──────────────────────────────────
if (useLocalStorage) {
  const catalogDir = path.join(process.cwd(), 'public', 'catalog', contentId);
  const catalogHls = path.join(catalogDir, 'hls');
  await fs.ensureDir(catalogHls);

  // Copy cover image to catalog
  const coverExt  = path.extname(coverImage.originalname) || '.jpg';
  const coverDest = path.join(catalogDir, 'cover' + coverExt);
  await fs.copy(coverImagePath, coverDest);
  contentMetadata.files.cover_image = `/catalog/${contentId}/cover${coverExt}`;

  if (isAudioOnly) {
    // Generate 30s preview
    await new Promise((resolve, reject) => {
      ffmpeg(inputPath)
        .outputOptions(['-t', '30', '-b:a', '128k', '-ar', '44100'])
        .audioCodec('libmp3lame')
        .output(previewPath)
        .on('end', resolve).on('error', reject)
        .run();
    }).catch(() => {});

    // Transcode to HLS (lo quality only in dev for speed)
    await new Promise((resolve, reject) => {
      ffmpeg(inputPath)
        .output(`${catalogHls}/lo.m3u8`)
          .audioCodec('aac').audioBitrate('128k').audioFrequency(44100)
          .format('hls')
          .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
```

```
        .addOption('-hls_segment_filename', `${catalogHls}/lo_%03d.ts`)
      .output(`${catalogHls}/hi.m3u8`)
        .audioCodec('aac').audioBitrate('320k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', `${catalogHls}/hi_%03d.ts`)
      .on('start', cmd => logger.info({ contentId }, 'FFmpeg DEV: ' + cmd))
      .on('end', resolve).on('error', reject)
      .run();
  });

  const master =
    '#EXTM3U\n#EXT-X-VERSION:3\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=128000,CODECS="mp4a.40.2"\nlo.m3u8\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=320000,CODECS="mp4a.40.2"\nhi.m3u8\n';
  await fs.writeFile(`${catalogHls}/master.m3u8`, master);

  if (await fs.pathExists(previewPath)) {
    await fs.copy(previewPath, path.join(catalogDir, 'preview.mp3'));
    contentMetadata.files.preview_url = `/catalog/${contentId}/preview.mp3`;
  }

  contentMetadata.ipfs_audio_url = `/catalog/${contentId}/hls/master.m3u8`;
  contentMetadata.files.hls_url  = contentMetadata.ipfs_audio_url;
} else {
  // Non-audio in dev — just copy file directly
  const mediaDest = path.join(catalogDir, 'media' + path.extname(audioFile.originalname));
  await fs.copy(inputPath, mediaDest);
  contentMetadata.files.media_url = `/catalog/${contentId}/media${path.extname(audioFile.origi
  contentMetadata.ipfs_audio_url  = contentMetadata.files.media_url;
  contentMetadata.files.hls_url   = contentMetadata.files.media_url;
}

// Write metadata JSON locally
const metadataStr    = JSON.stringify(contentMetadata, null, 2);
const sha256Metadata = crypto.createHash('sha256').update(metadataStr).digest('hex');
contentMetadata.integrityHashes.sha256Metadata = sha256Metadata;
await fs.writeFile(path.join(catalogDir, 'metadata.json'), JSON.stringify(contentMetadata, nul

// Store in profiles
const profiles2 = await loadProfiles();
if (profiles2[wallet]) {
  if (!profiles2[wallet].playlist_cids) profiles2[wallet].playlist_cids = [];
  profiles2[wallet].playlist_cids.push(`local:${contentId}`);
  await saveProfiles(profiles2);
}
```

```
  // Also save to catalog index
  const catalogIndexPath = path.resolve(process.cwd(), 'catalog.json');
  let catalogIndex = {};
  try { catalogIndex = JSON.parse(await fs.readFile(catalogIndexPath, 'utf8')); } catch (_) {}
  catalogIndex[contentId] = {
    contentId,
    title:        contentMetadata.title,
    artistName:   contentMetadata.creator.name,
    wallet,
    contentType,
    metadataUrl:  `/catalog/${contentId}/metadata.json`,
    hlsUrl:       contentMetadata.ipfs_audio_url,
    coverUrl:     contentMetadata.files.cover_image,
    previewUrl:   contentMetadata.files.preview_url || null,
    uploadedAt:   Date.now(),
  };
  await fs.writeFile(catalogIndexPath, JSON.stringify(catalogIndex, null, 2));

  await cleanup();
  logger.info({ contentId, contentType, wallet }, 'DEV upload complete — stored locally');

  return res.json({
    success:          true,
    contentId,
    contentType,
    hlsUrl:           contentMetadata.ipfs_audio_url,
    metadataUrl:      `/catalog/${contentId}/metadata.json`,
    metadataCid:      `local:${contentId}`,
    coverCid:         contentMetadata.files.cover_image,
    caSignature:      null,
    royalty_fee_rate: profile.royalty_fee_rate,
    mint_pending:     false,
    dev_mode:         true,
  });
}
// ── END DEV_MODE local storage ──────────────────────────────────────────

if (isArtStill) {
  const artAdd = await ipfs.add({ path: 'media' + path.extname(audioFile.originalname), content:
  contentMetadata.files.media_url = `ipfs://${artAdd.cid.toString()}`;

} else if (isAudioOnly) {
  await new Promise((resolve, reject) => {
    ffmpeg(inputPath)
      .outputOptions(['-t', '30', '-b:a', '128k', '-ar', '44100'])
      .audioCodec('libmp3lame')
      .output(previewPath)
```

```
      .on('end', resolve).on('error', reject)
      .run();
  });

  await new Promise((resolve, reject) => {
    ffmpeg(inputPath)
      .output(`${hlsDir}/lo.m3u8`)
        .audioCodec('aac').audioBitrate('128k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', `${hlsDir}/lo_%03d.ts`)
      .output(`${hlsDir}/mid.m3u8`)
        .audioCodec('aac').audioBitrate('256k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', `${hlsDir}/mid_%03d.ts`)
      .output(`${hlsDir}/hi.m3u8`)
        .audioCodec('aac').audioBitrate('320k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', `${hlsDir}/hi_%03d.ts`)
      .on('start', cmd => logger.info({ contentId }, 'FFmpeg: ' + cmd))
      .on('end', resolve).on('error', reject)
      .run();
  });

  const master =
    '#EXTM3U\n#EXT-X-VERSION:3\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=128000,CODECS="mp4a.40.2"\nlo.m3u8\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=256000,CODECS="mp4a.40.2"\nmid.m3u8\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=320000,CODECS="mp4a.40.2"\nhi.m3u8\n';
  await fs.writeFile(`${hlsDir}/master.m3u8`, master);

  const previewAdd = await ipfs.add({ path: 'preview.mp3', content: fs.createReadStream(previewP
  contentMetadata.files.preview_url = `ipfs://${previewAdd.cid.toString()}`;

  const { folderCid: fc } = await addDirectoryToIpfs(ipfs, hlsDir);
  folderCid = fc;
  contentMetadata.ipfs_audio_url  = `ipfs://${folderCid}/master.m3u8`;
  contentMetadata.files.hls_url   = contentMetadata.ipfs_audio_url;

} else {
  const srcH  = probeData?.videoStream?.height || 720;
  const ladder = [];
  if (srcH >= 1080) ladder.push({ h: 1080, w: 1920, vbr: '4000k', abr: '192k', name: '1080p' });
  if (srcH >= 720)  ladder.push({ h: 720,  w: 1280, vbr: '2500k', abr: '128k', name: '720p'  });
  ladder.push(       { h: 480,  w: 854,  vbr: '1200k', abr: '128k', name: '480p'  });
```

```
  await new Promise((resolve, reject) => {
    ffmpeg(inputPath)
      .outputOptions(['-ss', '0', '-t', '5', '-vf', 'scale=640:-1'])
      .output(previewPath)
      .on('end', resolve).on('error', e => { logger.warn('preview gen failed'); resolve(); })
      .run();
  }).catch(() => {});

  for (const rung of ladder) {
    await new Promise((resolve, reject) => {
      ffmpeg(inputPath)
        .videoCodec('libx264')
        .outputOptions([
          '-preset', 'veryfast', '-crf', '22',
          '-maxrate', rung.vbr, '-bufsize', `${parseInt(rung.vbr) * 2}k`,
          '-vf', `scale=${rung.w}:${rung.h}:force_original_aspect_ratio=decrease,pad=${rung.w}:$
          '-g', '48', '-keyint_min', '48',
        ])
        .audioCodec('aac').audioBitrate(rung.abr).audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '6').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', `${hlsDir}/${rung.name}_%03d.ts`)
        .output(`${hlsDir}/${rung.name}.m3u8`)
        .on('start', cmd => logger.info({ contentId, rung: rung.name }, 'FFmpeg: ' + cmd))
        .on('end', resolve).on('error', reject)
        .run();
    });
  }

  const bwMap  = { '1080p': 4500000, '720p': 2800000, '480p': 1400000 };
  const master = '#EXTM3U\n#EXT-X-VERSION:3\n' +
    ladder.map(r =>
      `#EXT-X-STREAM-INF:BANDWIDTH=${bwMap[r.name]},RESOLUTION=${r.w}x${r.h},CODECS="avc1.42e01e
    ).join('\n') + '\n';
  await fs.writeFile(`${hlsDir}/master.m3u8`, master);

  if (await fs.pathExists(previewPath)) {
    const previewAdd = await ipfs.add({ path: 'preview.mp4', content: fs.createReadStream(previe
    contentMetadata.files.preview_url = `ipfs://${previewAdd.cid.toString()}`;
  }

  const { folderCid: fc } = await addDirectoryToIpfs(ipfs, hlsDir);
  folderCid = fc;
  contentMetadata.ipfs_audio_url = `ipfs://${folderCid}/master.m3u8`;
  contentMetadata.files.hls_url  = contentMetadata.ipfs_audio_url;
}
```

```
const coverAdd = await ipfs.add({ path: 'cover' + path.extname(coverImage.originalname), content
contentMetadata.files.cover_image = `ipfs://${coverAdd.cid.toString()}`;

const metadataStr    = JSON.stringify(contentMetadata);
const sha256Metadata = crypto.createHash('sha256').update(metadataStr).digest('hex');
const blake3Metadata = blake3 ? blake3.hash(Buffer.from(metadataStr)).toString('hex') : null;
contentMetadata.integrityHashes.sha256Metadata = sha256Metadata;
if (blake3Metadata) contentMetadata.integrityHashes.blake3Metadata = blake3Metadata;

const { cid: metadataCid } = await ipfs.add(JSON.stringify(contentMetadata));
const metadataCidStr = metadataCid.toString();

let caSignature = null;
if (provider && mspWallet) {
  try {
    const network = await provider.getNetwork();
    const domain  = { name: 'ContentCA', version: '1', chainId: Number(network.chainId), verifyi
    const types   = { Certificate: [{ name: 'cid', type: 'string' }, { name: 'contentType', type
    caSignature   = await signEIP712(domain, types, { cid: metadataCidStr, contentType });
  } catch (e) {
    logger.warn({ contentId, err: e }, 'EIP-712 signing skipped (DEV_MODE or no provider)');
  }
}

if (profiles[wallet]) {
  if (!profiles[wallet].playlist_cids) profiles[wallet].playlist_cids = [];
  profiles[wallet].playlist_cids.push(metadataCidStr);
  await saveProfiles(profiles);
}

await cleanup();
logger.info({ contentId, contentType, wallet, metadataCidStr }, 'Upload complete');

res.json({
  success:          true,
  contentId,
  contentType,
  hlsUrl:           contentMetadata.ipfs_audio_url || contentMetadata.files?.media_url,
  metadataUrl:      `ipfs://${metadataCidStr}`,
  metadataCid:      metadataCidStr,
  coverCid:         contentMetadata.files.cover_image,
  caSignature,
  royalty_fee_rate: profile.royalty_fee_rate,
  mint_pending:     mintNft === 'true',
});
```

```
    } catch (err) {
      await cleanup();
      logger.error({ err }, 'Upload failed');
      res.status(400).json({ error: String(err.message || err) });
    }
  }
);

// ──────────────────────────── DJ set routes ──────────────────────────────────

const djSetsPath = path.resolve(process.cwd(), 'dj_sets.json');
fs.ensureFileSync(djSetsPath);

async function loadDjSets() {
  try { return JSON.parse(await fs.readFile(djSetsPath, 'utf8')); }
  catch { return {}; }
}
async function saveDjSets(sets) {
  await fs.writeFile(djSetsPath, JSON.stringify(sets, null, 2));
}

app.post('/api/start-dj-set', async (req, res) => {
  const { wallet, set_name, tips_enabled, dj_percent, artist_splits } = req.body || {};
  if (!wallet || !set_name) return res.status(400).json({ error: 'Missing wallet or set_name' });

  // DEV_WALLET bypass
  if (!isDevWallet(wallet)) {
    const profiles = await loadProfiles();
    const profile  = profiles[wallet];
    if (!profile) return res.status(403).json({ error: 'Profile not found' });

    const level = getCapabilityLevel(profile);
    const canDj = ['listener_2', 'listener_3', 'creator_active', 'nft_creator_active'].includes(level)
      || (profile.supporter_subaccount?.enabled && isSubscriptionActive(profile));

    if (!canDj) {
      return res.status(403).json({ error: 'A Tier 2 or higher subscription is required to host DJ set
    }
  }

  const profiles = await loadProfiles();
  const profile  = profiles[wallet] || {};
  const tipsForThisSet = typeof tips_enabled === 'boolean'
    ? tips_enabled
    : (profile.dj_settings?.tips_enabled_default ?? true);

  const setId = uuidv4();
```

```
  const sets  = await loadDjSets();
  sets[setId] = {
    set_id:        setId,
    dj_wallet:     wallet,
    set_name,
    tips_enabled:  tipsForThisSet,
    dj_percent:    dj_percent ?? 100,
    artist_splits: artist_splits || [],
    created_at:    Date.now(),
    active:        true,
  };
  await saveDjSets(sets);

  res.status(201).json({ success: true, set_id: setId, tips_enabled: tipsForThisSet });
});

app.post('/api/end-dj-set', async (req, res) => {
  const { wallet, set_id } = req.body || {};
  const sets = await loadDjSets();
  if (!sets[set_id] || sets[set_id].dj_wallet !== wallet) {
    return res.status(403).json({ error: 'Set not found or not owned by this wallet' });
  }
  sets[set_id].active = false;
  await saveDjSets(sets);
  res.json({ success: true });
});

// ──────────────────────────── Live concert route ──────────────────────────────

app.post('/api/start-live-encode', async (req, res) => {
  const { wallet, eventTitle, artistName, inputSource = 'rtmp://localhost/live/djset' } = req.body ||

  if (!wallet || !eventTitle || !artistName) {
    return res.status(400).json({ error: 'wallet, eventTitle, and artistName are required' });
  }

  // ── DEV BYPASS ──────────────────────────────────────────────────────────
  // DEV_WALLET skips the creator subscription check entirely.
  // Remove DEV_WALLET (set ETH_RPC + MSP_PRIVATE_KEY in .env) before production.
  if (!isDevWallet(wallet)) {
    const profiles = await loadProfiles();
    const profile  = profiles[wallet];
    const level    = getCapabilityLevel(profile);

    if (!['creator_active', 'nft_creator_active'].includes(level)) {
      return res.status(403).json({
        error: 'An active creator subscription is required to host live concerts.',
```

```
    });
  }
} else {
  logger.warn({ wallet }, 'DEV_WALLET bypass — skipping creator access check for live encode');
}
// ──────────────────────────────────────────────────────────────────────

const productionID = `${artistName.replace(/\s+/g, '_')}_${eventTitle.replace(/\s+/g, '_')}_${Date.n
const outputDir    = path.join(process.cwd(), 'public', 'live', productionID);
await fs.ensureDir(outputDir);

const ffmpegPath = process.env.FFMPEG_PATH || 'ffmpeg';
const ffmpegArgs = [
  '-i', inputSource,
  '-c:v', 'libx264', '-preset', 'veryfast',
  '-filter_complex', '[0:v]split=3[v1][v2][v3];[v1]scale=1280:720[v1out];[v2]scale=854:480[v2out];[v
  '-map', '[v1out]', '-b:v:0', '4000k', '-maxrate:v:0', '4000k', '-bufsize:v:0', '8000k',
  '-map', '[v2out]', '-b:v:1', '1500k', '-maxrate:v:1', '1500k', '-bufsize:v:1', '3000k',
  '-map', '[v3out]', '-b:v:2', '800k',  '-maxrate:v:2', '800k',  '-bufsize:v:2', '1600k',
  '-map', '0:a?', '-c:a', 'aac', '-b:a', '128k',
  '-f', 'hls', '-hls_time', '4', '-hls_playlist_type', 'event',
  '-hls_segment_filename', `${outputDir}/v%v_%03d.ts`,
  '-master_pl_name', 'master.m3u8',
  '-var_stream_map', 'v:0,a:0 v:1,a:0 v:2,a:0',
  '-hls_flags', 'independent_segments',
  `${outputDir}/v%v.m3u8`,
  '-vf', 'fps=1/10,scale=320:-1', '-update', '1', `${outputDir}/thumbnail.jpg`,
];

try {
  const { spawn } = require('child_process');
  const proc = spawn(ffmpegPath, ffmpegArgs, { stdio: ['pipe', 'pipe', 'pipe'] });
  proc.stderr.on('data', d => logger.debug(`FFmpeg: ${d}`));
  proc.on('error', err => logger.error({ productionID, err }, 'FFmpeg error'));
  proc.on('close', code => logger.info({ productionID, code }, 'Live encode closed'));
  logger.info({ productionID, inputSource }, 'Live encode started');

  res.status(201).json({
    success:      true,
    productionID,
    hlsUrl:       `/live/${productionID}/master.m3u8`,
    thumbnailUrl: `/live/${productionID}/thumbnail.jpg`,
  });
} catch (error) {
  logger.error({ productionID, err: error.message }, 'Failed to start live encode');
  res.status(500).json({ success: false, error: error.message });
}
```

```
});

// ──────────────────────────── Play token & proof ────────────────────────────

app.post('/api/request-play-token', async (req, res) => {
  const { cid, listener, live, playlistId } = req.body || {};
  if (!cid || !listener) return res.status(400).json({ error: 'Missing fields' });

  // DEV_WALLET bypass — skip subscription check
  if (!isDevWallet(listener)) {
    const profiles = await loadProfiles();
    const profile  = profiles[listener];
    const level    = getCapabilityLevel(profile);

    const canStream = ['listener_1', 'listener_2', 'listener_3',
                       'creator_active', 'nft_creator_active'].includes(level);
    if (!canStream) {
      return res.status(403).json({
        error:        'Active subscription required to stream.',
        pay_per_play: true,
      });
    }
  }

  const playId  = uuidv4();
  const payload = { playId, cid, listener, live: !!live, playlistId, iat: Math.floor(Date.now() / 1000
  const token   = jwt.sign(payload, process.env.PLAY_TOKEN_SECRET || 'dev-play-secret', { algorithm: '
  res.json({ playToken: token });
});

app.post('/api/submit-play-proof', async (req, res) => {
  const { playToken } = req.body || {};
  try {
    const decoded   = jwt.verify(playToken, process.env.PLAY_TOKEN_SECRET || 'dev-play-secret');
    const replayKey = `play:${decoded.playId}`;
    if (await redisClient?.exists(replayKey)) throw new Error('Replay detected');
    await redisClient?.set(replayKey, 'used', { EX: 86400 });

    // In DEV_MODE, local catalog CIDs start with 'local:' — skip IPFS fetch
    if (decoded.cid && decoded.cid.startsWith('local:')) {
      const contentId  = decoded.cid.replace('local:', '');
      const metaPath   = path.join(process.cwd(), 'public', 'catalog', contentId, 'metadata.json');
      try {
        const raw = await fs.readFile(metaPath, 'utf8');
        const fetchedMetadata = JSON.parse(raw);
        logger.info({ playId: decoded.playId, contentId }, 'DEV play proof — local catalog metadata lo
      } catch (e) {
```

```
        logger.warn({ contentId }, 'DEV play proof — local metadata not found, skipping integrity chec
      }
    } else {
      // Production path — fetch from IPFS gateway
      const gatewayUrl   = `${IPFS_GATEWAY}${decoded.cid}`;
      const metaResponse = await fetch(gatewayUrl);
      if (!metaResponse.ok) throw new Error(`IPFS fetch failed: ${metaResponse.status}`);
      const fetchedMetadata = await metaResponse.json();

      const recomputedSha = crypto.createHash('sha256').update(JSON.stringify(fetchedMetadata)).digest
      if (recomputedSha !== fetchedMetadata.integrityHashes?.sha256Metadata) {
        throw new Error('Metadata integrity failed (sha256 mismatch)');
      }
      if (blake3 && fetchedMetadata.integrityHashes?.blake3Metadata) {
        const recomputedB3 = blake3.hash(Buffer.from(JSON.stringify(fetchedMetadata))).toString('hex')
        if (recomputedB3 !== fetchedMetadata.integrityHashes.blake3Metadata) {
          throw new Error('Metadata integrity failed (blake3 mismatch)');
        }
      }
    }

    if (streamingContract) {
      const metadataHash = ethers.utils.keccak256(
        ethers.utils.defaultAbiCoder.encode(
          ['bytes32', 'string', 'address', 'bool'],
          [decoded.playId, decoded.cid, decoded.listener, !!decoded.live]
        )
      );
      const tx = await streamingContract.logPlay(
        ethers.utils.keccak256(ethers.utils.toUtf8Bytes(decoded.playId)),
        decoded.cid, decoded.listener, !!decoded.live, metadataHash
      );
      await tx.wait();
    } else {
      logger.warn({ playId: decoded.playId }, 'logPlay skipped — DEV_MODE (no chain connection)');
    }
    res.json({ success: true });
  } catch (error) {
    logger.error({ err: error }, 'Proof verification failed');
    res.status(400).json({ error: 'Invalid proof' });
  }
});

// ──────────────────────────── Utility routes ────────────────────────────────────

app.get('/api/playlists', async (req, res) => {
  const profiles = await loadProfiles();
```

```
  const playlists = Object.values(profiles)
    .flatMap(p => p.playlist_cids || [])
    .filter(cid => !cid.startsWith('local:'))
    .map(cid => ({ id: cid, name: `Playlist ${cid.slice(0, 8)}`, curator: 'User', cids: ['mockcid'] })
  res.json(playlists);
});

// ──────────────────────────── Catalog route (DEV_MODE local uploads) ───────────
app.get('/api/catalog', async (req, res) => {
  const catalogIndexPath = path.resolve(process.cwd(), 'catalog.json');
  try {
    const raw = await fs.readFile(catalogIndexPath, 'utf8');
    const index = JSON.parse(raw);
    res.json(Object.values(index).sort((a, b) => b.uploadedAt - a.uploadedAt));
  } catch (_) {
    res.json([]);
  }
});

app.get('/api/catalog/:contentId/metadata', async (req, res) => {
  const metaPath = path.join(process.cwd(), 'public', 'catalog', req.params.contentId, 'metadata.json'
  try {
    const raw = await fs.readFile(metaPath, 'utf8');
    res.json(JSON.parse(raw));
  } catch (_) {
    res.status(404).json({ error: 'Not found' });
  }
});

app.get('/api/live-concerts', async (req, res) => {
  // Return real active live sessions from the in-memory store
  const active = [];
  liveSessions.forEach((s) => {
    if (s.alive) {
      active.push({
        cid:             s.sessionId,   // used as the join key
        sessionId:       s.sessionId,
        artist:          s.artistName,
        artistWallet:    s.wallet,
        title:           s.title,
        contractAddress: null,
        live:            true,
        hlsUrl:          `/live/${s.sessionId}/master.m3u8`,
        viewerCount:     s.viewerCount,
        duration:        Math.floor((Date.now() - s.startTime) / 1000),
        thumbnailUrl:    `/live/${s.sessionId}/thumbnail.jpg`,
      });
```

```
    }
  });
  res.json(active);
});

app.post('/api/convert-currency', async (req, res) => {
  const { amountEth, to } = req.body || {};
  const rate     = parseFloat(process.env.ETH_USD_RATE || '2500');
  const ethFloat = parseFloat(amountEth || '0');
  const rates    = { usd: rate, btc: rate / 65000, sol: rate / 150, zec: rate / 20 };
  const amount   = ethFloat * (rates[(to || '').toLowerCase()] || 1);
  res.json({ [`amount${(to || '').toUpperCase()}`]: amount });
});

app.get('/api/fees', (req, res) => {
  res.json({
    royalty_standard_pct:   FEES.PLATFORM_ROYALTY_STANDARD * 100,
    royalty_nft_pct:        FEES.PLATFORM_ROYALTY_NFT * 100,
    tip_pct:                FEES.PLATFORM_TIP * 100,
    nft_sale_pct:           FEES.PLATFORM_NFT_SALE * 100,
    platform_nft_price_usd: FEES.PLATFORM_NFT_PRICE_USD,
    subscription_plans:     SUBSCRIPTION_PLANS,
  });
});

// ──────────────────────────── NFTs route ────────────────────────────────────

app.get('/api/nfts', async (req, res) => {
  try {
    const profiles = await loadProfiles();
    const nfts = [];
    for (const [wallet, profile] of Object.entries(profiles)) {
      if (!profile.nft_contract_address) continue;
      const cids = profile.playlist_cids || [];
      for (const cid of cids) {
        nfts.push({
          metadataCid:     cid,
          contractAddress: profile.nft_contract_address,
          artistWallet:    wallet,
          artist:          profile.name || 'Unknown',
          title:           `Track ${cid.slice(0, 8)}`,
          cover_image:     null,
          price_eth:       null,
          tokenId:         null,
        });
      }
    }
```

```
    res.json(nfts);
  } catch (err) {
    logger.error({ err }, '/api/nfts failed');
    res.status(500).json({ error: 'Failed to fetch NFTs' });
  }
});

// ──────────────────────────── Playlist routes ────────────────────────────────

app.post('/api/create-playlist', async (req, res) => {
  const { wallet, name, cids, sharePercent = 8 } = req.body || {};
  if (!wallet || !name || !Array.isArray(cids) || !cids.length) {
    return res.status(400).json({ error: 'Missing wallet, name, or cids' });
  }

  const profiles = await loadProfiles();
  if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

  const level = getCapabilityLevel(profiles[wallet]);
  if (!['listener_2','listener_3','creator_active','nft_creator_active'].includes(level)) {
    return res.status(403).json({ error: 'Tier 2 or higher subscription required to create playlists'
  }

  const playlistId = uuidv4();
  const playlist   = { id: playlistId, name, cids, wallet, sharePercent, createdAt: Date.now() };

  if (!profiles[wallet].playlists) profiles[wallet].playlists = [];
  profiles[wallet].playlists.push(playlist);
  await saveProfiles(profiles);

  logger.info({ wallet, playlistId, name }, 'Playlist created');
  res.status(201).json({ success: true, playlist });
});

// ──────────────────────────── Favorites routes ───────────────────────────────
//
// Favorites are private — never exposed in public profile responses.
// Favoriting alone generates NO royalty event.
// Any complete playthrough of a favorited track IS royalty-eligible (same as any play).
//
// SPEC: Favorites are available to ALL user roles — no subscription gate.

app.get('/api/favorites/:wallet', async (req, res) => {
  const { wallet } = req.params;
  const profiles   = await loadProfiles();
  const profile    = profiles[wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });
```

```
  res.json({ favorites: profile.favorites || [] });
});

app.post('/api/favorites/add', async (req, res) => {
  const { wallet, cid } = req.body || {};
  if (!wallet || !cid) return res.status(400).json({ error: 'Missing wallet or cid' });

  const profiles = await loadProfiles();
  if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

  // NO subscription gate — favorites are available to all user roles per spec.
  // The only requirement is a valid profile (wallet registered on MSP).

  if (!profiles[wallet].favorites) profiles[wallet].favorites = [];
  if (!profiles[wallet].favorites.includes(cid)) {
    profiles[wallet].favorites.push(cid);
    await saveProfiles(profiles);
  }

  res.json({ success: true, favorites: profiles[wallet].favorites });
});

app.post('/api/favorites/remove', async (req, res) => {
  const { wallet, cid } = req.body || {};
  if (!wallet || !cid) return res.status(400).json({ error: 'Missing wallet or cid' });

  const profiles = await loadProfiles();
  if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

  profiles[wallet].favorites = (profiles[wallet].favorites || []).filter(c => c !== cid);
  await saveProfiles(profiles);
  res.json({ success: true, favorites: profiles[wallet].favorites });
});

app.post('/api/favorites/convert-to-playlist', async (req, res) => {
  const { wallet, name, cids } = req.body || {};
  if (!wallet || !name || !Array.isArray(cids) || !cids.length) {
    return res.status(400).json({ error: 'Missing wallet, name, or cids' });
  }

  const profiles = await loadProfiles();
  if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

  const level = getCapabilityLevel(profiles[wallet]);
  if (!['listener_2','listener_3','creator_active','nft_creator_active'].includes(level)) {
    return res.status(403).json({ error: 'Tier 2 or higher required to create playlists from Favorites
  }
```

```
  const playlistId = uuidv4();
  const playlist   = { id: playlistId, name, cids, wallet, sharePercent: 8, fromFavorites: true, creat

  if (!profiles[wallet].playlists) profiles[wallet].playlists = [];
  profiles[wallet].playlists.push(playlist);
  await saveProfiles(profiles);

  logger.info({ wallet, playlistId, name, cidsCount: cids.length }, 'Favorites converted to playlist')
  res.status(201).json({ success: true, playlist });
});

// ──────────────────────────── Live session routes ──────────────────────────────
//
// In-memory session store — sessions are transient (lost on server restart).
// Each session: { sessionId, wallet, title, artistName, quality, startTime,
//                 chunks:[], chunkCount, viewerCount, peakViewers, tipsTotal }

const liveSessions = new Map();

// Accept binary chunk uploads — needs express.raw() for this route only
app.post('/api/live-ingest/:sessionId',
  express.raw({ type: 'application/octet-stream', limit: '20mb' }),
  (req, res) => {
    const { sessionId } = req.params;
    const session = liveSessions.get(sessionId);
    if (!session) return res.status(404).json({ error: 'Session not found' });
    if (!session.alive) return res.status(410).json({ status: 'ended_unexpectedly' });

    const elapsed     = Date.now() - session.startTime;
    const accountType = session.accountType || 'creator';
    const cap         = STREAM_DURATION_LIMITS[accountType] ?? STREAM_DURATION_LIMITS.creator;
    if (elapsed > cap) {
      session.alive = false; session.endTime = Date.now();
      broadcastToSession(session.sessionId, { type: 'stream_ended', sessionId: session.sessionId, reas
      return res.status(410).json({ status: 'duration_cap_reached',
        error: accountType === 'creator' ? 'Your 3-hour stream limit has been reached. Upgrade to Plat
        cap_ms: cap });
    }

    // Store chunk in memory (for archive/discard later)
    if (req.body && req.body.length) {
      session.chunks.push(req.body);
      session.chunkCount++;
    }
    res.json({ success: true, chunkCount: session.chunkCount });
  }
```

```
);

app.post('/api/live-start', async (req, res) => {
  const { wallet, title, artistName, quality = '720p' } = req.body || {};
  if (!wallet || !title || !artistName) {
    return res.status(400).json({ error: 'Missing wallet, title, or artistName' });
  }

  // Access guard — DEV_WALLET bypasses
  if (!isDevWallet(wallet)) {
    const profiles = await loadProfiles();
    const level    = getCapabilityLevel(profiles[wallet]);
    if (!['creator_active', 'nft_creator_active'].includes(level)) {
      return res.status(403).json({ error: 'Active creator subscription required to go live.' });
    }
  }

  const sessionId = uuidv4();
  liveSessions.set(sessionId, {
    sessionId,
    wallet,
    title,
    artistName,
    quality,
    startTime:   Date.now(),
    alive:       true,
    accountType: (await loadProfiles().then(p => p[wallet]?.account_type)) || 'creator',
    chunks:      [],
    chunkCount:  0,
    viewerCount: 0,
    peakViewers: 0,
    tipsTotal:   0,
  });

  logger.info({ sessionId, wallet, title }, 'Live session started');

  res.status(201).json({
    success:      true,
    sessionId,
    hlsUrl:       `/live/${sessionId}/master.m3u8`,
    thumbnailUrl: `/live/${sessionId}/thumbnail.jpg`,
  });
});

app.post('/api/live-end/:sessionId', async (req, res) => {
  const { sessionId } = req.params;
  const { wallet }    = req.body || {};
```

```
  const session       = liveSessions.get(sessionId);

  if (!session) return res.status(404).json({ error: 'Session not found' });
  if (session.wallet !== wallet && !isDevWallet(wallet)) {
    return res.status(403).json({ error: 'Not your session' });
  }

  session.alive    = false;
  session.endTime  = Date.now();
  const duration   = Math.floor((session.endTime - session.startTime) / 1000);

  // Notify all WebSocket clients in this session that the stream ended
  broadcastToSession(sessionId, { type: 'stream_ended', sessionId });

  logger.info({ sessionId, duration, chunkCount: session.chunkCount }, 'Live session ended');

  res.json({
    success:     true,
    sessionId,
    duration,
    chunkCount:  session.chunkCount,
    tipsTotal:   session.tipsTotal,
    peakViewers: session.peakViewers,
    archiveCid:  null,   // populated after /api/live-archive
  });
});

app.get('/api/live-sessions', (req, res) => {
  const active = [];
  liveSessions.forEach((s) => {
    if (s.alive) {
      active.push({
        sessionId:    s.sessionId,
        title:        s.title,
        artistName:   s.artistName,
        viewerCount:  s.viewerCount,
        duration:     Math.floor((Date.now() - s.startTime) / 1000),
        thumbnailUrl: `/live/${s.sessionId}/thumbnail.jpg`,
      });
    }
  });
  res.json(active);
});

app.post('/api/live-archive', async (req, res) => {
  const { sessionId, wallet, title, description, tags, mintNft } = req.body || {};
  const session = liveSessions.get(sessionId);
```

```
  if (!session) return res.status(404).json({ error: 'Session not found' });
  if (session.wallet !== wallet && !isDevWallet(wallet)) {
    return res.status(403).json({ error: 'Not your session' });
  }
  if (!ipfs) {
    // IPFS not running — return a stub response so the modal closes cleanly
    logger.warn({ sessionId }, 'live-archive: IPFS not configured — returning stub');
    return res.json({
      success:    true,
      sessionId,
      archiveCid: 'QmDevStub_' + sessionId.slice(0, 8),
      mintNft:    !!mintNft,
      note:       'DEV MODE — IPFS not running, archive not actually stored',
    });
  }

  try {
    // Assemble all chunks into one buffer
    const total  = session.chunks.reduce((n, c) => n + c.length, 0);
    const merged = Buffer.concat(session.chunks, total);
    const added  = await ipfs.add({ path: `${sessionId}.webm`, content: merged });
    const archiveCid = added.cid.toString();

    logger.info({ sessionId, archiveCid, mintNft }, 'Live session archived');
    res.json({ success: true, sessionId, archiveCid, mintNft: !!mintNft });
  } catch (err) {
    logger.error({ err, sessionId }, 'Live archive failed');
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/live-discard', async (req, res) => {
  const { sessionId, wallet } = req.body || {};
  const session = liveSessions.get(sessionId);
  if (!session) return res.status(404).json({ error: 'Session not found' });
  if (session.wallet !== wallet && !isDevWallet(wallet)) {
    return res.status(403).json({ error: 'Not your session' });
  }

  session.chunks = [];   // free memory
  liveSessions.delete(sessionId);
  logger.info({ sessionId }, 'Live session discarded');
  res.json({ success: true });
});
// ──────────────────────────── Download recording ──────────────────────────────
// Concatenates recorded WebM chunks and returns as a file download.
app.post('/api/live-download/:sessionId', (req, res) => {
```

```
  const { sessionId } = req.params;
  const { wallet }    = req.body || {};
  const session       = liveSessions.get(sessionId);

  if (!session) return res.status(404).json({ error: 'Session not found' });
  if (session.wallet !== wallet && !isDevWallet(wallet)) {
    return res.status(403).json({ error: 'Not your session' });
  }
  if (!session.chunks || !session.chunks.length) {
    return res.status(404).json({ error: 'No recording data available' });
  }

  const total    = session.chunks.reduce((n, c) => n + c.length, 0);
  const merged   = Buffer.concat(session.chunks, total);
  const safeName = (session.title || 'stream').replace(/[^a-z0-9_\-\s]/gi, '_').replace(/\s+/g, '_');
  const filename = safeName + '_' + new Date().toISOString().slice(0, 10) + '.webm';

  logger.info({ sessionId, bytes: total, filename }, 'Live recording download served');
  res.setHeader('Content-Type', 'video/webm');
  res.setHeader('Content-Disposition', `attachment; filename="${filename}"`);
  res.setHeader('Content-Length', total);
  res.send(merged);
});


// ──────────────────────────── Stream key routes ────────────────────────────

app.get('/api/stream-key/:wallet', async (req, res) => {
  const profiles = await loadProfiles();
  const profile  = profiles[req.params.wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
    return res.status(403).json({ error: 'Creator account required for stream key' });
  }

  // Generate a key if none exists
  if (!profile.stream_key) {
    profile.stream_key = uuidv4().replace(/-/g, '');
    profiles[req.params.wallet] = profile;
    await saveProfiles(profiles);
  }

  const rtmpHost = process.env.RTMP_HOST || 'rtmp://localhost/live';
  res.json({
    streamKey:   profile.stream_key,
    rtmpUrl:     rtmpHost,
```

```
    fullUrl:     `${rtmpHost}/${profile.stream_key}`,
    playbackUrl: `/live/${profile.stream_key}/master.m3u8`,
  });
});

app.post('/api/stream-key/regenerate', async (req, res) => {
  const { wallet } = req.body || {};
  if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  profile.stream_key = uuidv4().replace(/-/g, '');
  profiles[wallet]   = profile;
  await saveProfiles(profiles);

  const rtmpHost = process.env.RTMP_HOST || 'rtmp://localhost/live';
  logger.info({ wallet }, 'Stream key regenerated');
  res.json({
    streamKey:   profile.stream_key,
    rtmpUrl:     rtmpHost,
    fullUrl:     `${rtmpHost}/${profile.stream_key}`,
    playbackUrl: `/live/${profile.stream_key}/master.m3u8`,
  });
});

// ──────────────────────────── Error handlers ──────────────────────────────

app.use((req, res) => res.status(404).json({ error: 'Not found' }));
// eslint-disable-next-line no-unused-vars
app.use((err, req, res, next) => {
  logger.error({ err }, 'Unhandled error');
  res.status(500).json({ error: 'Internal server error' });
});

// ──────────────────────────── WebSocket helpers ───────────────────────────

// wsSessionClients: sessionId -> Set of ws connections
const wsSessionClients = new Map();

function broadcastToSession(sessionId, msg) {
  const clients = wsSessionClients.get(sessionId);
  if (!clients) return;
  const text = JSON.stringify(msg);
  clients.forEach((ws) => {
    if (ws.readyState === 1 /* OPEN */) ws.send(text);
```

```
  });
}

function attachWebSocket(server) {
  let WebSocketServer;
  try {
    ({ WebSocketServer } = require('ws'));
  } catch (e) {
    try {
      // Some versions export the constructor directly
      const ws = require('ws');
      WebSocketServer = ws.Server || ws;
    } catch (e2) {
      logger.warn('ws package not found — WebSocket disabled. Run: npm install ws');
      return;
    }
  }

  const wss = new WebSocketServer({ server, path: '/ws' });
  logger.info('WebSocket server attached at /ws');

  wss.on('connection', (ws) => {
    let sessionId = null;
    let role      = 'viewer';

    ws.on('message', (raw) => {
      let msg;
      try { msg = JSON.parse(raw); } catch { return; }

      if (msg.type === 'join_session') {
        sessionId = msg.sessionId;
        role      = msg.name === 'HOST' ? 'host' : 'viewer';

        if (!wsSessionClients.has(sessionId)) wsSessionClients.set(sessionId, new Set());
        wsSessionClients.get(sessionId).add(ws);

        // Update viewer count
        if (role === 'viewer') {
          const session = liveSessions.get(sessionId);
          if (session) {
            session.viewerCount++;
            if (session.viewerCount > session.peakViewers) session.peakViewers = session.viewerCount;
            broadcastToSession(sessionId, { type: 'viewer_count', viewerCount: session.viewerCount });
          }
        }
        return;
      }
```

```
      if (msg.type === 'ping') {
        ws.send(JSON.stringify({ type: 'pong' }));
        return;
      }

      // Route chat / reactions / tips to everyone in the session
      if (sessionId && ['chat', 'reaction', 'tip', 'tip_alert'].includes(msg.type)) {
        if (msg.type === 'tip' && sessionId) {
          const session = liveSessions.get(sessionId);
          if (session && msg.amount) session.tipsTotal += parseFloat(msg.amount) || 0;
        }
        broadcastToSession(sessionId, msg);
      }
    });

    ws.on('close', () => {
      if (!sessionId) return;
      const clients = wsSessionClients.get(sessionId);
      if (clients) {
        clients.delete(ws);
        if (!clients.size) wsSessionClients.delete(sessionId);
      }
      // Decrement viewer count
      if (role === 'viewer') {
        const session = liveSessions.get(sessionId);
        if (session && session.viewerCount > 0) {
          session.viewerCount--;
          broadcastToSession(sessionId, { type: 'viewer_count', viewerCount: session.viewerCount });
        }
      }
    });

    ws.on('error', () => ws.close());
  });
}

// ──────────────────────────── Server start ────────────────────────────────────

const PORT = Number(process.env.PORT || 3001);

(async () => {
  try {
    const net = require('net');
    const isPortInUse = (port) => new Promise(resolve => {
      const t = net.createServer();
      t.once('error', () => resolve(true));
```

```
      t.once('listening', () => { t.close(); resolve(false); });
      t.listen(port);
    });

    if (process.env.TLS_KEY_PATH && process.env.TLS_CERT_PATH) {
      if (!fs.existsSync(process.env.TLS_KEY_PATH) || !fs.existsSync(process.env.TLS_CERT_PATH)) {
        throw new Error('TLS certificate files not found');
      }
      const tlsOptions = {
        key:  fs.readFileSync(process.env.TLS_KEY_PATH),
        cert: fs.readFileSync(process.env.TLS_CERT_PATH),
        minVersion:       'TLSv1.3',
        ciphers:          'TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128_GCM_SHA256'
        honorCipherOrder: true,
      };
      const server = require('https').createServer(tlsOptions, app);
      attachWebSocket(server);
      server.listen(443, () => logger.info('HTTPS server started on port 443'));

      const port80InUse = await isPortInUse(80);
      if (!port80InUse) {
        require('http').createServer((req, res) => {
          res.writeHead(301, { Location: `https://${req.headers.host}${req.url}` });
          res.end();
        }).listen(80, () => logger.info('HTTP redirect on port 80'));
      }
    } else {
      throw new Error('TLS not configured — falling back to HTTP');
    }
  } catch (e) {
    logger.warn(`${e.message} — starting HTTP on port ${PORT}`);
    const httpServer = require('http').createServer(app);
    attachWebSocket(httpServer);
    httpServer.listen(PORT, () => logger.info(`HTTP server on http://localhost:${PORT}`));
  }
})();

module.exports = { app, FEES, SUBSCRIPTION_PLANS };
```

`server_live.js` (26.6 KB)

```javascript
/**
 * MSP LIVE STREAMING ADDITIONS
 * Merge these blocks into server.cjs at the marked positions.
 *
```

```
 * NEW DEPENDENCIES (npm install):
 *   ws           — WebSocket server
 *
 * BUILT-IN (no install):
 *   stream.PassThrough
 *   child_process.spawn
 *   http.createServer  (replace app.listen with this)
 *
 * ─── INTEGRATION CHECKLIST ──────────────────────────────────────────────
 *  [1] Add requires at top of server.cjs
 *  [2] Add activeSessions Map after requires
 *  [3] Add WebSocket server setup (replaces app.listen at bottom)
 *  [4] Add all new routes (live-start, live-ingest, live-end, live-archive, live-discard)
 *  [5] Replace /api/live-concerts stub with real /api/live-sessions
 *  [6] Add broadcastToSession and broadcastToAll helpers
 */

'use strict';

// ═══════════════════════════════════════════════════════════════════════════
// [1]  NEW REQUIRES — add near top of server.cjs after existing requires
// ═══════════════════════════════════════════════════════════════════════════

const { PassThrough }  = require('stream');
const http             = require('http');
const { WebSocketServer } = require('ws');
const { spawn }        = require('child_process');

// ═══════════════════════════════════════════════════════════════════════════
// [2]  IN-MEMORY SESSION STORE — add after constants (FEES / SUBSCRIPTION_PLANS)
// ═══════════════════════════════════════════════════════════════════════════

/**
 * activeSessions — keyed by sessionId (uuid)
 * Each session:
 * {
 *   sessionId, wallet, title, artistName, startTime,
 *   hlsDir, hlsUrl, thumbnailUrl,
 *   passThrough,       ← Node stream piped to ffmpeg stdin
 *   ffmpegProc,        ← spawned ffmpeg child process
 *   viewers: Map(ws → { wallet, name }),
 *   chatHistory: [],   ← last 200 messages, sent to late joiners
 *   tipsTotal: 0,
 *   peakViewers: 0,
 *   status: 'live' | 'ended_clean' | 'ended_unexpectedly' | 'archived' | 'discarded',
 *   archiveCid: null,
 *   endTime: null,
```

```
 *   chunkCount: 0,
 * }
 */
const activeSessions = new Map();

// Persist session metadata to disk so recovery survives server restarts
const liveSessionsPath = path.resolve(process.cwd(), 'live_sessions.json');
fs.ensureFileSync(liveSessionsPath);

async function loadLiveSessions() {
  try { return JSON.parse(await fs.readFile(liveSessionsPath, 'utf8')); }
  catch { return {}; }
}
async function saveLiveSession(sessionId, meta) {
  const all = await loadLiveSessions();
  all[sessionId] = meta;
  await fs.writeFile(liveSessionsPath, JSON.stringify(all, null, 2));
}

// ═══════════════════════════════════════════════════════════════════════
// [3]  WEBSOCKET HELPERS — add after activeSessions, before routes
// ═══════════════════════════════════════════════════════════════════════

// wss is defined in [6] (server startup). Forward-declared here for route use.
let wss = null;

/** Broadcast a JSON message to all viewers in a session */
function broadcastToSession(sessionId, msg) {
  const session = activeSessions.get(sessionId);
  if (!session) return;
  const payload = JSON.stringify(msg);
  for (const [ws] of session.viewers) {
    if (ws.readyState === 1 /* OPEN */) ws.send(payload);
  }
}

/** Broadcast to ALL connected WebSocket clients (e.g. new session alert) */
function broadcastToAll(msg) {
  if (!wss) return;
  const payload = JSON.stringify(msg);
  wss.clients.forEach(ws => {
    if (ws.readyState === 1) ws.send(payload);
  });
}

// ═══════════════════════════════════════════════════════════════════════
// [4]  LIVE ROUTES — add before the error handlers at the bottom
```

```
// ═══════════════════════════════════════════════════════════════════════

// ──────────────────────────── Start Live Session ──────────────────────────────

app.post('/api/live-start', async (req, res) => {
  const { wallet, title, artistName, quality = '720p' } = req.body || {};
  if (!wallet || !title || !artistName) {
    return res.status(400).json({ error: 'wallet, title, and artistName are required' });
  }

  // Access guard
  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  const level    = getCapabilityLevel(profile || {});
  if (!['creator_active', 'nft_creator_active'].includes(level)) {
    return res.status(403).json({ error: 'An active Creator subscription is required to go live.' });
  }

  const sessionId  = uuidv4();
  const hlsDir     = path.join(process.cwd(), 'public', 'live', sessionId);
  await fs.ensureDir(hlsDir);

  // Quality presets
  const qualityMap = {
    '1080p': { w: 1920, h: 1080, vbr: '4500k', preset: 'veryfast' },
    '720p':  { w: 1280, h: 720,  vbr: '2800k', preset: 'veryfast' },
    '480p':  { w: 854,  h: 480,  vbr: '1200k', preset: 'ultrafast' },
    '360p':  { w: 640,  h: 360,  vbr: '600k',  preset: 'ultrafast' },
  };
  const q = qualityMap[quality] || qualityMap['720p'];

  // PassThrough: browser chunks → ffmpeg stdin
  const passThrough = new PassThrough();

  const ffmpegPath = process.env.FFMPEG_PATH || 'ffmpeg';
  const ffmpegArgs = [
    '-fflags', 'nobuffer',
    '-flags', 'low_delay',
    '-i', 'pipe:0',
    // Video
    '-c:v', 'libx264',
    '-preset', q.preset,
    '-tune', 'zerolatency',
    '-b:v', q.vbr,
    '-maxrate', q.vbr,
    '-bufsize', `${parseInt(q.vbr) * 2}k`,
    '-vf', `scale=${q.w}:${q.h}`,
```

```
  '-g', '48', '-keyint_min', '48',
  '-sc_threshold', '0',
  // Audio
  '-c:a', 'aac', '-b:a', '128k', '-ar', '44100',
  // HLS
  '-f', 'hls',
  '-hls_time', '2',
  '-hls_list_size', '0',         // keep all segments (for full archive)
  '-hls_flags', 'append_list',
  '-hls_segment_type', 'mpegts',
  '-hls_segment_filename', `${hlsDir}/seg_%05d.ts`,
  // Thumbnail every 10s
  '-vf', `scale=${q.w}:${q.h},fps=1/10`,
  '-update', '1', `${hlsDir}/thumb.jpg`,
  `${hlsDir}/stream.m3u8`,
];

const ffmpegProc = spawn(ffmpegPath, ffmpegArgs, { stdio: ['pipe', 'pipe', 'pipe'] });
passThrough.pipe(ffmpegProc.stdin);

ffmpegProc.stderr.on('data', d => logger.debug({ sessionId }, `FFmpeg: ${d}`));
ffmpegProc.on('error', err => {
  logger.error({ sessionId, err }, 'FFmpeg error during live session');
  const session = activeSessions.get(sessionId);
  if (session && session.status === 'live') {
    session.status = 'ended_unexpectedly';
    session.endTime = Date.now();
    broadcastToSession(sessionId, { type: 'stream_ended', sessionId, reason: 'encoder_error' });
  }
});
ffmpegProc.on('close', code => {
  logger.info({ sessionId, code }, 'FFmpeg process closed');
});

const sessionMeta = {
  sessionId,
  wallet,
  title,
  artistName,
  quality,
  startTime: Date.now(),
  hlsDir,
  hlsUrl:        `/live/${sessionId}/stream.m3u8`,
  thumbnailUrl:  `/live/${sessionId}/thumb.jpg`,
  passThrough,
  ffmpegProc,
  viewers:     new Map(),
```

```
    chatHistory: [],
    tipsTotal:   0,
    peakViewers: 0,
    status:      'live',
    archiveCid:  null,
    endTime:     null,
    chunkCount:  0,
  };
  activeSessions.set(sessionId, sessionMeta);

  // Persist metadata (sans non-serializable fields)
  await saveLiveSession(sessionId, {
    sessionId, wallet, title, artistName, quality,
    startTime: sessionMeta.startTime,
    hlsUrl: sessionMeta.hlsUrl,
    thumbnailUrl: sessionMeta.thumbnailUrl,
    status: 'live',
  });

  // Notify all connected viewers
  broadcastToAll({
    type:       'session_started',
    sessionId,
    title,
    artistName,
    thumbnailUrl: sessionMeta.thumbnailUrl,
    hlsUrl:       sessionMeta.hlsUrl,
  });

  logger.info({ sessionId, wallet, title, quality }, 'Live session started');
  res.status(201).json({
    sessionId,
    hlsUrl:      sessionMeta.hlsUrl,
    thumbnailUrl: sessionMeta.thumbnailUrl,
  });
});

// ──────────────────────────── Ingest Chunks ────────────────────────────────

// Raw binary POST — each MediaRecorder chunk pushed here
app.post('/api/live-ingest/:sessionId',
  express.raw({ type: 'application/octet-stream', limit: '10mb' }),
  async (req, res) => {
    const session = activeSessions.get(req.params.sessionId);
    if (!session) {
      return res.status(404).json({ error: 'Session not found or already ended' });
    }
```

```
    if (session.status !== 'live') {
      return res.status(409).json({ error: 'Session is not active', status: session.status });
    }
    if (!req.body || !req.body.length) {
      return res.status(400).json({ error: 'Empty chunk' });
    }

    try {
      session.passThrough.write(req.body);
      session.chunkCount++;

      // Update viewer count broadcast every 5 chunks (~10s)
      if (session.chunkCount % 5 === 0) {
        broadcastToSession(session.sessionId, {
          type: 'stats',
          sessionId: session.sessionId,
          viewerCount: session.viewers.size,
          duration: Math.floor((Date.now() - session.startTime) / 1000),
          tipsTotal: session.tipsTotal,
        });
      }

      res.json({ ok: true, chunkCount: session.chunkCount });
    } catch (err) {
      logger.error({ sessionId: req.params.sessionId, err }, 'Chunk write error');
      res.status(500).json({ error: 'Chunk write failed' });
    }
  }
);

// ──────────────────────────── End Live Session ────────────────────────────────

app.post('/api/live-end/:sessionId', async (req, res) => {
  const { wallet } = req.body || {};
  const session    = activeSessions.get(req.params.sessionId);

  if (!session) return res.status(404).json({ error: 'Session not found' });
  if (session.wallet !== wallet) return res.status(403).json({ error: 'Not your session' });
  if (session.status !== 'live') return res.status(409).json({ error: 'Session already ended' });

  session.status  = 'ended_clean';
  session.endTime = Date.now();

  // Close the PassThrough — tells FFmpeg stdin to finish
  session.passThrough.end();

  // Wait up to 8s for FFmpeg to flush remaining segments
```

```
  await new Promise(resolve => {
    const timeout = setTimeout(resolve, 8000);
    session.ffmpegProc.on('close', () => { clearTimeout(timeout); resolve(); });
  });

  const duration = Math.floor((session.endTime - session.startTime) / 1000);

  // Notify viewers
  broadcastToSession(session.sessionId, {
    type:     'stream_ended',
    sessionId: session.sessionId,
    reason:   'creator_ended',
    duration,
  });

  await saveLiveSession(session.sessionId, {
    sessionId:   session.sessionId,
    wallet:      session.wallet,
    title:       session.title,
    artistName:  session.artistName,
    startTime:   session.startTime,
    endTime:     session.endTime,
    duration,
    hlsUrl:      session.hlsUrl,
    thumbnailUrl: session.thumbnailUrl,
    tipsTotal:   session.tipsTotal,
    peakViewers: session.peakViewers,
    status:      'ended_clean',
  });

  broadcastToAll({ type: 'session_ended', sessionId: session.sessionId });

  res.json({
    success:     true,
    duration,
    tipsTotal:   session.tipsTotal,
    peakViewers: session.peakViewers,
    chunkCount:  session.chunkCount,
    hlsDir:      session.hlsDir,
  });
});

// ──────────────────────────── Archive to IPFS + Catalog ────────────────────────

app.post('/api/live-archive', async (req, res) => {
  const { sessionId, wallet, title, description, tags, mintNft } = req.body || {};
  if (!sessionId || !wallet) return res.status(400).json({ error: 'Missing sessionId or wallet' });
```

```
const session = activeSessions.get(sessionId) || {};
const savedMeta = (await loadLiveSessions())[sessionId];
if (!savedMeta) return res.status(404).json({ error: 'Session not found' });
if (savedMeta.wallet !== wallet) return res.status(403).json({ error: 'Not your session' });

const hlsDir = session.hlsDir || path.join(process.cwd(), 'public', 'live', sessionId);
if (!fs.existsSync(hlsDir)) return res.status(404).json({ error: 'Recording files not found' });

try {
  let archiveCid = null;

  if (ipfs) {
    // Archive the HLS folder to IPFS
    const { folderCid } = await addDirectoryToIpfs(ipfs, hlsDir);
    archiveCid = folderCid;
    logger.info({ sessionId, archiveCid }, 'Live session archived to IPFS');
  } else {
    logger.warn({ sessionId }, 'IPFS not configured — archiving locally only');
    archiveCid = `local:${sessionId}`;
  }

  // Build catalog metadata (same schema as uploaded content)
  const duration  = savedMeta.duration || Math.floor(((session.endTime || Date.now()) - session.star
  const parsedTags = (tags || '').split(',').map(t => t.trim()).filter(Boolean);
  const profiles   = await loadProfiles();
  const profile    = profiles[wallet];

  const liveMetadata = {
    id:               sessionId,
    title:            title || savedMeta.title,
    description:      description || '',
    creator: {
      name:           savedMeta.artistName,
      wallet_address: wallet,
    },
    content_type:      'live_recording',
    availability_type: 'on_demand',
    duration_seconds:  duration,
    recorded_live:     true,
    release_date:      new Date().toISOString().split('T')[0],
    tags:              parsedTags.length ? parsedTags : ['live', 'stream'],
    files: {
      hls_url:   archiveCid ? `ipfs://${archiveCid}/stream.m3u8` : session.hlsUrl,
      thumbnail: archiveCid ? `ipfs://${archiveCid}/thumb.jpg`   : session.thumbnailUrl,
    },
    live_stats: {
```

```
    peak_viewers: savedMeta.peakViewers || 0,
    tips_total:   savedMeta.tipsTotal   || 0,
    chunk_count:  session.chunkCount    || 0,
  },
  royalty_fee_rate: profile?.royalty_fee_rate || FEES.PLATFORM_ROYALTY_STANDARD,
};

let metadataCid = null;
if (ipfs) {
  const { cid } = await ipfs.add(JSON.stringify(liveMetadata));
  metadataCid = cid.toString();
}

// Save to creator's catalog (playlist_cids is the catalog store for now)
if (profile && metadataCid) {
  profile.playlist_cids = profile.playlist_cids || [];
  if (!profile.playlist_cids.includes(metadataCid)) {
    profile.playlist_cids.push(metadataCid);
  }
  // Also track in a dedicated live_recordings field
  profile.live_recordings = profile.live_recordings || [];
  profile.live_recordings.push({
    sessionId,
    metadataCid,
    archiveCid,
    title: liveMetadata.title,
    date:  new Date().toISOString(),
  });
  profiles[wallet] = profile;
  await saveProfiles(profiles);
}

// Update session status
if (activeSessions.has(sessionId)) {
  activeSessions.get(sessionId).status     = 'archived';
  activeSessions.get(sessionId).archiveCid = archiveCid;
}
await saveLiveSession(sessionId, { ...savedMeta, status: 'archived', archiveCid, metadataCid });

res.json({
  success:     true,
  archiveCid,
  metadataCid,
  hlsUrl:      liveMetadata.files.hls_url,
  mintPending: !!mintNft,
  message:     mintNft
    ? 'Archived to IPFS. Use metadataCid to mint the NFT on-chain.'
```

```
        : 'Live recording saved to your catalog.',
    });
  } catch (err) {
    logger.error({ sessionId, err }, 'Archive failed');
    res.status(500).json({ error: err.message });
  }
});

// ──────────────────────────── Discard Recording ────────────────────────────

app.post('/api/live-discard', async (req, res) => {
  const { sessionId, wallet } = req.body || {};
  if (!sessionId || !wallet) return res.status(400).json({ error: 'Missing sessionId or wallet' });

  const savedMeta = (await loadLiveSessions())[sessionId];
  if (!savedMeta) return res.status(404).json({ error: 'Session not found' });
  if (savedMeta.wallet !== wallet) return res.status(403).json({ error: 'Not your session' });

  const hlsDir = path.join(process.cwd(), 'public', 'live', sessionId);
  await fs.remove(hlsDir).catch(() => {});

  if (activeSessions.has(sessionId)) {
    activeSessions.get(sessionId).status = 'discarded';
    activeSessions.delete(sessionId);
  }
  await saveLiveSession(sessionId, { ...savedMeta, status: 'discarded' });

  logger.info({ sessionId, wallet }, 'Live recording discarded');
  res.json({ success: true, message: 'Recording deleted.' });
});

// ──────────────────────────── Live Sessions List (replaces stub) ──────────────

// REMOVE the old: app.get('/api/live-concerts', ...) stub and replace with:
app.get('/api/live-sessions', async (req, res) => {
  const sessions = [];
  for (const [id, s] of activeSessions) {
    if (s.status !== 'live') continue;
    sessions.push({
      sessionId:    id,
      title:        s.title,
      artistName:   s.artistName,
      wallet:       s.wallet,
      hlsUrl:       s.hlsUrl,
      thumbnailUrl: s.thumbnailUrl,
      viewerCount:  s.viewers.size,
      startTime:    s.startTime,
```

```
      tipsTotal:    s.tipsTotal,
      duration:     Math.floor((Date.now() - s.startTime) / 1000),
    });
  }
  res.json(sessions);
});

// Keep backward compat alias
app.get('/api/live-concerts', async (req, res) => {
  const r = await fetch(`http://localhost:${process.env.PORT || 3001}/api/live-sessions`);
  res.json(r.ok ? await r.json() : []);
});

// ═══════════════════════════════════════════════════════════════════════
// [5]  WEBSOCKET SERVER + SERVER STARTUP
//      Replace the existing app.listen(PORT) block at the bottom of server.cjs
//      with this entire block.
// ═══════════════════════════════════════════════════════════════════════

const PORT = Number(process.env.PORT || 3001);

(async () => {
  try {
    const httpServer = http.createServer(app);

    // ── WebSocket Server ──────────────────────────────────────────────
    wss = new WebSocketServer({ server: httpServer, path: '/ws' });

    wss.on('connection', (ws, req) => {
      ws._sessionId = null;
      ws._wallet    = null;
      ws._name      = null;

      ws.on('message', async (raw) => {
        let msg;
        try { msg = JSON.parse(raw); } catch { return; }

        switch (msg.type) {

          // ── Viewer joins a live session ──────────────────────────
          case 'join_session': {
            const session = activeSessions.get(msg.sessionId);
            if (!session || session.status !== 'live') {
              ws.send(JSON.stringify({ type: 'error', error: 'Session not found or not live' }));
              return;
            }
            // Leave previous session if any
```

```
  if (ws._sessionId && ws._sessionId !== msg.sessionId) {
    const prev = activeSessions.get(ws._sessionId);
    if (prev) prev.viewers.delete(ws);
  }
  ws._sessionId = msg.sessionId;
  ws._wallet    = msg.wallet   || null;
  ws._name      = msg.name     || 'Listener';
  session.viewers.set(ws, { wallet: ws._wallet, name: ws._name });
  session.peakViewers = Math.max(session.peakViewers, session.viewers.size);

  // Send recent chat history + current stats
  ws.send(JSON.stringify({
    type:        'session_state',
    sessionId:   msg.sessionId,
    title:       session.title,
    artistName:  session.artistName,
    hlsUrl:      session.hlsUrl,
    viewerCount: session.viewers.size,
    duration:    Math.floor((Date.now() - session.startTime) / 1000),
    tipsTotal:   session.tipsTotal,
    chatHistory: session.chatHistory.slice(-50),
  }));

  // Broadcast updated count
  broadcastToSession(msg.sessionId, {
    type:        'viewer_count',
    sessionId:   msg.sessionId,
    viewerCount: session.viewers.size,
  });
  break;
}

// ── Leave session ──────────────────────────────────────────
case 'leave_session': {
  const session = activeSessions.get(ws._sessionId);
  if (session) {
    session.viewers.delete(ws);
    broadcastToSession(ws._sessionId, {
      type:        'viewer_count',
      sessionId:   ws._sessionId,
      viewerCount: session.viewers.size,
    });
  }
  ws._sessionId = null;
  break;
}
```

```
// ── Chat message ──────────────────────────────────────────
case 'chat': {
  if (!ws._sessionId) return;
  const session = activeSessions.get(ws._sessionId);
  if (!session || session.status !== 'live') return;
  const text = String(msg.text || '').trim().slice(0, 300);
  if (!text) return;
  const chatMsg = {
    type:      'chat',
    sessionId: ws._sessionId,
    name:      ws._name     || 'Listener',
    wallet:    ws._wallet   || null,
    text,
    ts:        Date.now(),
  };
  session.chatHistory.push(chatMsg);
  if (session.chatHistory.length > 200) session.chatHistory.shift();
  broadcastToSession(ws._sessionId, chatMsg);
  break;
}

// ── Emoji reaction ────────────────────────────────────────
case 'reaction': {
  if (!ws._sessionId) return;
  const ALLOWED_REACTIONS = ['🔥', '❤️', '👏', '🎵', '💎', '🚀'];
  const emoji = ALLOWED_REACTIONS.includes(msg.emoji) ? msg.emoji : '🔥';
  broadcastToSession(ws._sessionId, {
    type:      'reaction',
    sessionId: ws._sessionId,
    emoji,
    name:      ws._name || 'Listener',
  });
  break;
}

// ── Tip alert (after tip API call succeeds, client sends this) ──
case 'tip_alert': {
  if (!ws._sessionId) return;
  const session = activeSessions.get(ws._sessionId);
  if (!session) return;
  const amount = parseFloat(msg.amountEth) || 0;
  session.tipsTotal += amount;
  broadcastToSession(ws._sessionId, {
    type:      'tip_alert',
    sessionId: ws._sessionId,
    name:      ws._name  || 'Listener',
    amountEth: amount,
```

```
          tipsTotal: session.tipsTotal,
        });
        break;
      }

      // ── Creator heartbeat (keep session alive) ────────────────────
      case 'ping': {
        ws.send(JSON.stringify({ type: 'pong', ts: Date.now() }));
        break;
      }
    }
  });

  ws.on('close', () => {
    if (ws._sessionId) {
      const session = activeSessions.get(ws._sessionId);
      if (session) {
        session.viewers.delete(ws);
        broadcastToSession(ws._sessionId, {
          type:        'viewer_count',
          sessionId:   ws._sessionId,
          viewerCount: session.viewers.size,
        });
      }
    }
  });

  ws.on('error', (err) => logger.warn({ err }, 'WebSocket error'));
});

// ── TLS or HTTP ──────────────────────────────────────────────────────
if (process.env.TLS_KEY_PATH && process.env.TLS_CERT_PATH) {
  if (fs.existsSync(process.env.TLS_KEY_PATH) && fs.existsSync(process.env.TLS_CERT_PATH)) {
    const tls = require('tls');
    const tlsOptions = {
      key:  fs.readFileSync(process.env.TLS_KEY_PATH),
      cert: fs.readFileSync(process.env.TLS_CERT_PATH),
      minVersion:       'TLSv1.3',
      honorCipherOrder: true,
    };
    require('https').createServer(tlsOptions, app).listen(443, () =>
      logger.info('HTTPS + WSS server on port 443'));
    require('http').createServer((req, res) => {
      res.writeHead(301, { Location: `https://${req.headers.host}${req.url}` });
      res.end();
    }).listen(80);
    return;
```

```
      }
    }

    httpServer.listen(PORT, () => {
      logger.info(`HTTP + WS server on http://localhost:${PORT}`);
      console.log(`[READY] Server running on port ${PORT} — WebSocket at ws://localhost:${PORT}/ws`);
    });

  } catch (e) {
    logger.error({ err: e }, 'Server startup failed');
    console.error('[STARTUP ERROR]', e.message);
    process.exit(1);
  }
})();

module.exports = { app, activeSessions, FEES, SUBSCRIPTION_PLANS };
```

`server_gst_additions.js` (22.5 KB)

```javascript
'use strict';
/**
 * MSP — Server additions for GStreamer + Nginx integration
 * Merge into server.cjs.
 *
 * ─── INTEGRATION CHECKLIST ──────────────────────────────────────────────
 *  [1] Add to requires at top of server.cjs:
 *      const { GstPipeline, detectCapabilities, MODES, STREAMS_ROOT, HLS_ROOT }
 *            = require('./gst_pipeline');
 *
 *  [2] Add to startup (after logger init):
 *      let gstCaps = { gstreamer: false, ffmpeg: true };
 *      detectCapabilities().then(c => {
 *        gstCaps = c;
 *        logger.info({ ...c }, 'Media capabilities');
 *      }).catch(() => {});
 *
 *  [3] Add the routes below before the error handlers.
 *
 *  [4] Add to .env:
 *      STREAMS_ROOT=/var/www/msp/streams
 *      HLS_ROOT=/var/www/msp/live
 *      STREAM_HOST=your-server-ip-or-domain
 *
 *  [5] npm install --save ws node-fetch
 * ────────────────────────────────────────────────────────────────────────
```

```
 */

// ────────────────────────────────────────────────────────────────────────────
//  [A]  CATALOG PIPELINE  —  Your exact proposed flow:
//
//  Artist upload/mint → IPFS CID
//    ↓
//  Backend downloads CID → GStreamer transcodes to HLS variants
//    ↓
//  HLS written to /var/www/msp/streams/{cid}/  (lo/mid/hi + master.m3u8)
//    ↓
//  Nginx serves https://stream.michie.com/{cid}/master.m3u8
//    ↓
//  Frontend hls.js → subscription check → plays
//    ↓
//  Play completes → StreamingRegistry.logPlay() + royalty events
// ────────────────────────────────────────────────────────────────────────────

/**
 * POST /api/catalog-transcode
 *
 * Triggered automatically after a successful /api/upload, OR manually
 * by a creator who uploaded content that hasn't been transcoded yet.
 *
 * Body: { cid, wallet, contentType, title, mode? }
 *   cid         — IPFS content ID of the uploaded media
 *   wallet      — creator wallet (for access guard)
 *   contentType — 'music' | 'podcast' | 'video' | 'art_animated'
 *   mode        — 'production' (default) | 'social'
 */
app.post('/api/catalog-transcode', async (req, res) => {
  const { cid, wallet, contentType = 'music', mode = MODES.PRODUCTION } = req.body || {};
  if (!cid || !wallet) return res.status(400).json({ error: 'Missing cid or wallet' });

  // ── Access guard ──────────────────────────────────────────────────────────
  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) return res.status(403).json({ error: 'Profile not found' });
  if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
    return res.status(403).json({ error: 'Creator account required' });
  }

  // ── Check if already transcoded ───────────────────────────────────────────
  const outDir     = path.join(STREAMS_ROOT, cid);
  const masterPath = path.join(outDir, 'master.m3u8');
  if (await fs.pathExists(masterPath)) {
    return res.json({
```

```
      success:  true,
      cached:   true,
      hlsUrl:   `/streams/${cid}/master.m3u8`,
      thumbUrl: `/streams/${cid}/thumb.jpg`,
    });
  }

  // ── Reply immediately — transcode is async ──────────────────────────────
  res.json({
    success:   true,
    queued:    true,
    cid,
    statusUrl: `/api/stream-ready/${cid}`,
    message:   'Transcode queued — poll /api/stream-ready/:cid for status',
  });

  // ── Background: download from IPFS + transcode ──────────────────────────
  _runCatalogTranscode({ cid, wallet, contentType, mode, profile, profiles }).catch(err => {
    logger.error({ cid, err }, 'Catalog transcode failed');
  });
});

/**
 * Internal — download from IPFS and run GStreamer.
 */
async function _runCatalogTranscode({ cid, wallet, contentType, mode, profile, profiles }) {
  const outDir  = path.join(STREAMS_ROOT, cid);
  const tempDir = path.join(__dirname, '../temp', `transcode_${cid}`);
  await fs.ensureDir(outDir);
  await fs.ensureDir(tempDir);

  const gateway    = process.env.IPFS_GATEWAY || 'https://ipfs.io/ipfs/';
  const mediaUrl   = gateway + cid;
  const localFile  = path.join(tempDir, `source${_extForType(contentType)}`);

  logger.info({ cid, contentType, mode }, 'Catalog transcode: downloading from IPFS');

  // Download the file
  const { default: nodeFetch } = require('node-fetch').catch ? { default: require('node-fetch') } : re
  try {
    const resp = await nodeFetch(mediaUrl, { timeout: 120000 });
    if (!resp.ok) throw new Error(`IPFS download failed: ${resp.status}`);
    const dest = require('fs').createWriteStream(localFile);
    await new Promise((res, rej) => {
      resp.body.pipe(dest);
      resp.body.on('error', rej);
      dest.on('finish', res);
```

```
    });
  } catch (err) {
    logger.error({ cid, err }, 'IPFS download failed — catalog transcode aborted');
    await fs.remove(tempDir);
    return;
  }

  logger.info({ cid, localFile }, 'IPFS download complete — starting GStreamer');

  // Detect source height (for video)
  let sourceHeight = 0;
  if (contentType === 'video' || contentType === 'art_animated') {
    try {
      const probeOut = await _probeVideo(localFile);
      sourceHeight   = probeOut.height || 720;
    } catch (_) {
      sourceHeight = 720;
    }
  }

  // Run pipeline
  const pipe = new GstPipeline({
    id:     cid,
    mode,
    hlsDir: outDir,
    caps:   gstCaps,
    logger,
  });

  try {
    await pipe.transcodeFile({ inputPath: localFile, contentType, sourceHeight });
  } catch (err) {
    logger.error({ cid, err }, 'GStreamer transcode error');
    await fs.remove(tempDir);
    return;
  }

  // Cleanup temp download
  await fs.remove(tempDir);

  logger.info({ cid, outDir }, 'Catalog transcode complete — HLS ready');

  // Emit notification if WebSocket server is available
  if (typeof broadcastToAll === 'function') {
    broadcastToAll({ type: 'transcode_complete', cid, hlsUrl: `/streams/${cid}/master.m3u8` });
  }
}
```

```
// ────────────────────────────────────────────────────────────────────────────
//  [B]  STREAM READY CHECK
// ────────────────────────────────────────────────────────────────────────────

/**
 * GET /api/stream-ready/:cid
 *
 * Returns whether the HLS output for a given CID is ready to serve.
 * The frontend polls this after triggering a transcode, or calls it
 * before attempting to play a catalog asset.
 */
app.get('/api/stream-ready/:cid', async (req, res) => {
  const { cid } = req.params;
  const masterPath = path.join(STREAMS_ROOT, cid, 'master.m3u8');
  const thumbPath  = path.join(STREAMS_ROOT, cid, 'thumb.jpg');
  const ready      = await fs.pathExists(masterPath);

  // Check how many segments are available (rough progress indicator)
  let segCount = 0;
  if (ready) {
    try {
      const files = await fs.readdir(path.join(STREAMS_ROOT, cid));
      segCount = files.filter(f => f.endsWith('.ts')).length;
    } catch (_) {}
  }

  res.json({
    cid,
    ready,
    hlsUrl:   ready ? `/streams/${cid}/master.m3u8` : null,
    thumbUrl: (await fs.pathExists(thumbPath)) ? `/streams/${cid}/thumb.jpg` : null,
    segCount,
  });
});

// ────────────────────────────────────────────────────────────────────────────
//  [C]  DUAL-MODE LIVE  —  Replaces the single-mode /api/live-start
// ────────────────────────────────────────────────────────────────────────────

/**
 * POST /api/live-start
 *
 * Body: {
 *   wallet,
 *   title,
 *   artistName,
```

```
 *   mode:    'production' | 'social'   ← NEW: creator chooses
 *   quality: '1080p' | '720p' | '480p' | '360p'
 *   audioOnly: boolean
 * }
 *
 * Mode implications:
 *   production — multi-bitrate, tee pipeline, archived to IPFS on end, royalty-eligible
 *   social     — single quality, 1s segments, no archive by default, fast
 */
app.post('/api/live-start', async (req, res) => {
  const {
    wallet, title, artistName,
    mode     = MODES.PRODUCTION,
    quality  = '720p',
    audioOnly = false,
  } = req.body || {};

  if (!wallet || !title || !artistName) {
    return res.status(400).json({ error: 'wallet, title, and artistName are required' });
  }

  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  const level    = getCapabilityLevel(profile || {});

  if (!['creator_active', 'nft_creator_active'].includes(level)) {
    return res.status(403).json({ error: 'Active creator subscription required' });
  }

  const sessionId  = uuidv4();
  const hlsDir     = path.join(HLS_ROOT, sessionId);
  await fs.ensureDir(hlsDir);

  // Quality → rung list
  const qualityOrder = ['1080p', '720p', '480p', '360p'];
  const qi           = qualityOrder.indexOf(quality);
  const qualities    = mode === MODES.SOCIAL
    ? [quality]
    : qualityOrder.slice(Math.max(0, qi));   // production: quality + all below

  const hlsUrl   = `/live/${sessionId}/master.m3u8`;
  const thumbUrl = `/live/${sessionId}/thumb.jpg`;

  // Build GstPipeline — will be used by the ingest path
  const { PassThrough } = require('stream');
  const passThrough = new PassThrough();
```

```
const gstPipe = new GstPipeline({
  id:     sessionId,
  mode,
  hlsDir,
  caps:   gstCaps,
  logger,
});

// Start browser-pipe path immediately
gstPipe.startBrowserLive({ passThrough, audioOnly, qualities });
gstPipe.on('error',          err  => logger.error({ sessionId, err }, 'Pipeline error'));
gstPipe.on('pipeline_closed', e   => logger.warn({ sessionId, ...e }, 'Pipeline closed unexpectedly'
gstPipe.on('all_dead',        ()  => {
  const s = activeSessions.get(sessionId);
  if (s && s.status === 'live') {
    s.status = 'ended_unexpectedly';
    broadcastToSession(sessionId, { type: 'stream_ended', sessionId, reason: 'pipeline_error' });
  }
});

const sessionMeta = {
  sessionId, wallet, title, artistName,
  mode, quality, audioOnly, qualities,
  startTime: Date.now(),
  hlsDir, hlsUrl, thumbnailUrl: thumbUrl,
  passThrough,
  gstPipe,
  ffmpegProc: null,
  viewers:    new Map(),
  chatHistory: [],
  tipsTotal:  0,
  peakViewers: 0,
  chunkCount: 0,
  status:     'live',
  archiveCid: null,
  endTime:    null,
  source:     'browser',
};

activeSessions.set(sessionId, sessionMeta);

await saveLiveSession(sessionId, {
  sessionId, wallet, title, artistName,
  mode, source: 'browser',
  startTime: sessionMeta.startTime,
  hlsUrl, thumbnailUrl: thumbUrl,
  status: 'live',
```

```
  });

  broadcastToAll({
    type: 'session_started',
    sessionId, title, artistName,
    mode,
    thumbnailUrl: thumbUrl,
    hlsUrl,
    isSocial: mode === MODES.SOCIAL,
  });

  logger.info({
    sessionId, wallet, mode,
    engine: gstCaps.gstreamer ? 'gstreamer' : 'ffmpeg',
    qualities,
  }, 'Live session started');

  res.status(201).json({
    sessionId,
    hlsUrl,
    thumbnailUrl: thumbUrl,
    mode,
    qualities,
    engine: gstCaps.gstreamer ? 'gstreamer' : 'ffmpeg',
  });
});

// ────────────────────────────────────────────────────────────────────────────
//  [D]  STREAM KEY + RTMP ROUTES  (unchanged from previous version)
//       These power OBS/Larix/Streamlabs ingest → nginx-rtmp → gst-transcode.sh
// ────────────────────────────────────────────────────────────────────────────

app.get('/api/stream-key/:wallet', async (req, res) => {
  const { wallet } = req.params;
  const profiles   = await loadProfiles();
  const profile    = profiles[wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  const level = getCapabilityLevel(profile);
  if (!['creator_active', 'nft_creator_active'].includes(level)) {
    return res.status(403).json({ error: 'Active creator subscription required' });
  }

  if (!profile.stream_key) {
    profile.stream_key = uuidv4();
    profiles[wallet]   = profile;
    await saveProfiles(profiles);
```

```
  }

  const host = process.env.STREAM_HOST || 'YOUR_SERVER_IP';
  res.json({
    stream_key:  profile.stream_key,
    rtmp_server: `rtmp://${host}:1935/live`,
    rtmp_url:    `rtmp://${host}:1935/live/${profile.stream_key}`,
    hls_preview: `https://${host}/live/[sessionId]/master.m3u8`,
    instructions: {
      obs:       `Settings → Stream → Service: Custom RTMP | Server: rtmp://${host}:1935/live | Key: $
      larix:     `Connections → Add → URL: rtmp://${host}:1935/live/${profile.stream_key}`,
      streamlabs:`RTMP Server: rtmp://${host}:1935/live | Stream Key: ${profile.stream_key}`,
    },
  });
});

app.post('/api/stream-key/regenerate', async (req, res) => {
  const { wallet } = req.body || {};
  if (!wallet) return res.status(400).json({ error: 'Missing wallet' });
  const profiles = await loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) return res.status(404).json({ error: 'Profile not found' });

  const level = getCapabilityLevel(profile);
  if (!['creator_active', 'nft_creator_active'].includes(level)) {
    return res.status(403).json({ error: 'Active creator subscription required' });
  }

  const oldKey       = profile.stream_key;
  profile.stream_key = uuidv4();
  profiles[wallet]   = profile;
  await saveProfiles(profiles);

  if (oldKey && rtmpSessions && rtmpSessions.has(oldKey)) {
    const s = rtmpSessions.get(oldKey);
    if (s) broadcastToSession(s.sessionId, { type: 'stream_ended', reason: 'key_regenerated' });
    rtmpSessions.delete(oldKey);
  }

  logger.info({ wallet }, 'Stream key regenerated');
  res.json({ stream_key: profile.stream_key });
});

// nginx-rtmp on_publish — validate stream key, allow or deny
app.post('/api/rtmp-auth', async (req, res) => {
  const streamKey = req.body?.name || req.query?.name;
  if (!streamKey) return res.status(403).send('DENY: no stream key');
```

```
  const profiles = await loadProfiles();
  const entry    = Object.entries(profiles).find(([, p]) => p.stream_key === streamKey);
  if (!entry) return res.status(403).send('DENY: unknown key');

  const [wallet, profile] = entry;
  const level             = getCapabilityLevel(profile);
  if (!['creator_active', 'nft_creator_active'].includes(level)) {
    logger.warn({ wallet }, 'RTMP auth denied: subscription inactive');
    return res.status(403).send('DENY: subscription inactive');
  }

  logger.info({ wallet, streamKey }, 'RTMP auth: ALLOW');
  res.status(200).send('OK');
});

// gst-transcode.sh calls this after nginx-rtmp accepts the stream
app.post('/api/rtmp-publish', async (req, res) => {
  const { streamKey, mode = MODES.PRODUCTION } = req.body || {};
  if (!streamKey) return res.status(400).json({ error: 'Missing streamKey' });

  const profiles = await loadProfiles();
  const entry    = Object.entries(profiles).find(([, p]) => p.stream_key === streamKey);
  if (!entry) return res.status(403).json({ error: 'Invalid stream key' });

  const [wallet, profile] = entry;
  const sessionId         = uuidv4();
  const hlsDir            = path.join(HLS_ROOT, sessionId);
  await fs.ensureDir(hlsDir);

  const sessionMeta = {
    sessionId, wallet,
    artistName:  profile.name || 'Creator',
    title:       `Live — ${profile.name || 'Creator'}`,
    mode,
    startTime:   Date.now(),
    hlsDir,
    hlsUrl:      `/live/${sessionId}/master.m3u8`,
    thumbnailUrl: `/live/${sessionId}/thumb.jpg`,
    source:      'rtmp',
    passThrough: null,
    gstPipe:     null,
    viewers:     new Map(),
    chatHistory: [],
    tipsTotal:   0,
    peakViewers: 0,
    chunkCount:  0,
```

```
    status:      'starting',
    archiveCid:  null,
    endTime:     null,
  };

  activeSessions.set(sessionId, sessionMeta);
  if (typeof rtmpSessions !== 'undefined') rtmpSessions.set(streamKey, { sessionId, wallet, startedAt:

  await saveLiveSession(sessionId, {
    sessionId, wallet, title: sessionMeta.title,
    artistName: sessionMeta.artistName, mode, source: 'rtmp',
    startTime: sessionMeta.startTime, hlsUrl: sessionMeta.hlsUrl,
    thumbnailUrl: sessionMeta.thumbnailUrl, status: 'starting',
  });

  broadcastToAll({ type: 'session_started', sessionId, title: sessionMeta.title,
    artistName: sessionMeta.artistName, thumbnailUrl: sessionMeta.thumbnailUrl,
    hlsUrl: sessionMeta.hlsUrl, mode });

  logger.info({ sessionId, wallet, mode }, 'RTMP publish session created');
  res.json({
    sessionId,
    hlsDir,
    qualities: mode === MODES.SOCIAL ? ['480p'] : ['720p', '480p'],
    audioOnly: false,
    mode,
  });
});

// gst-transcode.sh calls this when GStreamer pipelines are running
app.post('/api/rtmp-live-ready', async (req, res) => {
  const { sessionId } = req.body || {};
  const session = activeSessions.get(sessionId);
  if (!session) return res.status(404).json({ error: 'Session not found' });
  session.status = 'live';
  await saveLiveSession(sessionId, { ...((await loadLiveSessions())[sessionId] || {}), status: 'live'
  broadcastToSession(sessionId, { type: 'stream_ready', sessionId, hlsUrl: session.hlsUrl });
  res.json({ success: true });
});

// nginx-rtmp on_done + gst-transcode-done.sh
app.post('/api/rtmp-done', async (req, res) => {
  const streamKey = req.body?.name || req.body?.streamKey;
  if (!streamKey) return res.status(400).json({ error: 'Missing stream key' });

  const rtmpInfo  = typeof rtmpSessions !== 'undefined' ? rtmpSessions.get(streamKey) : null;
  if (!rtmpInfo) return res.json({ success: true, sessionId: null });
```

```
  const { sessionId, wallet } = rtmpInfo;
  const session               = activeSessions.get(sessionId);

  if (session && session.status === 'live') {
    session.status  = 'ended_clean';
    session.endTime = Date.now();
    const duration  = Math.floor((session.endTime - session.startTime) / 1000);

    broadcastToSession(sessionId, { type: 'stream_ended', sessionId, reason: 'creator_ended', duration
    await saveLiveSession(sessionId, {
      sessionId, wallet, title: session.title, artistName: session.artistName,
      startTime: session.startTime, endTime: session.endTime, duration,
      hlsUrl: session.hlsUrl, tipsTotal: session.tipsTotal,
      peakViewers: session.peakViewers, status: 'ended_clean', mode: session.mode,
    });
    broadcastToAll({ type: 'session_ended', sessionId });
  }

  if (typeof rtmpSessions !== 'undefined') rtmpSessions.delete(streamKey);
  logger.info({ sessionId, wallet }, 'RTMP done');
  res.json({ success: true, sessionId });
});

// ────────────────────────────────────────────────────────────────────────
//  [E]  MEDIA CAPABILITIES endpoint
// ────────────────────────────────────────────────────────────────────────

app.get('/api/media-capabilities', (req, res) => {
  res.json({
    gstreamer:   gstCaps.gstreamer   || false,
    gstVersion:  gstCaps.gstVersion  || null,
    hwAccel:     gstCaps.nvenc       ? 'nvidia'
               : gstCaps.vaapi       ? 'vaapi'
               : gstCaps.videotoolbox ? 'videotoolbox'
               : 'software',
    hlssink2:    gstCaps.hlssink2    || false,
    level:       gstCaps.level       || false,
    jpegenc:     gstCaps.jpegenc     || false,
    rtmpsrc:     gstCaps.rtmpsrc     || false,
    ffmpeg:      gstCaps.ffmpeg      || false,
    ffmpegVersion: gstCaps.ffmpegVersion || null,
    liveEngine:  gstCaps.gstreamer   ? 'gstreamer' : 'ffmpeg',
    catalogEngine: gstCaps.gstreamer ? 'gstreamer' : 'ffmpeg',
  });
});
```

```
// ────────────────────────────────────────────────────────────────────────────
//  INTERNAL HELPERS
// ────────────────────────────────────────────────────────────────────────────

function _extForType(contentType) {
  switch (contentType) {
    case 'music':        return '.mp3';
    case 'podcast':      return '.mp3';
    case 'video':        return '.mp4';
    case 'art_animated': return '.mp4';
    default:             return '.bin';
  }
}

async function _probeVideo(filePath) {
  const { default: nodeFetch } = require('node-fetch');
  // Use ffprobe for height detection
  const { spawn: _spawn } = require('child_process');
  return new Promise((resolve) => {
    const chunks = [];
    const p = _spawn(process.env.FFPROBE_PATH || 'ffprobe', [
      '-v', 'quiet', '-print_format', 'json', '-show_streams', filePath,
    ], { stdio: ['ignore', 'pipe', 'pipe'] });
    p.stdout.on('data', d => chunks.push(d));
    p.on('close', () => {
      try {
        const data    = JSON.parse(Buffer.concat(chunks).toString());
        const vStream = data.streams?.find(s => s.codec_type === 'video');
        resolve({ height: vStream?.height || 720, width: vStream?.width || 1280 });
      } catch (_) {
        resolve({ height: 720, width: 1280 });
      }
    });
    p.on('error', () => resolve({ height: 720, width: 1280 }));
  });
}
```

BACKEND — MODULES (Routes / Services)

`access.js` (2.8 KB)

```javascript
'use strict';

const express        = require('express');
```

```
const profileService = require('../services/profileService');

const router = express.Router();

// GET /api/access/:wallet
router.get('/:wallet', async (req, res, next) => {
  try {
    const profiles = await profileService.loadProfiles();
    const profile  = profiles[req.params.wallet];
    const level    = profileService.getCapabilityLevel(profile);
    const tier     = profileService.getListenerTier(profile);

    res.json({
      level,
      tier,
      account_type:        profile?.account_type      || null,
      subscription_expiry: profile?.subscription_expiry || null,
      active:              profileService.isSubscriptionActive(profile),
      royalty_fee_rate:    profile?.royalty_fee_rate   || null,
      dj_tips_default:     profile?.dj_settings?.tips_enabled_default ?? true,
      supporter_enabled:   profile?.supporter_subaccount?.enabled || false,
    });
  } catch (err) { next(err); }
});

// POST /api/add-supporter-subaccount
router.post('/add-supporter-subaccount', async (req, res, next) => {
  try {
    const { wallet } = req.body || {};
    if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

    const profiles = await profileService.loadProfiles();
    const profile  = profiles[wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });

    if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
      return res.status(403).json({ error: 'Only creator accounts can add a supporter sub-account' });
    }

    profile.supporter_subaccount = {
      enabled:                true,
      linked_creator_wallet:  wallet,
      royalty_beneficiary_of: profile.supporter_subaccount?.royalty_beneficiary_of || [],
    };

    profiles[wallet] = profile;
    await profileService.saveProfiles(profiles);
```

```
    res.json({ success: true, supporter_subaccount: profile.supporter_subaccount });
  } catch (err) { next(err); }
});

// POST /api/toggle-supporter-subaccount
router.post('/toggle-supporter-subaccount', async (req, res, next) => {
  try {
    const { wallet, enabled } = req.body || {};
    if (!wallet || typeof enabled !== 'boolean') {
      return res.status(400).json({ error: 'Missing wallet or enabled flag' });
    }

    const profiles = await profileService.loadProfiles();
    const profile  = profiles[wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });

    if (!profile.supporter_subaccount) {
      return res.status(400).json({ error: 'No supporter sub-account. Call /api/add-supporter-subaccou
    }

    profile.supporter_subaccount.enabled = enabled;
    profiles[wallet] = profile;
    await profileService.saveProfiles(profiles);
    res.json({ success: true, enabled });
  } catch (err) { next(err); }
});

module.exports = router;
```

`catalog.js` (2.4 KB)

```javascript
'use strict';

const express        = require('express');
const path           = require('path');
const fs             = require('fs-extra');
const logger         = require('../config/logger');
const catalogService = require('../services/catalogService');
const ownershipGuard = require('../middleware/ownershipGuard');

const router = express.Router();

// GET /api/catalog
router.get('/', async (req, res, next) => {
  try {
```

```
    const catalog = await catalogService.loadCatalog();
    const entries = Object.values(catalog).sort((a, b) => b.uploadedAt - a.uploadedAt);
    res.json(entries);
  } catch { res.json([]); }
});

// GET /api/catalog/:contentId/metadata
router.get('/:contentId/metadata', async (req, res, next) => {
  try {
    const metaPath = path.join(process.cwd(), 'public', 'catalog', req.params.contentId, 'metadata.jso
    const raw      = await fs.readFile(metaPath, 'utf8');
    res.json(JSON.parse(raw));
  } catch {
    res.status(404).json({ error: 'Not found' });
  }
});

// POST /api/catalog/:contentId/supporter-royalty
// Toggles the supporterRoyaltyEnabled flag on a catalog entry.
// ownershipGuard verifies wallet owns the asset before this handler runs.
router.post('/:contentId/supporter-royalty', ownershipGuard, async (req, res, next) => {
  try {
    const { contentId } = req.params;
    const { enabled }   = req.body || {};

    const updated = await catalogService.patchCatalogEntry(contentId, {
      supporterRoyaltyEnabled:   !!enabled,
      supporterRoyaltyChangedAt: Date.now(),
    });

    logger.info({ contentId, enabled }, 'Supporter royalty flag updated');

    // TODO (production): when enabled === false, notify supporters who have
    // this asset in playlists so they can remove it. Hook in here.

    res.json({ success: true, contentId, supporterRoyaltyEnabled: !!enabled });
  } catch (err) { next(err); }
});

// POST /api/catalog/:contentId/privacy
// Toggles the isPrivate flag on a catalog entry.
router.post('/:contentId/privacy', ownershipGuard, async (req, res, next) => {
  try {
    const { contentId } = req.params;
    const { isPrivate } = req.body || {};

    const updated = await catalogService.patchCatalogEntry(contentId, {
```

```
      isPrivate: !!isPrivate,
    });

    logger.info({ contentId, isPrivate }, 'Asset privacy updated');
    res.json({ success: true, contentId, isPrivate: !!isPrivate });
  } catch (err) { next(err); }
});

module.exports = router;
```

`catalogService.js` (1.1 KB)

```javascript
'use strict';

const path = require('path');
const fs   = require('fs-extra');

const CATALOG_PATH = path.resolve(process.cwd(), 'catalog.json');

async function loadCatalog() {
  try {
    return JSON.parse(await fs.readFile(CATALOG_PATH, 'utf8'));
  } catch {
    return {};
  }
}

async function saveCatalog(catalog) {
  await fs.writeFile(CATALOG_PATH, JSON.stringify(catalog, null, 2));
}

/**
 * Applies a partial patch to a single catalog entry and persists.
 * Throws if the entry does not exist.
 */
async function patchCatalogEntry(contentId, patch) {
  const catalog = await loadCatalog();
  if (!catalog[contentId]) throw new Error(`Catalog entry not found: ${contentId}`);
  catalog[contentId] = Object.assign({}, catalog[contentId], patch);
  await saveCatalog(catalog);
  return catalog[contentId];
}

/**
 * Adds a new entry to the catalog index.
```

```
 */
async function addCatalogEntry(contentId, entry) {
  const catalog = await loadCatalog();
  catalog[contentId] = { ...entry, uploadedAt: Date.now() };
  await saveCatalog(catalog);
}

module.exports = { loadCatalog, saveCatalog, patchCatalogEntry, addCatalogEntry };
```

`djSets.js` (2.5 KB)

```javascript
'use strict';

const express        = require('express');
const { v4: uuidv4 } = require('uuid');
const logger         = require('../config/logger');
const { DJ_CAPABLE_LEVELS } = require('../config/constants');
const { isDevWallet } = require('../middleware/devBypass');
const profileService  = require('../services/profileService');
const djService       = require('../services/djService');

const router = express.Router();

// POST /api/start-dj-set
router.post('/start', async (req, res, next) => {
  try {
    const { wallet, set_name, tips_enabled, dj_percent, artist_splits } = req.body || {};
    if (!wallet || !set_name) return res.status(400).json({ error: 'Missing wallet or set_name' });

    if (!isDevWallet(wallet)) {
      const profiles = await profileService.loadProfiles();
      const profile  = profiles[wallet];
      if (!profile) return res.status(403).json({ error: 'Profile not found' });

      const level = profileService.getCapabilityLevel(profile);
      const canDj = DJ_CAPABLE_LEVELS.has(level)
        || (profile.supporter_subaccount?.enabled && profileService.isSubscriptionActive(profile));
      if (!canDj) {
        return res.status(403).json({ error: 'A Tier 2 or higher subscription is required to host DJ s
      }
    }

    const profiles       = await profileService.loadProfiles();
    const profile        = profiles[wallet] || {};
    const tipsForThisSet = typeof tips_enabled === 'boolean'
```

```
      ? tips_enabled
      : (profile.dj_settings?.tips_enabled_default ?? true);

    const setId = uuidv4();
    const sets  = await djService.loadDjSets();
    sets[setId] = {
      set_id:        setId,
      dj_wallet:     wallet,
      set_name,
      tips_enabled:  tipsForThisSet,
      dj_percent:    dj_percent ?? 100,
      artist_splits: artist_splits || [],
      created_at:    Date.now(),
      active:        true,
    };
    await djService.saveDjSets(sets);

    res.status(201).json({ success: true, set_id: setId, tips_enabled: tipsForThisSet });
  } catch (err) { next(err); }
});

// POST /api/end-dj-set
router.post('/end', async (req, res, next) => {
  try {
    const { wallet, set_id } = req.body || {};
    const sets = await djService.loadDjSets();
    if (!sets[set_id] || sets[set_id].dj_wallet !== wallet) {
      return res.status(403).json({ error: 'Set not found or not owned by this wallet' });
    }
    sets[set_id].active = false;
    await djService.saveDjSets(sets);
    res.json({ success: true });
  } catch (err) { next(err); }
});

module.exports = router;
```

`djService.js` (0.5 KB)

```javascript
'use strict';

const path = require('path');
const fs   = require('fs-extra');

const DJ_SETS_PATH = path.resolve(process.cwd(), 'dj_sets.json');
```

```
fs.ensureFileSync(DJ_SETS_PATH);

async function loadDjSets() {
  try {
    return JSON.parse(await fs.readFile(DJ_SETS_PATH, 'utf8'));
  } catch {
    return {};
  }
}

async function saveDjSets(sets) {
  await fs.writeFile(DJ_SETS_PATH, JSON.stringify(sets, null, 2));
}

module.exports = { loadDjSets, saveDjSets };
```

`ethService.js` (4.2 KB)

```javascript
'use strict';

const { ethers }    = require('ethers');
const logger        = require('../config/logger');
const { DEV_MODE }  = require('../middleware/devBypass');
const { awsKmsSignEIP712 } = require('./module_aws_shim');

// ── Contract addresses ──────────────────────────────────────────────
const ADDRESSES = {
  contentCA:         process.env.CONTENT_CA_ADDRESS          || '0x0000000000000000000000000000000000000
  streamingRegistry: process.env.STREAMING_REGISTRY_ADDRESS  || '0x0000000000000000000000000000000000000
  royaltyPayout:     process.env.ROYALTY_PAYOUT_ADDRESS      || '0x0000000000000000000000000000000000000
  escrow:            process.env.ESCROW_CONTRACT_ADDRESS      || '0x0000000000000000000000000000000000000
  platformNft:       process.env.PLATFORM_NFT_ADDRESS         || '0x0000000000000000000000000000000000000
};

// Minimal ERC-721 ABI — only what we need for ownership checks
const ERC721_ABI = [
  'function balanceOf(address owner) view returns (uint256)',
  'function ownerOf(uint256 tokenId) view returns (address)',
];

// ── Provider + wallet initialization ────────────────────────────────
let provider         = null;
let mspWallet        = null;
let streamingContract = null;
```

```
if (!DEV_MODE) {
  try {
    provider   = new ethers.providers.JsonRpcProvider(process.env.ETH_RPC);
    mspWallet  = new ethers.Wallet(process.env.MSP_PRIVATE_KEY, provider);

    const streamingRegistryABI = global.STREAMING_REGISTRY_ABI || [];
    streamingContract = new ethers.Contract(ADDRESSES.streamingRegistry, streamingRegistryABI, mspWall
    logger.info('Ethereum provider, wallet, and contracts initialized');
  } catch (err) {
    logger.warn({ err }, 'Ethereum provider failed to init — running without on-chain calls');
  }
}

// ── Platform NFT ownership check ──────────────────────────────────────────

async function hasPlatformNft(walletAddress) {
  if (!provider) return false;
  try {
    const contract = new ethers.Contract(ADDRESSES.platformNft, ERC721_ABI, provider);
    const balance  = await contract.balanceOf(walletAddress);
    return balance.gt(0);
  } catch (err) {
    logger.warn({ err }, 'Platform NFT check failed');
    return false;
  }
}

// ── EIP-712 signing ───────────────────────────────────────────────────────

function derSigToRSV(derHex, digestHex) {
  const buf = Buffer.from(derHex.replace(/^0x/, ''), 'hex');
  if (buf[0] !== 0x30) throw new Error('Unexpected DER prefix');

  let offset = 2;
  if (buf[offset] !== 0x02) throw new Error('DER: missing r marker');
  offset++;
  const rLen = buf[offset++];
  const r    = buf.slice(offset, offset + rLen);
  offset    += rLen;
  if (buf[offset] !== 0x02) throw new Error('DER: missing s marker');
  offset++;
  const sLen = buf[offset++];
  const s    = buf.slice(offset, offset + sLen);

  const r32 = Buffer.alloc(32); r.copy(r32, 32 - r.length);
  const s32 = Buffer.alloc(32); s.copy(s32, 32 - s.length);
  const rHex = '0x' + r32.toString('hex');
```

```
  const sHex = '0x' + s32.toString('hex');

  const digest = ethers.utils.arrayify(digestHex);
  for (const v of [27, 28]) {
    try {
      const addr = ethers.utils.recoverAddress(digest, { r: rHex, s: sHex, v });
      if (addr && ethers.utils.isAddress(addr)) {
        return ethers.utils.joinSignature({ r: rHex, s: sHex, v });
      }
    } catch { /* try next v */ }
  }
  return ethers.utils.joinSignature({ r: rHex, s: sHex, v: 27 });
}

async function signEIP712(domain, types, value) {
  const maybeSig = await awsKmsSignEIP712(domain, types, value, mspWallet);
  const digest   = ethers.utils._TypedDataEncoder.hash(domain, types, value);
  return /^0x30/i.test(maybeSig) ? derSigToRSV(maybeSig, digest) : maybeSig;
}

module.exports = {
  ADDRESSES,
  provider,
  mspWallet,
  streamingContract,
  hasPlatformNft,
  signEIP712,
  ethers,
};
```

`ipfsService.js` (1.4 KB)

```javascript
'use strict';

const path   = require('path');
const fs     = require('fs-extra');
const { create: createIpfs } = require('ipfs-http-client');
const logger = require('../config/logger');

const IPFS_GATEWAY = process.env.IPFS_GATEWAY || 'https://ipfs.io/ipfs/';

let ipfs = null;
try {
  ipfs = createIpfs({ url: process.env.IPFS_ENDPOINT || 'http://127.0.0.1:5001' });
  logger.info('IPFS client initialized');
```

```
} catch (err) {
  logger.warn({ err }, 'IPFS client not configured');
}

/**
 * Adds an entire directory to IPFS and returns the folder CID.
 */
async function addDirectoryToIpfs(ipfsClient, dir) {
  const entries = [];

  async function walk(base) {
    for (const name of await fs.readdir(base)) {
      const full = path.join(base, name);
      const stat = await fs.stat(full);
      if (stat.isDirectory()) {
        await walk(full);
      } else {
        const rel = path.relative(dir, full).split(path.sep).join('/');
        entries.push({ path: rel, content: fs.createReadStream(full) });
      }
    }
  }

  await walk(dir);

  const added = [];
  for await (const result of ipfsClient.addAll(entries, { wrapWithDirectory: true })) {
    added.push(result);
  }

  const dirEntry = added.find((r) => r.path === '');
  if (!dirEntry) throw new Error('IPFS folder CID not found');

  return { folderCid: dirEntry.cid.toString(), files: added };
}

module.exports = { ipfs, IPFS_GATEWAY, addDirectoryToIpfs };
```

`live.js` (7.8 KB)

```javascript
'use strict';

const express        = require('express');
const path           = require('path');
const fs             = require('fs-extra');
```

```
const { v4: uuidv4 } = require('uuid');
const logger         = require('../config/logger');
const { STREAM_DURATION_LIMITS } = require('../config/constants');
const { isDevWallet } = require('../middleware/devBypass');
const profileService  = require('../services/profileService');
const sessionStore    = require('../state/liveSessions');

const router = express.Router();

// POST /api/live-start
router.post('/start', async (req, res, next) => {
  try {
    const { wallet, title, artistName, quality = '720p' } = req.body || {};
    if (!wallet || !title || !artistName) {
      return res.status(400).json({ error: 'Missing wallet, title, or artistName' });
    }

    if (!isDevWallet(wallet)) {
      const profiles = await profileService.loadProfiles();
      const level    = profileService.getCapabilityLevel(profiles[wallet]);
      if (!['creator_active', 'nft_creator_active'].includes(level)) {
        return res.status(403).json({ error: 'Active creator subscription required to go live.' });
      }
    }

    const profiles    = await profileService.loadProfiles();
    const accountType = profiles[wallet]?.account_type || 'creator';
    const sessionId   = uuidv4();

    sessionStore.createSession(sessionId, { wallet, title, artistName, quality, accountType });
    logger.info({ sessionId, wallet, title }, 'Live session started');

    res.status(201).json({
      success:      true,
      sessionId,
      hlsUrl:       `/live/${sessionId}/master.m3u8`,
      thumbnailUrl: `/live/${sessionId}/thumbnail.jpg`,
    });
  } catch (err) { next(err); }
});

// POST /api/live-end/:sessionId
router.post('/end/:sessionId', async (req, res, next) => {
  try {
    const { sessionId } = req.params;
    const { wallet }    = req.body || {};
    const session       = sessionStore.getSession(sessionId);
```

```
    if (!session) return res.status(404).json({ error: 'Session not found' });
    if (session.wallet !== wallet && !isDevWallet(wallet)) {
      return res.status(403).json({ error: 'Not your session' });
    }

    const ended    = sessionStore.endSession(sessionId);
    const duration = Math.floor((ended.endTime - ended.startTime) / 1000);

    logger.info({ sessionId, duration, peakViewers: ended.peakViewers, tipsTotal: ended.tipsTotal }, '
    res.json({ success: true, sessionId, duration, peakViewers: ended.peakViewers, tipsTotal: ended.ti
  } catch (err) { next(err); }
});

// POST /api/live-ingest/:sessionId  (binary chunk upload)
router.post('/ingest/:sessionId',
  express.raw({ type: 'application/octet-stream', limit: '20mb' }),
  (req, res) => {
    const session = sessionStore.getSession(req.params.sessionId);
    if (!session) return res.status(404).json({ error: 'Session not found' });
    if (!session.alive) return res.status(410).json({ status: 'ended_unexpectedly' });

    const elapsed = Date.now() - session.startTime;
    const cap     = STREAM_DURATION_LIMITS[session.accountType] ?? STREAM_DURATION_LIMITS.creator;

    if (elapsed > cap) {
      sessionStore.endSession(req.params.sessionId);
      return res.status(410).json({
        status: 'duration_cap_reached',
        error:  session.accountType === 'creator'
          ? 'Your 3-hour stream limit has been reached. Upgrade to Platform NFT Creator for unlimited
          : 'Stream duration limit reached.',
        cap_ms: cap,
      });
    }

    if (req.body?.length) {
      session.chunks.push(req.body);
      session.chunkCount++;
    }
    res.json({ success: true, chunkCount: session.chunkCount });
  }
);

// GET /api/live-concerts
router.get('/concerts', (req, res) => {
  res.json(sessionStore.getAllActive().map((s) => ({
```

```
    cid:             s.sessionId,
    sessionId:       s.sessionId,
    artist:          s.artistName,
    artistWallet:    s.wallet,
    title:           s.title,
    contractAddress: null,
    live:            true,
    hlsUrl:          `/live/${s.sessionId}/master.m3u8`,
    viewerCount:     s.viewerCount,
    duration:        Math.floor((Date.now() - s.startTime) / 1000),
    thumbnailUrl:    `/live/${s.sessionId}/thumbnail.jpg`,
  })));
});

// GET /api/live-recording/:sessionId  (download archived stream)
router.get('/recording/:sessionId', (req, res) => {
  const session = sessionStore.getSession(req.params.sessionId);
  const { wallet } = req.query;

  if (!session) return res.status(404).json({ error: 'Session not found' });
  if (session.wallet !== wallet && !isDevWallet(wallet)) {
    return res.status(403).json({ error: 'Not your session' });
  }
  if (session.alive) return res.status(409).json({ error: 'Stream still live — end it first.' });
  if (!session.chunks?.length) return res.status(404).json({ error: 'No recording data available.' });

  const total    = session.chunks.reduce((n, c) => n + c.length, 0);
  const merged   = Buffer.concat(session.chunks, total);
  const safeName = (session.title || 'stream').replace(/[^a-z0-9_\-\s]/gi, '_').replace(/\s+/g, '_');
  const filename = `${safeName}_${new Date().toISOString().slice(0, 10)}.webm`;

  logger.info({ sessionId: req.params.sessionId, bytes: total, filename }, 'Live recording download se
  res.setHeader('Content-Type', 'video/webm');
  res.setHeader('Content-Disposition', `attachment; filename="${filename}"`);
  res.setHeader('Content-Length', total);
  res.send(merged);
});

// POST /api/start-live-encode  (external FFmpeg encoder)
router.post('/start-encode', async (req, res, next) => {
  try {
    const { wallet, eventTitle, artistName, inputSource = 'rtmp://localhost/live/djset' } = req.body |
    if (!wallet || !eventTitle || !artistName) {
      return res.status(400).json({ error: 'wallet, eventTitle, and artistName are required' });
    }

    if (!isDevWallet(wallet)) {
```

```
      const profiles = await profileService.loadProfiles();
      const level    = profileService.getCapabilityLevel(profiles[wallet]);
      if (!['creator_active', 'nft_creator_active'].includes(level)) {
        return res.status(403).json({ error: 'An active creator subscription is required to host live
      }
    }

    const ffmpegPath  = process.env.FFMPEG_PATH || 'ffmpeg';
    const productionID = `${artistName.replace(/\s+/g, '_')}_${eventTitle.replace(/\s+/g, '_')}_${Date
    const outputDir   = path.join(process.cwd(), 'public', 'live', productionID);
    await fs.ensureDir(outputDir);

    const ffmpegArgs = buildLiveEncodeArgs(inputSource, outputDir);
    const { spawn }  = require('child_process');
    const proc       = spawn(ffmpegPath, ffmpegArgs, { stdio: ['pipe', 'pipe', 'pipe'] });

    proc.stderr.on('data', (d) => logger.debug(`FFmpeg: ${d}`));
    proc.on('error', (err) => logger.error({ productionID, err }, 'FFmpeg error'));
    proc.on('close',  (code) => logger.info({ productionID, code }, 'Live encode closed'));

    logger.info({ productionID, inputSource }, 'Live encode started');
    res.status(201).json({
      success:      true,
      productionID,
      hlsUrl:       `/live/${productionID}/master.m3u8`,
      thumbnailUrl: `/live/${productionID}/thumbnail.jpg`,
    });
  } catch (err) {
    logger.error({ err }, 'Failed to start live encode');
    res.status(500).json({ success: false, error: err.message });
  }
});

function buildLiveEncodeArgs(inputSource, outputDir) {
  return [
    '-i', inputSource,
    '-c:v', 'libx264', '-preset', 'veryfast', '-crf', '23',
    '-c:a', 'aac', '-b:a', '128k',
    '-f', 'hls', '-hls_time', '4', '-hls_list_size', '10',
    '-hls_flags', 'delete_segments+independent_segments',
    '-master_pl_name', 'master.m3u8',
    '-var_stream_map', 'v:0,a:0',
    `${outputDir}/v%v.m3u8`,
    '-vf', 'fps=1/10,scale=320:-1', '-update', '1', `${outputDir}/thumbnail.jpg`,
  ];
}
```

```
module.exports = router;
```

`liveSessions.js` (1.7 KB)

```javascript
'use strict';

/**
 * In-memory live session store.
 * Shared by live routes and the WebSocket server.
 * Sessions are transient — lost on server restart (by design for MVP).
 *
 * Each session shape:
 *   { sessionId, wallet, title, artistName, quality, accountType,
 *     startTime, endTime?, alive, chunks[], chunkCount,
 *     viewerCount, peakViewers, tipsTotal }
 */
const liveSessions = new Map();

function getSession(sessionId) {
  return liveSessions.get(sessionId) || null;
}

function createSession(sessionId, data) {
  const session = {
    sessionId,
    chunks:      [],
    chunkCount:  0,
    viewerCount: 0,
    peakViewers: 0,
    tipsTotal:   0,
    alive:       true,
    startTime:   Date.now(),
    ...data,
  };
  liveSessions.set(sessionId, session);
  return session;
}

function endSession(sessionId) {
  const session = liveSessions.get(sessionId);
  if (!session) return null;
  session.alive   = false;
  session.endTime = Date.now();
  return session;
}
```

```javascript
function incrementViewer(sessionId) {
  const session = liveSessions.get(sessionId);
  if (!session) return;
  session.viewerCount++;
  if (session.viewerCount > session.peakViewers) session.peakViewers = session.viewerCount;
}

function decrementViewer(sessionId) {
  const session = liveSessions.get(sessionId);
  if (!session || session.viewerCount <= 0) return;
  session.viewerCount--;
}

function addTip(sessionId, amount) {
  const session = liveSessions.get(sessionId);
  if (!session) return;
  session.tipsTotal += parseFloat(amount) || 0;
}

function getAllActive() {
  return Array.from(liveSessions.values()).filter((s) => s.alive);
}

module.exports = {
  liveSessions,
  getSession,
  createSession,
  endSession,
  incrementViewer,
  decrementViewer,
  addTip,
  getAllActive,
};
```

`nftPlatform.js` (3.2 KB)

```javascript
'use strict';

const express        = require('express');
const logger         = require('../config/logger');
const { FEES }       = require('../config/constants');
const profileService = require('../services/profileService');
const { hasPlatformNft, ADDRESSES } = require('../services/ethService');
```

```
const router = express.Router();

// POST /api/claim-platform-nft
router.post('/claim-platform-nft', async (req, res, next) => {
  try {
    const { wallet } = req.body || {};
    if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

    const holdsNft = await hasPlatformNft(wallet);
    if (!holdsNft) {
      return res.status(403).json({ error: 'Platform NFT not detected in this wallet. Purchase it firs
    }

    const profiles   = await profileService.loadProfiles();
    if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

    const profile    = profiles[wallet];
    const wasCreator = profile.account_type === 'creator';

    profile.account_type          = 'platform_nft_creator';
    profile.platform_nft_address  = ADDRESSES.platformNft;
    profile.royalty_fee_rate      = FEES.PLATFORM_ROYALTY_NFT;

    if (!profile.supporter_subaccount) {
      profile.supporter_subaccount = {
        enabled:                false,
        linked_creator_wallet:  wallet,
        royalty_beneficiary_of: [],
      };
    }

    profiles[wallet] = profile;
    await profileService.saveProfiles(profiles);

    if (wasCreator && profileService.isSubscriptionActive(profile)) {
      logger.info({ wallet }, 'Creator upgraded to Platform NFT — subscription continues at NFT rate')
    }

    res.json({
      success:          true,
      account_type:     profile.account_type,
      royalty_fee_rate: profile.royalty_fee_rate,
      message:          'Platform NFT claimed. You now have creator capabilities with a 1.5% royalty f
    });
  } catch (err) { next(err); }
});
```

```
// POST /api/check-platform-nft
router.post('/check-platform-nft', async (req, res, next) => {
  try {
    const { wallet } = req.body || {};
    if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

    const profiles = await profileService.loadProfiles();
    const profile  = profiles[wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });

    const holdsNft = await hasPlatformNft(wallet);

    if (holdsNft && profile.account_type !== 'platform_nft_creator') {
      profile.account_type          = 'platform_nft_creator';
      profile.royalty_fee_rate      = FEES.PLATFORM_ROYALTY_NFT;
      profile.platform_nft_address  = ADDRESSES.platformNft;
      profiles[wallet] = profile;
      await profileService.saveProfiles(profiles);
    } else if (!holdsNft && profile.account_type === 'platform_nft_creator') {
      profile.account_type          = 'creator';
      profile.royalty_fee_rate      = FEES.PLATFORM_ROYALTY_STANDARD;
      profile.platform_nft_address  = null;
      profiles[wallet] = profile;
      await profileService.saveProfiles(profiles);
    }

    res.json({
      holds_nft:        holdsNft,
      account_type:     profile.account_type,
      royalty_fee_rate: profile.royalty_fee_rate,
    });
  } catch (err) { next(err); }
});

module.exports = router;
```

`playTokens.js` (4.4 KB)

```javascript
'use strict';

const express  = require('express');
const path     = require('path');
const fs       = require('fs-extra');
const crypto   = require('crypto');
const jwt      = require('jsonwebtoken');
```

```
const logger   = require('../config/logger');
const { isDevWallet } = require('../middleware/devBypass');
const profileService  = require('../services/profileService');
const redisService    = require('../services/redisService');
const { IPFS_GATEWAY } = require('../services/ipfsService');
const { streamingContract, ethers } = require('../services/ethService');
const { v4: uuidv4 } = require('uuid');

let blake3;
try { blake3 = require('blake3'); } catch { /* optional */ }

const PLAY_TOKEN_SECRET = process.env.PLAY_TOKEN_SECRET || 'dev-play-secret';

const STREAMABLE_LEVELS = new Set([
  'listener_1', 'listener_2', 'listener_3', 'creator_active', 'nft_creator_active',
]);

const router = express.Router();

// POST /api/request-play-token
router.post('/request', async (req, res, next) => {
  try {
    const { cid, listener, live, playlistId } = req.body || {};
    if (!cid || !listener) return res.status(400).json({ error: 'Missing fields' });

    if (!isDevWallet(listener)) {
      const profiles = await profileService.loadProfiles();
      const level    = profileService.getCapabilityLevel(profiles[listener]);
      if (!STREAMABLE_LEVELS.has(level)) {
        return res.status(403).json({ error: 'Active subscription required to stream.', pay_per_play:
      }
    }

    const playId  = uuidv4();
    const payload = { playId, cid, listener, live: !!live, playlistId, iat: Math.floor(Date.now() / 10
    const token   = jwt.sign(payload, PLAY_TOKEN_SECRET, { algorithm: 'HS256', expiresIn: '10m' });
    res.json({ playToken: token });
  } catch (err) { next(err); }
});

// POST /api/submit-play-proof
router.post('/proof', async (req, res, next) => {
  try {
    const { playToken } = req.body || {};
    const decoded       = jwt.verify(playToken, PLAY_TOKEN_SECRET);
    const replayKey     = `play:${decoded.playId}`;
```

```
    const redis = redisService.getClient();
    if (redis && await redis.exists(replayKey)) throw new Error('Replay detected');
    if (redis) await redis.set(replayKey, 'used', { EX: 86400 });

    await verifyMetadataIntegrity(decoded.cid);

    if (streamingContract) {
      const metadataHash = ethers.utils.keccak256(
        ethers.utils.defaultAbiCoder.encode(
          ['bytes32', 'string', 'address', 'bool'],
          [decoded.playId, decoded.cid, decoded.listener, !!decoded.live]
        )
      );
      const tx = await streamingContract.logPlay(
        ethers.utils.keccak256(ethers.utils.toUtf8Bytes(decoded.playId)),
        decoded.cid, decoded.listener, !!decoded.live, metadataHash
      );
      await tx.wait();
    } else {
      logger.warn({ playId: decoded.playId }, 'logPlay skipped — DEV_MODE');
    }

    res.json({ success: true });
  } catch (err) {
    logger.error({ err }, 'Proof verification failed');
    res.status(400).json({ error: 'Invalid proof' });
  }
});

async function verifyMetadataIntegrity(cid) {
  if (!cid) return;

  // DEV_MODE: local CIDs start with 'local:'
  if (cid.startsWith('local:')) {
    const contentId = cid.replace('local:', '');
    const metaPath  = path.join(process.cwd(), 'public', 'catalog', contentId, 'metadata.json');
    try {
      await fs.readFile(metaPath, 'utf8');
      logger.info({ contentId }, 'DEV play proof — local metadata verified');
    } catch {
      logger.warn({ contentId }, 'DEV play proof — local metadata not found');
    }
    return;
  }

  // Production: fetch from IPFS gateway and verify hashes
  const response = await fetch(`${IPFS_GATEWAY}${cid}`);
```

```
  if (!response.ok) throw new Error(`IPFS fetch failed: ${response.status}`);

  const fetchedMetadata = await response.json();
  const recomputedSha   = crypto.createHash('sha256').update(JSON.stringify(fetchedMetadata)).digest('

  if (recomputedSha !== fetchedMetadata.integrityHashes?.sha256Metadata) {
    throw new Error('Metadata integrity failed (sha256 mismatch)');
  }
  if (blake3 && fetchedMetadata.integrityHashes?.blake3Metadata) {
    const recomputedB3 = blake3.hash(Buffer.from(JSON.stringify(fetchedMetadata))).toString('hex');
    if (recomputedB3 !== fetchedMetadata.integrityHashes.blake3Metadata) {
      throw new Error('Metadata integrity failed (blake3 mismatch)');
    }
  }
}

module.exports = router;
```

`playlists.js` (1.8 KB)

```javascript
'use strict';

const express        = require('express');
const { v4: uuidv4 } = require('uuid');
const logger         = require('../config/logger');
const { PLAYLIST_CAPABLE_LEVELS } = require('../config/constants');
const profileService = require('../services/profileService');

const router = express.Router();

// GET /api/playlists
router.get('/', async (req, res, next) => {
  try {
    const profiles  = await profileService.loadProfiles();
    const playlists = Object.values(profiles)
      .flatMap((p) => p.playlists || [])
      .filter((pl) => !pl.isPrivate);
    res.json(playlists);
  } catch (err) { next(err); }
});

// POST /api/create-playlist
router.post('/', async (req, res, next) => {
  try {
    const { wallet, name, cids, sharePercent = 8 } = req.body || {};
```

```
    if (!wallet || !name || !Array.isArray(cids) || !cids.length) {
      return res.status(400).json({ error: 'Missing wallet, name, or cids' });
    }

    const profiles = await profileService.loadProfiles();
    if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

    const level = profileService.getCapabilityLevel(profiles[wallet]);
    if (!PLAYLIST_CAPABLE_LEVELS.has(level)) {
      return res.status(403).json({ error: 'Tier 2 or higher subscription required to create playlists
    }

    const playlistId = uuidv4();
    const playlist   = { id: playlistId, name, cids, wallet, sharePercent, createdAt: Date.now() };

    if (!profiles[wallet].playlists) profiles[wallet].playlists = [];
    profiles[wallet].playlists.push(playlist);
    await profileService.saveProfiles(profiles);

    logger.info({ wallet, playlistId, name }, 'Playlist created');
    res.status(201).json({ success: true, playlist });
  } catch (err) { next(err); }
});

module.exports = router;
```

`profiles.js` (2.9 KB)

```javascript
'use strict';

const express        = require('express');
const { v4: uuidv4 } = require('uuid');
const logger         = require('../config/logger');
const { FEES }       = require('../config/constants');
const profileService = require('../services/profileService');
const { hasPlatformNft } = require('../services/ethService');

const router = express.Router();

// GET /api/profile/:wallet
router.get('/:wallet', async (req, res, next) => {
  try {
    const profiles = await profileService.loadProfiles();
    const profile  = profiles[req.params.wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });
```

```
    // Auto-downgrade if Platform NFT was sold
    if (profile.account_type === 'platform_nft_creator') {
      const stillHolds = await hasPlatformNft(req.params.wallet);
      if (!stillHolds) {
        profile.account_type      = 'creator';
        profile.royalty_fee_rate  = FEES.PLATFORM_ROYALTY_STANDARD;
        profile.platform_nft_address = null;
        profiles[req.params.wallet] = profile;
        await profileService.saveProfiles(profiles);
        logger.info({ wallet: req.params.wallet }, 'Platform NFT no longer held — downgraded to creato
      }
    }

    res.json(profile);
  } catch (err) { next(err); }
});

// POST /api/create-profile
router.post('/create-profile', async (req, res, next) => {
  try {
    const { wallet: rawWallet, name, account_type = 'listener' } = req.body || {};
    if (!rawWallet || !name) return res.status(400).json({ error: 'Missing wallet or name' });
    if (!['listener', 'creator'].includes(account_type)) {
      return res.status(400).json({ error: 'account_type must be listener or creator' });
    }

    const wallet   = rawWallet.toLowerCase();
    const profiles = await profileService.loadProfiles();
    if (profiles[wallet]) return res.status(409).json({ error: 'Profile already exists' });

    profiles[wallet] = profileService.createProfile(wallet, name, account_type);
    await profileService.saveProfiles(profiles);
    res.json(profiles[wallet]);
  } catch (err) { next(err); }
});

// POST /api/update-profile
router.post('/update-profile', async (req, res, next) => {
  try {
    const { wallet, nftContractAddress, playlistCid, djTipsDefault, name } = req.body || {};
    const profiles = await profileService.loadProfiles();
    if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

    if (nftContractAddress) profiles[wallet].nft_contract_address = nftContractAddress;
    if (playlistCid) profiles[wallet].playlist_cids = (profiles[wallet].playlist_cids || []).concat(pl
    if (typeof djTipsDefault === 'boolean') profiles[wallet].dj_settings.tips_enabled_default = djTips
```

```
    if (name) profiles[wallet].name = name;

    await profileService.saveProfiles(profiles);
    res.json({ success: true });
  } catch (err) { next(err); }
});

module.exports = router;
```

`profileService.js` (3.5 KB)

```javascript
'use strict';

const path = require('path');
const fs   = require('fs-extra');
const { v4: uuidv4 } = require('uuid');
const { FEES } = require('../config/constants');

const PROFILES_PATH = path.resolve(process.cwd(), 'profiles.json');
fs.ensureFileSync(PROFILES_PATH);

// ── Persistence ──────────────────────────────────────────────────────

async function loadProfiles() {
  try {
    const raw = JSON.parse(await fs.readFile(PROFILES_PATH, 'utf8'));
    // Normalize all keys to lowercase so mixed-case checksummed addresses always match
    const normalized = {};
    for (const [key, value] of Object.entries(raw)) {
      normalized[key.toLowerCase()] = value;
    }
    return normalized;
  } catch {
    return {};
  }
}

async function saveProfiles(profiles) {
  await fs.writeFile(PROFILES_PATH, JSON.stringify(profiles, null, 2));
}

// ── Profile factory ──────────────────────────────────────────────────

function createProfile(wallet, name, accountType) {
  return {
```

```
    user_id:              uuidv4(),
    name,
    wallet_address:       wallet,
    account_type:         accountType,
    listener_tier:        accountType === 'listener' ? 1 : null,
    listener_plan:        null,
    subscription_start:   null,
    subscription_expiry:  null,
    platform_nft_address: null,
    royalty_fee_rate:     accountType === 'creator' ? FEES.PLATFORM_ROYALTY_STANDARD : null,
    nft_contract_address: null,
    playlist_cids:        [],
    favorites:            [],
    dj_settings: { tips_enabled_default: true },
    supporter_subaccount: {
      enabled:                false,
      linked_creator_wallet:  wallet,
      royalty_beneficiary_of: [],
    },
  };
}

// ── Capability helpers ──────────────────────────────────────────────────

function isSubscriptionActive(profile) {
  if (!profile) return false;
  return !!(profile.subscription_expiry && Date.now() < profile.subscription_expiry);
}

function getListenerTier(profile) {
  if (!profile || profile.account_type !== 'listener') return 0;
  if (!isSubscriptionActive(profile)) return 0;
  return profile.listener_tier || 0;
}

function getCapabilityLevel(profile) {
  if (!profile) return 'none';
  const { account_type: type } = profile;
  const active = isSubscriptionActive(profile);

  if (type === 'platform_nft_creator') return active ? 'nft_creator_active' : 'nft_creator_passive';
  if (type === 'creator')              return active ? 'creator_active'      : 'creator_inactive';
  if (type === 'listener') {
    const tier = active ? (profile.listener_tier || 1) : 0;
    return tier > 0 ? `listener_${tier}` : 'none';
  }
  if (type === 'admin') return 'admin';
```

```
  return 'none';
}

// ── Currency helper ──────────────────────────────────────────────────────

function usdToEth(usdAmount) {
  const rate = parseFloat(process.env.ETH_USD_RATE || '2500');
  return (usdAmount / rate).toFixed(8);
}

module.exports = {
  loadProfiles,
  saveProfiles,
  createProfile,
  isSubscriptionActive,
  getListenerTier,
  getCapabilityLevel,
  usdToEth,
};
```

`redisService.js` (0.6 KB)

```javascript
'use strict';

const redis  = require('redis');
const logger = require('../config/logger');

let client = null;

async function connect() {
  try {
    client = redis.createClient({ url: process.env.REDIS_URL || 'redis://127.0.0.1:6379' });
    client.on('error', (err) => logger.error({ err }, 'Redis error'));
    await client.connect();
    logger.info('Redis connected');
  } catch (err) {
    logger.warn({ err }, 'Redis unavailable — continuing without cache');
    client = null;
  }
}

function getClient() {
  return client;
}
```

```
module.exports = { connect, getClient };
```

`royalties.js` (6.9 KB)

```javascript
'use strict';

const express        = require('express');
const logger         = require('../config/logger');
const { FEES }       = require('../config/constants');
const profileService = require('../services/profileService');
const djService      = require('../services/djService');

const router = express.Router();

// POST /api/set-royalty-splits
router.post('/set-royalty-splits', async (req, res, next) => {
  try {
    const { wallet, cid, splits } = req.body || {};
    if (!wallet || !cid || !splits) return res.status(400).json({ error: 'Missing fields' });

    const profiles = await profileService.loadProfiles();
    const profile  = profiles[wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });

    if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
      return res.status(403).json({ error: 'Only creators can set royalty splits' });
    }

    const passiveTotal = (splits.passive || []).reduce((sum, p) => sum + (p.percent || 0), 0);
    const total = (splits.artist || 0) + (splits.nft_holders || 0) +
                  (splits.activity_pool || 0) + passiveTotal;

    if (Math.abs(total - 100) > 0.01) {
      return res.status(400).json({ error: `Splits must sum to 100%. Got: ${total}%` });
    }

    for (const recipient of (splits.passive || [])) {
      const recipientProfile = profiles[recipient.wallet];
      if (!recipientProfile) continue;
      const level       = profileService.getCapabilityLevel(recipientProfile);
      const isTier3     = level === 'listener_3';
      const isSupporter = recipientProfile.supporter_subaccount?.enabled;
      if (!isTier3 && !isSupporter) {
        return res.status(400).json({
          error: `Passive split recipient ${recipient.wallet} must be a Tier 3 listener or active supp
```

```
        });
      }
    }

    if (!profile.royalty_splits) profile.royalty_splits = {};
    profile.royalty_splits[cid] = splits;
    profiles[wallet] = profile;
    await profileService.saveProfiles(profiles);

    res.json({ success: true, splits });
  } catch (err) { next(err); }
});

// POST /api/royalty-splits  (asset-manager.html uses this endpoint)
router.post('/royalty-splits', async (req, res, next) => {
  try {
    const { wallet, cid, splits } = req.body || {};
    if (!wallet || !cid || !splits) return res.status(400).json({ error: 'Missing fields' });

    const total = Object.values(splits).reduce((s, v) => s + (parseFloat(v) || 0), 0);
    if (Math.abs(total - 100) > 0.01) {
      return res.status(400).json({ error: `Splits must sum to 100%. Got: ${total.toFixed(1)}%` });
    }

    const profiles = await profileService.loadProfiles();
    const profile  = profiles[wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });

    if (!profile.royalty_splits) profile.royalty_splits = {};
    profile.royalty_splits[cid] = splits;
    profiles[wallet] = profile;
    await profileService.saveProfiles(profiles);

    logger.info({ wallet, cid }, 'Royalty splits saved');
    res.json({ success: true, splits });
  } catch (err) { next(err); }
});

// POST /api/tip
router.post('/tip', async (req, res, next) => {
  try {
    const { from_wallet, to_wallet, tip_type, amount_eth, dj_set_id, artist_splits, dj_percent } = req

    if (!from_wallet || !tip_type || !amount_eth) {
      return res.status(400).json({ error: 'Missing from_wallet, tip_type, or amount_eth' });
    }
```

```
const grossEth    = parseFloat(amount_eth);
const platformCut = grossEth * FEES.PLATFORM_TIP;
const remaining   = grossEth - platformCut;
const distribution = [];

if (tip_type === 'artist') {
  if (!to_wallet) return res.status(400).json({ error: 'Missing to_wallet for artist tip' });
  distribution.push({ wallet: to_wallet, amount_eth: remaining, role: 'artist' });

} else if (tip_type === 'dj') {
  if (!dj_set_id) return res.status(400).json({ error: 'Missing dj_set_id for DJ tip' });

  const djSets = await djService.loadDjSets();
  const set    = djSets[dj_set_id];
  if (set?.tips_enabled === false) {
    return res.status(403).json({ error: 'This DJ set has tips disabled.' });
  }

  const djCut         = remaining * ((dj_percent || 100) / 100);
  const artistPoolCut = remaining - djCut;
  distribution.push({ wallet: to_wallet || set?.dj_wallet, amount_eth: djCut, role: 'dj' });

  if (artistPoolCut > 0 && artist_splits?.length) {
    const totalArtistPct = artist_splits.reduce((s, a) => s + (a.percent || 0), 0);
    for (const a of artist_splits) {
      distribution.push({
        wallet:     a.wallet,
        amount_eth: artistPoolCut * ((a.percent || 0) / totalArtistPct),
        role:       'artist_from_dj_tip',
      });
    }
  }
} else {
  return res.status(400).json({ error: 'tip_type must be artist or dj' });
}

const profiles   = await profileService.loadProfiles();
const tipper     = profiles[from_wallet];
const recognized = tipper && tipper.account_type !== null && profileService.isSubscriptionActive(t

logger.info({ tip_type, from_wallet, gross_eth: grossEth, platform_cut_eth: platformCut, distribut

res.json({
  success:      true,
  gross_eth:    grossEth,
  platform_cut: platformCut,
  distribution,
```

```
      recognized,
      message: recognized ? 'Tip sent — you will be credited as a supporter.' : 'Tip sent anonymously.
    });
  } catch (err) { next(err); }
});

// POST /api/nft-sale-fee
router.post('/nft-sale-fee', async (req, res, next) => {
  try {
    const { sale_price_eth, nft_type, seller_wallet, is_primary } = req.body || {};
    if (!sale_price_eth || !nft_type || !seller_wallet) {
      return res.status(400).json({ error: 'Missing sale_price_eth, nft_type, or seller_wallet' });
    }

    const priceEth = parseFloat(sale_price_eth);

    if (nft_type === 'platform' && is_primary === true) {
      return res.json({ success: true, platform_fee: 0, seller_gets: priceEth, note: 'Platform NFT pri
    }

    if (nft_type === 'platform') {
      const floorEth = parseFloat(profileService.usdToEth(FEES.PLATFORM_NFT_PRICE_USD));
      if (priceEth < floorEth) {
        return res.status(400).json({
          error:     `Platform NFT cannot be sold below $${FEES.PLATFORM_NFT_PRICE_USD} USD (${floorEt
          floor_eth: floorEth,
        });
      }
    }

    const platformFee = priceEth * FEES.PLATFORM_NFT_SALE;
    const sellerGets  = priceEth - platformFee;

    logger.info({ nft_type, seller_wallet, sale_price_eth, platform_fee: platformFee }, 'NFT sale fee
    res.json({
      success:          true,
      sale_price_eth:   priceEth,
      platform_fee_pct: FEES.PLATFORM_NFT_SALE * 100,
      platform_fee_eth: platformFee,
      seller_gets_eth:  sellerGets,
    });
  } catch (err) { next(err); }
});

module.exports = router;
```

`streamKeys.js` (1.9 KB)

```javascript
'use strict';

const express        = require('express');
const { v4: uuidv4 } = require('uuid');
const logger         = require('../config/logger');
const profileService = require('../services/profileService');

const router = express.Router();

function buildStreamKeyResponse(profile) {
  const rtmpHost = process.env.RTMP_HOST || 'rtmp://localhost/live';
  return {
    streamKey:   profile.stream_key,
    rtmpUrl:     rtmpHost,
    fullUrl:     `${rtmpHost}/${profile.stream_key}`,
    playbackUrl: `/live/${profile.stream_key}/master.m3u8`,
  };
}

// GET /api/stream-key/:wallet
router.get('/:wallet', async (req, res, next) => {
  try {
    const profiles = await profileService.loadProfiles();
    const profile  = profiles[req.params.wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });

    if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
      return res.status(403).json({ error: 'Creator account required for stream key' });
    }

    if (!profile.stream_key) {
      profile.stream_key = uuidv4().replace(/-/g, '');
      profiles[req.params.wallet] = profile;
      await profileService.saveProfiles(profiles);
    }

    res.json(buildStreamKeyResponse(profile));
  } catch (err) { next(err); }
});

// POST /api/stream-key/regenerate
router.post('/regenerate', async (req, res, next) => {
  try {
    const { wallet } = req.body || {};
```

```
    if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

    const profiles = await profileService.loadProfiles();
    const profile  = profiles[wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });

    profile.stream_key = uuidv4().replace(/-/g, '');
    profiles[wallet]   = profile;
    await profileService.saveProfiles(profiles);

    logger.info({ wallet }, 'Stream key regenerated');
    res.json(buildStreamKeyResponse(profile));
  } catch (err) { next(err); }
});

module.exports = router;
```

`subscriptions.js` (2.4 KB)

```javascript
'use strict';

const express        = require('express');
const logger         = require('../config/logger');
const { SUBSCRIPTION_PLANS } = require('../config/constants');
const profileService = require('../services/profileService');

const router = express.Router();

// POST /api/subscribe
router.post('/', async (req, res, next) => {
  try {
    const { wallet, plan } = req.body || {};
    if (!wallet || !plan) return res.status(400).json({ error: 'Missing wallet or plan' });

    const planDef = SUBSCRIPTION_PLANS[plan];
    if (!planDef) {
      return res.status(400).json({
        error: `Unknown plan. Valid plans: ${Object.keys(SUBSCRIPTION_PLANS).join(', ')}`,
      });
    }

    const profiles = await profileService.loadProfiles();
    if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

    const profile = profiles[wallet];
```

```
    if (planDef.type === 'listener' && profile.account_type !== 'listener') {
      return res.status(400).json({ error: 'Listener plans are only for listener accounts' });
    }
    if (planDef.type === 'creator' && profile.account_type !== 'creator') {
      return res.status(400).json({ error: 'Creator plans are only for creator accounts' });
    }
    if (planDef.type === 'nft_creator' && profile.account_type !== 'platform_nft_creator') {
      return res.status(400).json({ error: 'NFT creator plans require a Platform NFT' });
    }

    const now      = Date.now();
    const msPerDay = 24 * 60 * 60 * 1000;
    const baseTime = (plan.endsWith('_rolling') || !profileService.isSubscriptionActive(profile))
      ? now
      : profile.subscription_expiry;

    profile.listener_plan       = plan.endsWith('_annual')  ? 'annual'
                                 : plan.endsWith('_rolling') ? 'rolling' : 'monthly';
    profile.subscription_start  = now;
    profile.subscription_expiry = baseTime + planDef.days * msPerDay;
    if (planDef.tier) profile.listener_tier = planDef.tier;
    profile.last_subscription_price_eth = profileService.usdToEth(planDef.price_usd);

    profiles[wallet] = profile;
    await profileService.saveProfiles(profiles);

    logger.info({ wallet, plan }, 'Subscription activated');
    res.json({
      success:   true,
      plan,
      tier:      profile.listener_tier,
      expiry:    profile.subscription_expiry,
      price_usd: planDef.price_usd,
      price_eth: profile.last_subscription_price_eth,
    });
  } catch (err) { next(err); }
});

module.exports = router;
```

`transcodeService.js` (8.8 KB)

```javascript
'use strict';
```

```
const path    = require('path');
const fs      = require('fs-extra');
const ffmpeg  = require('fluent-ffmpeg');
const ffprobe = require('ffprobe');
const ffprobeStatic = require('ffprobe-static');
const sharp   = require('sharp');
const logger  = require('../config/logger');

// ── FFmpeg path setup ──────────────────────────────────────────────
try {
  const installer = require('@ffmpeg-installer/ffmpeg');
  ffmpeg.setFfmpegPath(process.env.FFMPEG_PATH || installer.path);
} catch {
  if (process.env.FFMPEG_PATH) ffmpeg.setFfmpegPath(process.env.FFMPEG_PATH);
}

// ── Quality validation ─────────────────────────────────────────────

async function validateQuality(filePath, contentType, devMode = false) {
  const data = await new Promise((resolve, reject) =>
    ffprobe(filePath, { path: ffprobeStatic.path }, (err, info) =>
      err ? reject(err) : resolve(info))
  );

  const audioStream = data?.streams?.find((s) => s.codec_type === 'audio');
  const videoStream = data?.streams?.find((s) => s.codec_type === 'video');

  if (contentType === 'music' || contentType === 'podcast') {
    if (!audioStream) throw new Error('No audio stream found in uploaded file');
    if (!devMode) {
      const bitrate = parseInt(audioStream.bit_rate || '0', 10);
      if (bitrate && bitrate < 128000) {
        throw new Error(`Audio bitrate too low (${bitrate}bps — minimum 128 kbps)`);
      }
    }
  }

  if (contentType === 'video') {
    if (!videoStream) throw new Error('No video stream — upload an MP4, MOV, MKV, or WebM file');
    if (!audioStream) throw new Error('Video must include an audio track');
    if (!devMode) {
      const vBitrate = parseInt(videoStream.bit_rate || data?.format?.bit_rate || '0', 10);
      if (vBitrate && vBitrate < 500000) {
        throw new Error(`Video bitrate too low (${vBitrate}bps — minimum 500 kbps)`);
      }
    }
  }
```

```
  if (contentType === 'art_animated') {
    if (!videoStream) throw new Error('Animated art must be a video file (MP4, WebM)');
  }

  return { audioStream, videoStream, format: data?.format };
}

async function validateImage(filePath, devMode = false) {
  const meta  = await sharp(filePath).metadata();
  const stats = await fs.stat(filePath);

  if (stats.size > 10 * 1024 * 1024) throw new Error('Cover image must be under 10MB');

  if (!devMode) {
    if (meta.width < 1000 || meta.height < 1000) {
      throw new Error('Cover image must be at least 1000×1000px');
    }
    const aspectRatio = meta.width / meta.height;
    if (Math.abs(aspectRatio - 1) > 0.05) {
      throw new Error('Cover image must be square (1:1 aspect ratio)');
    }
  }
}

// ── Audio HLS transcoding — DEV (single-pass, lo+hi only for speed) ──────────

async function transcodeAudioDev(inputPath, outputDir, contentId) {
  const previewPath = path.join(outputDir, 'preview.mp3');

  await runFfmpeg(
    ffmpeg(inputPath)
      .outputOptions(['-t', '30', '-b:a', '128k', '-ar', '44100'])
      .audioCodec('libmp3lame')
      .output(previewPath)
  ).catch(() => {}); // preview failure is non-fatal

  await runFfmpeg(
    ffmpeg(inputPath)
      .output(path.join(outputDir, 'lo.m3u8'))
        .audioCodec('aac').audioBitrate('128k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', path.join(outputDir, 'lo_%03d.ts'))
      .output(path.join(outputDir, 'hi.m3u8'))
        .audioCodec('aac').audioBitrate('320k').audioFrequency(44100)
        .format('hls')
```

```
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', path.join(outputDir, 'hi_%03d.ts'))
      .on('start', (cmd) => logger.info({ contentId }, 'FFmpeg DEV audio: ' + cmd))
  );

  const master =
    '#EXTM3U\n#EXT-X-VERSION:3\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=128000,CODECS="mp4a.40.2"\nlo.m3u8\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=320000,CODECS="mp4a.40.2"\nhi.m3u8\n';
  await fs.writeFile(path.join(outputDir, 'master.m3u8'), master);

  return { previewPath: await fs.pathExists(previewPath) ? previewPath : null };
}

// ── Audio HLS transcoding — PRODUCTION (lo + mid + hi) ──────────────────────

async function transcodeAudioProd(inputPath, outputDir, contentId) {
  const previewPath = path.join(outputDir, 'preview.mp3');

  await runFfmpeg(
    ffmpeg(inputPath)
      .outputOptions(['-t', '30', '-b:a', '128k', '-ar', '44100'])
      .audioCodec('libmp3lame')
      .output(previewPath)
  );

  await runFfmpeg(
    ffmpeg(inputPath)
      .output(path.join(outputDir, 'lo.m3u8'))
        .audioCodec('aac').audioBitrate('128k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', path.join(outputDir, 'lo_%03d.ts'))
      .output(path.join(outputDir, 'mid.m3u8'))
        .audioCodec('aac').audioBitrate('256k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', path.join(outputDir, 'mid_%03d.ts'))
      .output(path.join(outputDir, 'hi.m3u8'))
        .audioCodec('aac').audioBitrate('320k').audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '10').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', path.join(outputDir, 'hi_%03d.ts'))
      .on('start', (cmd) => logger.info({ contentId }, 'FFmpeg audio: ' + cmd))
  );

  const master =
```

```
    '#EXTM3U\n#EXT-X-VERSION:3\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=128000,CODECS="mp4a.40.2"\nlo.m3u8\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=256000,CODECS="mp4a.40.2"\nmid.m3u8\n' +
    '#EXT-X-STREAM-INF:BANDWIDTH=320000,CODECS="mp4a.40.2"\nhi.m3u8\n';
  await fs.writeFile(path.join(outputDir, 'master.m3u8'), master);

  return { previewPath };
}

// ── Video HLS transcoding — adaptive ladder ──────────────────────────────

async function transcodeVideo(inputPath, outputDir, contentId, sourceHeight) {
  const previewPath = path.join(outputDir, 'preview.mp4');
  const ladder      = buildVideoLadder(sourceHeight || 720);
  const bwMap       = { '1080p': 4500000, '720p': 2800000, '480p': 1400000 };

  await runFfmpeg(
    ffmpeg(inputPath)
      .outputOptions(['-ss', '0', '-t', '5', '-vf', 'scale=640:-1'])
      .output(previewPath)
      .on('error', () => logger.warn({ contentId }, 'Video preview generation failed — non-fatal'))
  ).catch(() => {});

  for (const rung of ladder) {
    await runFfmpeg(
      ffmpeg(inputPath)
        .videoCodec('libx264')
        .outputOptions([
          '-preset', 'veryfast', '-crf', '22',
          '-maxrate', rung.vbr, '-bufsize', `${parseInt(rung.vbr) * 2}k`,
          '-vf', `scale=${rung.w}:${rung.h}:force_original_aspect_ratio=decrease,pad=${rung.w}:${rung.
          '-g', '48', '-keyint_min', '48',
        ])
        .audioCodec('aac').audioBitrate(rung.abr).audioFrequency(44100)
        .format('hls')
        .addOption('-hls_time', '6').addOption('-hls_list_size', '0')
        .addOption('-hls_segment_filename', path.join(outputDir, `${rung.name}_%03d.ts`))
        .output(path.join(outputDir, `${rung.name}.m3u8`))
        .on('start', (cmd) => logger.info({ contentId, rung: rung.name }, 'FFmpeg video: ' + cmd))
    );
  }

  const master =
    '#EXTM3U\n#EXT-X-VERSION:3\n' +
    ladder.map((r) =>
      `#EXT-X-STREAM-INF:BANDWIDTH=${bwMap[r.name]},RESOLUTION=${r.w}x${r.h},CODECS="avc1.42e01e,mp4a.
    ).join('\n') + '\n';
```

```
  await fs.writeFile(path.join(outputDir, 'master.m3u8'), master);

  return { previewPath: await fs.pathExists(previewPath) ? previewPath : null };
}

// ── Helpers ──────────────────────────────────────────────────────────────

function buildVideoLadder(sourceHeight) {
  const ladder = [];
  if (sourceHeight >= 1080) ladder.push({ h: 1080, w: 1920, vbr: '4000k', abr: '192k', name: '1080p' }
  if (sourceHeight >= 720)  ladder.push({ h: 720,  w: 1280, vbr: '2500k', abr: '128k', name: '720p' })
  ladder.push(               { h: 480,  w: 854,  vbr: '1200k', abr: '128k', name: '480p' });
  return ladder;
}

function runFfmpeg(command) {
  return new Promise((resolve, reject) => {
    command.on('end', resolve).on('error', reject).run();
  });
}

module.exports = {
  validateQuality,
  validateImage,
  transcodeAudioDev,
  transcodeAudioProd,
  transcodeVideo,
};
```

`upload.js` (16.8 KB)

```javascript
'use strict';

const express        = require('express');
const path           = require('path');
const fs             = require('fs-extra');
const crypto         = require('crypto');
const multer         = require('multer');
const { v4: uuidv4 } = require('uuid');
const logger         = require('../config/logger');
const { FEES, VALID_CONTENT_TYPES } = require('../config/constants');
const { DEV_MODE, isDevWallet } = require('../middleware/devBypass');
const profileService   = require('../services/profileService');
const catalogService   = require('../services/catalogService');
const transcodeService = require('../services/transcodeService');
```

```
const ipfsService      = require('../services/ipfsService');
const { ethers, ADDRESSES, signEIP712, mspWallet, provider } = require('../services/ethService');

let blake3;
try {
  blake3 = require('blake3');
} catch {
  if (process.env.FORCE_BLAKE3 === 'true') throw new Error('BLAKE3 required but failed to load');
}

// ── Multer configuration ──────────────────────────────────────────────────
const ALLOWED_MEDIA_MIMES = new Set([
  'audio/mpeg','audio/mp3','audio/wav','audio/x-wav','audio/wave',
  'audio/ogg','audio/flac','audio/x-flac','audio/aac','audio/mp4','audio/x-m4a',
  'video/mp4','video/quicktime','video/x-matroska','video/webm',
]);
const ALLOWED_MEDIA_EXTS  = /\.(mp3|wav|ogg|flac|aac|m4a|mp4|mov|mkv|webm)$/i;
const ALLOWED_IMAGE_MIMES = new Set(['image/png','image/jpeg','image/webp']);
const ALLOWED_IMAGE_EXTS  = /\.(png|jpg|jpeg|webp)$/i;

fs.ensureDirSync(path.join(__dirname, '..', '..', 'temp'));

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, path.join(__dirname, '..', '..', 'temp')),
  filename:    (req, file, cb) => cb(null, uuidv4() + path.extname(file.originalname)),
});

const upload = multer({
  storage,
  limits: { fileSize: 500 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    if (file.fieldname === 'audio-file') {
      return (ALLOWED_MEDIA_MIMES.has(file.mimetype) && ALLOWED_MEDIA_EXTS.test(file.originalname))
        ? cb(null, true)
        : cb(new Error(`Unsupported media type: ${file.mimetype}`));
    }
    if (file.fieldname === 'cover-image') {
      return (ALLOWED_IMAGE_MIMES.has(file.mimetype) && ALLOWED_IMAGE_EXTS.test(file.originalname))
        ? cb(null, true)
        : cb(new Error('Cover image must be PNG, JPG, or WebP'));
    }
    cb(null, true);
  },
});

// ── Route ─────────────────────────────────────────────────────────────────
const router = express.Router();
```

```
router.post('/', upload.fields([{ name: 'audio-file', maxCount: 1 }, { name: 'cover-image', maxCount:
  async (req, res, next) => {
    const tempFiles = [];
    const cleanup   = () => Promise.all(tempFiles.map((p) => fs.remove(p).catch(() => {})));

    try {
      const audioFile  = req.files?.['audio-file']?.[0];
      const coverImage = req.files?.['cover-image']?.[0];
      if (audioFile)  tempFiles.push(audioFile.path);
      if (coverImage) tempFiles.push(coverImage.path);

      const fields = parseFields(req.body);
      validateRequiredFields(audioFile, coverImage, fields);

      if (!isDevWallet(fields.wallet)) {
        await assertCreatorProfile(fields.wallet);
      }

      assertSaneFilename(audioFile.originalname);

      const probeData = await transcodeService.validateQuality(audioFile.path, fields.contentType, DEV
        .catch((err) => { throw new Error('Quality check: ' + err.message); });
      await transcodeService.validateImage(coverImage.path, DEV_MODE)
        .catch((err) => { throw new Error('Cover image: ' + err.message); });

      const profiles    = await profileService.loadProfiles();
      const profile     = profiles[fields.wallet] || { royalty_fee_rate: FEES.PLATFORM_ROYALTY_STANDAR
      const contentId   = uuidv4();
      const hashes      = await computeHashes(audioFile.path, coverImage.path);
      const metadata    = buildMetadata(contentId, fields, profile, probeData, hashes);
      const tempDir     = path.join(__dirname, '..', '..', 'temp', contentId);
      tempFiles.push(tempDir);
      await fs.ensureDir(tempDir);

      if (DEV_MODE && !ipfsService.ipfs) {
        const result = await handleDevUpload(contentId, fields, audioFile, coverImage, metadata, tempD
        await cleanup();
        return res.json(result);
      }

      const result = await handleProdUpload(contentId, fields, audioFile, coverImage, metadata, tempDi
      await cleanup();
      return res.json(result);

    } catch (err) {
      await cleanup();
```

```
      logger.error({ err }, 'Upload failed');
      res.status(400).json({ error: String(err.message || err) });
    }
  }
);

// ── Field parsing ──────────────────────────────────────────────────────
function parseFields(body = {}) {
  const rawType     = body.contentType || 'music';
  const contentType = VALID_CONTENT_TYPES.includes(rawType) ? rawType : 'music';
  return {
    contentType,
    isAudioOnly: contentType === 'music' || contentType === 'podcast',
    isVideoType: contentType === 'video' || contentType === 'art_animated',
    isArtStill:  contentType === 'art_still',
    songTitle:   body.songTitle,
    artistName:  body.artistName,
    description: body.description    || '',
    album:       body.album          || '',
    bpm:         body.bpm            || '',
    episodeNumber: body.episodeNumber || '',
    seriesName:  body.seriesName     || '',
    releaseDate: body.releaseDate,
    userId:      body.userId,
    wallet:      body.wallet,
    tags:        String(body.tags || '').split(',').map((t) => t.trim()).filter(Boolean),
    mlc_iswc:    body.mlc_iswc            || '',
    mlc_ipi:     body.mlc_ipi_name_number || '',
    isrc:        body.isrc               || '',
    mintNft:     body.mintNft,
  };
}

function validateRequiredFields(audioFile, coverImage, fields) {
  if (!audioFile)        throw new Error('Media file is required');
  if (!coverImage)       throw new Error('Cover image is required');
  if (!fields.songTitle) throw new Error('Title is required');
  if (!fields.artistName) throw new Error('Artist name is required');
  if (!fields.userId || !fields.wallet) throw new Error('Missing userId or wallet');
}

function assertSaneFilename(filename) {
  if (/^(track\d+|song\d+|test\d*|untitled)\.(mp3|mp4|wav|webm)$/i.test(filename)) {
    throw new Error('Please rename your file before uploading.');
  }
}
```

```
async function assertCreatorProfile(wallet) {
  const profiles = await profileService.loadProfiles();
  const profile  = profiles[wallet];
  if (!profile) throw new Error('Profile not found. Create a profile first.');
  if (!['creator', 'platform_nft_creator'].includes(profile.account_type)) {
    throw new Error('A creator account is required to upload content.');
  }
}

// ── Hashing ──────────────────────────────────────────────────────────────
async function computeHashes(audioPath, coverPath) {
  const audioData  = await fs.readFile(audioPath);
  const coverData  = await fs.readFile(coverPath);
  return {
    sha256Audio:  crypto.createHash('sha256').update(audioData).digest('hex'),
    sha256Cover:  crypto.createHash('sha256').update(coverData).digest('hex'),
    blake3Audio:  blake3 ? blake3.hash(audioData).toString('hex') : null,
    blake3Cover:  blake3 ? blake3.hash(coverData).toString('hex') : null,
  };
}

// ── Metadata builder ─────────────────────────────────────────────────────
function buildMetadata(contentId, fields, profile, probeData, hashes) {
  const base = {
    id:           contentId,
    title:        fields.songTitle,
    description:  fields.description,
    creator: {
      name:           fields.artistName,
      user_id:        fields.userId,
      wallet_address: ethers.utils.keccak256(ethers.utils.toUtf8Bytes(fields.wallet)),
    },
    content_type:      fields.contentType,
    availability_type: 'on_demand',
    release_date:      fields.releaseDate || new Date().toISOString().split('T')[0],
    tags:              fields.tags,
    files:             {},
    royalty_fee_rate:  profile.royalty_fee_rate,
    integrityHashes: {
      sha256Audio: hashes.sha256Audio,
      sha256Cover: hashes.sha256Cover,
      ...(hashes.blake3Audio ? { blake3Audio: hashes.blake3Audio } : {}),
      ...(hashes.blake3Cover ? { blake3Cover: hashes.blake3Cover } : {}),
    },
  };

  if (fields.isAudioOnly) {
```

```
    base.mlc_metadata = {
      work_title:      fields.songTitle,
      iswc:            fields.mlc_iswc,
      isrc:            fields.isrc,
      ipi_name_number: fields.mlc_ipi,
      writers: [{ name: fields.artistName, role: 'artist', ipi_name_number: fields.mlc_ipi, ownership_
      publishers: [],
    };
  }
  if (fields.contentType === 'music') {
    if (fields.album) base.album = fields.album;
    if (fields.bpm)   base.bpm   = parseInt(fields.bpm, 10) || null;
  }
  if (fields.contentType === 'podcast') {
    if (fields.episodeNumber) base.episode_number = parseInt(fields.episodeNumber, 10) || null;
    if (fields.seriesName)    base.series_name    = fields.seriesName;
  }
  if (fields.isVideoType) {
    base.video = {
      width:    probeData?.videoStream?.width    || null,
      height:   probeData?.videoStream?.height   || null,
      codec:    probeData?.videoStream?.codec_name || null,
      duration: parseFloat(probeData?.format?.duration || '0') || null,
    };
  }
  return base;
}

// ── DEV upload path ──────────────────────────────────────────────────────
async function handleDevUpload(contentId, fields, audioFile, coverImage, metadata, tempDir, profiles)
  const catalogDir = path.join(process.cwd(), 'public', 'catalog', contentId);
  const hlsDir     = path.join(catalogDir, 'hls');
  await fs.ensureDir(hlsDir);

  const coverExt  = path.extname(coverImage.originalname) || '.jpg';
  const coverDest = path.join(catalogDir, 'cover' + coverExt);
  await fs.copy(coverImage.path, coverDest);
  metadata.files.cover_image = `/catalog/${contentId}/cover${coverExt}`;

  if (fields.isAudioOnly) {
    const { previewPath } = await transcodeService.transcodeAudioDev(audioFile.path, hlsDir, contentId
    if (previewPath) {
      await fs.copy(previewPath, path.join(catalogDir, 'preview.mp3'));
      metadata.files.preview_url = `/catalog/${contentId}/preview.mp3`;
    }
    metadata.ipfs_audio_url  = `/catalog/${contentId}/hls/master.m3u8`;
    metadata.files.hls_url   = metadata.ipfs_audio_url;
```

```
  } else {
    const mediaDest = path.join(catalogDir, 'media' + path.extname(audioFile.originalname));
    await fs.copy(audioFile.path, mediaDest);
    metadata.files.media_url    = `/catalog/${contentId}/media${path.extname(audioFile.originalname)}`
    metadata.ipfs_audio_url     = metadata.files.media_url;
    metadata.files.hls_url      = metadata.files.media_url;
  }

  await writeMetadata(catalogDir, metadata);
  await updateProfilePlaylist(profiles, fields.wallet, `local:${contentId}`);

  await catalogService.addCatalogEntry(contentId, {
    contentId,
    title:       metadata.title,
    artistName:  metadata.creator.name,
    wallet:      fields.wallet,
    contentType: fields.contentType,
    metadataUrl: `/catalog/${contentId}/metadata.json`,
    hlsUrl:      metadata.ipfs_audio_url,
    coverUrl:    metadata.files.cover_image,
    previewUrl:  metadata.files.preview_url || null,
  });

  logger.info({ contentId, contentType: fields.contentType, wallet: fields.wallet }, 'DEV upload compl
  return {
    success:          true,
    contentId,
    contentType:      fields.contentType,
    hlsUrl:           metadata.ipfs_audio_url,
    metadataUrl:      `/catalog/${contentId}/metadata.json`,
    metadataCid:      `local:${contentId}`,
    coverCid:         metadata.files.cover_image,
    caSignature:      null,
    royalty_fee_rate: metadata.royalty_fee_rate,
    mint_pending:     false,
    dev_mode:         true,
  };
}

// ── Production upload path ──────────────────────────────────────────────
async function handleProdUpload(contentId, fields, audioFile, coverImage, metadata, tempDir, probeData
  const ipfs   = ipfsService.ipfs;
  const hlsDir = path.join(tempDir, 'hls');
  await fs.ensureDir(hlsDir);

  if (fields.isArtStill) {
    const result = await ipfs.add({ path: 'media' + path.extname(audioFile.originalname), content: fs.
```

```
  metadata.files.media_url = `ipfs://${result.cid.toString()}`;

} else if (fields.isAudioOnly) {
  const { previewPath } = await transcodeService.transcodeAudioProd(audioFile.path, hlsDir, contentI
  const previewAdd      = await ipfs.add({ path: 'preview.mp3', content: fs.createReadStream(preview
  metadata.files.preview_url = `ipfs://${previewAdd.cid.toString()}`;
  const { folderCid } = await ipfsService.addDirectoryToIpfs(ipfs, hlsDir);
  metadata.ipfs_audio_url = `ipfs://${folderCid}/master.m3u8`;
  metadata.files.hls_url  = metadata.ipfs_audio_url;

} else {
  const { previewPath } = await transcodeService.transcodeVideo(audioFile.path, hlsDir, contentId, p
  if (previewPath) {
    const previewAdd = await ipfs.add({ path: 'preview.mp4', content: fs.createReadStream(previewPat
    metadata.files.preview_url = `ipfs://${previewAdd.cid.toString()}`;
  }
  const { folderCid } = await ipfsService.addDirectoryToIpfs(ipfs, hlsDir);
  metadata.ipfs_audio_url = `ipfs://${folderCid}/master.m3u8`;
  metadata.files.hls_url  = metadata.ipfs_audio_url;
}

const coverAdd = await ipfs.add({ path: 'cover' + path.extname(coverImage.originalname), content: fs
metadata.files.cover_image = `ipfs://${coverAdd.cid.toString()}`;

const metaStr     = JSON.stringify(metadata);
const sha256Meta  = crypto.createHash('sha256').update(metaStr).digest('hex');
const blake3Meta  = blake3 ? blake3.hash(Buffer.from(metaStr)).toString('hex') : null;
metadata.integrityHashes.sha256Metadata = sha256Meta;
if (blake3Meta) metadata.integrityHashes.blake3Metadata = blake3Meta;

const { cid: metadataCid } = await ipfs.add(JSON.stringify(metadata));
const metadataCidStr = metadataCid.toString();

let caSignature = null;
if (provider && mspWallet) {
  try {
    const network   = await provider.getNetwork();
    const domain    = { name: 'ContentCA', version: '1', chainId: Number(network.chainId), verifying
    const types     = { Certificate: [{ name: 'cid', type: 'string' }, { name: 'contentType', type:
    caSignature     = await signEIP712(domain, types, { cid: metadataCidStr, contentType: fields.con
  } catch (err) {
    logger.warn({ contentId, err }, 'EIP-712 signing skipped');
  }
}

await updateProfilePlaylist(profiles, fields.wallet, metadataCidStr);
```

```
  logger.info({ contentId, contentType: fields.contentType, wallet: fields.wallet, metadataCidStr }, '
  return {
    success:          true,
    contentId,
    contentType:      fields.contentType,
    hlsUrl:           metadata.ipfs_audio_url || metadata.files?.media_url,
    metadataUrl:      `ipfs://${metadataCidStr}`,
    metadataCid:      metadataCidStr,
    coverCid:         metadata.files.cover_image,
    caSignature,
    royalty_fee_rate: profile.royalty_fee_rate,
    mint_pending:     fields.mintNft === 'true',
  };
}

// ── Shared helpers ──────────────────────────────────────────────────────
async function writeMetadata(dir, metadata) {
  const str  = JSON.stringify(metadata, null, 2);
  const hash = crypto.createHash('sha256').update(str).digest('hex');
  metadata.integrityHashes.sha256Metadata = hash;
  await fs.writeFile(path.join(dir, 'metadata.json'), JSON.stringify(metadata, null, 2));
}

async function updateProfilePlaylist(profiles, wallet, cid) {
  const fresh = await profileService.loadProfiles();
  if (!fresh[wallet]) return;
  if (!fresh[wallet].playlist_cids) fresh[wallet].playlist_cids = [];
  fresh[wallet].playlist_cids.push(cid);
  await profileService.saveProfiles(fresh);
}

module.exports = router;
```

`wsServer.js` (2.8 KB)

```javascript
'use strict';

const logger      = require('../config/logger');
const sessionStore = require('../state/liveSessions');

/**
 * Attaches a WebSocket server to an existing HTTP/HTTPS server.
 * All session state is managed through the shared liveSessions store.
 */
function attachWebSocket(httpServer) {
```

```
let WebSocketServer;
try {
  ({ WebSocketServer } = require('ws'));
} catch {
  try {
    const ws = require('ws');
    WebSocketServer = ws.Server || ws;
  } catch {
    logger.warn('ws package not found — WebSocket disabled. Run: npm install ws');
    return;
  }
}

const wss = new WebSocketServer({ server: httpServer, path: '/ws' });
logger.info('WebSocket server attached at /ws');

wss.on('connection', (ws) => {
  let sessionId = null;
  let role      = 'viewer';

  ws.on('message', (raw) => {
    let msg;
    try { msg = JSON.parse(raw); } catch { return; }

    if (msg.type === 'join_session') {
      sessionId = msg.sessionId;
      role      = msg.name === 'HOST' ? 'host' : 'viewer';
      registerClient(ws, sessionId, role);
      return;
    }

    if (msg.type === 'ping') {
      ws.send(JSON.stringify({ type: 'pong' }));
      return;
    }

    if (sessionId && ['chat', 'reaction', 'tip', 'tip_alert'].includes(msg.type)) {
      if (msg.type === 'tip' && msg.amount) {
        sessionStore.addTip(sessionId, msg.amount);
      }
      broadcast(sessionId, msg);
    }
  });

  ws.on('close', () => {
    if (!sessionId) return;
    unregisterClient(ws, sessionId, role);
```

```
    });

    ws.on('error', () => ws.close());
  });
}

// ── Client registry (session → Set of ws connections) ──────────────────────
const wsSessionClients = new Map();

function registerClient(ws, sessionId, role) {
  if (!wsSessionClients.has(sessionId)) wsSessionClients.set(sessionId, new Set());
  wsSessionClients.get(sessionId).add(ws);

  if (role === 'viewer') {
    sessionStore.incrementViewer(sessionId);
    broadcast(sessionId, {
      type:        'viewer_count',
      viewerCount: sessionStore.getSession(sessionId)?.viewerCount || 0,
    });
  }
}

function unregisterClient(ws, sessionId, role) {
  const clients = wsSessionClients.get(sessionId);
  if (clients) {
    clients.delete(ws);
    if (!clients.size) wsSessionClients.delete(sessionId);
  }

  if (role === 'viewer') {
    sessionStore.decrementViewer(sessionId);
    broadcast(sessionId, {
      type:        'viewer_count',
      viewerCount: sessionStore.getSession(sessionId)?.viewerCount || 0,
    });
  }
}

function broadcast(sessionId, msg) {
  const clients = wsSessionClients.get(sessionId);
  if (!clients) return;
  const text = JSON.stringify(msg);
  clients.forEach((ws) => {
    if (ws.readyState === 1 /* OPEN */) ws.send(text);
  });
}
```

```
module.exports = { attachWebSocket };
```

BACKEND — UTILITIES & HELPERS

`constants.js` (3.3 KB)

```javascript
'use strict';

// ── Platform fee rates ──────────────────────────────────────────────────
const FEES = {
  PLATFORM_ROYALTY_STANDARD:  0.05,    // 5%   — standard creator royalty fee
  PLATFORM_ROYALTY_NFT:       0.015,   // 1.5% — Platform NFT holder royalty fee
  PLATFORM_TIP:               0.03,    // 3%   — platform cut of all tips
  PLATFORM_NFT_SALE:          0.025,   // 2.5% — platform cut of all NFT sales
  PLATFORM_NFT_PRICE_USD:     10000,   // $10,000 USD — Platform NFT price floor
};

// ── Subscription plan definitions ──────────────────────────────────────
const SUBSCRIPTION_PLANS = {
  listener_tier1_monthly: { price_usd: 10.99,   days: 30,  type: 'listener',     tier: 1 },
  listener_tier1_annual:  { price_usd: 131.88,  days: 365, type: 'listener',     tier: 1 },
  listener_tier1_rolling: { price_usd: 10.99,   days: 3,   type: 'listener',     tier: 1 },
  listener_tier2_monthly: { price_usd: 19.99,   days: 30,  type: 'listener',     tier: 2 },
  listener_tier2_annual:  { price_usd: 239.88,  days: 365, type: 'listener',     tier: 2 },
  listener_tier2_rolling: { price_usd: 19.99,   days: 3,   type: 'listener',     tier: 2 },
  listener_tier3_monthly: { price_usd: 34.99,   days: 30,  type: 'listener',     tier: 3 },
  listener_tier3_annual:  { price_usd: 419.88,  days: 365, type: 'listener',     tier: 3 },
  listener_tier3_rolling: { price_usd: 34.99,   days: 3,   type: 'listener',     tier: 3 },
  creator_monthly:        { price_usd: 29.99,   days: 30,  type: 'creator',      tier: null },
  creator_annual:         { price_usd: 299.88,  days: 365, type: 'creator',      tier: null },
  nft_creator_monthly:    { price_usd: 14.99,   days: 30,  type: 'nft_creator',  tier: null },
  nft_creator_annual:     { price_usd: 179.88,  days: 365, type: 'nft_creator',  tier: null },
};

// ── Stream duration caps (milliseconds) ────────────────────────────────
const STREAM_DURATION_LIMITS = {
  creator:               3 * 60 * 60 * 1000,  // 3 hours
  platform_nft_creator:  Infinity,             // unlimited
};

// ── Valid content types ─────────────────────────────────────────────────
const VALID_CONTENT_TYPES = ['music', 'podcast', 'video', 'art_still', 'art_animated'];

// ── Capability levels that can DJ ──────────────────────────────────────
```

```javascript
const DJ_CAPABLE_LEVELS = new Set([
  'listener_2', 'listener_3', 'creator_active', 'nft_creator_active',
]);

// ── Capability levels that can create playlists ──────────────────────────
const PLAYLIST_CAPABLE_LEVELS = new Set([
  'listener_2', 'listener_3', 'creator_active', 'nft_creator_active',
]);

module.exports = {
  FEES,
  SUBSCRIPTION_PLANS,
  STREAM_DURATION_LIMITS,
  VALID_CONTENT_TYPES,
  DJ_CAPABLE_LEVELS,
  PLAYLIST_CAPABLE_LEVELS,
};
```

`logger.js` (0.3 KB)

```javascript
'use strict';

const path = require('path');
const fs   = require('fs-extra');
const pino = require('pino');

const logsDir = path.resolve(process.cwd(), 'logs');
fs.ensureDirSync(logsDir);

const logger = pino(
  { level: process.env.LOG_LEVEL || 'info' },
  pino.destination(path.join(logsDir, 'metrics.log'))
);

module.exports = logger;
```

`utility.js` (1.8 KB)

```javascript
'use strict';

const express        = require('express');
const { FEES, SUBSCRIPTION_PLANS } = require('../config/constants');
const profileService = require('../services/profileService');
```

```
const router = express.Router();

// GET /api/fees
router.get('/fees', (req, res) => {
  res.json({
    royalty_standard_pct:   FEES.PLATFORM_ROYALTY_STANDARD * 100,
    royalty_nft_pct:        FEES.PLATFORM_ROYALTY_NFT * 100,
    tip_pct:                FEES.PLATFORM_TIP * 100,
    nft_sale_pct:           FEES.PLATFORM_NFT_SALE * 100,
    platform_nft_price_usd: FEES.PLATFORM_NFT_PRICE_USD,
    subscription_plans:     SUBSCRIPTION_PLANS,
  });
});

// POST /api/convert-currency
router.post('/convert-currency', (req, res) => {
  const { amountEth, to } = req.body || {};
  const rate     = parseFloat(process.env.ETH_USD_RATE || '2500');
  const ethFloat = parseFloat(amountEth || '0');
  const rates    = { usd: rate, btc: rate / 65000, sol: rate / 150, zec: rate / 20 };
  const amount   = ethFloat * (rates[(to || '').toLowerCase()] || 1);
  res.json({ [`amount${(to || '').toUpperCase()}`]: amount });
});

// GET /api/nfts
router.get('/nfts', async (req, res, next) => {
  try {
    const profiles = await profileService.loadProfiles();
    const nfts = [];
    for (const [wallet, profile] of Object.entries(profiles)) {
      if (!profile.nft_contract_address) continue;
      for (const cid of (profile.playlist_cids || [])) {
        nfts.push({
          metadataCid:     cid,
          contractAddress: profile.nft_contract_address,
          artistWallet:    wallet,
          artist:          profile.name || 'Unknown',
          title:           `Track ${cid.slice(0, 8)}`,
          cover_image:     null,
          price_eth:       null,
          tokenId:         null,
        });
      }
    }
    res.json(nfts);
  } catch (err) { next(err); }
```

```
});

module.exports = router;
```

`devBypass.js` (1.1 KB)

```javascript
'use strict';

const logger = require('../config/logger');

// DEV_MODE is true when ETH_RPC or MSP_PRIVATE_KEY are absent.
// Set both in .env to disable DEV_MODE before production.
const DEV_MODE = !process.env.ETH_RPC || !process.env.MSP_PRIVATE_KEY;

// DEV_WALLET bypasses all profile and subscription checks for that address.
// REMOVE BEFORE PRODUCTION — provide ETH_RPC + MSP_PRIVATE_KEY in .env instead.
const DEV_WALLET = DEV_MODE
  ? (process.env.DEV_WALLET || '0xDe1f33Ce3E81bc54ebD0F4BC3Ce0a2F64d7Ef399')
  : null;

if (DEV_MODE) {
  logger.warn('Running in DEV MODE — on-chain calls disabled');
  if (DEV_WALLET) logger.warn({ DEV_WALLET }, 'DEV_WALLET active — permission checks bypassed for this
}

/**
 * Returns true if the given wallet is the DEV_WALLET and DEV_MODE is active.
 * Call at the top of any route that needs an access guard to short-circuit it.
 */
function isDevWallet(wallet) {
  return !!(DEV_WALLET && wallet && wallet.toLowerCase() === DEV_WALLET.toLowerCase());
}

module.exports = { DEV_MODE, DEV_WALLET, isDevWallet };
```

`ownershipGuard.js` (1.1 KB)

```javascript
'use strict';

const catalogService = require('../services/catalogService');
const { isDevWallet } = require('./devBypass');

/**
```

```
 * Express middleware — verifies that req.body.wallet owns the catalog asset
 * identified by req.params.contentId.
 *
 * Usage: router.post('/:contentId/something', ownershipGuard, handler)
 */
async function ownershipGuard(req, res, next) {
  const { contentId } = req.params;
  const { wallet }    = req.body || {};

  if (!wallet) return res.status(400).json({ error: 'Missing wallet' });

  // DEV_WALLET bypasses ownership check
  if (isDevWallet(wallet)) return next();

  try {
    const catalog = await catalogService.loadCatalog();
    const entry   = catalog[contentId];
    if (!entry) return res.status(404).json({ error: 'Asset not found' });
    if ((entry.wallet || '').toLowerCase() !== wallet.toLowerCase()) {
      return res.status(403).json({ error: 'Not your asset' });
    }
    // Attach the entry to the request so handlers don't re-read the catalog
    req.catalogEntry = entry;
    req.catalog      = catalog;
    next();
  } catch (err) {
    next(err);
  }
}

module.exports = ownershipGuard;
```

`walletNormalizer.js` (0.6 KB)

```javascript
'use strict';

/**
 * Normalizes all wallet address fields to lowercase on every request.
 * Ensures MetaMask/Coinbase checksummed addresses always match stored profiles.
 */
function walletNormalizer(req, res, next) {
  if (req.body) {
    const fields = ['wallet', 'listener', 'from_wallet', 'to_wallet'];
    for (const field of fields) {
      if (req.body[field]) req.body[field] = req.body[field].toLowerCase();
```

```
    }
  }
  if (req.params && req.params.wallet) {
    req.params.wallet = req.params.wallet.toLowerCase();
  }
  next();
}

module.exports = walletNormalizer;
```

`module_aws.js` (5.2 KB)

```javascript
'use strict';
/**
 * AWS module (CommonJS)
 * - KMS EIP-712 signing (KMS if configured, else wallet fallback)
 * - AWS SDK v3 clients (DynamoDB, S3, CloudFront)
 * - Helpers: ensureKmsKeyIsSecp256k1, getKmsPublicKey, getKmsEthAddress
 *
 * ENV:
 *  AWS_REGION=us-east-1
 *  KMS_KEY_ID=arn:aws:kms:...       (asymmetric key, ECC_SECG_P256K1)
 */

const { KMSClient, SignCommand, GetPublicKeyCommand, DescribeKeyCommand } = require('@aws-sdk/client-k
const { DynamoDBClient } = require('@aws-sdk/client-dynamodb');
const { DynamoDBDocumentClient } = require('@aws-sdk/lib-dynamodb');
const { S3Client } = require('@aws-sdk/client-s3');
const { CloudFrontClient } = require('@aws-sdk/client-cloudfront');
const { ethers } = require('ethers');

const region = process.env.AWS_REGION || 'us-east-1';
const kms = new KMSClient({ region });
const kmsKeyId = process.env.KMS_KEY_ID || null;

// v3 SDK clients
const dynamoClient = new DynamoDBClient({ region });
const dynamo = DynamoDBDocumentClient.from(dynamoClient);
const s3 = new S3Client({ region });
const cloudfront = new CloudFrontClient({ region });

/**
 * awsKmsSignEIP712(domain, types, value, wallet)
 * If KMS_KEY_ID is set: signs the EIP-712 digest with AWS KMS and returns a DER hex (0x…).
 * If KMS_KEY_ID is missing: falls back to wallet._signTypedData (returns RSV).
```

```
 *
 * NOTE: server.cjs converts DER → RSV automatically when it sees a DER prefix (0x30…).
 */
async function awsKmsSignEIP712(domain, types, value, wallet) {
  if (!kmsKeyId) {
    if (!wallet) throw new Error('No KMS key or wallet provided for signing');
    return wallet._signTypedData(domain, types, value);
  }
  const digest = ethers.utils._TypedDataEncoder.hash(domain, types, value); // 0x…32 bytes
  const params = {
    KeyId: kmsKeyId,
    Message: Buffer.from(digest.slice(2), 'hex'),
    SigningAlgorithm: 'ECDSA_SHA_256',
    MessageType: 'DIGEST',
  };
  const { Signature } = await kms.send(new SignCommand(params));
  return '0x' + Buffer.from(Signature).toString('hex'); // DER hex
}

/**
 * Ensure the configured KMS key is usable for Ethereum (ECC_SECG_P256K1).
 * Throws with a helpful message if not.
 */
async function ensureKmsKeyIsSecp256k1() {
  if (!kmsKeyId) return false;
  const desc = await kms.send(new DescribeKeyCommand({ KeyId: kmsKeyId }));
  const meta = desc?.KeyMetadata;
  if (!meta) throw new Error('KMS key metadata not found');

  if (meta.KeyState !== 'Enabled') {
    throw new Error(`KMS key is not enabled (state: ${meta.KeyState})`);
  }
  if (meta.KeyUsage !== 'SIGN_VERIFY') {
    throw new Error(`KMS key usage must be SIGN_VERIFY (got: ${meta.KeyUsage})`);
  }

  // The definitive check is GetPublicKey (gives KeySpec)
  const pub = await kms.send(new GetPublicKeyCommand({ KeyId: kmsKeyId }));
  if (pub.KeySpec !== 'ECC_SECG_P256K1') {
    throw new Error(`KMS KeySpec must be ECC_SECG_P256K1 for Ethereum (got: ${pub.KeySpec})`);
  }
  return true;
}

/**
 * Fetch the KMS public key (ASN.1 DER) and return it as a Buffer.
 */
```

```
async function getKmsPublicKey() {
  if (!kmsKeyId) throw new Error('KMS_KEY_ID not set');
  const { PublicKey } = await kms.send(new GetPublicKeyCommand({ KeyId: kmsKeyId }));
  return Buffer.from(PublicKey);
}

/**
 * Derive the Ethereum address from the KMS public key.
 * AWS returns an ASN.1 SubjectPublicKeyInfo. We extract the 65-byte uncompressed key, then keccak-256
 */
async function getKmsEthAddress() {
  const spki = await getKmsPublicKey(); // ASN.1 SPKI
  // Very small DER decoder to get uncompressed EC point from SPKI:
  // Look for the BIT STRING (0x03), skip 1 "unused bits" byte, remaining must start with 0x04 (uncomp
  let i = 0;
  if (spki[i++] !== 0x30) throw new Error('SPKI: bad sequence');
  // Skip length (could be short or long form)
  const lenByte = spki[i++];
  const lenLen = (lenByte & 0x80) ? (lenByte & 0x7f) : 0;
  if (lenLen) i += lenLen; // skip long-form length bytes

  // AlgorithmIdentifier sequence
  if (spki[i++] !== 0x30) throw new Error('SPKI: bad alg seq');
  const aLenByte = spki[i++];
  const aLenLen = (aLenByte & 0x80) ? (aLenByte & 0x7f) : 0;
  const aLen = aLenLen ? parseInt(spki.slice(i, i + aLenLen).toString('hex'), 16) : aLenByte;
  i += (aLenLen ? aLenLen : 0) + (aLenLen ? aLen : 0); // move past alg seq (approx; good enough for A

  // SubjectPublicKey BIT STRING
  if (spki[i++] !== 0x03) throw new Error('SPKI: missing BIT STRING');
  let bitLenByte = spki[i++];
  let bitLenLen = 0;
  if (bitLenByte & 0x80) {
    bitLenLen = bitLenByte & 0x7f;
    bitLenByte = parseInt(spki.slice(i, i + bitLenLen).toString('hex'), 16);
    i += bitLenLen;
  }
  const unusedBits = spki[i++]; // should be 0
  if (unusedBits !== 0x00) throw new Error('SPKI: unexpected unused bits');

  // Now we expect uncompressed EC point: 0x04 <32-byte X> <32-byte Y>
  if (spki[i] !== 0x04) throw new Error('SPKI: expected uncompressed EC point (0x04)');
  const uncompressed = spki.slice(i, i + 65);
  if (uncompressed.length !== 65) throw new Error('SPKI: bad EC point length');

  const pubkey = '0x' + uncompressed.toString('hex'); // 0x04 + X + Y
  const addr = ethers.utils.computeAddress(pubkey);
```

```
  return addr;
}

module.exports = {
  kms,
  kmsKeyId,
  awsKmsSignEIP712,
  ensureKmsKeyIsSecp256k1,
  getKmsPublicKey,
  getKmsEthAddress,
  dynamo,
  s3,
  cloudfront,
};
```

`module_aws_shim.js` (0.7 KB)

```javascript
'use strict';

/**
 * Shim so ethService.js can import this without knowing the relative path
 * to the real module_aws.js which sits at the project root.
 *
 * In production: place module_aws.js at src/services/module_aws.js
 * or update this path to wherever module_aws.js lives.
 */
let awsKmsSignEIP712;
try {
  ({ awsKmsSignEIP712 } = require('../../module_aws'));
} catch {
  // Fallback no-op — AWS KMS signing is optional
  awsKmsSignEIP712 = async (domain, types, value, wallet) => {
    if (wallet && wallet._signTypedData) {
      return wallet._signTypedData(domain, types, value);
    }
    return '0x';
  };
}

module.exports = { awsKmsSignEIP712 };
```

`validator.js` (4.0 KB)

```javascript
```

```
// server/validator.js (CommonJS)
'use strict';

const Ajv = require('ajv');

const ajv = new Ajv({ allErrors: true, strict: false });

// 64 hex chars (32-byte digests)
const HEX64 = '^[0-9a-fA-F]{64}$';

const metadataSchema = {
  type: 'object',
  additionalProperties: true,
  required: [
    'id',
    'title',
    'creator',
    'content_type',
    'availability_type',
    'release_date',
    'tags',
    'integrityHashes'
  ],
  properties: {
    id: { type: 'string' },
    title: { type: 'string', minLength: 1 },
    description: { type: 'string' },
    creator: {
      type: 'object',
      additionalProperties: true,
      required: ['name', 'user_id', 'wallet_address'],
      properties: {
        name: { type: 'string', minLength: 1 },
        user_id: { type: 'string', minLength: 1 },
        wallet_address: { type: 'string', minLength: 1 }
      }
    },
    content_type: { type: 'string', enum: ['music', 'art', 'podcast'] },
    availability_type: { type: 'string' },
    release_date: { type: 'string' },
    tags: { type: 'array', minItems: 5, items: { type: 'string', minLength: 1 } },
    mlc_metadata: {
      type: 'object',
      additionalProperties: true,
      properties: {
        work_title: { type: 'string' },
        iswc: { type: 'string' },
```

```
        isrc: { type: 'string' },
        ipi_name_number: { type: 'string' },
        writers: {
          type: 'array',
          items: {
            type: 'object',
            additionalProperties: true,
            required: ['name'],
            properties: {
              name: { type: 'string', minLength: 1 },
              role: { type: 'string' },
              ipi_name_number: { type: 'string' },
              ownership_percent: { type: ['number', 'integer'], minimum: 0, maximum: 100 }
            }
          }
        },
        publishers: { type: 'array' }
      }
    },
    integrityHashes: {
      type: 'object',
      additionalProperties: true,
      // SHA-256 is always required (BLAKE3 is optional)
      required: ['sha256Audio', 'sha256CoverImage'],
      properties: {
        sha256Audio:      { type: 'string', minLength: 64, maxLength: 64, pattern: HEX64 },
        sha256CoverImage: { type: 'string', minLength: 64, maxLength: 64, pattern: HEX64 },
        sha256Metadata:   { type: 'string', minLength: 64, maxLength: 64, pattern: HEX64 },

        // Optional BLAKE3 digests (same 32-byte hex by default)
        blake3Audio:        { type: 'string', minLength: 64, maxLength: 64, pattern: HEX64 },
        blake3CoverImage:   { type: 'string', minLength: 64, maxLength: 64, pattern: HEX64 },
        blake3Metadata:     { type: 'string', minLength: 64, maxLength: 64, pattern: HEX64 },
      }
    },
    files: { type: 'object', additionalProperties: true },
    ipfs_audio_url: { type: 'string' }
  }
};

const validateCompiled = ajv.compile(metadataSchema);

function validateMetadata(metadata, contentType) {
  validateMetadata.errors = null;
  const ok = validateCompiled(metadata);
  let errors = validateCompiled.errors ? [...validateCompiled.errors] : [];
```

```
  if (contentType && metadata && typeof metadata === 'object') {
    if (metadata.content_type && metadata.content_type !== contentType) {
      errors.push({
        instancePath: '/content_type',
        schemaPath: '#/properties/content_type/enum',
        keyword: 'enum',
        params: { allowedValues: ['music', 'art', 'podcast'] },
        message: `content_type must match the provided contentType argument (${contentType})`
      });
    }
  }

  if (!ok || errors.length) {
    validateMetadata.errors = errors;
    return false;
  }
  return true;
}

function validateOwnership(metadata) {
  const writers = metadata?.mlc_metadata?.writers;
  if (Array.isArray(writers) && writers.length) {
    const total = writers.reduce((sum, w) => sum + (Number(w.ownership_percent) || 0), 0);
    const rounded = Math.round(total * 1000) / 1000;
    if (Math.abs(rounded - 100) > 0.001) {
      throw new Error(`Ownership percentages must sum to 100 (got ${rounded}).`);
    }
  }
}

module.exports = { validateMetadata, validateOwnership };
```

FRONTEND — HTML PAGES

`index.html` (15.5 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- Favicon -->
  <link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
  <link rel="apple-touch-icon" href="assets/msp-vinyl-180.png">
```

```
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#F5F000">
  <title>Michie Stream Platform</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>
/* ── MSP Neon Banner ──────────────────────────────────────── */
.neon-banner {
  position: relative;
  z-index: 90;
  display: flex;
  align-items: center;
  width: 100%;
  padding: 14px 8px;
  background: rgba(8,8,10,0.6);
  border-bottom: 1px solid rgba(212,168,83,0.15);
  overflow: hidden;
}

.neon-banner .b-side {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 7px;
  flex-shrink: 0;
  width: 80px;
}

.neon-banner .b-slogan {
  flex: 1;
  text-align: center;
}

.neon-banner .b-line {
  font-family: 'Syne', 'Arial Black', sans-serif;
  font-weight: 800;
  font-size: clamp(13px, 2vw, 22px);
  letter-spacing: .04em;
  line-height: 1.3;
  color: #F5F000;
  animation:
    neon-boot    2.4s ease-out forwards,
    neon-flicker 5s  ease-in-out infinite 2.4s;
}

.neon-banner .b-line:last-child {
  animation:
```

```
    neon-boot    2.4s ease-out .15s forwards,
    neon-flicker 5s  ease-in-out infinite 2.55s;
}

.neon-banner svg.b-ic {
  opacity: 0;
  animation: b-icon-in 0.4s ease-out forwards, b-fl 3s ease-in-out infinite;
}

@keyframes b-icon-in { 0%{opacity:0;} 100%{opacity:.85;} }
@keyframes b-fl { 0%,100%{transform:translateY(0) scale(1);opacity:.85;} 50%{transform:translateY(-4px

@keyframes neon-boot {
  0%   {opacity:0;   text-shadow:none;}
  8%   {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8);}
  10%  {opacity:.1;  text-shadow:none;}
  14%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 20px #F5F000,0 0 40px rgba(245,240,0,.9);}
  16%  {opacity:.3;  text-shadow:none;}
  20%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000;}
  22%  {opacity:.05; text-shadow:none;}
  28%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
  30%  {opacity:.6;  text-shadow:none;}
  36%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
  100% {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
}

@keyframes neon-flicker {
  0%,18%,20%,22%,24%,53%,55%,100% {
    text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
    opacity:1;
  }
  19%,23%,54%{text-shadow:none;opacity:.82;}
}
</style>
</head>
<body style="padding-top:70px;">

  <!-- Navigation Bar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
    <div class="container-fluid">
      <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNa
        aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav me-auto">
```

```
        <li class="nav-item">
          <a class="nav-link active" aria-current="page" href="index.html">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="listen.html">Just Listen</a>
        </li>
        <li class="nav-item" id="nav-dashboard" style="display:none;">
          <a  class="nav-link" href="dashboard.html">Dashboard</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="creators.html">Creators Corner</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="marketplace.html">NFT Marketplace</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="profile.html">Profile</a>
        </li>
        <li class="nav-item" data-requires="hostConcert" style="display:none;">
          <a class="nav-link" href="live_studio.html">🔴 Live Studio</a>
        </li>
      </ul>
      <div class="d-flex align-items-center">
        <button class="btn btn-primary" data-connect-wallet>Connect Wallet</button>
        <button id="btn-disconnect" class="btn btn-outline-light ms-2 d-none">Disconnect</button>
        <span id="walletAddress" class="ms-2 small text-light"></span>
      </div>
    </div>
  </div>
</nav>
<!-- ═══ MSP NEON BANNER ═══════════════════════════════ -->
<div class="neon-banner">

  <!-- Left icons -->
  <div class="b-side">
    <svg class="b-ic" style="animation-delay:.1s,0s;width:24px;height:20px;filter:drop-shadow(0 0 5p
      <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
      <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
      <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
    </svg>
    <svg class="b-ic" style="animation-delay:.4s,.35s;width:36px;height:24px;filter:drop-shadow(0 0
      <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
      <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
      <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
      <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
```

```
  </svg>
  <svg class="b-ic" style="animation-delay:.7s,.7s;width:26px;height:18px;filter:drop-shadow(0 0 5
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.05s,1.05s;width:24px;height:20px;filter:drop-shadow(0
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.4s,1.4s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.75s,1.75s;width:38px;height:26px;filter:drop-shadow(0
    <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
    <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
    <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
    <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
    <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:2.1s,2.1s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
  </svg>
</div>

<!-- Slogan -->
<div class="b-slogan">
  <div class="b-line">Put the Needle on the Record</div>
  <div class="b-line">That is on a Blockchain</div>
</div>

<!-- Right icons (staggered offsets) -->
<div class="b-side">
  <svg class="b-ic" style="animation-delay:.2s,.2s;width:24px;height:20px;filter:drop-shadow(0 0 5
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
  </svg>
```

```
    <svg class="b-ic" style="animation-delay:.55s,.55s;width:36px;height:24px;filter:drop-shadow(0 0
      <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
      <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
      <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
      <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
    </svg>
    <svg class="b-ic" style="animation-delay:.9s,.9s;width:26px;height:18px;filter:drop-shadow(0 0 5
      <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
      <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
      <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
    </svg>
    <svg class="b-ic" style="animation-delay:1.25s,1.25s;width:24px;height:20px;filter:drop-shadow(0
      <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
      <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
      <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
    </svg>
    <svg class="b-ic" style="animation-delay:1.6s,1.6s;width:26px;height:18px;filter:drop-shadow(0 0
      <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
      <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
      <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
    </svg>
    <svg class="b-ic" style="animation-delay:1.95s,1.95s;width:38px;height:26px;filter:drop-shadow(0
      <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
      <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
      <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
      <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
      <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
    </svg>
    <svg class="b-ic" style="animation-delay:2.3s,2.3s;width:26px;height:18px;filter:drop-shadow(0 0
      <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
      <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
      <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
    </svg>
  </div>

</div>
<!-- ═══ END NEON BANNER ═══════════════════════════════ -->


<!-- Page Content -->
<div class="container py-5 mt-5">
  <section id="home" class="banner mb-5">

    <p>Welcome to Michie Stream Platform</p>
    <p>Explore music, podcasts, art, and short films on the blockchain. Listen to live DJ sets, crea
```

```
    <a href="listen.html" class="btn btn-primary">Start Listening</a>
    <a href="creators.html" class="btn btn-secondary">Create Content</a>
  </section>
</div>

<!-- Wallet Connect Modal -->
<div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidde
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content bg-dark text-light">
      <div class="modal-header border-0">
        <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="
      </div>
      <div class="modal-body pt-0">
        <div class="list-group">
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://cdn.jsdelivr.net/gh/MetaMask/brand-resources/SVG/metamask-fox.svg" alt
            <span>MetaMask (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/1885080?s=200&v=4" alt="" width="28" h
            <span>Coinbase Wallet (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/78782331?s=200&v=4" alt="" width="28"
            <span>Phantom (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/64856450?s=200&v=4" alt="" width="28"
            <span>Solflare (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/21111808?s=200&v=4" alt="" width="28"
            <span>Zcash Wallet / Hardware</span>
          </button>
        </div>
        <div class="small text-secondary mt-3" id="wallet-help"></div>
      </div>
    </div>
  </div>
</div>

<!-- Vendor scripts -->
<script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
<script src="vendor/ethers/ethers.umd.min.js"></script>
<script src="vendor/hls/hls.min.js"></script>
<script src="vendor/ipfs/index.min.js"></script>
```

```
  <!-- App scripts -->
  <script src="Scripts/wallets.js"></script>
  <script src="Scripts/common.js"></script>
  <script src="Scripts/main.js"></script>

</body>
</html>
```

`listen.html` (47.8 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Just Listen — Michie Stream Platform</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>

/* ──────────────────────────────────────────────────────────────────────────
   LISTEN PAGE — all local styles
   Depends on: styles/styles.css for CSS custom properties
────────────────────────────────────────────────────────────────────────────── */

body { padding-bottom: 90px; }

/* ── Off-screen stubs — common.js needs these IDs, they must never be visible.
   1×1px clipped, pointer-events:none, so nothing can interact with them.    */
#play-btn, #pause-btn, #stop-btn, #duration-display {
  position: absolute !important;
  width: 1px !important; height: 1px !important;
  overflow: hidden !important; clip: rect(0,0,0,0) !important;
  white-space: nowrap !important; border: 0 !important;
  pointer-events: none !important; opacity: 0 !important;
}

/* ═══ CONTENT TYPE TABS ═══════════════════════════════════════════════════ */
.ct-tabs {
  border-bottom: 1px solid var(--border-subtle);
  display: flex; gap: 4px; margin-bottom: 20px;
}
.ct-tab {
  background: none; border: none; border-bottom: 2px solid transparent;
  color: var(--text-secondary); cursor: pointer;
```

```
  font-family: var(--font-ui); font-size: 13px; font-weight: 700;
  letter-spacing: .06em; padding: 10px 18px; text-transform: uppercase;
  transition: color .15s, border-color .15s; white-space: nowrap;
}
.ct-tab:hover                  { color: var(--text-primary); }
.ct-tab.active                 { color: var(--teal);   border-bottom-color: var(--teal); }
.ct-tab.podcast-tab.active     { color: var(--violet); border-bottom-color: var(--violet); }
.ct-tab.live-tab.active        { color: var(--ember);  border-bottom-color: var(--ember); }

/* == SUB-FILTER PILLS ============================================================ */
.filter-pills { display: flex; flex-wrap: wrap; gap: 6px; margin-bottom: 16px; }
.filter-pill {
  background: var(--bg-raised); border: 1px solid var(--border-subtle);
  border-radius: 20px; color: var(--text-secondary); cursor: pointer;
  font-size: 12px; font-weight: 600; letter-spacing: .04em;
  padding: 4px 14px; text-transform: uppercase; transition: all .15s;
}
.filter-pill:hover              { border-color: var(--border-mid); color: var(--text-primary); }
.filter-pill.active             { background: var(--bg-hover); border-color: var(--teal); color: var(-
.filter-pill.fav-pill.active    { border-color: var(--ember); color: var(--ember); }

/* == VIEW TOGGLE ================================================================= */
.view-toggle {
  background: var(--bg-raised); border: 1px solid var(--border-subtle);
  border-radius: 6px; display: flex; gap: 2px; padding: 2px;
}
.view-btn {
  background: none; border: none; border-radius: 4px;
  color: var(--text-secondary); cursor: pointer; font-size: 14px;
  padding: 4px 9px; transition: all .15s;
}
.view-btn.active { background: var(--bg-hover); color: var(--teal); }

/* == MUSIC LIST ROWS ============================================================= */
.track-list { display: flex; flex-direction: column; }
.track-row {
  align-items: center; border-bottom: 1px solid var(--border-subtle);
  display: grid; gap: 0 8px;
  /* num | cover | info | duration | heart | three-dot */
  grid-template-columns: 28px 44px 1fr auto auto auto;
  padding: 8px 4px; position: relative; transition: background .12s;
}
.track-row:hover      { background: var(--bg-raised); border-radius: 6px; }
.track-row:last-child { border-bottom: none; }

.track-num {
  color: var(--text-muted); font-family: var(--font-mono);
```

```
  font-size: 11px; text-align: center;
}
.track-row:hover .track-num { display: none; }

.track-play-inline {
  align-items: center; background: none; border: none;
  color: var(--teal); cursor: pointer; display: none;
  font-size: 16px; justify-content: center; padding: 0; width: 28px;
}
.track-row:hover .track-play-inline { display: flex; }

.track-cover {
  border-radius: 4px; flex-shrink: 0; height: 44px; object-fit: cover; width: 44px;
}
.track-cover-placeholder {
  align-items: center; background: var(--bg-raised); border-radius: 4px;
  color: var(--text-muted); display: flex; flex-shrink: 0;
  font-size: 20px; height: 44px; justify-content: center; width: 44px;
}
.track-info { min-width: 0; }
.track-title {
  color: var(--text-primary); font-size: 13px; font-weight: 600;
  overflow: hidden; text-overflow: ellipsis; white-space: nowrap;
}
.track-row.is-playing .track-title { color: var(--teal); }
.track-artist {
  color: var(--text-secondary); font-size: 12px;
  overflow: hidden; text-overflow: ellipsis; white-space: nowrap;
}
.track-tags { display: none; }
@media (min-width: 768px) {
  .track-row  { grid-template-columns: 28px 44px 1fr 160px auto auto auto; }
  .track-tags { display: flex; gap: 4px; flex-wrap: nowrap; max-width: 160px; overflow: hidden; }
  .track-tag  {
    background: var(--bg-raised); border-radius: 3px;
    color: var(--text-muted); font-size: 10px; padding: 1px 6px; white-space: nowrap;
  }
}
.track-duration { color: var(--text-muted); font-family: var(--font-mono); font-size: 12px; white-spac

/* ── Vinyl badge — TOP-right corner of thumbnail ──────────────────────────── */
.track-cover-wrap {
  flex-shrink: 0;
  height: 44px;
  position: relative;
  width: 44px;
}
```

```
.track-cover-wrap .track-cover,
.track-cover-wrap .track-cover-placeholder { height: 44px; width: 44px; }
.vinyl-cover-badge {
  filter: drop-shadow(0 1px 3px rgba(0,0,0,.9));
  position: absolute;
  right: -4px;
  top: -4px;
}
.vinyl-inline-badge { opacity: .85; }

/* ── Tile — cover wrap fills full width ──────────────────────────────── */
.track-tile .track-cover-wrap {
  aspect-ratio: 1;
  display: block;
  height: auto;
  position: relative;
  width: 100%;
}
.track-tile .track-cover-wrap img,
.track-tile .track-cover-wrap .track-tile-placeholder {
  aspect-ratio: 1;
  display: block;
  height: auto;
  object-fit: cover;
  width: 100%;
}
.track-tile.is-playing { border-color: var(--teal) !important; }
.track-tile.is-playing .track-tile-title { color: var(--teal); }

/* ── Tile footer — heart · duration · three-dot ──────────────────────── */
.track-tile-footer {
  align-items: center;
  border-top: 1px solid var(--border-subtle);
  display: flex;
  gap: 2px;
  padding: 4px 6px;
}
.track-tile-footer .fav-heart-btn { font-size: 13px; padding: 3px 5px; }
.track-tile-footer .track-duration { flex: 1; font-size: 10px; text-align: center; }

/* ── Three-dot options button ────────────────────────────────────────── */
.track-options-btn {
  background: none;
  border: none;
  color: var(--text-muted);
  cursor: pointer;
  flex-shrink: 0;
```

```
  font-size: 18px;
  font-weight: 700;
  line-height: 1;
  padding: 2px 6px;
  transition: color .12s;
}
.track-options-btn:hover { color: var(--text-primary); }

/* ── Floating options menu (appended to <body>) ──────────────────────────── */
#track-options-menu {
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 10px;
  box-shadow: 0 12px 40px rgba(0,0,0,.8);
  display: none;
  min-width: 210px;
  overflow: hidden;
  position: fixed;
  z-index: 3000;
}
#track-options-menu.open { display: block; }
.opt-section { padding: 4px 0; }
.opt-divider { border: none; border-top: 1px solid var(--border-subtle); margin: 2px 0; }
.opt-item {
  align-items: center;
  background: none;
  border: none;
  color: var(--text-secondary);
  cursor: pointer;
  display: flex;
  font-family: var(--font-ui);
  font-size: 13px;
  gap: 10px;
  padding: 10px 16px;
  text-align: left;
  transition: background .1s, color .1s;
  width: 100%;
}
.opt-item:hover { background: var(--bg-hover); color: var(--text-primary); }
.opt-item.fav-active-track  { color: var(--ember); }
.opt-item.fav-active-artist { color: var(--gold); }
.opt-item.fav-active-album  { color: var(--violet); }
.opt-item.opt-disabled      { color: var(--text-muted); cursor: default; }
.opt-item.opt-disabled:hover { background: none; color: var(--text-muted); }
.opt-icon { flex-shrink: 0; font-style: normal; text-align: center; width: 18px; }
.opt-tier-badge {
  background: rgba(212,168,83,.15);
```

```
  border-radius: 4px;
  color: var(--gold);
  font-size: 9px;
  font-weight: 700;
  letter-spacing: .06em;
  margin-left: auto;
  padding: 1px 5px;
  text-transform: uppercase;
}
.opt-submenu-head {
  color: var(--text-muted);
  font-size: 10px;
  font-weight: 700;
  letter-spacing: .08em;
  padding: 8px 16px 4px;
  text-transform: uppercase;
}
.opt-pl-item {
  align-items: center;
  background: none;
  border: none;
  color: var(--text-secondary);
  cursor: pointer;
  display: flex;
  font-size: 12px;
  gap: 8px;
  padding: 8px 16px 8px 40px;
  text-align: left;
  transition: background .1s;
  width: 100%;
}
.opt-pl-item:hover { background: var(--bg-hover); color: var(--text-primary); }
.opt-pl-empty { color: var(--text-muted); font-size: 12px; padding: 8px 16px 8px 40px; }

/* == FAVORITES HEART + POPOVER ================================================ */
.fav-heart-btn {
  background: none; border: none; color: var(--text-muted); cursor: pointer;
  font-size: 16px; line-height: 1; padding: 4px 6px; position: relative;
  transition: color .15s, transform .15s;
}
.fav-heart-btn:hover      { color: var(--ember); transform: scale(1.2); }
.fav-heart-btn.fav-track  { color: var(--ember); }
.fav-heart-btn.fav-artist { color: var(--gold); }
.fav-heart-btn.fav-album  { color: var(--violet); }

.fav-popover {
  background: var(--bg-raised); border: 1px solid var(--border-mid);
```

```
  border-radius: 8px; bottom: calc(100% + 8px); box-shadow: 0 8px 32px rgba(0,0,0,.6);
  display: none; flex-direction: column; min-width: 150px; overflow: hidden;
  position: absolute; right: 0; z-index: 300;
}
.fav-popover.open { display: flex; }
.fav-pop-item {
  align-items: center; background: none; border: none; color: var(--text-secondary);
  cursor: pointer; display: flex; font-size: 12px; font-weight: 600; gap: 8px;
  letter-spacing: .04em; padding: 9px 14px; text-align: left;
  text-transform: uppercase; transition: background .1s, color .1s; width: 100%;
}
.fav-pop-item:hover                          { background: var(--bg-hover); color: var(--text-primary)
.fav-pop-item.active                         { color: var(--ember); }
.fav-pop-item[data-fav-type="artist"].active { color: var(--gold); }
.fav-pop-item[data-fav-type="album"].active  { color: var(--violet); }
.fav-pop-dot { border-radius: 50%; flex-shrink: 0; height: 6px; width: 6px; }
.fav-pop-item[data-fav-type="track"]  .fav-pop-dot { background: var(--ember); }
.fav-pop-item[data-fav-type="artist"] .fav-pop-dot { background: var(--gold); }
.fav-pop-item[data-fav-type="album"]  .fav-pop-dot { background: var(--violet); }

/* == GRID TILES ============================================================ */
.track-grid { display: grid; gap: 16px; grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));
.track-tile {
  background: var(--bg-surface); border: 1px solid var(--border-subtle);
  border-radius: 8px; cursor: pointer; overflow: hidden; position: relative;
  transition: border-color .15s, transform .15s;
}
.track-tile:hover { border-color: var(--border-mid); transform: translateY(-2px); }
.track-tile img   { aspect-ratio: 1; display: block; object-fit: cover; width: 100%; }
.track-tile-placeholder {
  align-items: center; aspect-ratio: 1; background: var(--bg-raised);
  color: var(--text-muted); display: flex; font-size: 36px; justify-content: center; width: 100%;
}
.track-tile-info   { padding: 10px 10px 8px; }
.track-tile-title  { color: var(--text-primary); font-size: 12px; font-weight: 600; overflow: hidden;
.track-tile-artist { color: var(--text-secondary); font-size: 11px; overflow: hidden; text-overflow: e
.track-tile-play {
  align-items: center; background: rgba(245,240,0,.9); border: none; border-radius: 50%;
  bottom: 52px; color: #08080A; cursor: pointer; display: flex; font-size: 14px;
  height: 34px; justify-content: center; opacity: 0; position: absolute;
  right: 8px; transition: opacity .15s; width: 34px;
}
.track-tile:hover .track-tile-play { opacity: 1; }

/* == PODCAST TILES ========================================================= */
.podcast-grid { display: grid; gap: 20px; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr))
.podcast-tile { background: var(--bg-surface); border: 1px solid var(--border-subtle); border-radius:
```

```
.podcast-tile:hover   { border-color: var(--violet); }
.podcast-tile img     { aspect-ratio: 1; display: block; object-fit: cover; width: 100%; }
.podcast-tile-info    { padding: 12px; }
.podcast-tile-title   { color: var(--text-primary); font-size: 13px; font-weight: 700; margin-bottom:
.podcast-tile-meta    { color: var(--text-secondary); font-size: 11px; }
.podcast-tile-play {
  background: var(--violet); border: none; border-radius: 6px; color: #fff;
  cursor: pointer; font-size: 12px; font-weight: 700; margin-top: 10px; padding: 6px 12px; width: 100%
}

/* == LIVE BANNERS ============================================================ */
.live-grid { display: flex; flex-direction: column; gap: 12px; }
.live-banner {
  align-items: center;
  background: linear-gradient(135deg, var(--bg-raised) 0%, var(--ember-dim) 100%);
  border: 1px solid var(--ember); border-radius: 10px; display: flex;
  gap: 16px; overflow: hidden; padding: 16px; transition: border-color .15s;
}
.live-banner:hover { border-color: var(--gold); }
.live-dot {
  animation: pulse-live 1.2s ease-in-out infinite; background: var(--ember);
  border-radius: 50%; flex-shrink: 0; height: 10px; width: 10px;
}
@keyframes pulse-live { 0%,100% { opacity:1; transform:scale(1); } 50% { opacity:.4; transform:scale(1
.live-info    { flex: 1; min-width: 0; }
.live-title   { color: var(--text-primary); font-size: 15px; font-weight: 700; }
.live-artist  { color: var(--ember); font-size: 12px; font-weight: 600; letter-spacing: .05em; text-tr
.live-viewers { color: var(--text-secondary); font-size: 11px; margin-top: 2px; }
.live-watch-btn {
  background: var(--ember); border: none; border-radius: 6px; color: #fff;
  cursor: pointer; flex-shrink: 0; font-size: 12px; font-weight: 700; padding: 8px 16px;
}

/* == FAVORITES PANEL ========================================================= */
.fav-section-header { align-items: center; display: flex; gap: 12px; margin-bottom: 12px; }
.fav-count-badge {
  background: var(--ember-dim); border-radius: 10px; color: var(--ember);
  font-size: 11px; font-weight: 700; padding: 1px 8px;
}

/* == FIXED BOTTOM PLAYER BAR ================================================= */
#msp-player-bar {
  align-items: center; background: var(--bg-raised);
  border-top: 1px solid var(--border-mid); bottom: 0; display: grid;
  gap: 0 16px; grid-template-columns: 220px 1fr 110px;
  height: 80px; left: 0; padding: 0 20px; position: fixed; right: 0; z-index: 1000;
}
```

```
@media (max-width: 600px) {
  #msp-player-bar { grid-template-columns: 160px 1fr; height: 72px; }
  .player-vol-col { display: none; }
}
.player-track-col { align-items: center; display: flex; gap: 12px; min-width: 0; overflow: hidden; }
#vinyl-icon { border-radius: 4px; flex-shrink: 0; height: 48px; object-fit: cover; width: 48px; }
body.audio-playing     #vinyl-icon { animation: spin 4s linear infinite; border-radius: 50%; }
body:not(.audio-playing) #vinyl-icon { animation: none; border-radius: 4px; }
@keyframes spin { to { transform: rotate(360deg); } }
.player-track-text { min-width: 0; }
#track-name {
  color: var(--text-primary); font-size: 13px; font-weight: 600;
  overflow: hidden; text-overflow: ellipsis; white-space: nowrap;
}
#player-artist-name {
  color: var(--text-secondary); font-size: 11px;
  overflow: hidden; text-overflow: ellipsis; white-space: nowrap;
}
.player-center-col { display: flex; flex-direction: column; gap: 6px; min-width: 0; }
.player-controls-row { align-items: center; display: flex; gap: 14px; justify-content: center; }
.player-ctrl-btn {
  align-items: center; background: none; border: none; color: var(--text-secondary);
  cursor: pointer; display: flex; justify-content: center; padding: 4px; transition: color .12s;
}
.player-ctrl-btn:hover { color: var(--text-primary); }

/* The real play/pause toggle — neon yellow, owned entirely by this page */
#msp-play-toggle {
  align-items: center; background: #F5F000; border: none; border-radius: 50%;
  box-shadow: 0 0 10px rgba(245,240,0,.45), 0 0 22px rgba(245,240,0,.2);
  cursor: pointer; display: flex; flex-shrink: 0; height: 40px;
  justify-content: center; padding: 0;
  transition: background .15s, box-shadow .15s, transform .1s; width: 40px;
}
#msp-play-toggle:hover {
  box-shadow: 0 0 14px rgba(245,240,0,.8), 0 0 36px rgba(245,240,0,.4);
  transform: scale(1.07);
}
#msp-play-toggle.is-playing {
  background: #08080A; border: 2px solid #F5F000;
  box-shadow: 0 0 10px rgba(245,240,0,.3), 0 0 22px rgba(245,240,0,.15);
}
#msp-play-toggle.is-playing:hover {
  background: #111108;
  box-shadow: 0 0 14px rgba(245,240,0,.6), 0 0 30px rgba(245,240,0,.3);
}
```

```
.player-progress-row { align-items: center; display: flex; gap: 8px; }
#time-display {
  color: var(--text-muted); flex-shrink: 0; font-family: var(--font-mono);
  font-size: 10px; min-width: 34px; text-align: right;
}
#progress-bar {
  --val: 0%;
  -webkit-appearance: none; appearance: none;
  background: linear-gradient(to right, var(--teal) var(--val), var(--border-mid) var(--val));
  border-radius: 3px; cursor: pointer; flex: 1; height: 3px; outline: none;
}
#progress-bar::-webkit-slider-thumb {
  -webkit-appearance: none; background: var(--teal-bright);
  border-radius: 50%; cursor: pointer; height: 12px; width: 12px;
}
.player-vol-col { align-items: center; display: flex; gap: 8px; }
.vol-icon       { color: var(--text-muted); font-size: 13px; }
#volume-bar {
  --val: 100%;
  -webkit-appearance: none; appearance: none;
  background: linear-gradient(to right, var(--text-secondary) var(--val), var(--border-mid) var(--val)
  border-radius: 3px; cursor: pointer; height: 3px; outline: none; width: 80px;
}
#volume-bar::-webkit-slider-thumb {
  -webkit-appearance: none; background: var(--text-primary);
  border-radius: 50%; cursor: pointer; height: 10px; width: 10px;
}

/* == STATUS BAR + NEON BANNER ================================================ */
.sub-status-bar {
  align-items: center; background: var(--bg-surface); border: 1px solid var(--border-subtle);
  border-radius: 8px; display: flex; flex-wrap: wrap; gap: 12px; margin-bottom: 24px; padding: 10px 16
}
.neon-banner {
  align-items: center; background: rgba(8,8,10,0.6);
  border-bottom: 1px solid rgba(212,168,83,0.15); display: flex;
  overflow: hidden; padding: 14px 8px; position: relative; width: 100%; z-index: 90;
}
.neon-banner .b-side   { align-items: center; display: flex; flex-direction: column; flex-shrink: 0; g
.neon-banner .b-slogan { flex: 1; text-align: center; }
.neon-banner .b-line {
  animation: neon-boot 2.4s ease-out forwards, neon-flicker 5s ease-in-out infinite 2.4s;
  color: #F5F000; font-family: 'Syne','Arial Black',sans-serif;
  font-size: clamp(13px,2vw,22px); font-weight: 800; letter-spacing: .04em; line-height: 1.3;
}
.neon-banner .b-line:last-child { animation: neon-boot 2.4s ease-out .15s forwards, neon-flicker 5s ea
.neon-banner svg.b-ic  { animation: b-icon-in 0.4s ease-out forwards, b-fl 3s ease-in-out infinite; op
```

```
@keyframes b-icon-in { to { opacity:.85; } }
@keyframes b-fl      { 0%,100%{opacity:.85;transform:translateY(0) scale(1);}  50%{opacity:1;transform
@keyframes neon-boot {
  0%{opacity:0;text-shadow:none;}
  8%{opacity:1;text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8);}
  10%{opacity:.1;text-shadow:none;}14%{opacity:1;text-shadow:0 0 4px #F5F000,0 0 20px #F5F000,0 0 40px
  16%{opacity:.3;text-shadow:none;}20%{opacity:1;text-shadow:0 0 4px #F5F000,0 0 12px #F5F000;}
  22%{opacity:.05;text-shadow:none;}28%{opacity:1;text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,
  30%{opacity:.6;text-shadow:none;}36%,100%{opacity:1;text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,24
}
@keyframes neon-flicker {
  0%,18%,20%,22%,24%,53%,55%,100%{opacity:1;text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba
  19%,23%,54%{opacity:.82;text-shadow:none;}
}

/* == EMPTY / LOADING STATES ================================================== */
.catalog-empty { color: var(--text-muted); font-size: 13px; padding: 40px 0; text-align: center; }
.catalog-empty a { color: var(--teal); }
.catalog-loading { align-items: center; color: var(--text-muted); display: flex; font-size: 13px; gap:
.catalog-loading .spinner {
  animation: spin .8s linear infinite; border: 2px solid var(--border-subtle);
  border-radius: 50%; border-top-color: var(--teal); flex-shrink: 0; height: 18px; width: 18px;
}
</style>
</head>
<body class="page-listen">

<!-- === NAVBAR ============================================================== -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
  <div class="container-fluid">
    <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
      data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle n
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
        <li class="nav-item"><a class="nav-link active" aria-current="page" href="listen.html">Just Li
        <li class="nav-item"><a class="nav-link" href="creators.html">Creators Corner</a></li>
        <li class="nav-item"><a class="nav-link" href="marketplace.html">NFT Marketplace</a></li>
        <li class="nav-item"><a class="nav-link" href="profile.html">Profile</a></li>
        <li class="nav-item" data-requires="hostConcert" style="display:none;">
          <a class="nav-link" href="live_studio.html">🔴 Live Studio</a>
        </li>
      </ul>
      <div class="d-flex align-items-center gap-2">
```

```
        <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
        <button id="btn-disconnect" class="btn btn-outline-light btn-sm d-none">Disconnect</button>
        <span id="walletAddress" class="small text-light wallet-address-display"></span>
        <span class="small text-warning fw-semibold user-name-display"></span>
      </div>
    </div>
  </div>
</nav>

<!-- ═══ NEON BANNER ═══════════════════════════════════════════════════ -->
<div class="neon-banner">
  <div class="b-side">
    <svg class="b-ic" style="animation-delay:.1s,0s;width:24px;height:20px;filter:drop-shadow(0 0 5px
    <svg class="b-ic" style="animation-delay:.4s,.35s;width:36px;height:24px;filter:drop-shadow(0 0 5p
    <svg class="b-ic" style="animation-delay:.7s,.7s;width:26px;height:18px;filter:drop-shadow(0 0 5px
    <svg class="b-ic" style="animation-delay:1.05s,1.05s;width:24px;height:20px;filter:drop-shadow(0 0
    <svg class="b-ic" style="animation-delay:1.4s,1.4s;width:26px;height:18px;filter:drop-shadow(0 0 5
    <svg class="b-ic" style="animation-delay:1.75s,1.75s;width:38px;height:26px;filter:drop-shadow(0 0
    <svg class="b-ic" style="animation-delay:2.1s,2.1s;width:26px;height:18px;filter:drop-shadow(0 0 5
  </div>
  <div class="b-slogan">
    <div class="b-line">Put the Needle on the Record</div>
    <div class="b-line">That is on a Blockchain</div>
  </div>
  <div class="b-side">
    <svg class="b-ic" style="animation-delay:.2s,.2s;width:24px;height:20px;filter:drop-shadow(0 0 5px
    <svg class="b-ic" style="animation-delay:.55s,.55s;width:36px;height:24px;filter:drop-shadow(0 0 5
    <svg class="b-ic" style="animation-delay:.9s,.9s;width:26px;height:18px;filter:drop-shadow(0 0 5px
    <svg class="b-ic" style="animation-delay:1.25s,1.25s;width:24px;height:20px;filter:drop-shadow(0 0
    <svg class="b-ic" style="animation-delay:1.6s,1.6s;width:26px;height:18px;filter:drop-shadow(0 0 5
    <svg class="b-ic" style="animation-delay:1.95s,1.95s;width:38px;height:26px;filter:drop-shadow(0 0
    <svg class="b-ic" style="animation-delay:2.3s,2.3s;width:26px;height:18px;filter:drop-shadow(0 0 5
  </div>
</div>
<!-- ═══ END NEON BANNER ═══════════════════════════════════════════════ -->


<!-- ═══ PAGE CONTENT ══════════════════════════════════════════════════ -->
<div class="container py-4">

  <!-- Subscription status bar -->
  <div class="sub-status-bar">
    <div class="d-flex align-items-center gap-2 flex-grow-1">
      <span id="subscription-status" class="fw-semibold small"></span>
      <span id="subscription-expiry" class="small text-muted"></span>
      <span id="platform-nft-badge" class="badge bg-warning text-dark" style="display:none;"></span>
    </div>
```

```
  <button class="btn btn-outline-secondary btn-sm"
    data-bs-toggle="collapse" data-bs-target="#subscribe-panel" aria-expanded="false">
    Plans &amp; Pricing ↓
  </button>
</div>

<!-- Collapsible subscription plans -->
<div class="collapse mb-4" id="subscribe-panel">
  <section id="subscribe" class="border border-secondary rounded p-4">
    <h5 class="mb-1">Choose Your Plan</h5>
    <p class="text-muted small mb-3">All plans include ad-free streaming. Cancel or change anytime.<
    <div class="btn-group btn-group-sm mb-3" role="group">
      <input type="radio" class="btn-check" name="billing-period" id="bp-monthly" value="monthly" ch
      <label class="btn btn-outline-secondary" for="bp-monthly">Monthly</label>
      <input type="radio" class="btn-check" name="billing-period" id="bp-annual" value="annual">
      <label class="btn btn-outline-secondary" for="bp-annual">Annual <span class="badge bg-success
      <input type="radio" class="btn-check" name="billing-period" id="bp-rolling" value="rolling">
      <label class="btn btn-outline-secondary" for="bp-rolling">3-Day</label>
    </div>
    <div class="row g-3">
      <div class="col-md-4">
        <div class="card h-100 border-secondary bg-dark text-light">
          <div class="card-header bg-secondary text-white text-center py-2"><strong>Tier 1 — Listene
          <div class="card-body d-flex flex-column py-3">
            <p class="fs-4 text-center fw-bold mb-2"><span class="price-t1">$10.99</span><small clas
            <ul class="list-unstyled small flex-grow-1">
              <li>✅ On-demand streaming</li><li>✅ Browse &amp; buy NFTs</li>
              <li>✅ Watch live concerts</li><li>✅ Anonymous tipping</li>
              <li class="text-muted">❌ Playlists</li><li class="text-muted">❌ Activity earnings</
            </ul>
            <button class="btn btn-secondary btn-sm w-100 mt-2"
              data-subscribe-plan="listener_tier1_monthly"
              data-plan-monthly="listener_tier1_monthly" data-plan-annual="listener_tier1_annual" da
              Subscribe — Tier 1</button>
          </div>
        </div>
      </div>
      <div class="col-md-4">
        <div class="card h-100 border-primary bg-dark text-light">
          <div class="card-header bg-primary text-white text-center py-2"><strong>Tier 2 — Active</s
          <div class="card-body d-flex flex-column py-3">
            <p class="fs-4 text-center fw-bold mb-2"><span class="price-t2">$19.99</span><small clas
            <ul class="list-unstyled small flex-grow-1">
              <li>✅ Everything in Tier 1</li><li>✅ Concert chat</li>
              <li>✅ Create playlists</li><li>✅ Host DJ sets</li>
              <li>✅ Activity royalty earning</li><li class="text-muted">❌ Passive earnings</li>
            </ul>
```

```
            <button class="btn btn-primary btn-sm w-100 mt-2"
              data-subscribe-plan="listener_tier2_monthly"
              data-plan-monthly="listener_tier2_monthly" data-plan-annual="listener_tier2_annual" da
              Subscribe — Tier 2</button>
          </div>
        </div>
      </div>
      <div class="col-md-4">
        <div class="card h-100 border-warning bg-dark text-light">
          <div class="card-header bg-warning text-dark text-center py-2"><strong>Tier 3 — Supporter<
          <div class="card-body d-flex flex-column py-3">
            <p class="fs-4 text-center fw-bold mb-2"><span class="price-t3">$34.99</span><small clas
            <ul class="list-unstyled small flex-grow-1">
              <li>✅ Everything in Tier 2</li><li>✅ Passive royalty split</li>
              <li>✅ Offline downloads</li><li>✅ Priority support</li>
            </ul>
            <button class="btn btn-warning btn-sm w-100 mt-2 text-dark"
              data-subscribe-plan="listener_tier3_monthly"
              data-plan-monthly="listener_tier3_monthly" data-plan-annual="listener_tier3_annual" da
              Subscribe — Tier 3</button>
          </div>
        </div>
      </div>
    </div>
    <div class="alert alert-dark mt-3 mb-0 small">
      <strong>Creator or artist?</strong> Plans on <a href="creators.html" class="alert-link">Creato
      Platform NFT holder? <a href="profile.html#claim-nft" class="alert-link">Claim on your profile
    </div>
  </section>
</div>

<!-- Content type tabs -->
<div class="ct-tabs" role="tablist">
  <button class="ct-tab active"      data-ct-tab="music"    role="tab" aria-selected="true">🎵 Music
  <button class="ct-tab podcast-tab" data-ct-tab="podcasts" role="tab" aria-selected="false">🎙 Podc
  <button class="ct-tab live-tab"    data-ct-tab="live"     role="tab" aria-selected="false">🔴 Live
</div>

<!-- MUSIC TAB -->
<div id="tab-music" class="tab-pane">
  <div class="d-flex align-items-center justify-content-between mb-3 gap-2 flex-wrap">
    <div class="filter-pills">
      <button class="filter-pill active"   data-music-filter="all">All</button>
      <button class="filter-pill"          data-music-filter="videos">Videos</button>
      <button class="filter-pill fav-pill" data-music-filter="favorites">♥ Favorites</button>
    </div>
    <div class="view-toggle">
```

```
        <button class="view-btn active" data-view="list" title="List view">≡</button>
        <button class="view-btn"        data-view="grid" title="Grid view">⊞</button>
      </div>
    </div>
    <!-- Catalog renders here. Named msp-catalog so main.js (which targets
         #library-list) never overwrites it. -->
    <div id="msp-catalog">
      <div class="catalog-loading"><div class="spinner"></div>Loading catalog…</div>
    </div>
    <!-- Favorites sub-panel — compact summary + link to full management -->
    <div id="favorites-section" style="display:none;">
      <div class="fav-section-header">
        <h5 class="mb-0" style="color:var(--ember);">♥ Your Favorites</h5>
        <a href="favorites.html" class="btn btn-sm btn-outline-warning ms-auto">Manage All →</a>
      </div>
      <!-- Per-category summary chips rendered by JS -->
      <div id="fav-summary-chips" style="display:flex;flex-wrap:wrap;gap:8px;margin:12px 0;"></div>
      <!-- Playable track quick-list -->
      <div id="favorites-list">
        <p class="text-muted small py-2">No favorites yet. Tap ♥ on any track.</p>
      </div>
    </div>
  </div>

  <!-- PODCASTS TAB -->
  <div id="tab-podcasts" class="tab-pane" style="display:none;">
    <div class="podcast-grid" id="podcast-list">
      <div class="catalog-empty">No podcasts yet. Check back soon.</div>
    </div>
  </div>

  <!-- LIVE TAB -->
  <div id="tab-live" class="tab-pane" style="display:none;">
    <div class="live-grid" id="live-concerts">
      <div class="catalog-empty">No live shows right now.</div>
    </div>
  </div>

</div><!-- /container -->


<!-- ═══ FIXED BOTTOM PLAYER BAR ═══════════════════════════════════════════
  RULE: Each of these IDs appears EXACTLY ONCE in the entire document.
  #audio-player  #play-btn  #pause-btn  #stop-btn  #duration-display
  All hidden off-screen via CSS. The visible player is #msp-play-toggle.
═══════════════════════════════════════════════════════════════════════ -->
<div id="msp-player-bar">
```

```
<!-- Hidden stubs for common.js — do not duplicate, do not remove -->
<audio id="audio-player"    preload="none" style="display:none;"></audio>
<button id="play-btn"       aria-hidden="true" tabindex="-1">▶</button>
<button id="pause-btn"      aria-hidden="true" tabindex="-1">⏸</button>
<button id="stop-btn"       aria-hidden="true" tabindex="-1">■</button>
<span   id="duration-display" aria-hidden="true"></span>

<!-- Left: cover art + track info -->
<div class="player-track-col">
  <img id="vinyl-icon"
    src="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 48 48'%3E%3Ccircl
    alt="Now playing" width="48" height="48">
  <div class="player-track-text">
    <div id="track-name" style="color:var(--text-secondary);font-style:italic;">No track selected</d
    <div id="player-artist-name" style="color:var(--text-muted);font-size:11px;">—</div>
  </div>
</div>

<!-- Centre: skip / play toggle / progress -->
<div class="player-center-col">
  <div class="player-controls-row">

    <button class="player-ctrl-btn" id="btn-skip-back" title="Back 15s">
      <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
        stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round">
        <polyline points="1 4 1 10 7 10"/>
        <path d="M3.51 15a9 9 0 1 0 .49-3.5"/>
        <text x="8" y="16" font-size="5.5" font-family="monospace" fill="currentColor" stroke="none"
      </svg>
    </button>

    <!-- Neon yellow ▶ / inverted ⏸ — wired in inline script, no Bootstrap -->
    <button id="msp-play-toggle" aria-label="Play">
      <svg id="icon-play"  width="16" height="16" viewBox="0 0 16 16" fill="none">
        <polygon points="3,1 15,8 3,15" fill="#08080A"/>
      </svg>
      <svg id="icon-pause" width="16" height="16" viewBox="0 0 16 16" fill="none" style="display:non
        <rect x="2"  y="1" width="4" height="14" rx="1" fill="#F5F000"/>
        <rect x="10" y="1" width="4" height="14" rx="1" fill="#F5F000"/>
      </svg>
    </button>

    <button class="player-ctrl-btn" id="btn-skip-fwd" title="Forward 15s">
      <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
        stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round">
        <polyline points="23 4 23 10 17 10"/>
```

```
          <path d="M20.49 15a9 9 0 1 1-.49-3.5"/>
          <text x="8" y="16" font-size="5.5" font-family="monospace" fill="currentColor" stroke="none"
        </svg>
      </button>

    </div>
    <div class="player-progress-row">
      <span id="time-display">0:00</span>
      <input type="range" id="progress-bar" min="0" max="100" value="0" step="0.1">
    </div>
  </div>

  <!-- Right: volume -->
  <div class="player-vol-col">
    <span class="vol-icon">🔊</span>
    <input type="range" id="volume-bar" min="0" max="1" step="0.01" value="1">
  </div>

</div><!-- /msp-player-bar -->


<!-- ═══ WALLET MODAL ═══════════════════════════════════════════════════ -->
<div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidden=
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content bg-dark text-light">
      <div class="modal-header border-0">
        <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Cl
      </div>
      <div class="modal-body pt-0">
        <div class="list-group">
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://cdn.jsdelivr.net/gh/MetaMask/brand-resources/SVG/metamask-fox.svg" alt="
            <span>MetaMask (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/1885080?s=200&v=4" alt="" width="28" hei
            <span>Coinbase Wallet (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/78782331?s=200&v=4" alt="" width="28" he
            <span>Phantom (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/64856450?s=200&v=4" alt="" width="28" he
            <span>Solflare (Solana)</span>
          </button>
```

```
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/21111808?s=200&v=4" alt="" width="28" he
            <span>Zcash Wallet / Hardware</span>
          </button>
        </div>
        <div class="small text-secondary mt-3" id="wallet-help"></div>
      </div>
    </div>
  </div>
</div>

<!-- ═══ FLOATING OPTIONS MENU ═══════════════════════════════════════════ -->
<div id="track-options-menu" role="menu" aria-label="Track options">
  <div class="opt-section">
    <button class="opt-item" data-opt="fav-track">
      <i class="opt-icon">♥</i> Favorite Track
    </button>
    <button class="opt-item" data-opt="fav-artist">
      <i class="opt-icon">★</i> Favorite Artist
    </button>
  </div>
  <hr class="opt-divider">
  <div class="opt-section">
    <div class="opt-submenu-head">Add to Playlist</div>
    <div id="opt-playlist-list">
      <span class="opt-pl-empty">No playlists yet</span>
    </div>
  </div>
  <hr class="opt-divider">
  <div class="opt-section">
    <button class="opt-item" data-opt="share">
      <i class="opt-icon">🔗</i> Share
    </button>
    <button class="opt-item" data-opt="download">
      <i class="opt-icon">⬇</i> Download
      <span class="opt-tier-badge" id="opt-download-badge">Tier 2+</span>
    </button>
    <button class="opt-item" data-opt="lyrics">
      <i class="opt-icon">🎤</i> Lyrics
    </button>
    <button class="opt-item" data-opt="info">
      <i class="opt-icon">ℹ</i> Info
    </button>
  </div>
</div>

<!-- ═══ VENDOR + APP SCRIPTS ═══════════════════════════════════════════ -->
```

```
<script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
<script src="vendor/ethers/ethers.umd.min.js"></script>
<script src="vendor/hls/hls.min.js"></script>
<script src="vendor/ipfs/index.min.js"></script>
<script src="Scripts/wallets.js"></script>
<script src="Scripts/common.js"></script>
<script src="Scripts/favorites.js"></script>
<script src="Scripts/live_broadcast.js"></script>
<script src="Scripts/main.js"></script>

<script src="Scripts/listen.js"></script>

</body>
</html>
```

`creators.html` (39.1 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- Favicon -->
  <link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
  <link rel="apple-touch-icon" href="assets/msp-vinyl-180.png">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#F5F000">
  <title>Creators Corner — Michie Stream Platform</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>
/* ── MSP Neon Banner ──────────────────────────────────────── */
.neon-banner {
  position: relative;
  z-index: 90;
  display: flex;
  align-items: center;
  width: 100%;
  padding: 14px 8px;
  background: rgba(8,8,10,0.6);
  border-bottom: 1px solid rgba(212,168,83,0.15);
  overflow: hidden;
}

.neon-banner .b-side {
```

```
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 7px;
  flex-shrink: 0;
  width: 80px;
}

.neon-banner .b-slogan {
  flex: 1;
  text-align: center;
}

.neon-banner .b-line {
  font-family: 'Syne', 'Arial Black', sans-serif;
  font-weight: 800;
  font-size: clamp(13px, 2vw, 22px);
  letter-spacing: .04em;
  line-height: 1.3;
  color: #F5F000;
  animation:
    neon-boot    2.4s ease-out forwards,
    neon-flicker 5s  ease-in-out infinite 2.4s;
}

.neon-banner .b-line:last-child {
  animation:
    neon-boot    2.4s ease-out .15s forwards,
    neon-flicker 5s  ease-in-out infinite 2.55s;
}

.neon-banner svg.b-ic {
  opacity: 0;
  animation: b-icon-in 0.4s ease-out forwards, b-fl 3s ease-in-out infinite;
}

@keyframes b-icon-in { 0%{opacity:0;} 100%{opacity:.85;} }
@keyframes b-fl { 0%,100%{transform:translateY(0) scale(1);opacity:.85;} 50%{transform:translateY(-4px

@keyframes neon-boot {
  0%   {opacity:0;   text-shadow:none;}
  8%   {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8);}
  10%  {opacity:.1;  text-shadow:none;}
  14%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 20px #F5F000,0 0 40px rgba(245,240,0,.9);}
  16%  {opacity:.3;  text-shadow:none;}
  20%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000;}
  22%  {opacity:.05; text-shadow:none;}
```

```
  28%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
  30%  {opacity:.6;  text-shadow:none;}
  36%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
  100% {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
}

@keyframes neon-flicker {
  0%,18%,20%,22%,24%,53%,55%,100% {
    text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
    opacity:1;
  }
  19%,23%,54%{text-shadow:none;opacity:.82;}
}
</style>
</head>
<body style="padding-top:70px;" class="page-creators">

  <!-- ═══════ NAVBAR ═══════ -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
    <div class="container-fluid">
      <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNa
        aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav me-auto">
          <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="listen.html">Just Listen</a></li>
          <li class="nav-item"><a class="nav-link active" aria-current="page" href="creators.html">Cre
          <li class="nav-item"><a class="nav-link" href="marketplace.html">NFT Marketplace</a></li>
          <li class="nav-item"><a class="nav-link" href="profile.html">Profile</a></li>
          <li class="nav-item" data-requires="hostConcert" style="display:none;"><a class="nav-link" h
        </ul>
        <div class="d-flex align-items-center gap-2">
          <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
          <button id="btn-disconnect" class="btn btn-outline-light btn-sm d-none">Disconnect</button>
          <span id="walletAddress" class="small text-light wallet-address-display"></span>
          <span class="small text-warning fw-semibold user-name-display"></span>
        </div>
      </div>
    </div>
  </nav>
  <!-- ═══ MSP NEON BANNER ═══════════════════════════════ -->
  <div class="neon-banner">

    <!-- Left icons -->
```

```
<div class="b-side">
  <svg class="b-ic" style="animation-delay:.1s,0s;width:24px;height:20px;filter:drop-shadow(0 0 5p
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.4s,.35s;width:36px;height:24px;filter:drop-shadow(0 0
    <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
    <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
    <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
    <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.7s,.7s;width:26px;height:18px;filter:drop-shadow(0 0 5
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.05s,1.05s;width:24px;height:20px;filter:drop-shadow(0
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.4s,1.4s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.75s,1.75s;width:38px;height:26px;filter:drop-shadow(0
    <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
    <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
    <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
    <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
    <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:2.1s,2.1s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
  </svg>
</div>

<!-- Slogan -->
<div class="b-slogan">
```

```
  <div class="b-line">Put the Needle on the Record</div>
  <div class="b-line">That is on a Blockchain</div>
</div>

<!-- Right icons (staggered offsets) -->
<div class="b-side">
  <svg class="b-ic" style="animation-delay:.2s,.2s;width:24px;height:20px;filter:drop-shadow(0 0 5
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.55s,.55s;width:36px;height:24px;filter:drop-shadow(0 0
    <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
    <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
    <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
    <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.9s,.9s;width:26px;height:18px;filter:drop-shadow(0 0 5
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.25s,1.25s;width:24px;height:20px;filter:drop-shadow(0
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.6s,1.6s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.95s,1.95s;width:38px;height:26px;filter:drop-shadow(0
    <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
    <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
    <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
    <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
    <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:2.3s,2.3s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
```

```
    </svg>
  </div>

</div>
<!-- ═══ END NEON BANNER ═══════════════════════════════ -->


<!-- ═══════ PAGE CONTENT ═══════ -->
<div class="container py-5 mt-5">

  <!-- ── Creator Status Bar ── -->
  <div class="d-flex align-items-center gap-3 mb-4 flex-wrap">
    <div>
      <span id="account-type-display" class="fw-semibold text-muted">Connect wallet to load</span>
      <span id="subscription-expiry" class="ms-2 small"></span>
    </div>
    <span id="platform-nft-badge" class="badge bg-warning text-dark" style="display:none;"></span>
    <div class="ms-auto small text-muted" id="fee-row" style="display:none;">
      Royalty platform fee: <strong><span id="royalty-fee-rate"></span></strong>
       ·  <a href="profile.html#claim-nft" class="text-muted">Manage NFT status</a>
    </div>
  </div>

  <!-- ── Creator Access Gate Message ── -->
  <div id="creator-gate-msg" class="alert alert-warning" style="display:none;">
    <strong>Creator account required.</strong>
    Upload, mint, and live tools are available to Standard Creator subscribers ($29.99/mo) and Platf
    <a href="listen.html#subscribe" class="alert-link">Subscribe here</a> or
    <a href="profile.html#claim-nft" class="alert-link">claim your Platform NFT</a>.
  </div>

  <!-- ═══════════════════════════════════════════
       UPLOAD FORM
  ═══════════════════════════════════════════ -->
  <section id="creators" class="mb-5">

    <!-- ── Content type tabs ── -->
    <div class="upload-type-bar" id="upload-type-bar">
      <button class="utype-btn active" data-type="music">
        <span class="utype-icon">♪</span>
        <span>Music</span>
      </button>
      <button class="utype-btn" data-type="podcast">
        <span class="utype-icon">🎙</span>
        <span>Podcast</span>
      </button>
      <button class="utype-btn" data-type="video">
```

```
    <span class="utype-icon">▶</span>
    <span>Video</span>
  </button>
  <button class="utype-btn" data-type="art_still">
    <span class="utype-icon">◈</span>
    <span>Art — Still</span>
  </button>
  <button class="utype-btn" data-type="art_animated">
    <span class="utype-icon">◉</span>
    <span>Art — Animated</span>
  </button>
</div>

<!-- ── Requirements line (changes with type) ── -->
<p class="upload-reqs" id="upload-reqs">
  Audio ≥ 128 kbps · 44.1 / 48 kHz · MP3, WAV, OGG, FLAC, AAC, M4A · max 500 MB
</p>

<form id="upload-form" enctype="multipart/form-data">
  <input type="hidden" id="content-type-field" name="contentType" value="music">

  <!-- ── Drop zone ── -->
  <div class="drop-zone" id="drag-drop-area">
    <div class="drop-zone-inner" id="drop-zone-inner">
      <div class="drop-icon" id="drop-icon">↑</div>
      <div class="drop-primary">Drop your file here</div>
      <div class="drop-secondary">or <button type="button" id="browse-btn" class="drop-browse">b
      <div class="drop-hint" id="drop-hint">MP3 · WAV · OGG · FLAC · AAC · M4A</div>
    </div>
    <input type="file" id="audio-file" name="audio-file"
           accept="audio/mpeg,audio/wav,audio/ogg,audio/flac,audio/aac,audio/mp4,audio/x-m4a"
           hidden>
  </div>

  <!-- ── Cover image ── -->
  <div class="upload-field-row">
    <div class="upload-field cover-field">
      <label class="ufield-label" for="cover-image">Cover Image <span class="req">*</span></labe
      <div class="cover-drop" id="cover-drop">
        <div class="cover-preview" id="cover-preview" style="display:none">
          <img id="cover-preview-img" src="" alt="cover">
          <button type="button" id="cover-clear" class="cover-clear">✕</button>
        </div>
        <div class="cover-placeholder" id="cover-placeholder">
          <span style="font-size:28px;opacity:0.3">◈</span>
          <span style="font-size:11px;color:var(--text-3)">PNG / JPG / WebP · any size in DEV mo
          <button type="button" id="cover-browse-btn" class="drop-browse" style="font-size:12px"
```

```
      </div>
      <input type="file" id="cover-image" name="cover-image"
             accept="image/png,image/jpeg,image/webp" hidden>
    </div>
  </div>

  <!-- ── Core fields ── -->
  <div class="upload-field flex-fields">
    <div class="ufield">
      <label class="ufield-label" for="song-title">Title <span class="req">*</span></label>
      <input class="ufield-input" type="text" id="song-title" name="songTitle" required placeh
    </div>
    <div class="ufield">
      <label class="ufield-label" for="artist-name">Artist / Creator <span class="req">*</span
      <input class="ufield-input" type="text" id="artist-name" name="artistName" required plac
    </div>
    <div class="ufield">
      <label class="ufield-label" for="description">Description</label>
      <textarea class="ufield-input" id="description" name="description" rows="2" placeholder=
    </div>
  </div>
</div>

<!-- ── Music-specific fields ── -->
<div class="type-fields" id="fields-music">
  <div class="ufield-group">
    <div class="ufield">
      <label class="ufield-label" for="album">Album / EP</label>
      <input class="ufield-input" type="text" id="album" name="album" placeholder="Leave blank
    </div>
    <div class="ufield ufield-sm">
      <label class="ufield-label" for="bpm">BPM</label>
      <input class="ufield-input" type="number" id="bpm" name="bpm" placeholder="120" min="40"
    </div>
  </div>
</div>

<!-- ── Podcast-specific fields ── -->
<div class="type-fields" id="fields-podcast" style="display:none">
  <div class="ufield-group">
    <div class="ufield">
      <label class="ufield-label" for="series-name">Series / Show Name</label>
      <input class="ufield-input" type="text" id="series-name" name="seriesName" placeholder="
    </div>
    <div class="ufield ufield-sm">
      <label class="ufield-label" for="episode-number">Episode #</label>
      <input class="ufield-input" type="number" id="episode-number" name="episodeNumber" place
```

```
    </div>
  </div>
</div>

<!-- ── Dates & registration ── -->
<div class="ufield-group">
  <div class="ufield">
    <label class="ufield-label" for="release-date">Release Date</label>
    <input class="ufield-input" type="date" id="release-date" name="releaseDate">
  </div>
  <div class="ufield" style="display:none">
    <label class="ufield-label">Date Created <span class="ufield-auto">auto-filled</span></lab
    <input class="ufield-input" type="date" id="date-created" name="dateCreated" readonly>
  </div>
  <div class="ufield">
    <label class="ufield-label" for="withdraw-address">Withdraw Address <span class="ufield-op
    <input class="ufield-input ufield-mono" type="text" id="withdraw-address" name="withdrawAd
  </div>
</div>

<!-- ── Rights metadata (music/podcast only) ── -->
<div class="type-fields" id="fields-rights">
  <details class="rights-details">
    <summary class="rights-summary">Rights & Registration <span class="rights-opt">optional —
    <div class="ufield-group" style="margin-top:14px">
      <div class="ufield">
        <label class="ufield-label" for="mlc-iswc">ISWC</label>
        <input class="ufield-input ufield-mono" type="text" id="mlc-iswc" name="mlc_iswc" plac
      </div>
      <div class="ufield">
        <label class="ufield-label" for="mlc-isrc">ISRC</label>
        <input class="ufield-input ufield-mono" type="text" id="mlc-isrc" name="isrc" placehol
      </div>
      <div class="ufield">
        <label class="ufield-label" for="mlc-ipi">IPI Name Number</label>
        <input class="ufield-input ufield-mono" type="text" id="mlc-ipi" name="mlc_ipi_name_nu
      </div>
    </div>
  </details>
</div>

<!-- ── Tags ── -->
<div class="ufield">
  <label class="ufield-label" for="tags">Tags <span class="ufield-opt">optional, comma-separat
  <input class="ufield-input" type="text" id="tags" name="tags"
         placeholder="e.g. electronic, deep house, original">
  <div class="tags-preview" id="tags-preview"></div>
```

```
    </div>

    <!-- ── Mint + chain selector ── -->
    <div class="mint-row">
      <div class="mint-toggle-wrap">
        <label class="mint-switch">
          <input type="checkbox" id="mint-nft" name="mint_nft" checked>
          <span class="mint-slider"></span>
        </label>
        <div class="mint-label-wrap">
          <span class="mint-label-main">Mint as NFT on-chain</span>
          <span class="mint-label-sub">Registers on your connected wallet after upload</span>
        </div>
      </div>
      <div class="mint-chain-wrap" id="mint-chain-wrap">
        <label class="ufield-label" for="mint-chain" style="margin-bottom:4px">Chain</label>
        <select class="ufield-input ufield-select" id="mint-chain" name="mintChain" style="width:a
          <option value="ethereum">Ethereum</option>
          <option value="polygon">Polygon</option>
          <option value="base">Base</option>
        </select>
      </div>
    </div>

    <!-- ── Copyright confirmation ── -->
    <label class="copyright-check">
      <input type="checkbox" id="copyright-check" required>
      <span>I confirm this is original content and I own all rights, or have proper licensing.</sp
    </label>

    <!-- ── Submit ── -->
    <div class="upload-submit-row">
      <button type="submit" id="upload-btn" class="btn-upload">
        <span id="upload-btn-text">↑ Upload &amp; Mint</span>
      </button>
      <div class="upload-feedback">
        <progress id="upload-progress" value="0" max="100" style="display:none"></progress>
        <div id="upload-status" style="display:none"></div>
      </div>
    </div>

  </form>
</section>

<style>
/* ── Upload section — SIGNAL design ──────────────────── */
.upload-type-bar {
```

```
  display: flex;
  gap: 8px;
  margin-bottom: 12px;
  flex-wrap: wrap;
}
.utype-btn {
  display: flex;
  align-items: center;
  gap: 7px;
  padding: 8px 16px;
  background: #1C1C21;
  border: 1px solid #2E2E36;
  border-radius: 5px;
  color: #8A8A98;
  font-family: 'Syne', sans-serif;
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: all .15s;
}
.utype-btn:hover { border-color: #E85D3A; color: #EEEAE4; }
.utype-btn.active { border-color: #E85D3A; color: #E85D3A; background: rgba(232,93,58,.08); }
.utype-icon { font-size: 15px; }

.upload-reqs {
  font-size: 11px;
  color: #484852;
  font-family: 'Space Mono', monospace;
  letter-spacing: .04em;
  margin-bottom: 20px;
}

.drop-zone {
  background: #1C1C21;
  border: 1.5px dashed #2E2E36;
  border-radius: 10px;
  padding: 32px 24px;
  text-align: center;
  transition: all .2s;
  cursor: pointer;
  margin-bottom: 20px;
}
.drop-zone.drag-over,
.drop-zone:hover { border-color: #E85D3A; background: rgba(232,93,58,.04); }
.drop-zone.has-file { border-style: solid; border-color: #00D4BB; background: rgba(0,212,187,.04);
.drop-icon {
  font-size: 36px;
```

```
  color: #484852;
  line-height: 1;
  margin-bottom: 8px;
  transition: color .2s;
}
.drop-zone:hover .drop-icon,
.drop-zone.drag-over .drop-icon { color: #E85D3A; }
.drop-zone.has-file .drop-icon { color: #00D4BB; font-size: 24px; }
.drop-primary { font-size: 14px; font-weight: 600; color: #EEEAE4; margin-bottom: 4px; }
.drop-secondary { font-size: 12px; color: #8A8A98; margin-bottom: 8px; }
.drop-hint { font-family: 'Space Mono', monospace; font-size: 10px; color: #484852; letter-spacing
.drop-browse {
  background: none; border: none;
  color: #E85D3A; cursor: pointer;
  font-size: inherit; font-family: inherit; font-weight: 600;
  text-decoration: underline;
  padding: 0;
}

.upload-field-row { display: grid; grid-template-columns: 180px 1fr; gap: 20px; margin-bottom: 20p
@media (max-width: 600px) { .upload-field-row { grid-template-columns: 1fr; } }

.cover-drop {
  background: #1C1C21;
  border: 1px solid #2E2E36;
  border-radius: 8px;
  aspect-ratio: 1;
  display: flex; align-items: center; justify-content: center;
  cursor: pointer;
  overflow: hidden;
  position: relative;
  transition: border-color .15s;
}
.cover-drop:hover { border-color: #E85D3A; }
.cover-placeholder { display: flex; flex-direction: column; align-items: center; gap: 6px; padding
.cover-preview { width: 100%; height: 100%; position: relative; }
.cover-preview img { width: 100%; height: 100%; object-fit: cover; display: block; }
.cover-clear {
  position: absolute; top: 6px; right: 6px;
  background: rgba(8,8,10,.7); border: none; color: #EEEAE4;
  width: 24px; height: 24px; border-radius: 50%;
  cursor: pointer; font-size: 12px;
  display: flex; align-items: center; justify-content: center;
}

.flex-fields { display: flex; flex-direction: column; gap: 12px; }
```

```
.ufield { display: flex; flex-direction: column; gap: 5px; flex: 1; }
.ufield-sm { max-width: 100px; }
.ufield-group { display: flex; gap: 12px; margin-bottom: 16px; flex-wrap: wrap; }
.ufield-label {
  font-family: 'Space Mono', monospace;
  font-size: 9.5px; letter-spacing: .15em; text-transform: uppercase;
  color: #484852;
}
.req { color: #E85D3A; }
.ufield-opt { color: #484852; font-weight: 400; text-transform: none; letter-spacing: 0; }
.ufield-auto { color: #484852; font-weight: 400; text-transform: none; letter-spacing: 0; font-siz
.ufield-input {
  background: #1C1C21;
  border: 1px solid #2E2E36;
  border-radius: 5px;
  padding: 9px 13px;
  font-family: 'Syne', sans-serif;
  font-size: 13px; color: #EEEAE4;
  outline: none;
  transition: border-color .15s;
  width: 100%;
  resize: vertical;
}
.ufield-input:focus { border-color: #D4A853; box-shadow: 0 0 0 3px rgba(212,168,83,.1); }
.ufield-input::placeholder { color: #484852; }
.ufield-mono { font-family: 'Space Mono', monospace; font-size: 11px; }
.ufield-select { appearance: none; cursor: pointer; }

.type-fields { margin-bottom: 16px; }

.rights-details { border: 1px solid #2E2E36; border-radius: 6px; padding: 0; overflow: hidden; mar
.rights-summary {
  padding: 11px 14px; cursor: pointer;
  font-family: 'Space Mono', monospace; font-size: 10px;
  letter-spacing: .1em; text-transform: uppercase; color: #484852;
  user-select: none; list-style: none;
  display: flex; align-items: center; gap: 12px;
}
.rights-summary::-webkit-details-marker { display: none; }
.rights-summary::before { content: '+'; font-size: 14px; color: #484852; }
details[open] .rights-summary::before { content: '−'; }
details[open] { background: rgba(28,28,33,.6); }
details[open] .rights-summary { border-bottom: 1px solid #2E2E36; }
.rights-opt { color: #484852; font-weight: 400; text-transform: none; letter-spacing: 0; font-size
details .ufield-group { padding: 0 14px 14px; margin-bottom: 0; }

.tags-preview { display: flex; flex-wrap: wrap; gap: 6px; margin-top: 8px; min-height: 0; }
```

```
.tag-chip {
  background: rgba(0,212,187,.1); border: 1px solid rgba(0,212,187,.3);
  border-radius: 3px; padding: 3px 8px;
  font-family: 'Space Mono', monospace; font-size: 10px; color: #00D4BB;
}

.mint-row {
  display: flex; align-items: center; justify-content: space-between;
  padding: 16px; background: #1C1C21; border: 1px solid #2E2E36;
  border-radius: 8px; margin-bottom: 14px; flex-wrap: wrap; gap: 12px;
}
.mint-toggle-wrap { display: flex; align-items: center; gap: 12px; }
.mint-switch { position: relative; display: inline-block; width: 40px; height: 22px; }
.mint-switch input { opacity: 0; width: 0; height: 0; }
.mint-slider {
  position: absolute; inset: 0; border-radius: 11px;
  background: #2E2E36; border: 1px solid #484852;
  transition: .2s; cursor: pointer;
}
.mint-slider::before {
  content: ''; position: absolute;
  width: 16px; height: 16px; border-radius: 50%;
  background: #484852; left: 2px; top: 2px;
  transition: .2s;
}
.mint-switch input:checked + .mint-slider { background: rgba(139,92,246,.25); border-color: #8B5CF
.mint-switch input:checked + .mint-slider::before { transform: translateX(18px); background: #8B5C
.mint-label-main { font-size: 13px; font-weight: 600; color: #EEEAE4; }
.mint-label-sub { font-size: 11px; color: #484852; display: block; margin-top: 2px; }

.copyright-check {
  display: flex; align-items: flex-start; gap: 10px;
  font-size: 12px; color: #484852; cursor: pointer;
  margin-bottom: 20px;
}
.copyright-check input { margin-top: 2px; accent-color: #D4A853; flex-shrink: 0; }

.upload-submit-row { display: flex; flex-direction: column; gap: 10px; }
.btn-upload {
  background: #D4A853; color: #0F0F12;
  border: none; border-radius: 6px;
  padding: 14px 28px;
  font-family: 'Syne', sans-serif; font-size: 13px; font-weight: 700;
  letter-spacing: .05em; text-transform: uppercase;
  cursor: pointer; transition: all .2s;
  align-self: flex-start;
}
```

```
.btn-upload:hover:not(:disabled) { background: #F0C060; transform: translateY(-1px); box-shadow: 0
.btn-upload:disabled { opacity: .5; cursor: not-allowed; }

#upload-progress { width: 100%; height: 4px; appearance: none; border-radius: 2px; border: none; }
#upload-progress::-webkit-progress-bar { background: #2E2E36; border-radius: 2px; }
#upload-progress::-webkit-progress-value { background: #D4A853; border-radius: 2px; transition: wi
#upload-status { font-family: 'Space Mono', monospace; font-size: 11px; padding: 8px 12px; border-
#upload-status.success { background: rgba(0,212,187,.1); color: #00D4BB; border: 1px solid rgba(0,
#upload-status.error   { background: rgba(232,93,58,.1);  color: #E85D3A; border: 1px solid rgba(2
</style>

<!-- ═══════════════════════════════════════════
     DJ SET  (Tier 2+ or active creator)
═══════════════════════════════════════════ -->
<section class="mb-5" data-requires="hostDjSet">
  <div class="card border-0 shadow-sm">
    <div class="card-header">
      <h4 class="mb-0">🎛️ Start a DJ Set</h4>
    </div>
    <div class="card-body">
      <p class="text-muted small mb-3">
        Host a live DJ set. Tips are on by default — you set your own split between yourself
        and the artists whose tracks you played.
        Platform takes 3% of all tips before your split.
      </p>
      <form id="dj-set-form">
        <div class="mb-3">
          <label for="dj-set-name" class="form-label">Set Name <span class="text-danger">*</span><
          <input type="text" class="form-control" id="dj-set-name" placeholder="Friday Night Mix V
        </div>

        <div class="row g-3 mb-3">
          <div class="col-sm-6">
            <label for="dj-tip-percent" class="form-label">Your cut of tips (%)</label>
            <input type="number" class="form-control" id="dj-tip-percent" value="70" min="0" max="
            <small class="form-text text-muted">Remaining goes to the artists whose tracks were pl
          </div>
          <div class="col-sm-6 d-flex align-items-end pb-1">
            <div class="form-check form-switch">
              <input class="form-check-input" type="checkbox" role="switch" id="dj-tips-enabled" c
              <label class="form-check-label" for="dj-tips-enabled">Accept tips for this set</labe
            </div>
          </div>
        </div>

        <button type="submit" class="btn btn-primary">Start DJ Set</button>
        <p id="dj-set-status" class="mt-2 small"></p>
```

```
        </form>
      </div>
    </div>
  </section>

  <!-- ═══════════════════════════════════════════
       LIVE CONCERT  (active creator only)
  ═══════════════════════════════════════════ -->
  <section class="mb-5" data-requires="hostConcert">
    <div class="card border-0 shadow-sm">
      <div class="card-header">
        <h4 class="mb-0">🔴 Start Live Concert</h4>
      </div>
      <div class="card-body">
        <p class="text-muted small mb-3">
          Push an RTMP stream to the server and start a live HLS encode.
          Tier 1 listeners can watch. Tier 2+ can interact in the chat.
        </p>
        <form id="liveEncodeForm">
          <div class="mb-3">
            <label for="eventTitle" class="form-label">Event Title <span class="text-danger">*</span
            <input type="text" class="form-control" id="eventTitle" placeholder="Summer Night Live"
          </div>
          <div class="mb-3">
            <label for="artistName" class="form-label">Artist / Band Name <span class="text-danger">
            <input type="text" class="form-control" id="artistName" placeholder="DJ Michie" required
          </div>
          <button type="submit" class="btn btn-danger">🔴 Go Live</button>
          <p id="liveStatus" class="mt-2 small"></p>
        </form>
      </div>
    </div>
  </section>

</div><!-- /container -->

<!-- ═══════ WALLET MODAL ═══════ -->
<div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidde
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content bg-dark text-light">
      <div class="modal-header border-0">
        <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="
      </div>
      <div class="modal-body pt-0">
        <div class="list-group">
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
```

```
            <img src="https://cdn.jsdelivr.net/gh/MetaMask/brand-resources/SVG/metamask-fox.svg" alt
            <span>MetaMask (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/1885080?s=200&v=4" alt="" width="28" h
            <span>Coinbase Wallet (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/78782331?s=200&v=4" alt="" width="28"
            <span>Phantom (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/64856450?s=200&v=4" alt="" width="28"
            <span>Solflare (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/21111808?s=200&v=4" alt="" width="28"
            <span>Zcash Wallet / Hardware</span>
          </button>
        </div>
        <div class="small text-secondary mt-3" id="wallet-help"></div>
      </div>
    </div>
  </div>
</div>

<!-- Vendor scripts -->
<script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
<script src="vendor/ethers/ethers.umd.min.js"></script>
<script src="vendor/hls/hls.min.js"></script>
<script src="vendor/ipfs/index.min.js"></script>
<!-- App scripts -->
<script src="Scripts/wallets.js"></script>
<script src="Scripts/common.js"></script>
<script src="Scripts/live_broadcast.js"></script>
<script src="Scripts/main.js"></script>

<!-- Show/hide creator gate message and fee row after access loads -->
<script>
  document.addEventListener('walletConnected', function() {
    // Brief delay to allow main.js applyCapabilityGates() to run first
    setTimeout(function() {
      var feeEl  = document.getElementById('royalty-fee-rate');
      var feeRow = document.getElementById('fee-row');
      if (feeEl && feeRow) feeRow.style.display = feeEl.textContent.trim() ? '' : 'none';

      // Show gate message if upload section is not accessible
```

```
        var uploadBtn = document.getElementById('upload-btn');
        var gateMsg   = document.getElementById('creator-gate-msg');
        if (gateMsg && uploadBtn) gateMsg.style.display = uploadBtn.disabled ? '' : 'none';
      }, 300);
    });
  </script>

</body>
</html>
```

`marketplace.html` (18.8 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <!-- Favicon -->
  <link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
  <link rel="apple-touch-icon" href="assets/msp-vinyl-180.png">
  <link rel="manifest" href="manifest.json">
  <title>NFT Marketplace — Michie Stream Platform</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>
.fav-btn { background:none; border:none; cursor:pointer; font-size:18px; line-height:1; padding:2px 5p
.fav-btn:hover { color:#dc3545; transform:scale(1.2); }
.fav-btn.fav-active { color:#dc3545; }
</style>
<style>
/* ── MSP Neon Banner ──────────────────────────────────────── */
.neon-banner {
  position: relative;
  z-index: 90;
  display: flex;
  align-items: center;
  width: 100%;
  padding: 14px 8px;
  background: rgba(8,8,10,0.6);
  border-bottom: 1px solid rgba(212,168,83,0.15);
  overflow: hidden;
}

.neon-banner .b-side {
  display: flex;
```

```
  flex-direction: column;
  align-items: center;
  gap: 7px;
  flex-shrink: 0;
  width: 80px;
}

.neon-banner .b-slogan {
  flex: 1;
  text-align: center;
}

.neon-banner .b-line {
  font-family: 'Syne', 'Arial Black', sans-serif;
  font-weight: 800;
  font-size: clamp(13px, 2vw, 22px);
  letter-spacing: .04em;
  line-height: 1.3;
  color: #F5F000;
  animation:
    neon-boot    2.4s ease-out forwards,
    neon-flicker 5s  ease-in-out infinite 2.4s;
}

.neon-banner .b-line:last-child {
  animation:
    neon-boot    2.4s ease-out .15s forwards,
    neon-flicker 5s  ease-in-out infinite 2.55s;
}

.neon-banner svg.b-ic {
  opacity: 0;
  animation: b-icon-in 0.4s ease-out forwards, b-fl 3s ease-in-out infinite;
}

@keyframes b-icon-in { 0%{opacity:0;} 100%{opacity:.85;} }
@keyframes b-fl { 0%,100%{transform:translateY(0) scale(1);opacity:.85;} 50%{transform:translateY(-4px

@keyframes neon-boot {
  0%   {opacity:0;   text-shadow:none;}
  8%   {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8);}
  10%  {opacity:.1;  text-shadow:none;}
  14%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 20px #F5F000,0 0 40px rgba(245,240,0,.9);}
  16%  {opacity:.3;  text-shadow:none;}
  20%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000;}
  22%  {opacity:.05; text-shadow:none;}
  28%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
```

```
  30%  {opacity:.6;  text-shadow:none;}
  36%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
  100% {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
}

@keyframes neon-flicker {
  0%,18%,20%,22%,24%,53%,55%,100% {
    text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
    opacity:1;
  }
  19%,23%,54%{text-shadow:none;opacity:.82;}
}
</style>
</head>
<body style="padding-top:70px;" class="page-marketplace">

  <!-- ═══════ NAVBAR ═══════ -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
    <div class="container-fluid">
      <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNa
        aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav me-auto">
          <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="listen.html">Just Listen</a></li>
          <li class="nav-item"><a class="nav-link" href="creators.html">Creators Corner</a></li>
          <li class="nav-item"><a class="nav-link active" aria-current="page" href="marketplace.html">
          <li class="nav-item"><a class="nav-link" href="profile.html">Profile</a></li>
          <li class="nav-item" data-requires="hostConcert" style="display:none;"><a class="nav-link" h
        </ul>
        <div class="d-flex align-items-center gap-2">
          <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
          <button id="btn-disconnect" class="btn btn-outline-light btn-sm d-none">Disconnect</button>
          <span id="walletAddress" class="small text-light wallet-address-display"></span>
          <span class="small text-warning fw-semibold user-name-display"></span>
        </div>
      </div>
    </div>
  </nav>
  <!-- ═══ MSP NEON BANNER ═══════════════════════════════ -->
  <div class="neon-banner">

    <!-- Left icons -->
    <div class="b-side">
```

```
  <svg class="b-ic" style="animation-delay:.1s,0s;width:24px;height:20px;filter:drop-shadow(0 0 5p
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.4s,.35s;width:36px;height:24px;filter:drop-shadow(0 0
    <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
    <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
    <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
    <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.7s,.7s;width:26px;height:18px;filter:drop-shadow(0 0 5
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.05s,1.05s;width:24px;height:20px;filter:drop-shadow(0
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.4s,1.4s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.75s,1.75s;width:38px;height:26px;filter:drop-shadow(0
    <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
    <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
    <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
    <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
    <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:2.1s,2.1s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
  </svg>
</div>

<!-- Slogan -->
<div class="b-slogan">
  <div class="b-line">Put the Needle on the Record</div>
```

```
  <div class="b-line">That is on a Blockchain</div>
</div>

<!-- Right icons (staggered offsets) -->
<div class="b-side">
  <svg class="b-ic" style="animation-delay:.2s,.2s;width:24px;height:20px;filter:drop-shadow(0 0 5
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.55s,.55s;width:36px;height:24px;filter:drop-shadow(0 0
    <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
    <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
    <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
    <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.9s,.9s;width:26px;height:18px;filter:drop-shadow(0 0 5
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.25s,1.25s;width:24px;height:20px;filter:drop-shadow(0
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.6s,1.6s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.95s,1.95s;width:38px;height:26px;filter:drop-shadow(0
    <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
    <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
    <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
    <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
    <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:2.3s,2.3s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
  </svg>
```

```
  </div>

</div>
<!-- ═══ END NEON BANNER ═══════════════════════════════ -->


<!-- ═══════ PAGE CONTENT ═══════ -->
<div class="container py-5 mt-5">

  <div class="text-center mb-5">
    <h1>🎧 NFT Marketplace</h1>
    <p class="text-muted">
      Browse and purchase music NFTs. Own the content, earn royalties, support artists directly.
    </p>
    <div class="d-flex justify-content-center gap-4 small text-muted">
      <span>Total Plays: <strong id="total-plays">—</strong></span>
      <span>New Uploads: <strong id="new-uploads">—</strong></span>
    </div>
  </div>

  <!-- NFT grid — main.js replaces this if user is not subscribed -->
  <section id="marketplace-section" class="mb-5">
    <div class="d-flex align-items-center justify-content-between mb-3">
      <h2 class="mb-0">Browse NFTs</h2>
      <span id="platform-nft-badge" class="badge bg-warning text-dark" style="display:none;"></span>
    </div>

    <!-- Platform NFT callout — always visible -->
    <div class="alert alert-warning mb-4">
      <div class="d-flex align-items-start gap-3">
        <span style="font-size:1.8rem;">🎖️</span>
        <div>
          <strong>Platform NFT — $10,000 USD</strong>
          <p class="mb-1 small">
            One-time purchase. Grants full creator capabilities with a 1.5% royalty fee (vs 5% stand
            Price floor enforced on-chain — cannot be resold for less than $10,000 USD.
            2.5% platform fee on secondary resales.
          </p>
          <a href="profile.html#claim-nft" class="btn btn-sm btn-warning text-dark">
            Learn More &amp; Claim
          </a>
        </div>
      </div>
    </div>

    <div id="marketplace-list" class="row">
      <!-- Populated by main.js loadNFTs() -->
```

```
      <div class="col-12 text-center py-4 text-muted">
        <p>Connect your wallet to browse available NFTs.</p>
        <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
      </div>
    </div>
  </section>

  <!-- Player -->
  <section id="player-section" class="mb-5">
    <h4>Now Playing</h4>
    <audio id="audio-player" controls class="w-100"></audio>
    <p id="track-name" class="mt-2 text-muted small">Select a track above to play</p>
    <p id="duration-display" class="small text-muted">Duration: 0:00</p>
    <input type="range" id="progress-bar" class="form-range" min="0" max="100" value="0">
    <div class="d-flex align-items-center gap-3">
      <span id="time-display" class="small text-muted">0:00</span>
      <input type="range" id="volume-bar" class="form-range" min="0" max="1" step="0.01" value="1" s
      <span class="small text-muted">Volume</span>
    </div>
  </section>

  <!-- Fee schedule reference -->
  <section class="mb-5">
    <h4 class="mb-3">Fee Schedule</h4>
    <div class="table-responsive">
      <table class="table table-sm table-bordered text-center">
        <thead class="table-dark">
          <tr>
            <th>Transaction</th>
            <th>Platform Fee</th>
          </tr>
        </thead>
        <tbody>
          <tr><td>Streaming royalties — Standard Creator</td><td>5%</td></tr>
          <tr><td>Streaming royalties — Platform NFT Creator</td><td>1.5%</td></tr>
          <tr><td>DJ tips</td><td>3%</td></tr>
          <tr><td>NFT sales — primary &amp; secondary</td><td>2.5%</td></tr>
          <tr class="table-warning"><td>Platform NFT primary purchase</td><td><strong>0%</strong></t
          <tr><td>Platform NFT secondary resale</td><td>2.5%</td></tr>
        </tbody>
      </table>
    </div>
  </section>

</div><!-- /container -->

<!-- ═══════ WALLET MODAL ═══════ -->
```

```
  <div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidde
    <div class="modal-dialog modal-dialog-centered">
      <div class="modal-content bg-dark text-light">
        <div class="modal-header border-0">
          <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
          <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="
        </div>
        <div class="modal-body pt-0">
          <div class="list-group">
            <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
              <img src="https://cdn.jsdelivr.net/gh/MetaMask/brand-resources/SVG/metamask-fox.svg" alt
              <span>MetaMask (EVM)</span>
            </button>
            <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
              <img src="https://avatars.githubusercontent.com/u/1885080?s=200&v=4" alt="" width="28" h
              <span>Coinbase Wallet (EVM)</span>
            </button>
            <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
              <img src="https://avatars.githubusercontent.com/u/78782331?s=200&v=4" alt="" width="28"
              <span>Phantom (Solana)</span>
            </button>
            <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
              <img src="https://avatars.githubusercontent.com/u/64856450?s=200&v=4" alt="" width="28"
              <span>Solflare (Solana)</span>
            </button>
            <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
              <img src="https://avatars.githubusercontent.com/u/21111808?s=200&v=4" alt="" width="28"
              <span>Zcash Wallet / Hardware</span>
            </button>
          </div>
          <div class="small text-secondary mt-3" id="wallet-help"></div>
        </div>
      </div>
    </div>
  </div>

  <!-- Vendor scripts -->
  <script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
  <script src="vendor/ethers/ethers.umd.min.js"></script>
  <script src="vendor/hls/hls.min.js"></script>
  <script src="vendor/ipfs/index.min.js"></script>
  <!-- App scripts -->
  <script src="Scripts/wallets.js"></script>
  <script src="Scripts/common.js"></script>
  <script src="Scripts/main.js"></script>

</body>
```

```
</html>
```

`profile.html` (31.2 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Profile — Michie Stream Platform</title>
  <link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
  <link rel="apple-touch-icon" href="assets/msp-vinyl-180.png">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#F5F000">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>
/* ── Quick-link cards ──────────────────────────────────────────────── */
.profile-quick-links {
  display: grid;
  gap: 12px;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  margin-bottom: 32px;
}
.quick-link-card {
  align-items: flex-start;
  background: var(--bg-surface);
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  display: flex;
  flex-direction: column;
  gap: 6px;
  padding: 16px;
  text-decoration: none;
  transition: border-color .15s, transform .12s;
}
.quick-link-card:hover { border-color: var(--border-mid); transform: translateY(-2px); }
.quick-link-icon { font-size: 22px; }
.quick-link-label {
  color: var(--text-primary);
  font-size: 13px;
  font-weight: 700;
}
.quick-link-sub { color: var(--text-secondary); font-size: 11px; }
.quick-link-card.ql-teal   { border-left: 3px solid var(--teal); }
```

```
.quick-link-card.ql-gold   { border-left: 3px solid var(--gold); }
.quick-link-card.ql-ember  { border-left: 3px solid var(--ember); }
.quick-link-card.ql-violet { border-left: 3px solid var(--violet); }

/* ── Recent uploads mini-list ──────────────────────────────────────── */
.recent-asset-row {
  align-items: center;
  border-bottom: 1px solid var(--border-subtle);
  display: flex;
  gap: 10px;
  padding: 8px 0;
}
.recent-asset-row:last-child { border-bottom: none; }
.recent-asset-cover {
  border-radius: 4px;
  flex-shrink: 0;
  height: 36px;
  object-fit: cover;
  width: 36px;
}
.recent-asset-cover-ph {
  align-items: center;
  background: var(--bg-raised);
  border-radius: 4px;
  color: var(--text-muted);
  display: flex;
  flex-shrink: 0;
  font-size: 14px;
  height: 36px;
  justify-content: center;
  width: 36px;
}
.recent-asset-info { flex: 1; min-width: 0; }
.recent-asset-title {
  color: var(--text-primary);
  font-size: 12px;
  font-weight: 600;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.recent-asset-meta { color: var(--text-secondary); font-size: 10px; }
.recent-asset-play {
  background: none;
  border: none;
  color: var(--teal);
  cursor: pointer;
```

```
  flex-shrink: 0;
  font-size: 13px;
  padding: 3px 6px;
}
.recent-asset-plays {
  color: var(--text-muted);
  flex-shrink: 0;
  font-family: var(--font-mono);
  font-size: 10px;
  white-space: nowrap;
}
</style>
<style>
/* ── MSP Neon Banner ──────────────────────────────────────── */
.neon-banner {
  position: relative;
  z-index: 90;
  display: flex;
  align-items: center;
  width: 100%;
  padding: 14px 8px;
  background: rgba(8,8,10,0.6);
  border-bottom: 1px solid rgba(212,168,83,0.15);
  overflow: hidden;
}

.neon-banner .b-side {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 7px;
  flex-shrink: 0;
  width: 80px;
}

.neon-banner .b-slogan {
  flex: 1;
  text-align: center;
}

.neon-banner .b-line {
  font-family: 'Syne', 'Arial Black', sans-serif;
  font-weight: 800;
  font-size: clamp(13px, 2vw, 22px);
  letter-spacing: .04em;
  line-height: 1.3;
  color: #F5F000;
```

```
  animation:
    neon-boot    2.4s ease-out forwards,
    neon-flicker 5s  ease-in-out infinite 2.4s;
}

.neon-banner .b-line:last-child {
  animation:
    neon-boot    2.4s ease-out .15s forwards,
    neon-flicker 5s  ease-in-out infinite 2.55s;
}

.neon-banner svg.b-ic {
  opacity: 0;
  animation: b-icon-in 0.4s ease-out forwards, b-fl 3s ease-in-out infinite;
}

@keyframes b-icon-in { 0%{opacity:0;} 100%{opacity:.85;} }
@keyframes b-fl { 0%,100%{transform:translateY(0) scale(1);opacity:.85;} 50%{transform:translateY(-4px

@keyframes neon-boot {
  0%   {opacity:0;   text-shadow:none;}
  8%   {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8);}
  10%  {opacity:.1;  text-shadow:none;}
  14%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 20px #F5F000,0 0 40px rgba(245,240,0,.9);}
  16%  {opacity:.3;  text-shadow:none;}
  20%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 12px #F5F000;}
  22%  {opacity:.05; text-shadow:none;}
  28%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
  30%  {opacity:.6;  text-shadow:none;}
  36%  {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
  100% {opacity:1;   text-shadow:0 0 4px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
}

@keyframes neon-flicker {
  0%,18%,20%,22%,24%,53%,55%,100% {
    text-shadow:0 0 4px #F5F000,0 0 12px #F5F000,0 0 28px rgba(245,240,0,.8),0 0 60px rgba(245,240,0,.
    opacity:1;
  }
  19%,23%,54%{text-shadow:none;opacity:.82;}
}
</style>
</head>
<body style="padding-top:70px;" class="page-profile">

  <!-- ═══════ NAVBAR ═══════ -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
    <div class="container-fluid">
```

```
    <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNa
      aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="listen.html">Just Listen</a></li>
        <li class="nav-item"><a class="nav-link" href="creators.html">Creators Corner</a></li>
        <li class="nav-item"><a class="nav-link" href="marketplace.html">NFT Marketplace</a></li>
        <li class="nav-item"><a class="nav-link active" aria-current="page" href="profile.html">Prof
        <li class="nav-item" data-requires="hostConcert" style="display:none;"><a class="nav-link" h
      </ul>
      <div class="d-flex align-items-center gap-2">
        <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
        <button id="btn-disconnect" class="btn btn-outline-light btn-sm d-none">Disconnect</button>
        <span id="walletAddress" class="small text-light wallet-address-display"></span>
        <span class="small text-warning fw-semibold user-name-display"></span>
      </div>
    </div>
  </div>
</nav>
<!-- ═══ MSP NEON BANNER ═══════════════════════════════ -->
<div class="neon-banner">

  <!-- Left icons -->
  <div class="b-side">
    <svg class="b-ic" style="animation-delay:.1s,0s;width:24px;height:20px;filter:drop-shadow(0 0 5p
      <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
      <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
      <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
    </svg>
    <svg class="b-ic" style="animation-delay:.4s,.35s;width:36px;height:24px;filter:drop-shadow(0 0
      <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
      <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
      <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
      <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
    </svg>
    <svg class="b-ic" style="animation-delay:.7s,.7s;width:26px;height:18px;filter:drop-shadow(0 0 5
      <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
      <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
      <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
    </svg>
    <svg class="b-ic" style="animation-delay:1.05s,1.05s;width:24px;height:20px;filter:drop-shadow(0
```

```
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.4s,1.4s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
  </svg>
  <svg class="b-ic" style="animation-delay:1.75s,1.75s;width:38px;height:26px;filter:drop-shadow(0
    <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
    <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
    <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
    <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
    <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:2.1s,2.1s;width:26px;height:18px;filter:drop-shadow(0 0
    <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
    <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
    <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
  </svg>
</div>

<!-- Slogan -->
<div class="b-slogan">
  <div class="b-line">Put the Needle on the Record</div>
  <div class="b-line">That is on a Blockchain</div>
</div>

<!-- Right icons (staggered offsets) -->
<div class="b-side">
  <svg class="b-ic" style="animation-delay:.2s,.2s;width:24px;height:20px;filter:drop-shadow(0 0 5
    <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="4.5" stroke="#00D4BB" stroke-width="1.7"/>
    <circle cx="14" cy="15" r="1.8" fill="#00D4BB"/>
    <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#00D4BB" stroke-widt
    <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#00D4BB"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.55s,.55s;width:36px;height:24px;filter:drop-shadow(0 0
    <path d="M10 22V10l24-4v12" stroke="#F5F000" stroke-width="1.8" fill="none" stroke-linecap="ro
    <line x1="10" y1="10" x2="34" y2="6" stroke="#F5F000" stroke-width="1.8"/>
    <circle cx="7.5" cy="22" r="3.2" fill="#F5F000"/>
    <circle cx="31.5" cy="18" r="3.2" fill="#F5F000"/>
  </svg>
  <svg class="b-ic" style="animation-delay:.9s,.9s;width:26px;height:18px;filter:drop-shadow(0 0 5
```

```
      <rect x="2" y="5" width="18" height="13" rx="2" stroke="#E85D3A" stroke-width="1.7"/>
      <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#E85D3A" stroke-width="1.7" fill="none" stroke-line
      <circle cx="8" cy="11" r="2" fill="#E85D3A" opacity=".6"/>
    </svg>
    <svg class="b-ic" style="animation-delay:1.25s,1.25s;width:24px;height:20px;filter:drop-shadow(0
      <rect x="2" y="7" width="24" height="15" rx="2.5" stroke="#8B5CF6" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="4.5" stroke="#8B5CF6" stroke-width="1.7"/>
      <circle cx="14" cy="15" r="1.8" fill="#8B5CF6"/>
      <path d="M10 7V5.5C10 4.7 10.7 4 11.5 4h5C17.3 4 18 4.7 18 5.5V7" stroke="#8B5CF6" stroke-widt
      <rect x="20" y="10" width="3" height="2.5" rx=".6" fill="#8B5CF6"/>
    </svg>
    <svg class="b-ic" style="animation-delay:1.6s,1.6s;width:26px;height:18px;filter:drop-shadow(0 0
      <rect x="2" y="5" width="18" height="13" rx="2" stroke="#00D4BB" stroke-width="1.7"/>
      <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#00D4BB" stroke-width="1.7" fill="none" stroke-line
      <circle cx="8" cy="11" r="2" fill="#00D4BB" opacity=".6"/>
    </svg>
    <svg class="b-ic" style="animation-delay:1.95s,1.95s;width:38px;height:26px;filter:drop-shadow(0
      <path d="M8 24V11l32-5v13" stroke="#F5F000" stroke-width="1.7" fill="none" stroke-linecap="rou
      <line x1="8" y1="11" x2="40" y2="6" stroke="#F5F000" stroke-width="1.7"/>
      <circle cx="5.5" cy="24" r="3" fill="#F5F000"/>
      <circle cx="21" cy="19.5" r="3" fill="#F5F000"/>
      <circle cx="37" cy="19.5" r="3" fill="#F5F000"/>
    </svg>
    <svg class="b-ic" style="animation-delay:2.3s,2.3s;width:26px;height:18px;filter:drop-shadow(0 0
      <rect x="2" y="5" width="18" height="13" rx="2" stroke="#8B5CF6" stroke-width="1.7"/>
      <path d="M20 8.5l8-4v13l-8-4V8.5z" stroke="#8B5CF6" stroke-width="1.7" fill="none" stroke-line
      <circle cx="8" cy="11" r="2" fill="#8B5CF6" opacity=".6"/>
    </svg>
  </div>

</div>
<!-- ═══ END NEON BANNER ═══════════════════════════════ -->


<!-- ═══════ PAGE CONTENT ═══════ -->
<div class="container py-5 mt-5">

  <!-- ── Account Overview ──────────────────────────────────────── -->
  <section class="mb-4">
    <div class="d-flex align-items-start gap-3 flex-wrap">
      <div class="flex-grow-1">
        <h2 class="mb-1">Your Profile</h2>
        <p id="profile-address" class="text-muted small mb-1 font-monospace"></p>
        <div class="d-flex flex-wrap gap-3 align-items-center mt-2">
          <span>
            <strong class="small">Account:</strong>
            <span id="account-type-display" class="ms-1 small">Connect wallet to load</span>
```

```
        </span>
        <span id="expiry-row">
          <strong class="small">Subscription:</strong>
          <span id="subscription-expiry" class="ms-1 small text-muted"></span>
        </span>
        <span id="fee-row" style="display:none;">
          <strong class="small">Platform fee:</strong>
          <span id="royalty-fee-rate" class="ms-1 badge bg-dark"></span>
        </span>
        <span id="platform-nft-badge" class="badge bg-warning text-dark" style="display:none;"></s
      </div>
    </div>
    <div class="text-end flex-shrink-0">
      <a href="listen.html#subscribe" class="btn btn-sm btn-outline-primary">Manage Subscription</
    </div>
  </div>
</section>

<!-- ── Quick Links ──────────────────────────────────────────────────── -->
<div class="profile-quick-links" id="profile-quick-links">

  <!-- Favorites — all users -->
  <a href="favorites.html" class="quick-link-card ql-ember">
    <span class="quick-link-icon">♥</span>
    <span class="quick-link-label">My Favorites</span>
    <span class="quick-link-sub" id="ql-fav-count">Manage your favorites lists</span>
  </a>

  <!-- Asset Manager — creator only -->
  <a href="asset-manager.html" class="quick-link-card ql-teal" data-requires="upload" style="displ
    <span class="quick-link-icon">🎵</span>
    <span class="quick-link-label">My Assets</span>
    <span class="quick-link-sub" id="ql-asset-count">Manage uploads, royalties &amp; privacy</span
  </a>

  <!-- Playlists — Tier 2+ -->
  <a href="favorites.html" class="quick-link-card ql-gold" data-requires="createPlaylist" style="d
    <span class="quick-link-icon">🎛</span>
    <span class="quick-link-label">My Playlists</span>
    <span class="quick-link-sub" id="ql-playlist-count">Create &amp; manage playlists</span>
  </a>

  <!-- NFT Profile — creators with NFTs -->
  <a href="marketplace.html" class="quick-link-card ql-violet" id="ql-nfts">
    <span class="quick-link-icon">🖼</span>
    <span class="quick-link-label">NFT Portfolio</span>
    <span class="quick-link-sub" id="ql-nft-count">View &amp; manage your NFTs</span>
```

```
  </a>

</div>

<!-- ── Recent Uploads ── (creator only) ──────────────────────────── -->
<section class="mb-5" data-requires="upload" id="recent-uploads-section" style="display:none;">
  <div class="d-flex align-items-center justify-content-between mb-3">
    <h5 class="mb-0" style="color:var(--teal);">🎵 Recent Uploads</h5>
    <a href="asset-manager.html" class="btn btn-sm btn-outline-secondary">View All →</a>
  </div>
  <div class="card" style="background:var(--bg-surface);border:1px solid var(--border-subtle);">
    <div class="card-body p-3" id="recent-uploads-list">
      <p class="text-muted small mb-0">Loading your uploads…</p>
    </div>
  </div>
</section>

<!-- ── Platform NFT ── ──────────────────────────────────────────── -->
<section id="claim-nft" class="mb-5">
  <div class="card border-warning">
    <div class="card-header bg-warning text-dark">
      <h5 class="mb-0">🎖 Platform NFT</h5>
    </div>
    <div class="card-body">
      <p>
        The Platform NFT grants creator capabilities with a <strong>1.5% royalty fee</strong>
        (vs 5% for Standard Creator). One-time purchase — $10,000 USD. Price floor enforced on-cha
      </p>
      <ul class="mb-3">
        <li><strong>Without subscription:</strong> passive only — upload, mint, set splits, collec
        <li><strong>With subscription ($14.99/mo):</strong> full active tools — concerts, ads, lis
        <li>If you already hold the Platform NFT in your wallet, click below to activate it on you
      </ul>
      <button id="claim-platform-nft-btn" class="btn btn-warning text-dark">Claim Platform NFT</bu
      <p class="small text-muted mt-2">
        The NFT must already be in your connected wallet. Purchase is handled on the
        <a href="marketplace.html">NFT Marketplace</a>.
      </p>
    </div>
  </div>
</section>

<!-- ── Your NFTs ── ──────────────────────────────────────────────── -->
<section class="mb-5">
  <h5 class="mb-3">Your NFTs</h5>
  <div class="row" id="user-nfts">
    <p class="text-muted small">Connect wallet to load your NFTs.</p>
```

```
    </div>
  </section>

  <!-- ── DJ Settings ── ──────────────────────────────────────────────── -->
  <section class="mb-5">
    <div class="card" style="background:var(--bg-surface);border:1px solid var(--border-subtle);">
      <div class="card-header">
        <h5 class="mb-0">🎧 DJ Settings</h5>
      </div>
      <div class="card-body">
        <p class="text-muted small mb-3">
          Control whether tips are accepted by default on your DJ sets.
          3% of all tips go to the platform; you set your own split of the remaining 97%.
        </p>
        <div class="form-check form-switch">
          <input class="form-check-input" type="checkbox" role="switch" id="dj-tips-default-toggle"
          <label class="form-check-label small" for="dj-tips-default-toggle">Accept tips by default
        </div>
      </div>
    </div>
  </section>

  <!-- ── Supporter Sub-Account ── (creator only) ──────────────────────── -->
  <section class="mb-5" data-requires="supporterSub">
    <div class="card" style="background:var(--bg-surface);border:1px solid var(--border-subtle);">
      <div class="card-header">
        <h5 class="mb-0">👥 Supporter Sub-Account</h5>
      </div>
      <div class="card-body">
        <p class="text-muted small mb-3">
          Attach one supporter sub-account to your creator account.
          The sub-account inherits your subscription level and can stream all content,
          buy NFTs, join concerts, create playlists, earn royalty shares —
          but cannot stream your own creator content or upload/mint/host lives.
        </p>
        <div class="form-check form-switch mb-2">
          <input class="form-check-input" type="checkbox" role="switch" id="supporter-subaccount-tog
          <label class="form-check-label small" for="supporter-subaccount-toggle">Enable Supporter S
        </div>
        <p id="supporter-subaccount-status" class="small text-muted mb-0"></p>
      </div>
    </div>
  </section>

</div><!-- /container -->

<!-- ═══════ WALLET MODAL ═══════ -->
```

```
<div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidde
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content bg-dark text-light">
      <div class="modal-header border-0">
        <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="
      </div>
      <div class="modal-body pt-0">
        <div class="list-group">
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://cdn.jsdelivr.net/gh/MetaMask/brand-resources/SVG/metamask-fox.svg" alt
            <span>MetaMask (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/1885080?s=200&v=4" alt="" width="28" h
            <span>Coinbase Wallet (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/78782331?s=200&v=4" alt="" width="28"
            <span>Phantom (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/64856450?s=200&v=4" alt="" width="28"
            <span>Solflare (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id=
            <img src="https://avatars.githubusercontent.com/u/21111808?s=200&v=4" alt="" width="28"
            <span>Zcash Wallet / Hardware</span>
          </button>
        </div>
        <div class="small text-secondary mt-3" id="wallet-help"></div>
      </div>
    </div>
  </div>
</div>

<!-- Vendor scripts -->
<script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
<script src="vendor/ethers/ethers.umd.min.js"></script>
<script src="vendor/hls/hls.min.js"></script>
<script src="vendor/ipfs/index.min.js"></script>
<!-- App scripts -->
<script src="Scripts/wallets.js"></script>
<script src="Scripts/common.js"></script>
<script src="Scripts/favorites.js"></script>
<script src="Scripts/main.js"></script>
```

```
<script>
(function () {
  'use strict';

  // Fee row observer (unchanged)
  var feeEl  = document.getElementById('royalty-fee-rate');
  var feeRow = document.getElementById('fee-row');
  if (feeEl && feeRow) {
    new MutationObserver(function () {
      feeRow.style.display = feeEl.textContent.trim() ? '' : 'none';
    }).observe(feeEl, { childList: true, characterData: true, subtree: true });
  }

  // ── Profile boot — runs after wallet connects ──────────────────────
  function bootProfile() {
    var F = window.MSPFavorites;

    // ── Favorites count chip ──────────────────────────────────────────
    var qlFavCount = document.getElementById('ql-fav-count');
    if (qlFavCount && F) {
      var total = F.totalCount();
      qlFavCount.textContent = total > 0
        ? total + ' item' + (total !== 1 ? 's' : '') + ' saved'
        : 'Manage your favorites lists';
    }

    // ── Show creator quick links ──────────────────────────────────────
    var qlAssets   = document.getElementById('ql-assets');
    var qlPlaylists = document.getElementById('ql-playlists');
    var recentSec  = document.getElementById('recent-uploads-section');

    // CAN.upload() is set by main.js capability gates
    if (typeof window.CAN !== 'undefined') {
      if (CAN.upload && CAN.upload()) {
        if (qlAssets)  qlAssets.style.display   = '';
        if (recentSec) recentSec.style.display  = '';
        loadRecentUploads();
      }
      if (CAN.createPlaylist && CAN.createPlaylist()) {
        if (qlPlaylists) qlPlaylists.style.display = '';
      }
    }
  }

  // ── Load recent uploads from /api/catalog filtered by wallet ────────
  async function loadRecentUploads() {
    var listEl = document.getElementById('recent-uploads-list');
```

```
if (!listEl || !window.walletAddress) return;

try {
  var r = await fetch('/api/catalog');
  if (!r.ok) throw new Error('catalog fetch failed');
  var all = await r.json();

  // Filter to this wallet's uploads
  var mine = all.filter(function (item) {
    return (item.wallet || '').toLowerCase() === window.walletAddress.toLowerCase();
  }).slice(0, 5); // show 5 most recent

  var assetCountEl = document.getElementById('ql-asset-count');
  if (assetCountEl) {
    var total = all.filter(function (item) {
      return (item.wallet || '').toLowerCase() === window.walletAddress.toLowerCase();
    }).length;
    assetCountEl.textContent = total + ' asset' + (total !== 1 ? 's' : '') + ' uploaded';
  }

  if (!mine.length) {
    listEl.innerHTML = '<p class="text-muted small mb-0">No uploads yet. ' +
      '<a href="creators.html" style="color:var(--teal);">Upload your first track →</a></p>';
    return;
  }

  listEl.innerHTML = mine.map(function (item) {
    var typeIcon = { music:'🎵', podcast:'🎙', video:'🎬', art_still:'🖼', art_animated:'🎨' };
    var icon = typeIcon[item.contentType] || '🎵';
    var cover = item.coverUrl
      ? '<img class="recent-asset-cover" src="' + item.coverUrl + '" alt="">'
      : '<div class="recent-asset-cover-ph">' + icon + '</div>';
    var royaltyBadge = item.supporterRoyaltyEnabled
      ? '<img src="assets/msp-vinyl.svg" width="12" height="12" style="vertical-align:middle;mar
      : '';
    var plays = item.plays !== undefined ? item.plays : '—';
    return '<div class="recent-asset-row">' +
      cover +
      '<div class="recent-asset-info">' +
        '<div class="recent-asset-title">' + _esc(item.title) + royaltyBadge + '</div>' +
        '<div class="recent-asset-meta">' + _esc(item.artistName || '—') + ' · ' + (item.content
      '</div>' +
      '<span class="recent-asset-plays" title="Plays">▶ ' + plays + '</span>' +
      '<button class="recent-asset-play" title="Preview"' +
        ' data-hlsurl="' + (item.hlsUrl || '') + '"' +
        ' data-title="' + _esc(item.title) + '"' +
        ' data-artist="' + _esc(item.artistName || '') + '"' +
```

```
            ' data-cover="' + (item.coverUrl || '') + '">▶</button>' +
          '<a href="asset-manager.html#' + item.contentId + '" class="btn btn-sm btn-link p-0" style
        '</div>';
      }).join('');

      // Wire preview buttons
      listEl.querySelectorAll('.recent-asset-play').forEach(function (btn) {
        btn.addEventListener('click', function () {
          if (typeof window.playHls === 'function') {
            window.playHls(btn.dataset.hlsurl, '');
            var tn = document.getElementById('track-name');
            var an = document.getElementById('player-artist-name');
            if (tn) tn.textContent = btn.dataset.title || '';
            if (an) an.textContent = btn.dataset.artist || '';
          }
        });
      });

    } catch (err) {
      if (listEl) listEl.innerHTML = '<p class="text-muted small mb-0">Could not load uploads.</p>';
    }
  }

  function _esc(str) {
    return String(str || '').replace(/&/g,'&amp;').replace(/"/g,'&quot;').replace(/</g,'&lt;').repla
  }

  // Run after wallet connects (main.js fires walletConnected)
  document.addEventListener('walletConnected', function () {
    setTimeout(bootProfile, 400); // slight delay so main.js capability gates run first
  });
  if (window.walletAddress) setTimeout(bootProfile, 400);

 })();
 </script>

</body>
</html>
```

`dashboard.html` (15.6 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard — Michie Stream Platform</title>
  <link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#F5F000">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>
body { padding-bottom: 40px; }

/* ── Dashboard grid ──────────────────────────────────────────────── */
.dash-grid {
  display: grid;
  gap: 16px;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  margin-bottom: 32px;
}

/* ── Stat cards ──────────────────────────────────────────────────── */
.stat-card {
  background: var(--bg-surface);
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  padding: 20px;
  transition: border-color .15s;
}
.stat-card:hover { border-color: var(--border-mid); }
.stat-card-label {
  color: var(--text-muted);
  font-family: var(--font-mono);
  font-size: 10px;
  letter-spacing: .1em;
  margin-bottom: 8px;
  text-transform: uppercase;
}
.stat-card-value {
  color: var(--text-primary);
  font-family: var(--font-mono);
  font-size: 28px;
  font-weight: 700;
  line-height: 1;
}
.stat-card-value.gold   { color: var(--gold); }
.stat-card-value.teal   { color: var(--teal); }
.stat-card-value.ember  { color: var(--ember); }
.stat-card-value.violet { color: var(--violet); }
.stat-card-sub {
```

```
  color: var(--text-secondary);
  font-size: 11px;
  margin-top: 6px;
}
.stat-card-link {
  color: var(--teal);
  font-size: 11px;
  margin-top: 8px;
  text-decoration: none;
}
.stat-card-link:hover { text-decoration: underline; }

/* ── Section panels ──────────────────────────────────────────────── */
.dash-panel {
  background: var(--bg-surface);
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  margin-bottom: 20px;
  overflow: hidden;
}
.dash-panel-head {
  align-items: center;
  border-bottom: 1px solid var(--border-subtle);
  display: flex;
  gap: 10px;
  justify-content: space-between;
  padding: 14px 18px;
}
.dash-panel-title {
  color: var(--text-primary);
  font-family: var(--font-ui);
  font-size: 13px;
  font-weight: 700;
}
.dash-panel-body { padding: 0 18px 14px; }

/* ── Royalty earnings chart bar ──────────────────────────────────── */
.earnings-bar-wrap {
  align-items: center;
  display: flex;
  gap: 10px;
  margin: 10px 0 6px;
}
.earnings-bar-label {
  color: var(--text-secondary);
  font-size: 11px;
  min-width: 60px;
```

```
  text-align: right;
}
.earnings-bar-track {
  background: var(--bg-raised);
  border-radius: 3px;
  flex: 1;
  height: 6px;
  overflow: hidden;
}
.earnings-bar-fill {
  background: var(--gold);
  border-radius: 3px;
  height: 100%;
  transition: width .4s ease-out;
}
.earnings-bar-val {
  color: var(--gold);
  font-family: var(--font-mono);
  font-size: 10px;
  min-width: 60px;
}

/* ── Recent activity feed ──────────────────────────────────────────── */
.activity-item {
  align-items: flex-start;
  border-bottom: 1px solid var(--border-subtle);
  display: flex;
  gap: 10px;
  padding: 10px 0;
}
.activity-item:last-child { border-bottom: none; }
.activity-dot {
  border-radius: 50%;
  flex-shrink: 0;
  height: 7px;
  margin-top: 5px;
  width: 7px;
}
.activity-text { color: var(--text-secondary); flex: 1; font-size: 12px; line-height: 1.5; }
.activity-text strong { color: var(--text-primary); font-weight: 600; }
.activity-time { color: var(--text-muted); flex-shrink: 0; font-family: var(--font-mono); font-size: 1

/* ── Playlist mini-rows ────────────────────────────────────────────── */
.pl-row {
  align-items: center;
  border-bottom: 1px solid var(--border-subtle);
  display: flex;
```

```
  gap: 10px;
  padding: 9px 0;
}
.pl-row:last-child { border-bottom: none; }
.pl-name   { color: var(--text-primary); flex: 1; font-size: 13px; font-weight: 600; }
.pl-meta   { color: var(--text-secondary); font-size: 11px; }
.pl-badge  { border-radius: 4px; font-size: 9px; font-weight: 700; padding: 2px 6px; text-transform: u
.pl-badge.pub  { background: rgba(0,212,187,.12); color: var(--teal); }
.pl-badge.priv { background: rgba(139,92,246,.12); color: var(--violet); }

/* ── Notification banner ──────────────────────────────────────────── */
.dash-notice {
  align-items: flex-start;
  background: rgba(232,93,58,.08);
  border: 1px solid rgba(232,93,58,.25);
  border-radius: 8px;
  display: flex;
  gap: 10px;
  margin-bottom: 16px;
  padding: 12px 14px;
}
.dash-notice-icon { color: var(--ember); flex-shrink: 0; font-size: 16px; }
.dash-notice-text { color: var(--text-secondary); font-size: 12px; line-height: 1.5; }
.dash-notice-text strong { color: var(--ember); }
.dash-notice-dismiss {
  background: none; border: none; color: var(--text-muted);
  cursor: pointer; flex-shrink: 0; font-size: 14px; padding: 0;
}

/* ── Wallet prompt ────────────────────────────────────────────────── */
#dash-wallet-prompt {
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  color: var(--text-secondary);
  font-size: 13px;
  padding: 60px 20px;
  text-align: center;
}
</style>
</head>
<body class="page-dashboard">

<!-- ═══ NAVBAR ═══════════════════════════════════════════════════════ -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
  <div class="container-fluid">
    <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
```

```
      data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle n
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="listen.html">Just Listen</a></li>
        <li class="nav-item" id="nav-dashboard" style="display:none;"><a class="nav-link" href="dashbo
        <li class="nav-item"><a class="nav-link" href="creators.html">Creators Corner</a></li>
        <li class="nav-item"><a class="nav-link" href="marketplace.html">NFT Marketplace</a></li>
        <li class="nav-item"><a class="nav-link active" href="dashboard.html">Dashboard</a></li>
        <li class="nav-item"><a class="nav-link" href="profile.html">Profile</a></li>
        <li class="nav-item" data-requires="hostConcert" style="display:none;">
          <a class="nav-link" href="live_studio.html">🔴 Live Studio</a>
        </li>
      </ul>
      <div class="d-flex align-items-center gap-2">
        <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
        <button id="btn-disconnect" class="btn btn-outline-light btn-sm d-none">Disconnect</button>
        <span id="walletAddress" class="small text-light wallet-address-display"></span>
        <span class="small text-warning fw-semibold user-name-display"></span>
      </div>
    </div>
  </div>
</nav>

<!-- ═══ PAGE CONTENT ═══════════════════════════════════════════════════ -->
<div class="container py-4">

  <!-- Wallet not connected -->
  <div id="dash-wallet-prompt" style="display:none;">
    <p class="mb-3">Connect your wallet to view your dashboard.</p>
    <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
  </div>

  <!-- Main dashboard — hidden until wallet loads -->
  <div id="dash-main" style="display:none;">

    <!-- Page header -->
    <div class="d-flex align-items-center justify-content-between flex-wrap gap-2 mb-4">
      <div>
        <h1 style="font-family:var(--font-display);font-size:clamp(22px,3.5vw,32px);font-weight:300;">
          Dashboard
        </h1>
        <p class="text-muted small mb-0" id="dash-subtitle">Loading…</p>
      </div>
      <a href="profile.html" class="btn btn-sm btn-outline-secondary">View Profile →</a>
```

```
</div>

<!-- Notifications -->
<div id="dash-notices"></div>

<!-- ── Stat cards ──────────────────────────────────────────────── -->
<div class="dash-grid" id="dash-stats">
  <!-- populated by JS -->
</div>

<!-- ── Two-column layout: earnings + activity ────────────────────── -->
<div class="row g-4 mb-4">

  <!-- Royalty earnings breakdown -->
  <div class="col-lg-6" id="dash-earnings-col">
    <div class="dash-panel" id="dash-earnings-panel">
      <div class="dash-panel-head">
        <span class="dash-panel-title">💰 Royalty Earnings</span>
        <a href="asset-manager.html" class="btn btn-sm btn-outline-secondary" data-requires="uploa
      </div>
      <div class="dash-panel-body" id="dash-earnings-body">
        <p class="text-muted small py-3">No earnings data yet.</p>
      </div>
    </div>
  </div>

  <!-- Favorites summary -->
  <div class="col-lg-6">
    <div class="dash-panel">
      <div class="dash-panel-head">
        <span class="dash-panel-title">♥ Favorites</span>
        <a href="favorites.html" class="btn btn-sm btn-outline-secondary">Manage →</a>
      </div>
      <div class="dash-panel-body" id="dash-fav-body">
        <p class="text-muted small py-3">No favorites yet.</p>
      </div>
    </div>
  </div>

</div>

<!-- ── Playlists ─────────────────────────────────────────────────── -->
<div class="dash-panel" id="dash-playlists-panel" style="display:none;" data-requires="createPlayl
  <div class="dash-panel-head">
    <span class="dash-panel-title">🎛 My Playlists</span>
    <a href="favorites.html" class="btn btn-sm btn-outline-secondary">Create New →</a>
  </div>
```

```
      <div class="dash-panel-body" id="dash-playlists-body">
        <p class="text-muted small py-3">No playlists yet.</p>
      </div>
    </div>

    <!-- ── Recent uploads (creator only) ──────────────────────────────── -->
    <div class="dash-panel" id="dash-uploads-panel" style="display:none;" data-requires="upload">
      <div class="dash-panel-head">
        <span class="dash-panel-title">🎵 Recent Uploads</span>
        <div class="d-flex gap-2">
          <a href="creators.html" class="btn btn-sm btn-primary">+ Upload</a>
          <a href="asset-manager.html" class="btn btn-sm btn-outline-secondary">All Assets →</a>
        </div>
      </div>
      <div class="dash-panel-body" id="dash-uploads-body">
        <p class="text-muted small py-3">No uploads yet.</p>
      </div>
    </div>

    <!-- ── Subscription status ────────────────────────────────────────── -->
    <div class="dash-panel">
      <div class="dash-panel-head">
        <span class="dash-panel-title">📋 Subscription</span>
        <a href="listen.html#subscribe" class="btn btn-sm btn-outline-secondary">Manage →</a>
      </div>
      <div class="dash-panel-body py-3" id="dash-sub-body">
        <p class="text-muted small">Loading subscription…</p>
      </div>
    </div>

  </div><!-- /dash-main -->

</div><!-- /container -->

<!-- ═══ WALLET MODAL ═══════════════════════════════════════════════════ -->
<div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidden=
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content bg-dark text-light">
      <div class="modal-header border-0">
        <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Cl
      </div>
      <div class="modal-body pt-0">
        <div class="list-group">
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://cdn.jsdelivr.net/gh/MetaMask/brand-resources/SVG/metamask-fox.svg" alt="
            <span>MetaMask (EVM)</span>
```

```
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/1885080?s=200&v=4" alt="" width="28" hei
            <span>Coinbase Wallet (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/78782331?s=200&v=4" alt="" width="28" he
            <span>Phantom (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/64856450?s=200&v=4" alt="" width="28" he
            <span>Solflare (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/21111808?s=200&v=4" alt="" width="28" he
            <span>Zcash Wallet / Hardware</span>
          </button>
        </div>
        <div class="small text-secondary mt-3" id="wallet-help"></div>
      </div>
    </div>
  </div>
</div>

<!-- ═══ SCRIPTS ═══════════════════════════════════════════════════════════ -->
<script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
<script src="vendor/ethers/ethers.umd.min.js"></script>
<script src="vendor/hls/hls.min.js"></script>
<script src="vendor/ipfs/index.min.js"></script>
<script src="Scripts/wallets.js"></script>
<script src="Scripts/common.js"></script>
<script src="Scripts/favorites.js"></script>
<script src="Scripts/main.js"></script>
<script src="Scripts/dashboard.js"></script>

</body>
</html>
```

`favorites.html` (28.9 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <!-- Favicon -->
  <link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
  <link rel="apple-touch-icon" href="assets/msp-vinyl-180.png">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#F5F000">
  <title>My Favorites — Michie Stream Platform</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>

/* ────────────────────────────────────────────────────────────────────────────
   FAVORITES PAGE — SIGNAL Design System
──────────────────────────────────────────────────────────────────────────── */
body { padding-bottom: 40px; }

/* ── Page header ──────────────────────────────────────────────────────── */
.fav-page-header {
  align-items: center;
  border-bottom: 1px solid var(--border-subtle);
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin-bottom: 32px;
  padding-bottom: 20px;
}
.fav-page-title {
  color: var(--text-primary);
  flex: 1;
  font-family: var(--font-display);
  font-size: clamp(24px, 4vw, 36px);
  font-weight: 300;
  letter-spacing: .04em;
}
.fav-total-badge {
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 20px;
  color: var(--text-secondary);
  font-family: var(--font-mono);
  font-size: 11px;
  padding: 4px 14px;
}
.fav-total-badge span { color: var(--gold); font-weight: 700; }

/* ── Section groups ───────────────────────────────────────────────────── */
.fav-group {
  margin-bottom: 12px;
```

```
}
.fav-group-label {
  color: var(--text-muted);
  font-family: var(--font-mono);
  font-size: 9px;
  font-weight: 700;
  letter-spacing: .12em;
  margin-bottom: 8px;
  text-transform: uppercase;
}

/* ── Accordion section ──────────────────────────────────────────── */
.fav-section {
  background: var(--bg-surface);
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  margin-bottom: 8px;
  overflow: hidden;
  transition: border-color .15s;
}
.fav-section:has(.fav-section-body.open) {
  border-color: var(--border-mid);
}

.fav-section-head {
  align-items: center;
  cursor: pointer;
  display: flex;
  gap: 10px;
  padding: 14px 16px;
  user-select: none;
}
.fav-section-head:hover { background: var(--bg-raised); }

.fav-section-icon { flex-shrink: 0; font-size: 16px; }
.fav-section-label {
  flex: 1;
  font-family: var(--font-ui);
  font-size: 13px;
  font-weight: 700;
}
.fav-section-count {
  border-radius: 10px;
  font-family: var(--font-mono);
  font-size: 10px;
  font-weight: 700;
  min-width: 22px;
```

```
  padding: 2px 8px;
  text-align: center;
}
.fav-section-chevron {
  color: var(--text-muted);
  flex-shrink: 0;
  font-size: 12px;
  transition: transform .2s;
}
.fav-section-body.open ~ * .fav-section-chevron,
.fav-section-head[aria-expanded="true"] .fav-section-chevron {
  transform: rotate(180deg);
}

.fav-section-body {
  border-top: 1px solid var(--border-subtle);
  display: none;
  padding: 0 16px 12px;
}
.fav-section-body.open { display: block; }

/* ── Playlist action bar (inside section body for eligible types) ────────── */
.fav-playlist-bar {
  align-items: center;
  background: var(--bg-raised);
  border: 1px solid var(--border-subtle);
  border-radius: 8px;
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin: 12px 0;
  padding: 10px 14px;
}
.fav-playlist-bar input[type="text"] {
  background: var(--bg-surface);
  border: 1px solid var(--border-mid);
  border-radius: 5px;
  color: var(--text-primary);
  flex: 1;
  font-family: var(--font-ui);
  font-size: 12px;
  min-width: 140px;
  outline: none;
  padding: 5px 10px;
  transition: border-color .15s;
}
.fav-playlist-bar input[type="text"]:focus { border-color: var(--gold); }
```

```
.fav-playlist-bar input[type="text"]::placeholder { color: var(--text-muted); }

/* Public / Private toggle */
.fav-visibility-toggle {
  align-items: center;
  display: flex;
  gap: 6px;
}
.fav-vis-btn {
  background: var(--bg-surface);
  border: 1px solid var(--border-mid);
  border-radius: 5px;
  color: var(--text-muted);
  cursor: pointer;
  font-size: 11px;
  font-weight: 600;
  padding: 5px 10px;
  transition: all .12s;
}
.fav-vis-btn.active.public  { background: rgba(0,212,187,.15); border-color: var(--teal);  color: var(
.fav-vis-btn.active.private { background: rgba(139,92,246,.15); border-color: var(--violet); color: va

.fav-royalty-note {
  color: var(--text-muted);
  font-size: 10px;
  width: 100%;
}
.fav-royalty-note.public-mode { color: var(--gold); }

.fav-create-btn {
  background: var(--gold);
  border: none;
  border-radius: 6px;
  color: #000;
  cursor: pointer;
  font-size: 11px;
  font-weight: 700;
  letter-spacing: .04em;
  padding: 6px 16px;
  text-transform: uppercase;
  transition: background .12s, transform .1s;
  white-space: nowrap;
}
.fav-create-btn:hover { background: var(--gold-bright); transform: translateY(-1px); }
.fav-create-btn:disabled { opacity: .4; cursor: default; transform: none; }

/* ── Favorite item rows ──────────────────────────────────────────────── */
```

```
.fav-item-list { display: flex; flex-direction: column; }
.fav-item {
  align-items: center;
  border-bottom: 1px solid var(--border-subtle);
  display: flex;
  gap: 10px;
  padding: 9px 0;
}
.fav-item:last-child { border-bottom: none; }
.fav-item:hover { background: rgba(255,255,255,.01); }

.fav-item-dot {
  border-radius: 50%;
  flex-shrink: 0;
  height: 7px;
  width: 7px;
}
.fav-item-cover {
  border-radius: 4px;
  flex-shrink: 0;
  height: 36px;
  object-fit: cover;
  width: 36px;
}
.fav-item-cover-placeholder {
  align-items: center;
  background: var(--bg-raised);
  border-radius: 4px;
  color: var(--text-muted);
  display: flex;
  flex-shrink: 0;
  font-size: 16px;
  height: 36px;
  justify-content: center;
  width: 36px;
}
.fav-item-text { flex: 1; min-width: 0; }
.fav-item-label {
  color: var(--text-primary);
  font-size: 13px;
  font-weight: 600;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.fav-item-sub {
  color: var(--text-secondary);
```

```
  font-size: 11px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.fav-item-play {
  background: none;
  border: none;
  color: var(--teal);
  cursor: pointer;
  flex-shrink: 0;
  font-size: 13px;
  padding: 3px 7px;
  transition: color .12s, transform .1s;
}
.fav-item-play:hover { color: var(--teal-bright); transform: scale(1.1); }

.fav-item-remove {
  background: none;
  border: none;
  color: var(--text-muted);
  cursor: pointer;
  flex-shrink: 0;
  font-size: 13px;
  padding: 3px 6px;
  transition: color .12s;
}
.fav-item-remove:hover { color: var(--ember); }

/* Empty state per section */
.fav-empty {
  color: var(--text-muted);
  font-size: 12px;
  padding: 16px 0 8px;
  text-align: center;
}

/* ── Section color themes ──────────────────────────────────────────── */
.fav-section[data-color="ember"] .fav-section-label { color: var(--ember); }
.fav-section[data-color="ember"] .fav-section-count { background: rgba(232,93,58,.15); color: var(--em
.fav-section[data-color="gold"]  .fav-section-label { color: var(--gold); }
.fav-section[data-color="gold"]  .fav-section-count { background: rgba(212,168,83,.15); color: var(--g
.fav-section[data-color="teal"]  .fav-section-label { color: var(--teal); }
.fav-section[data-color="teal"]  .fav-section-count { background: rgba(0,212,187,.15); color: var(--te
.fav-section[data-color="violet"].fav-section-label { color: var(--violet); }
.fav-section[data-color="violet"].fav-section-count { background: rgba(139,92,246,.15); color: var(--v
```

```
/* ── Empty page state ──────────────────────────────────────────────── */
#fav-empty-state {
  border: 1px dashed var(--border-mid);
  border-radius: 12px;
  color: var(--text-muted);
  display: none;
  font-size: 14px;
  padding: 60px 20px;
  text-align: center;
}
#fav-empty-state a { color: var(--teal); }

/* ── Wallet prompt ─────────────────────────────────────────────────── */
#fav-wallet-prompt {
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  color: var(--text-secondary);
  font-size: 13px;
  padding: 40px 20px;
  text-align: center;
}
</style>
</head>
<body class="page-favorites">

<!-- ═══ NAVBAR ═══════════════════════════════════════════════════════ -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
  <div class="container-fluid">
    <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
      data-bs-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle n
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="listen.html">Just Listen</a></li>
        <li class="nav-item"><a class="nav-link" href="creators.html">Creators Corner</a></li>
        <li class="nav-item"><a class="nav-link" href="marketplace.html">NFT Marketplace</a></li>
        <li class="nav-item"><a class="nav-link active" href="favorites.html">♥ Favorites</a></li>
        <li class="nav-item"><a class="nav-link" href="profile.html">Profile</a></li>
        <li class="nav-item" data-requires="hostConcert" style="display:none;">
          <a class="nav-link" href="live_studio.html">🔴 Live Studio</a>
        </li>
      </ul>
      <div class="d-flex align-items-center gap-2">
```

```
        <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
        <button id="btn-disconnect" class="btn btn-outline-light btn-sm d-none">Disconnect</button>
        <span id="walletAddress" class="small text-light wallet-address-display"></span>
        <span class="small text-warning fw-semibold user-name-display"></span>
      </div>
    </div>
  </div>
</nav>

<!-- ═══ PAGE CONTENT ═══════════════════════════════════════════════════════ -->
<div class="container py-4">

  <!-- Page header -->
  <div class="fav-page-header">
    <h1 class="fav-page-title">♥ My Favorites</h1>
    <span class="fav-total-badge" id="fav-total-badge">
      <span id="fav-total-count">0</span> saved
    </span>
  </div>

  <!-- Wallet not connected -->
  <div id="fav-wallet-prompt" style="display:none;">
    <p class="mb-3">Connect your wallet to see your favorites.</p>
    <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
  </div>

  <!-- Empty state -->
  <div id="fav-empty-state">
    <p class="mb-2" style="font-size:32px;">♥</p>
    <p class="mb-3">You haven't favorited anything yet.</p>
    <p>Browse <a href="listen.html">music</a> or the <a href="marketplace.html">marketplace</a> and ta
  </div>

  <!-- ── LISTEN PAGE FAVORITES ──────────────────────────────────────────── -->
  <div id="fav-listen-group" class="fav-group" style="display:none;">
    <div class="fav-group-label">🎵 Listen — Music &amp; Audio</div>
    <div id="fav-sections-listen"></div>
  </div>

  <!-- ── MARKETPLACE FAVORITES ──────────────────────────────────────────── -->
  <div id="fav-market-group" class="fav-group" style="display:none;">
    <div class="fav-group-label">🖼 Marketplace — NFT Assets</div>
    <div id="fav-sections-market"></div>
  </div>

</div><!-- /container -->
```

```
<!-- ═══ WALLET MODAL ═══════════════════════════════════════════════════ -->
<div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidden=
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content bg-dark text-light">
      <div class="modal-header border-0">
        <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Cl
      </div>
      <div class="modal-body pt-0">
        <div class="list-group">
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://cdn.jsdelivr.net/gh/MetaMask/brand-resources/SVG/metamask-fox.svg" alt="
            <span>MetaMask (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/1885080?s=200&v=4" alt="" width="28" hei
            <span>Coinbase Wallet (EVM)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/78782331?s=200&v=4" alt="" width="28" he
            <span>Phantom (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/64856450?s=200&v=4" alt="" width="28" he
            <span>Solflare (Solana)</span>
          </button>
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
            <img src="https://avatars.githubusercontent.com/u/21111808?s=200&v=4" alt="" width="28" he
            <span>Zcash Wallet / Hardware</span>
          </button>
        </div>
        <div class="small text-secondary mt-3" id="wallet-help"></div>
      </div>
    </div>
  </div>
</div>

<!-- ═══ VENDOR + APP SCRIPTS ═══════════════════════════════════════════ -->
<script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
<script src="vendor/ethers/ethers.umd.min.js"></script>
<script src="vendor/hls/hls.min.js"></script>
<script src="vendor/ipfs/index.min.js"></script>
<script src="Scripts/wallets.js"></script>
<script src="Scripts/common.js"></script>
<script src="Scripts/favorites.js"></script>
<script src="Scripts/live_broadcast.js"></script>
```

```
<script src="Scripts/main.js"></script>

<!-- ═══ PAGE LOGIC ═══════════════════════════════════════════════════════ -->
<script>
(function () {
  'use strict';

  var F = window.MSPFavorites; // shorthand

  // ── HTML escape ──────────────────────────────────────────────────
  function esc(str) {
    return String(str || '').replace(/&/g,'&amp;').replace(/"/g,'&quot;').replace(/</g,'&lt;').replace
  }

  // ── Type config for each section — order determines render order ──────────
  var LISTEN_TYPES  = ['track','video','album','artist','dj','podcast','livestream','concert','nft_mus
  var MARKET_TYPES  = ['nft_artwork','nft_artist','nft_collection','nft_music','nft_video'];

  // ── Build one accordion section ──────────────────────────────────────────
  function buildSection(type, catalogData) {
    var meta    = F.FAV_TYPES[type];
    var ids     = F.getAll(type);
    var count   = ids.length;

    // Playlist bar HTML (only for eligible types)
    var playlistBar = '';
    if (meta.canPlaylist) {
      playlistBar =
        '<div class="fav-playlist-bar" id="pbar-' + type + '">' +
          '<input type="text" id="pname-' + type + '" placeholder="Name this playlist…" maxlength="80"
          '<div class="fav-visibility-toggle">' +
            '<button class="fav-vis-btn active public"  data-vis="public"  data-type="' + type + '">🌐
            '<button class="fav-vis-btn private"        data-vis="private" data-type="' + type + '">🔒
          '</div>' +
          '<button class="fav-create-btn" data-type="' + type + '"' + (count === 0 ? ' disabled' : '')
          '<p class="fav-royalty-note public-mode" id="rnote-' + type + '">' +
            '🟡 Public playlists earn you <strong>supporter royalties</strong> for plays of tracks ena
            'Look for the ' +
            '<img src="assets/msp-vinyl.svg" width="14" height="14" style="display:inline-block;vertic
            ' vinyl icon on tracks and videos — those are the ones that will earn you supporter royalt
          '</p>' +
        '</div>';
    }

    // Item rows
    var itemsHtml = '';
    if (!count) {
```

```
    itemsHtml = '<div class="fav-empty">Nothing here yet — tap ♥ on any ' + meta.label.toLowerCase()
  } else {
    itemsHtml = '<div class="fav-item-list">';
    ids.forEach(function (id) {
      // Try to look up richer data from catalog if available
      var item    = catalogData ? catalogData.find(function (c) { return c.contentId === id; }) : nu
      var label   = item ? (item.title || item.artistName || id.slice(0,20)) : id.slice(0, 24) + (id
      var sub     = item ? (item.artistName || '') : '';
      var coverUrl = item ? (item.coverUrl || '') : '';

      var cover = coverUrl
        ? '<img class="fav-item-cover" src="' + esc(coverUrl) + '" alt="">'
        : '<div class="fav-item-cover-placeholder">' + meta.icon + '</div>';

      var playBtn = (item && item.hlsUrl)
        ? '<button class="fav-item-play" data-hlsurl="' + esc(item.hlsUrl) + '" data-metaurl="' + es
        : '';

      itemsHtml +=
        '<div class="fav-item" data-fav-type="' + esc(type) + '" data-fav-id="' + esc(id) + '">' +
          '<span class="fav-item-dot" style="background:' + meta.cssColor + ';"></span>' +
          cover +
          '<div class="fav-item-text">' +
            '<div class="fav-item-label">' + esc(label) + '</div>' +
            (sub ? '<div class="fav-item-sub">' + esc(sub) + '</div>' : '') +
          '</div>' +
          playBtn +
          '<button class="fav-item-remove" data-fav-type="' + esc(type) + '" data-fav-id="' + esc(id
        '</div>';
    });
    itemsHtml += '</div>';
  }

  return '<div class="fav-section" data-color="' + meta.color + '" data-type="' + type + '" id="fsec
    '<div class="fav-section-head" data-type="' + type + '">' +
      '<span class="fav-section-icon">' + meta.icon + '</span>' +
      '<span class="fav-section-label">' + esc(meta.label) + '</span>' +
      '<span class="fav-section-count" id="fcnt-' + type + '">' + count + '</span>' +
      '<span class="fav-section-chevron">▾</span>' +
    '</div>' +
    '<div class="fav-section-body" id="fbody-' + type + '">' +
      playlistBar +
      itemsHtml +
    '</div>' +
  '</div>';
}
```

```
// ── Render all sections ──────────────────────────────────────────────
function renderAll(catalogData) {
  var listenEl = document.getElementById('fav-sections-listen');
  var marketEl = document.getElementById('fav-sections-market');
  var listenGroup = document.getElementById('fav-listen-group');
  var marketGroup = document.getElementById('fav-market-group');
  var emptyState  = document.getElementById('fav-empty-state');
  var totalEl     = document.getElementById('fav-total-count');

  if (listenEl) listenEl.innerHTML = LISTEN_TYPES.map(function (t) { return buildSection(t, catalogD
  if (marketEl) marketEl.innerHTML = MARKET_TYPES.map(function (t) { return buildSection(t, catalogD

  var total = F.totalCount();
  if (totalEl) totalEl.textContent = total;

  var listenHasAny = LISTEN_TYPES.some(function (t) { return F.countOf(t) > 0; });
  var marketHasAny = MARKET_TYPES.some(function (t) { return F.countOf(t) > 0; });
  if (listenGroup) listenGroup.style.display = listenHasAny ? '' : 'none';
  if (marketGroup) marketGroup.style.display = marketHasAny ? '' : 'none';
  if (emptyState)  emptyState.style.display  = (!listenHasAny && !marketHasAny) ? '' : 'none';

  wireAll();
}

// ── Wire accordion toggles ───────────────────────────────────────────
function wireAll() {
  // Accordion toggle
  document.querySelectorAll('.fav-section-head').forEach(function (head) {
    head.addEventListener('click', function () {
      var type = head.dataset.type;
      var body = document.getElementById('fbody-' + type);
      if (!body) return;
      var isOpen = body.classList.contains('open');
      body.classList.toggle('open', !isOpen);
      head.setAttribute('aria-expanded', String(!isOpen));
    });
  });

  // Remove buttons
  document.querySelectorAll('.fav-item-remove').forEach(function (btn) {
    btn.addEventListener('click', function (e) {
      e.stopPropagation();
      F.remove(btn.dataset.favType, btn.dataset.favId);
      // Re-render just the body of this section
      var sec  = btn.closest('.fav-section');
      var type = sec ? sec.dataset.type : null;
      if (!type) return;
```

```
    var body = document.getElementById('fbody-' + type);
    var wasOpen = body && body.classList.contains('open');
    renderAll(window._favCatalogCache);
    // Re-open the section that was open
    if (wasOpen && type) {
      var newBody = document.getElementById('fbody-' + type);
      if (newBody) newBody.classList.add('open');
    }
  });
});

// Play buttons
document.querySelectorAll('.fav-item-play').forEach(function (btn) {
  btn.addEventListener('click', function (e) {
    e.stopPropagation();
    if (typeof window.playHls === 'function') {
      window.playHls(btn.dataset.hlsurl, btn.dataset.metaurl || '');
      // Update player bar if elements exist
      var tn = document.getElementById('track-name');
      var an = document.getElementById('player-artist-name');
      var vi = document.getElementById('vinyl-icon');
      if (tn) { tn.textContent = btn.dataset.title || ''; tn.style.fontStyle = ''; }
      if (an) an.textContent = btn.dataset.artist || '';
      if (vi && btn.dataset.cover) vi.src = btn.dataset.cover;
    }
  });
});

// Visibility toggle buttons
document.querySelectorAll('.fav-vis-btn').forEach(function (btn) {
  btn.addEventListener('click', function () {
    var type    = btn.dataset.type;
    var vis     = btn.dataset.vis;
    var bar     = document.getElementById('pbar-' + type);
    if (!bar) return;
    bar.querySelectorAll('.fav-vis-btn').forEach(function (b) { b.classList.remove('active'); });
    btn.classList.add('active');
    var note = document.getElementById('rnote-' + type);
    var vinylSvg = '<img src="assets/msp-vinyl.svg" width="14" height="14" style="display:inline-b
    if (note) {
      note.classList.toggle('public-mode', vis === 'public');
      note.innerHTML = vis === 'public'
        ? '🟡 Public playlists earn you <strong>supporter royalties</strong> for plays of tracks e
        : '🔒 Private playlists do not earn royalties and are only visible to you.';
    }
  });
});
```

```
// Create playlist buttons
document.querySelectorAll('.fav-create-btn').forEach(function (btn) {
  btn.addEventListener('click', async function () {
    var type    = btn.dataset.type;
    var nameEl  = document.getElementById('pname-' + type);
    var bar     = document.getElementById('pbar-' + type);
    var name    = nameEl ? nameEl.value.trim() : '';
    if (!name) { if (nameEl) nameEl.focus(); return; }

    var visBtn  = bar ? bar.querySelector('.fav-vis-btn.active') : null;
    var isPublic = visBtn ? visBtn.dataset.vis === 'public' : true;
    var ids      = F.getAll(type);
    if (!ids.length) return;

    btn.disabled    = true;
    btn.textContent = 'Creating…';

    try {
      var res = await fetch('/api/favorites/convert-to-playlist', {
        method:  'POST',
        headers: { 'Content-Type': 'application/json' },
        body:    JSON.stringify({
          wallet:    window.walletAddress,
          name:      name,
          cids:      ids,
          isPublic:  isPublic,
          type:      type,
          royaltyEligible: isPublic, // backend gates actual royalties
        }),
      });
      var data = await res.json();
      if (!res.ok) throw new Error(data.error || 'Failed');
      btn.textContent = '✓ Created!';
      btn.style.background = 'var(--teal)';
      if (nameEl) nameEl.value = '';
      setTimeout(function () {
        btn.textContent = 'Create Playlist';
        btn.style.background = '';
        btn.disabled = false;
      }, 2500);
    } catch (err) {
      btn.textContent = 'Error: ' + err.message;
      btn.style.background = 'var(--ember)';
      setTimeout(function () {
        btn.textContent = 'Create Playlist';
        btn.style.background = '';
```

```
          btn.disabled = false;
        }, 3000);
      }
    });
  });
}

// ── Boot ──────────────────────────────────────────────────────────
async function boot() {
  if (!window.walletAddress) {
    var prompt = document.getElementById('fav-wallet-prompt');
    if (prompt) prompt.style.display = '';
    // Re-check when wallet connects
    document.addEventListener('walletConnected', function () {
      if (prompt) prompt.style.display = 'none';
      boot();
    });
    return;
  }

  // Fetch catalog for richer item display
  var catalog = [];
  try {
    var r = await fetch('/api/catalog');
    if (r.ok) catalog = await r.json();
  } catch (_) {}
  window._favCatalogCache = catalog;

  renderAll(catalog);

  // Auto-open sections that have items
  Object.keys(F.FAV_TYPES).forEach(function (type) {
    if (F.countOf(type) > 0) {
      var body = document.getElementById('fbody-' + type);
      if (body) body.classList.add('open');
    }
  });
}

// Wait for wallet module to fire before booting, but also try immediately
document.addEventListener('walletConnected', function (e) {
  boot();
});
// If wallet already connected (page reload with session)
if (window.walletAddress) boot();

})();
```

```
</script>

</body>
</html>
```

`live_studio.html` (75.1 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- Favicon -->
<link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
<link rel="apple-touch-icon" href="assets/msp-vinyl-180.png">
<link rel="manifest" href="manifest.json">
<title>MSP — Live Studio</title>
<!-- FIX #12: Added gstatic preconnect + display=swap already present in URL -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Cormorant:ital,wght@0,300;0,400;1,300&family=Syne
<!-- MSPLive broadcaster / viewer library -->
<script src="Scripts/live_broadcast.js"></script>
<style>

/* ── Design tokens ──────────────────────────────────────────── */
:root {
  --bg-void:      #08080A;
  --bg-base:      #0F0F12;
  --bg-surface:   #151518;
  --bg-raised:    #1C1C21;
  --bg-hover:     #222228;
  --border-sub:   #222228;
  --border-mid:   #2E2E36;
  --gold:         #D4A853;
  --gold-bright:  #F0C060;
  --teal:         #00D4BB;
  --ember:        #E85D3A;
  --violet:       #8B5CF6;
  --text-1:       #EEEAE4;
  --text-2:       #8A8A98;
  --text-3:       #484852;
  --font-d:       'Cormorant', Georgia, serif;
  --font-u:       'Syne', sans-serif;
  --font-m:       'Space Mono', monospace;
```

```
  --ease:         cubic-bezier(0.16, 1, 0.3, 1);
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
body {
  background: var(--bg-void);
  color: var(--text-1);
  font-family: var(--font-u);
  font-size: 14px;
  line-height: 1.6;
}

/* ═══════════════════════════════════════════════════════════
   BROADCASTER PANEL
   ═══════════════════════════════════════════════════════════ */

.live-studio {
  background: var(--bg-surface);
  border: 1px solid var(--border-sub);
  border-radius: 12px;
  overflow: hidden;
}

/* ── Access gate overlay ──────────────────────────────── */
/* Rendered INSTEAD of the studio body when the user lacks hostConcert */
.studio-gate {
  padding: 48px 32px;
  display: flex;
  flex-direction: column;
  align-items: center;
  text-align: center;
  gap: 16px;
}

.studio-gate-icon {
  font-size: 40px;
  line-height: 1;
  opacity: 0.5;
}

.studio-gate-title {
  font-family: var(--font-d);
  font-size: 26px;
  font-weight: 300;
  color: var(--text-1);
  line-height: 1.1;
}
```

```
.studio-gate-sub {
  font-family: var(--font-m);
  font-size: 11px;
  color: var(--text-3);
  letter-spacing: 0.08em;
  max-width: 360px;
  line-height: 1.7;
}

.studio-gate-pills {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  justify-content: center;
  margin: 4px 0;
}

.studio-gate-pill {
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 20px;
  padding: 5px 14px;
  font-family: var(--font-m);
  font-size: 10px;
  letter-spacing: 0.08em;
  color: var(--text-2);
}

.studio-gate-pill.active {
  border-color: var(--ember);
  color: var(--ember);
  background: rgba(232,93,58,0.08);
}

.studio-gate-btn {
  background: var(--ember);
  color: white;
  border: none;
  border-radius: 6px;
  padding: 13px 28px;
  font-family: var(--font-u);
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  cursor: pointer;
```

```
  transition: all 0.2s var(--ease);
  text-decoration: none;
  display: inline-block;
}

.studio-gate-btn:hover { background: #ff7050; transform: translateY(-1px); box-shadow: 0 8px 24px rgba

.studio-gate-btn.secondary {
  background: transparent;
  border: 1px solid var(--border-mid);
  color: var(--text-2);
  padding: 12px 24px;
}
.studio-gate-btn.secondary:hover { border-color: var(--teal); color: var(--teal); box-shadow: none; tr

/* ── Studio header ──────────────────────────────────────── */
.studio-header {
  padding: 20px 24px;
  border-bottom: 1px solid var(--border-sub);
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.studio-title {
  font-family: var(--font-d);
  font-size: 26px;
  font-weight: 300;
  color: var(--text-1);
  letter-spacing: -0.01em;
}

.studio-status {
  font-family: var(--font-m);
  font-size: 11px;
  letter-spacing: 0.1em;
  text-transform: uppercase;
  color: var(--text-3);
  display: flex;
  align-items: center;
  gap: 8px;
}

.studio-status.live { color: var(--ember); }

.status-dot {
  width: 8px; height: 8px;
```

```
  border-radius: 50%;
  background: var(--text-3);
  flex-shrink: 0;
}

.studio-status.live .status-dot {
  background: var(--ember);
  box-shadow: 0 0 0 3px rgba(232,93,58,0.25);
  animation: pulse-live 1.4s ease-in-out infinite;
}

@keyframes pulse-live {
  0%, 100% { box-shadow: 0 0 0 3px rgba(232,93,58,0.25); }
  50%       { box-shadow: 0 0 0 7px rgba(232,93,58,0.05); }
}

/* FIX #9: Spinner ──────────────────────────────────────────── */
.status-spinner {
  width: 10px; height: 10px;
  border: 2px solid rgba(232,93,58,0.3);
  border-top-color: var(--ember);
  border-radius: 50%;
  animation: spin 0.7s linear infinite;
  flex-shrink: 0;
  display: none;
}

.studio-status.spinning .status-spinner { display: block; }
.studio-status.spinning .status-dot     { display: none; }

@keyframes spin { to { transform: rotate(360deg); } }

/* ── Main studio layout ──────────────────────────────────── */
.studio-body {
  display: grid;
  grid-template-columns: 1fr 340px;
  min-height: 480px;
}

.studio-preview-area {
  background: #000;
  position: relative;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```
.studio-video-preview {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}

.studio-no-camera {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 12px;
  color: var(--text-3);
  font-family: var(--font-m);
  font-size: 12px;
  letter-spacing: 0.08em;
}

.studio-no-camera-icon {
  font-size: 48px;
  opacity: 0.4;
}

/* Camera overlay controls */
.studio-overlay-controls {
  position: absolute;
  bottom: 16px;
  left: 50%;
  transform: translateX(-50%);
  display: flex;
  gap: 12px;
  z-index: 10;
  opacity: 0;
  transition: opacity 0.2s;
}

.studio-preview-area:hover .studio-overlay-controls { opacity: 1; }
.studio-preview-area.live-active .studio-overlay-controls { opacity: 1; }

.ov-btn {
  width: 44px; height: 44px;
  border-radius: 50%;
  border: none;
  cursor: pointer;
  font-size: 18px;
  display: flex; align-items: center; justify-content: center;
  transition: all 0.2s var(--ease);
```

```
  background: rgba(0,0,0,0.65);
  color: white;
  backdrop-filter: blur(4px);
}

.ov-btn:hover { transform: scale(1.1); background: rgba(0,0,0,0.85); }
.ov-btn.muted, .ov-btn.cam-off { background: var(--ember); }

/* FIX #14: Bitrate/resolution stats bar (visible during live) */
.studio-bitrate-bar {
  position: absolute;
  bottom: 70px;
  left: 12px;
  background: rgba(0,0,0,0.7);
  backdrop-filter: blur(4px);
  border-radius: 5px;
  padding: 5px 10px;
  font-family: var(--font-m);
  font-size: 10px;
  color: var(--teal);
  letter-spacing: 0.04em;
  display: none;
  gap: 12px;
}

.studio-bitrate-bar.visible { display: flex; }
.bitrate-val { color: var(--text-1); }

/* HLS indicator */
.studio-hls-url {
  position: absolute;
  top: 12px; left: 12px; right: 12px;
  background: rgba(0,0,0,0.75);
  backdrop-filter: blur(4px);
  border-radius: 6px;
  padding: 8px 12px;
  font-family: var(--font-m);
  font-size: 10px;
  color: var(--teal);
  letter-spacing: 0.04em;
  display: none;
  word-break: break-all;
}

.studio-hls-url.visible { display: block; }

/* ── Side panel ──────────────────────────────────────────────── */
```

```
.studio-panel {
  border-left: 1px solid var(--border-sub);
  display: flex;
  flex-direction: column;
}

/* Setup form */
.studio-setup {
  padding: 20px;
  flex: 1;
  overflow-y: auto;
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.studio-setup.hidden { display: none; }

.s-label {
  font-family: var(--font-m);
  font-size: 10px;
  letter-spacing: 0.15em;
  text-transform: uppercase;
  color: var(--text-3);
  display: block;
  margin-bottom: 6px;
}

.s-input, .s-select {
  width: 100%;
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 5px;
  padding: 10px 13px;
  font-family: var(--font-u);
  font-size: 13px;
  color: var(--text-1);
  outline: none;
  transition: border-color 0.15s;
  appearance: none;
}

.s-input:focus, .s-select:focus { border-color: var(--ember); box-shadow: 0 0 0 3px rgba(232,93,58,0.1

/* Source picker */
.source-picker {
  display: grid;
```

```
  grid-template-columns: 1fr 1fr;
  gap: 8px;
}

.source-btn {
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 6px;
  padding: 10px 8px;
  cursor: pointer;
  font-family: var(--font-u);
  font-size: 12px;
  font-weight: 500;
  color: var(--text-2);
  transition: all 0.15s;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
}

.source-btn:hover { border-color: var(--ember); color: var(--text-1); }
.source-btn.active { border-color: var(--ember); color: var(--ember); background: rgba(232,93,58,0.08)

/* Go live button */
.btn-go-live {
  width: 100%;
  background: var(--ember);
  color: white;
  border: none;
  border-radius: 6px;
  padding: 14px;
  font-family: var(--font-u);
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  cursor: pointer;
  transition: all 0.2s var(--ease);
}

.btn-go-live:hover:not(:disabled) { background: #ff7050; transform: translateY(-1px); box-shadow: 0 8p
.btn-go-live:disabled { opacity: 0.5; cursor: not-allowed; }

/* Live stats (shown during stream) */
.studio-live-stats {
  padding: 16px 20px;
```

```
  border-bottom: 1px solid var(--border-sub);
  display: none;
  gap: 16px;
}

.studio-live-stats.visible {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

.ls-stat { text-align: center; }

.ls-val {
  font-family: var(--font-d);
  font-size: 22px;
  font-weight: 400;
  color: var(--text-1);
  display: block;
  line-height: 1;
}

.ls-lbl {
  font-family: var(--font-m);
  font-size: 9px;
  color: var(--text-3);
  letter-spacing: 0.12em;
  text-transform: uppercase;
  margin-top: 4px;
  display: block;
}

/* Live chat (shown during stream) */
.studio-live-chat {
  display: none;
  flex-direction: column;
  flex: 1;
  overflow: hidden;
}

.studio-live-chat.visible { display: flex; }

.live-chat-log {
  flex: 1;
  overflow-y: auto;
  padding: 12px 16px;
  display: flex;
  flex-direction: column;
```

```
  gap: 6px;
}

.live-chat-input-row {
  padding: 10px 16px;
  border-top: 1px solid var(--border-sub);
  display: flex;
  gap: 8px;
}

.live-chat-input {
  flex: 1;
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 4px;
  padding: 8px 11px;
  font-family: var(--font-u);
  font-size: 12px;
  color: var(--text-1);
  outline: none;
}

.live-chat-input:focus { border-color: var(--gold); }

.live-chat-send {
  background: var(--bg-hover);
  border: 1px solid var(--border-mid);
  border-radius: 4px;
  padding: 8px 12px;
  color: var(--text-2);
  cursor: pointer;
  font-size: 14px;
  transition: all 0.15s;
}

.live-chat-send:hover { color: var(--text-1); border-color: var(--gold); }

/* End stream button */
.studio-end-row {
  padding: 12px 16px;
  border-top: 1px solid var(--border-sub);
  display: none;
}

.studio-end-row.visible { display: block; }

.btn-end-stream {
```

```
  width: 100%;
  background: transparent;
  border: 1px solid var(--ember);
  border-radius: 5px;
  padding: 10px;
  color: var(--ember);
  font-family: var(--font-u);
  font-size: 12px;
  font-weight: 600;
  letter-spacing: 0.05em;
  text-transform: uppercase;
  cursor: pointer;
  transition: all 0.15s;
}

.btn-end-stream:hover { background: rgba(232,93,58,0.1); }

/* ═══════════════════════════════════════════════════════════
   VIEWER PANEL
   ═══════════════════════════════════════════════════════════ */

.live-sessions-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 14px;
  margin-top: 16px;
}

.live-card {
  background: var(--bg-raised);
  border: 1px solid var(--border-sub);
  border-radius: 10px;
  overflow: hidden;
  cursor: pointer;
  transition: all 0.25s var(--ease);
}

.live-card:hover {
  border-color: var(--ember);
  transform: translateY(-3px);
  box-shadow: 0 16px 40px rgba(232,93,58,0.15);
}

.live-thumb {
  height: 130px;
  background: #111;
  position: relative;
```

```
  overflow: hidden;
}

.live-thumb img { width: 100%; height: 100%; object-fit: cover; }

.live-badge {
  position: absolute;
  top: 8px; left: 8px;
  background: var(--ember);
  color: white;
  font-family: var(--font-m);
  font-size: 9px;
  letter-spacing: 0.1em;
  padding: 3px 7px;
  border-radius: 3px;
  animation: pulse-badge 1.4s ease-in-out infinite;
}

@keyframes pulse-badge { 0%,100%{opacity:1} 50%{opacity:0.75} }

.live-viewers {
  position: absolute;
  bottom: 8px; right: 8px;
  background: rgba(0,0,0,0.7);
  color: var(--text-1);
  font-family: var(--font-m);
  font-size: 10px;
  padding: 3px 7px;
  border-radius: 3px;
}

.live-info { padding: 12px 14px 8px; }
.live-title { font-weight: 600; font-size: 13px; margin-bottom: 3px; white-space: nowrap; overflow: hi
.live-artist { font-size: 11px; color: var(--text-2); }
.live-dur { font-family: var(--font-m); font-size: 10px; color: var(--text-3); margin-top: 4px; }
.live-join-btn {
  display: block;
  width: calc(100% - 28px);
  margin: 0 14px 12px;
  background: var(--ember);
  color: white;
  border: none;
  border-radius: 4px;
  padding: 8px;
  font-family: var(--font-u);
  font-size: 12px;
  font-weight: 600;
```

```
  letter-spacing: 0.05em;
  text-transform: uppercase;
  cursor: pointer;
  transition: background 0.15s;
}
.live-join-btn:hover { background: #ff7050; }
.live-empty { color: var(--text-3); font-family: var(--font-m); font-size: 12px; padding: 32px 0; text

/* ── Viewer player overlay ──────────────────────────────────── */
.viewer-overlay {
  position: fixed;
  inset: 0;
  background: rgba(8,8,10,0.96);
  z-index: 500;
  display: none;
  grid-template-columns: 1fr 360px;
  grid-template-rows: 1fr;
}

.viewer-overlay.open { display: grid; }

/* FIX #6: Mobile responsive viewer overlay */
@media (max-width: 900px) {
  .viewer-overlay {
    grid-template-columns: 1fr;
    grid-template-rows: auto 1fr;
  }
  .viewer-engage-col {
    border-left: none !important;
    border-top: 1px solid var(--border-sub);
    max-height: 240px;
  }
}

.viewer-video-col {
  display: flex;
  flex-direction: column;
  position: relative;
}

/* FIX #10: viewer video gets playsinline + muted for iOS autoplay */
.viewer-video {
  flex: 1;
  width: 100%;
  object-fit: contain;
  background: #000;
}
```

```
/* Stream stats bar */
.viewer-stats-bar {
  position: absolute;
  top: 0; left: 0; right: 0;
  padding: 12px 20px;
  background: linear-gradient(to bottom, rgba(8,8,10,0.85), transparent);
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.viewer-stream-title {
  font-family: var(--font-d);
  font-size: 20px;
  font-weight: 300;
  color: var(--text-1);
}

.viewer-live-pill {
  background: var(--ember);
  color: white;
  font-family: var(--font-m);
  font-size: 10px;
  letter-spacing: 0.1em;
  padding: 4px 10px;
  border-radius: 3px;
  animation: pulse-badge 1.4s ease-in-out infinite;
}

.viewer-meta-bar {
  display: flex;
  gap: 20px;
  align-items: center;
  font-family: var(--font-m);
  font-size: 10px;
  color: var(--text-2);
}

.viewer-meta-bar .viewer-count { color: var(--teal); }
.viewer-meta-bar .tips-total { color: var(--gold); }

/* Close viewer button */
.viewer-close {
  position: absolute;
  top: 12px; right: 12px;
  background: rgba(0,0,0,0.6);
```

```
  border: none;
  color: var(--text-2);
  width: 36px; height: 36px;
  border-radius: 50%;
  cursor: pointer;
  font-size: 18px;
  display: flex; align-items: center; justify-content: center;
  transition: all 0.15s;
  z-index: 10;
}

.viewer-close:hover { color: var(--text-1); background: rgba(0,0,0,0.85); }

/* FIX #13: Fullscreen button */
.viewer-fullscreen-btn {
  position: absolute;
  bottom: 80px; right: 16px;
  background: rgba(0,0,0,0.65);
  backdrop-filter: blur(4px);
  border: 1px solid rgba(255,255,255,0.08);
  border-radius: 8px;
  width: 38px; height: 38px;
  color: var(--text-2);
  cursor: pointer;
  font-size: 16px;
  display: flex; align-items: center; justify-content: center;
  transition: all 0.15s var(--ease);
  z-index: 10;
}
.viewer-fullscreen-btn:hover { color: var(--text-1); border-color: rgba(255,255,255,0.2); transform: s

/* Floating reactions layer */
.reactions-layer {
  position: absolute;
  inset: 0;
  pointer-events: none;
  overflow: hidden;
}

/* FIX #7: Floating reactions with organic x-offset + rotation */
.floating-reaction {
  position: absolute;
  bottom: 80px;
  font-size: 28px;
  animation: float-up 3s ease-out forwards;
  pointer-events: none;
  /* --rx is set inline by JS */
```

```
  transform-origin: center bottom;
}

@keyframes float-up {
  0%   { transform: translateY(0)      translateX(0)       rotate(0deg)   scale(1);   opacity: 1; }
  30%  { transform: translateY(-80px)  translateX(var(--rx, 0px))  rotate(var(--rr, 4deg))  scale(1.15
  70%  { transform: translateY(-160px) translateX(calc(var(--rx, 0px) * 1.4)) rotate(calc(var(--rr, 4d
  100% { transform: translateY(-240px) translateX(calc(var(--rx, 0px) * 1.8)) rotate(calc(var(--rr, 4d
}

/* Reaction buttons */
.reaction-bar {
  position: absolute;
  bottom: 20px;
  left: 50%;
  transform: translateX(-50%);
  display: flex;
  gap: 10px;
  align-items: center;
}

.react-btn {
  background: rgba(0,0,0,0.65);
  backdrop-filter: blur(4px);
  border: 1px solid rgba(255,255,255,0.08);
  border-radius: 50%;
  width: 44px; height: 44px;
  font-size: 20px;
  cursor: pointer;
  transition: all 0.15s var(--ease);
  display: flex; align-items: center; justify-content: center;
}

.react-btn:hover { transform: scale(1.2); border-color: rgba(255,255,255,0.2); }

/* Tip button */
.viewer-tip-btn {
  background: rgba(212,168,83,0.15);
  border: 1px solid var(--gold);
  border-radius: 22px;
  padding: 8px 16px;
  color: var(--gold);
  font-family: var(--font-u);
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  display: flex;
```

```
  align-items: center;
  gap: 6px;
  transition: all 0.15s;
}

.viewer-tip-btn:hover { background: rgba(212,168,83,0.25); }

/* FIX #5: Tip Modal ──────────────────────────────────────── */
.tip-modal-overlay {
  position: fixed;
  inset: 0;
  background: rgba(8,8,10,0.85);
  backdrop-filter: blur(8px);
  z-index: 700;
  display: none;
  align-items: center;
  justify-content: center;
  padding: 20px;
}

.tip-modal-overlay.open { display: flex; }

.tip-modal {
  background: var(--bg-surface);
  border: 1px solid var(--border-mid);
  border-radius: 14px;
  width: 100%;
  max-width: 380px;
  padding: 28px;
  box-shadow: 0 40px 100px rgba(0,0,0,0.7);
  animation: modal-in 0.25s var(--ease);
}

.tip-modal-header { margin-bottom: 20px; }
.tip-modal-eyebrow { font-family: var(--font-m); font-size: 9px; letter-spacing: 0.2em; text-transform
.tip-modal-title   { font-family: var(--font-d); font-size: 26px; font-weight: 300; color: var(--text-

.tip-presets {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 8px;
  margin-bottom: 16px;
}

.tip-preset-btn {
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
```

```
  border-radius: 6px;
  padding: 10px 4px;
  cursor: pointer;
  font-family: var(--font-m);
  font-size: 11px;
  color: var(--text-2);
  text-align: center;
  transition: all 0.15s;
}
.tip-preset-btn:hover,
.tip-preset-btn.selected { border-color: var(--gold); color: var(--gold); background: rgba(212,168,83,

.tip-custom-row { margin-bottom: 16px; }
.tip-custom-label { font-family: var(--font-m); font-size: 10px; letter-spacing: 0.12em; text-transfor
.tip-custom-input {
  width: 100%;
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 5px;
  padding: 10px 13px;
  font-family: var(--font-m);
  font-size: 14px;
  color: var(--gold);
  outline: none;
  transition: border-color 0.15s;
}
.tip-custom-input:focus { border-color: var(--gold); box-shadow: 0 0 0 3px rgba(212,168,83,0.1); }
.tip-custom-input::placeholder { color: var(--text-3); }

.tip-status {
  min-height: 18px;
  font-family: var(--font-m);
  font-size: 11px;
  margin-bottom: 14px;
  color: var(--text-3);
}
.tip-status.error   { color: var(--ember); }
.tip-status.success { color: var(--teal); }

.tip-actions { display: flex; gap: 8px; }
.tip-cancel-btn {
  flex: 1;
  background: transparent;
  border: 1px solid var(--border-mid);
  border-radius: 6px;
  padding: 11px;
  color: var(--text-3);
```

```
  font-family: var(--font-u);
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.15s;
}
.tip-cancel-btn:hover { border-color: var(--ember); color: var(--ember); }

.tip-send-btn {
  flex: 2;
  background: var(--gold);
  border: none;
  border-radius: 6px;
  padding: 11px;
  color: #0F0F12;
  font-family: var(--font-u);
  font-size: 12px;
  font-weight: 700;
  letter-spacing: 0.05em;
  text-transform: uppercase;
  cursor: pointer;
  transition: all 0.15s var(--ease);
}
.tip-send-btn:hover:not(:disabled) { background: var(--gold-bright); transform: translateY(-1px); box-
.tip-send-btn:disabled { opacity: 0.5; cursor: not-allowed; }

/* ── Engagement panel ──────────────────────────────────────── */
.viewer-engage-col {
  border-left: 1px solid var(--border-sub);
  display: flex;
  flex-direction: column;
  background: var(--bg-surface);
}

.engage-header {
  padding: 16px 18px;
  border-bottom: 1px solid var(--border-sub);
  font-family: var(--font-m);
  font-size: 10px;
  letter-spacing: 0.15em;
  text-transform: uppercase;
  color: var(--text-3);
}

.engage-chat-log {
  flex: 1;
  overflow-y: auto;
```

```
  padding: 12px 14px;
  display: flex;
  flex-direction: column;
  gap: 5px;
  scrollbar-width: thin;
  scrollbar-color: var(--border-mid) transparent;
}

.chat-row {
  font-size: 12px;
  line-height: 1.5;
  word-break: break-word;
}

.chat-name {
  font-weight: 700;
  color: var(--teal);
  margin-right: 4px;
}

.chat-row.chat-tip .chat-name { color: var(--gold); }
.chat-row.chat-system { color: var(--text-3); font-family: var(--font-m); font-size: 11px; }
.chat-text { color: var(--text-2); }
.chat-row.chat-tip .chat-text { color: var(--gold); }

.engage-input-row {
  padding: 12px 14px;
  border-top: 1px solid var(--border-sub);
  display: flex;
  gap: 8px;
}

.engage-chat-input {
  flex: 1;
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 4px;
  padding: 9px 11px;
  font-family: var(--font-u);
  font-size: 12px;
  color: var(--text-1);
  outline: none;
  transition: border-color 0.15s;
}

.engage-chat-input:focus { border-color: var(--teal); }
.engage-chat-input::placeholder { color: var(--text-3); }
```

```
.engage-send {
  background: var(--teal);
  border: none;
  border-radius: 4px;
  padding: 9px 14px;
  color: #0F0F12;
  font-family: var(--font-u);
  font-size: 13px;
  font-weight: 700;
  cursor: pointer;
  transition: background 0.15s;
}

.engage-send:hover { background: #20f0d8; }

/* ═══════════════════════════════════════════════════════════
   POST-STREAM MODAL
   ═══════════════════════════════════════════════════════════ */

.psm-overlay {
  position: fixed;
  inset: 0;
  background: rgba(8,8,10,0.9);
  backdrop-filter: blur(6px);
  z-index: 900;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
}

.psm-modal {
  background: var(--bg-surface);
  border: 1px solid var(--border-mid);
  border-radius: 14px;
  width: 100%;
  max-width: 520px;
  padding: 32px;
  box-shadow: 0 40px 120px rgba(0,0,0,0.7);
  animation: modal-in 0.3s var(--ease);
}

@keyframes modal-in {
  from { transform: translateY(24px); opacity: 0; }
  to   { transform: translateY(0);    opacity: 1; }
}
```

```
.psm-header { margin-bottom: 24px; }
.psm-eyebrow { font-family: var(--font-m); font-size: 10px; letter-spacing: 0.2em; text-transform: upp
.psm-title { font-family: var(--font-d); font-size: 30px; font-weight: 300; color: var(--text-1); line

.psm-stats {
  display: grid;
  grid-template-columns: repeat(3,1fr);
  gap: 12px;
  background: var(--bg-raised);
  border-radius: 8px;
  padding: 16px;
  margin-bottom: 24px;
}

.psm-stat { text-align: center; }
.psm-stat-val { font-family: var(--font-d); font-size: 26px; font-weight: 400; color: var(--gold); dis
.psm-stat-lbl { font-family: var(--font-m); font-size: 9px; color: var(--text-3); letter-spacing: 0.12

.psm-fields { display: flex; flex-direction: column; gap: 14px; margin-bottom: 16px; }
.psm-label { font-family: var(--font-m); font-size: 10px; letter-spacing: 0.12em; text-transform: uppe
.psm-input {
  width: 100%;
  background: var(--bg-raised);
  border: 1px solid var(--border-mid);
  border-radius: 5px;
  padding: 10px 13px;
  font-family: var(--font-u);
  font-size: 13px;
  color: var(--text-1);
  outline: none;
  resize: vertical;
  transition: border-color 0.15s;
}
.psm-input:focus { border-color: var(--gold); box-shadow: 0 0 0 3px rgba(212,168,83,0.1); }

.psm-status {
  min-height: 20px;
  font-family: var(--font-m);
  font-size: 11px;
  margin-bottom: 16px;
  transition: color 0.2s;
}

.psm-actions { display: flex; flex-direction: column; gap: 8px; }
.psm-btn {
  width: 100%;
```

```
  border-radius: 6px;
  padding: 12px 20px;
  font-family: var(--font-u);
  font-size: 13px;
  font-weight: 600;
  letter-spacing: 0.04em;
  cursor: pointer;
  transition: all 0.15s var(--ease);
  border: none;
  text-align: center;
}
.psm-btn:disabled { opacity: 0.4; cursor: not-allowed; }
.psm-btn-primary   { background: var(--violet); color: white; }
.psm-btn-primary:hover:not(:disabled) { background: #a070ff; transform: translateY(-1px); box-shadow:
.psm-btn-secondary { background: var(--teal); color: #0f0f12; }
.psm-btn-secondary:hover:not(:disabled) { background: #20f0d8; transform: translateY(-1px); }
.psm-btn-warn      { background: transparent; border: 1px solid var(--gold); color: var(--gold); }
.psm-btn-warn:hover:not(:disabled) { background: rgba(212,168,83,0.1); }
.psm-btn-ghost     { background: transparent; border: 1px solid var(--border-mid); color: var(--text-3
.psm-btn-ghost:hover:not(:disabled) { border-color: var(--ember); color: var(--ember); }


/* ── RTMP Stream Key Panel ──────────────────────────────── */
.rtmp-panel { padding: 4px 0; }
.rtmp-eyebrow { font-family: var(--font-m); font-size: 9px; letter-spacing: .2em; text-transform: uppe
.rtmp-row { margin-bottom: 12px; }
.rtmp-label { font-family: var(--font-m); font-size: 10px; letter-spacing: .12em; text-transform: uppe
.rtmp-private { color: var(--ember); text-transform: none; letter-spacing: 0; font-size: 9px; }
.rtmp-value-row { display: flex; gap: 8px; align-items: center; }
.rtmp-code { font-family: var(--font-m); font-size: 11px; color: var(--teal); background: var(--bg-rai
.rtmp-key-masked { filter: blur(5px); user-select: none; }
.rtmp-copy-btn, .rtmp-reveal-btn { background: var(--bg-hover); border: 1px solid var(--border-mid); b
.rtmp-copy-btn:hover, .rtmp-reveal-btn:hover { border-color: var(--teal); color: var(--teal); }
.rtmp-instructions { border: 1px solid var(--border-sub); border-radius: 5px; margin: 12px 0; overflow
.rtmp-instructions summary { padding: 9px 13px; cursor: pointer; font-family: var(--font-m); font-size
.rtmp-instructions summary::-webkit-details-marker { display: none; }
.rtmp-inst-body { padding: 12px 14px; font-size: 12px; color: var(--text-2); line-height: 1.8; border-
.rtmp-inst-body code { font-family: var(--font-m); font-size: 10px; color: var(--teal); background: va
.rtmp-regen-btn { background: transparent; border: 1px solid var(--border-mid); border-radius: 4px; pa
.rtmp-regen-btn:hover { border-color: var(--ember); color: var(--ember); }

/* FIX #8: RTMP Skeleton loading state */
.rtmp-skeleton {
  display: flex;
  flex-direction: column;
  gap: 14px;
  padding: 4px 0;
```

```
}

.rtmp-skeleton-row { display: flex; flex-direction: column; gap: 7px; }

.rtmp-skeleton-label {
  width: 80px;
  height: 9px;
  background: var(--bg-raised);
  border-radius: 3px;
  animation: skeleton-pulse 1.6s ease-in-out infinite;
}

.rtmp-skeleton-bar {
  height: 34px;
  background: var(--bg-raised);
  border-radius: 4px;
  border: 1px solid var(--border-sub);
  animation: skeleton-pulse 1.6s ease-in-out infinite;
}

.rtmp-skeleton-bar:nth-child(2) { animation-delay: 0.2s; }

@keyframes skeleton-pulse {
  0%, 100% { opacity: 0.5; }
  50%       { opacity: 1; }
}

/* ── Studio mode tabs ──────────────────────────────────────────── */
.studio-mode-tabs { display: flex; border-bottom: 1px solid var(--border-sub); }
.studio-mode-tab { flex: 1; padding: 11px 8px; background: none; border: none; border-bottom: 2px soli
.studio-mode-tab:hover { color: var(--text-2); }
.studio-mode-tab.active { color: var(--ember); border-bottom-color: var(--ember); }
.studio-tab-panel { display: none; flex: 1; flex-direction: column; overflow: hidden; }
.studio-tab-panel.active { display: flex; flex-direction: column; }

/* FIX #11: Respect reduced-motion preference */
@media (prefers-reduced-motion: reduce) {
  .status-dot,
  .studio-status.live .status-dot,
  .live-badge,
  .viewer-live-pill,
  .floating-reaction,
  .rtmp-skeleton-label,
  .rtmp-skeleton-bar,
  .status-spinner {
    animation: none !important;
  }
```

```
  .psm-modal,
  .tip-modal {
    animation: none !important;
  }
  .live-card:hover,
  .btn-go-live:hover:not(:disabled),
  .ov-btn:hover,
  .react-btn:hover,
  .viewer-fullscreen-btn:hover,
  .tip-send-btn:hover:not(:disabled) {
    transform: none !important;
  }
}

/* ── DEV MODE BANNER ──────────────────────────────────────── */
.dev-banner {
  position: fixed;
  bottom: 0;
  left: 0; right: 0;
  z-index: 9999;
  background: repeating-linear-gradient(
    -45deg,
    #E85D3A,
    #E85D3A 12px,
    #1a0a06 12px,
    #1a0a06 24px
  );
  border-top: 3px solid #ff9070;
  padding: 10px 24px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  box-shadow: 0 -4px 40px rgba(232,93,58,0.5);
  animation: dev-pulse 2.5s ease-in-out infinite;
}

@media (prefers-reduced-motion: reduce) {
  .dev-banner { animation: none !important; }
}

@keyframes dev-pulse {
  0%, 100% { box-shadow: 0 -4px 40px rgba(232,93,58,0.5); }
  50%       { box-shadow: 0 -4px 60px rgba(232,93,58,0.85); }
}

.dev-banner-left {
```

```
  display: flex;
  align-items: center;
  gap: 12px;
}

.dev-banner-icon {
  font-size: 22px;
  line-height: 1;
  flex-shrink: 0;
}

.dev-banner-label {
  font-family: var(--font-m);
  font-size: 11px;
  font-weight: 700;
  letter-spacing: 0.2em;
  text-transform: uppercase;
  color: white;
  text-shadow: 0 1px 4px rgba(0,0,0,0.6);
}

.dev-banner-wallet {
  font-family: var(--font-m);
  font-size: 11px;
  color: rgba(255,255,255,0.75);
  letter-spacing: 0.04em;
  background: rgba(0,0,0,0.35);
  padding: 4px 10px;
  border-radius: 4px;
  border: 1px solid rgba(255,255,255,0.15);
  cursor: pointer;
  transition: background 0.15s;
  word-break: break-all;
}

.dev-banner-wallet:hover { background: rgba(0,0,0,0.55); }
.dev-banner-wallet:active { background: rgba(0,0,0,0.7); }

.dev-banner-right {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-shrink: 0;
}

.dev-banner-copied {
  font-family: var(--font-m);
```

```
  font-size: 10px;
  color: var(--teal);
  letter-spacing: 0.08em;
  opacity: 0;
  transition: opacity 0.2s;
  pointer-events: none;
}

.dev-banner-copied.visible { opacity: 1; }

.dev-banner-dismiss {
  background: rgba(0,0,0,0.4);
  border: 1px solid rgba(255,255,255,0.2);
  border-radius: 4px;
  padding: 5px 12px;
  font-family: var(--font-u);
  font-size: 11px;
  font-weight: 700;
  color: white;
  cursor: pointer;
  letter-spacing: 0.04em;
  transition: background 0.15s;
}
.dev-banner-dismiss:hover { background: rgba(0,0,0,0.65); }

</style>
</head>
<body class="page-live-studio">

<!-- ═══════════════════════════════════════════════════════
     BROADCASTER PANEL
     ═══════════════════════════════════════════════════════ -->

<div style="max-width:900px;margin:40px auto;padding:20px;">

<div class="live-studio" id="live-studio">

  <!-- Header -->
  <div class="studio-header">
    <div>
      <div class="studio-title">Live <em style="font-style:italic;color:var(--ember)">Studio</em></div
    </div>
    <div class="studio-status" id="studio-status-badge">
      <!-- FIX #9: spinner element alongside the dot -->
      <div class="status-spinner" id="studio-status-spinner"></div>
      <div class="status-dot"></div>
      <span id="studio-status-text">Offline</span>
```

```
  </div>
</div>

<!-- Access gate — shown when user lacks hostConcert permission.
     JS: checkStudioAccess() toggles studio-gate vs studio-body -->
<div class="studio-gate" id="studio-gate" style="display:none;" aria-live="polite">
  <div class="studio-gate-icon">🔒</div>
  <div class="studio-gate-title" id="studio-gate-title">Creator Account Required</div>
  <p class="studio-gate-sub" id="studio-gate-sub">
    Going live on MSP requires an active Standard Creator or Platform NFT Creator subscription.
  </p>
  <div class="studio-gate-pills">
    <span class="studio-gate-pill">Listener T1 — watch only</span>
    <span class="studio-gate-pill">Listener T2/T3 — watch + chat</span>
    <span class="studio-gate-pill active">Standard Creator ✓</span>
    <span class="studio-gate-pill active">NFT Creator (active) ✓</span>
  </div>
  <a href="listen.html#subscribe" class="studio-gate-btn" id="studio-gate-cta">Upgrade to Creator</a
  <button class="studio-gate-btn secondary" id="studio-gate-connect" style="display:none;" aria-labe
</div>

<!-- Body: preview + panel (hidden by gate until access confirmed) -->
<div class="studio-body" id="studio-body">

  <!-- Camera preview -->
  <div class="studio-preview-area" id="studio-preview-area">
    <div class="studio-no-camera" id="studio-no-camera">
      <div class="studio-no-camera-icon">◎</div>
      <span>Camera not started</span>
      <span style="font-size:10px;color:var(--text-3)">Select a source below to preview</span>
    </div>
    <!-- FIX #10: broadcaster preview already has muted + playsinline ✓ -->
    <video class="studio-video-preview" id="studio-video-preview" autoplay muted playsinline style="

    <!-- HLS URL display -->
    <div class="studio-hls-url" id="studio-hls-url"></div>

    <!-- FIX #14: Bitrate / resolution bar (populated by JS during live) -->
    <div class="studio-bitrate-bar" id="studio-bitrate-bar">
      <span>RES <span class="bitrate-val" id="stat-resolution">—</span></span>
      <span>BIT <span class="bitrate-val" id="stat-bitrate">—</span></span>
      <span>FPS <span class="bitrate-val" id="stat-fps">—</span></span>
    </div>

    <!-- Overlay controls (shown on hover / during live) -->
    <div class="studio-overlay-controls" id="studio-overlay-controls">
      <!-- FIX #10: aria-labels on control buttons -->
```

```
    <button class="ov-btn" id="ov-mute" aria-label="Toggle Microphone" title="Toggle Microphone">🎤
    <button class="ov-btn" id="ov-cam"  aria-label="Toggle Camera" title="Toggle Camera">📷</butto
  </div>
</div>

<!-- Side panel -->
<div class="studio-panel">

  <!-- Live stats (hidden until live) -->
  <div class="studio-live-stats" id="studio-live-stats">
    <div class="ls-stat"><span class="ls-val duration">00:00</span><span class="ls-lbl">Duration</
    <div class="ls-stat"><span class="ls-val viewer-count">0</span><span class="ls-lbl">Watching</
    <div class="ls-stat"><span class="ls-val tips-total">0</span><span class="ls-lbl">ETH Tipped</
  </div>

  <!-- Mode selector tabs -->
  <div class="studio-mode-tabs" id="studio-mode-tabs">
    <button class="studio-mode-tab active" data-mode="browser" aria-label="Browser live mode">📷 B
    <button class="studio-mode-tab" data-mode="rtmp" aria-label="OBS / App stream mode">🎥 OBS / A
  </div>

  <!-- Browser mode: camera/screen + go-live setup -->
  <div class="studio-tab-panel active" id="tab-browser">
    <!-- Setup form -->
    <div class="studio-setup" id="studio-setup">

      <div>
        <label class="s-label">Stream Source</label>
        <div class="source-picker">
          <button class="source-btn" id="src-camera" aria-label="Use camera">📷 Camera</button>
          <button class="source-btn" id="src-screen" aria-label="Use screen share">🖥 Screen</butt
        </div>
      </div>

      <div>
        <label class="s-label" for="live-title">Stream Title</label>
        <input class="s-input" id="live-title" type="text" placeholder="What are you performing to
      </div>

      <div>
        <label class="s-label" for="live-artist">Artist Name</label>
        <input class="s-input" id="live-artist" type="text" placeholder="Your artist / DJ name">
      </div>

      <div>
        <label class="s-label" for="live-quality">Quality</label>
        <select class="s-select" id="live-quality">
```

```
        <option value="1080p">1080p — Best quality</option>
        <option value="720p" selected>720p — Recommended</option>
        <option value="480p">480p — Lower bandwidth</option>
        <option value="360p">360p — Minimal bandwidth</option>
      </select>
    </div>

    <button class="btn-go-live" id="btn-go-live" disabled aria-label="Start live stream">
      ● Go Live
    </button>

  </div><!-- /setup -->
</div><!-- /tab-browser -->

<!-- RTMP mode: stream key for OBS/mobile -->
<div class="studio-tab-panel" id="tab-rtmp" style="padding:20px;overflow-y:auto;">
  <!-- FIX #8: Skeleton shown while wallet not connected; replaced by JS when ready -->
  <div id="rtmp-key-panel">
    <div id="rtmp-wallet-prompt" style="display:block;">
      <p style="font-family:var(--font-m);font-size:11px;color:var(--text-3);margin-bottom:16px;
        Connect your wallet to reveal your stream key.
      </p>
      <!-- Skeleton placeholder -->
      <div class="rtmp-skeleton">
        <div class="rtmp-skeleton-row">
          <div class="rtmp-skeleton-label"></div>
          <div class="rtmp-skeleton-bar"></div>
        </div>
        <div class="rtmp-skeleton-row">
          <div class="rtmp-skeleton-label" style="width:60px;"></div>
          <div class="rtmp-skeleton-bar"></div>
        </div>
        <div class="rtmp-skeleton-row">
          <div class="rtmp-skeleton-label" style="width:100px;"></div>
          <div class="rtmp-skeleton-bar" style="height:22px;"></div>
        </div>
      </div>
    </div>
  </div>
</div>

<!-- Live chat (shown during stream) -->
<div class="studio-live-chat" id="studio-live-chat">
  <div class="live-chat-log" id="live-chat-log" aria-live="polite" aria-label="Live chat message
  <div class="live-chat-input-row">
    <input class="live-chat-input" id="live-chat-input" placeholder="Chat with your viewers…" ma
    <button class="live-chat-send" id="live-chat-send" aria-label="Send chat message">↑</button>
```

```
        </div>
      </div>

      <!-- End stream row -->
      <div class="studio-end-row" id="studio-end-row">
        <button class="btn-end-stream" id="btn-end-stream" aria-label="End live stream">■ End Stream</
      </div>

    </div><!-- /panel -->
  </div><!-- /body -->
</div><!-- /live-studio -->
</div><!-- /max-width wrapper -->


<!-- ═══════════════════════════════════════════════════════════
     VIEWER OVERLAY
     (One instance in listen.html, shown/hidden by JS)
     ═══════════════════════════════════════════════════════════ -->

<!-- FIX #1: Single root container, no redundant wrappers -->
<div class="viewer-overlay open" id="viewer-overlay" style="position:fixed;">

  <!-- Video column -->
  <!-- FIX #2: id added so JS can reference it correctly -->
  <div class="viewer-video-col" id="viewer-video-col">

    <!-- FIX #10: playsinline + muted on viewer video for iOS autoplay -->
    <video class="viewer-video" id="viewer-video" controls playsinline muted aria-label="Live stream v

    <!-- Stats bar -->
    <div class="viewer-stats-bar">
      <div>
        <div class="viewer-stream-title" id="viewer-stream-title">Midnight Protocol — Live</div>
        <div class="viewer-meta-bar">
          <span class="viewer-count">247 watching</span>
          <span class="duration">00:42:18</span>
          <span class="tips-total">1.204 ETH tipped</span>
        </div>
      </div>
      <div class="viewer-live-pill" aria-label="Stream is live">● LIVE</div>
    </div>

    <!-- Close button -->
    <button class="viewer-close" id="viewer-close" aria-label="Close viewer">✕</button>

    <!-- FIX #13: Fullscreen button -->
    <button class="viewer-fullscreen-btn" id="viewer-fullscreen-btn" aria-label="Toggle fullscreen" ti
```

```
    <!-- Floating reactions layer -->
    <div class="reactions-layer" id="reactions-layer" aria-hidden="true"></div>

    <!-- Reaction bar + tip -->
    <div class="reaction-bar">
      <button class="react-btn" data-emoji="🔥" aria-label="Fire reaction">🔥</button>
      <button class="react-btn" data-emoji="❤️" aria-label="Heart reaction">❤️</button>
      <button class="react-btn" data-emoji="👏" aria-label="Clap reaction">👏</button>
      <button class="react-btn" data-emoji="🎵" aria-label="Music note reaction">🎵</button>
      <button class="react-btn" data-emoji="💎" aria-label="Diamond reaction">💎</button>
      <button class="viewer-tip-btn" id="viewer-tip-btn" aria-label="Tip the artist">💸 Tip Artist</bu
    </div>

  </div><!-- /viewer-video-col -->

  <!-- Engagement column -->
  <div class="viewer-engage-col">
    <div class="engage-header">Live Chat</div>

    <div class="engage-chat-log" id="engage-chat-log" aria-live="polite" aria-label="Live chat">
      <!-- Sample messages -->
      <div class="chat-row"><span class="chat-name">Wavelength</span><span class="chat-text">this drop
      <div class="chat-row chat-tip"><span class="chat-name">💸 Nova</span><span class="chat-text">ser
      <div class="chat-row"><span class="chat-name">CipherX</span><span class="chat-text">that bass li
      <div class="chat-row"><span class="chat-name">Listener99</span><span class="chat-text">first tim
      <div class="chat-row"><span class="chat-name">vera.eth</span><span class="chat-text">next track?
    </div>

    <div class="engage-input-row">
      <input class="engage-chat-input" id="engage-chat-input" placeholder="Say something…" maxlength="
      <button class="engage-send" id="engage-send" aria-label="Send message">↑</button>
    </div>
  </div>

</div><!-- /viewer-overlay -->


<!-- FIX #5: Tip Modal HTML ──────────────────────────────────── -->
<div class="tip-modal-overlay" id="tip-modal-overlay" role="dialog" aria-modal="true" aria-labelledby=
  <div class="tip-modal">
    <div class="tip-modal-header">
      <div class="tip-modal-eyebrow">Support the Artist</div>
      <div class="tip-modal-title" id="tip-modal-title">Send a Tip</div>
    </div>

    <div class="tip-presets" id="tip-presets">
```

```
      <button class="tip-preset-btn" data-amount="0.005">0.005</button>
      <button class="tip-preset-btn" data-amount="0.01">0.01</button>
      <button class="tip-preset-btn selected" data-amount="0.05">0.05</button>
      <button class="tip-preset-btn" data-amount="0.1">0.1</button>
    </div>

    <div class="tip-custom-row">
      <label class="tip-custom-label" for="tip-custom-input">Amount (ETH)</label>
      <input class="tip-custom-input" id="tip-custom-input" type="number" step="0.001" min="0.001" pla
    </div>

    <div class="tip-status" id="tip-status"></div>

    <div class="tip-actions">
      <button class="tip-cancel-btn" id="tip-cancel-btn">Cancel</button>
      <button class="tip-send-btn" id="tip-send-btn">💸 Send Tip</button>
    </div>
  </div>
</div>


<script>
// ────────────────────────────────────────────────────────────
// BROADCASTER UI WIRING
// ────────────────────────────────────────────────────────────

(function () {

  // ── DEV MODE — REMOVE BEFORE PRODUCTION ──────────────────
  var DEV_MODE   = true;
  var DEV_WALLET = '0xDe1f33Ce3E81bc54ebD0F4BC3Ce0a2F64d7Ef399';
  // ────────────────────────────────────────────────────────────

  var broadcaster   = null;
  var selectedSource = null;
  var bitrateInterval = null;

  // ── Access gate ──────────────────────────────────────────
  // Checks wallet connection + CAN.hostConcert() from main.js and
  // either shows the studio or replaces it with a locked gate UI.
  var gateEl    = document.getElementById('studio-gate');
  var bodyEl    = document.getElementById('studio-body');
  var gateTitle = document.getElementById('studio-gate-title');
  var gateSub   = document.getElementById('studio-gate-sub');
  var gateConn  = document.getElementById('studio-gate-connect');

  function showDevBanner() {
```

```
  if (document.getElementById('dev-banner')) return; // already mounted
  var banner = document.createElement('div');
  banner.id        = 'dev-banner';
  banner.className = 'dev-banner';
  banner.setAttribute('role', 'alert');
  banner.innerHTML =
    '<div class="dev-banner-left">' +
      '<span class="dev-banner-icon">⚠️</span>' +
      '<span class="dev-banner-label">DEV MODE — Not for production</span>' +
      '<span class="dev-banner-wallet" id="dev-banner-wallet" title="Click to copy wallet address">'
        DEV_WALLET +
      '</span>' +
      '<span class="dev-banner-copied" id="dev-banner-copied">Copied!</span>' +
    '</div>' +
    '<div class="dev-banner-right">' +
      '<span class="dev-banner-label" style="font-size:10px;opacity:0.7;">All permission checks bypa
      '<button class="dev-banner-dismiss" id="dev-banner-dismiss" aria-label="Dismiss dev banner">Di
    '</div>';
  document.body.appendChild(banner);

  // Add bottom padding to body so banner doesn't cover page content
  document.body.style.paddingBottom = banner.offsetHeight + 'px';

  // Copy wallet to clipboard on click
  var walletEl = document.getElementById('dev-banner-wallet');
  var copiedEl = document.getElementById('dev-banner-copied');
  if (walletEl) walletEl.addEventListener('click', function () {
    navigator.clipboard && navigator.clipboard.writeText(DEV_WALLET).then(function () {
      if (copiedEl) {
        copiedEl.classList.add('visible');
        setTimeout(function () { copiedEl.classList.remove('visible'); }, 1800);
      }
    });
  });

  // Dismiss collapses the banner (DEV_MODE stays true — page reload restores it)
  var dismissBtn = document.getElementById('dev-banner-dismiss');
  if (dismissBtn) dismissBtn.addEventListener('click', function () {
    banner.style.transition = 'transform 0.25s ease, opacity 0.25s ease';
    banner.style.transform  = 'translateY(100%)';
    banner.style.opacity    = '0';
    document.body.style.paddingBottom = '0';
    setTimeout(function () { banner.remove(); }, 280);
  });
}

function checkStudioAccess() {
```

```
  // ── DEV BYPASS ──────────────────────────────────────────
  if (DEV_MODE && DEV_WALLET) {
    window.walletAddress = DEV_WALLET;
    hideGate();
    showDevBanner();
    return;
  }
  // ────────────────────────────────────────────────────────

  var wallet = window.walletAddress || null;

  // No wallet connected at all
  if (!wallet) {
    showGate(
      'Connect Your Wallet',
      'Connect your wallet so MSP can verify your creator account before going live.',
      true   // show the connect-wallet button
    );
    return;
  }

  // Wallet connected — check capability
  // CAN.hostConcert() is defined in main.js; falls back to false if not loaded yet
  var canHost = (typeof CAN !== 'undefined' && typeof CAN.hostConcert === 'function')
    ? CAN.hostConcert()
    : false;

  if (!canHost) {
    // Determine if they're a creator with an expired sub vs a listener
    var level = (typeof getAccess === 'function') ? (getAccess().level || 'none') : 'none';
    var isExpiredCreator = level === 'creator_inactive';

    showGate(
      isExpiredCreator ? 'Subscription Expired' : 'Creator Account Required',
      isExpiredCreator
        ? 'Your creator subscription has expired. Renew to resume live streaming on MSP.'
        : 'Going live requires an active Standard Creator or Platform NFT Creator account.',
      false
    );
    return;
  }

  // Access granted — show the studio
  hideGate();
}

function showGate(title, sub, showConnect) {
```

```
  if (gateTitle) gateTitle.textContent = title;
  if (gateSub)   gateSub.textContent   = sub;
  if (gateConn)  gateConn.style.display = showConnect ? 'inline-block' : 'none';
  if (gateEl)    gateEl.style.display   = '';
  if (bodyEl)    bodyEl.style.display   = 'none';
}

function hideGate() {
  if (gateEl)  gateEl.style.display  = 'none';
  if (bodyEl)  bodyEl.style.display  = '';
}

// Connect-wallet button delegates to main.js connectWallet()
if (gateConn) gateConn.addEventListener('click', function () {
  if (typeof connectWallet === 'function') {
    connectWallet().then(checkStudioAccess).catch(function (e) {
      if (gateSub) gateSub.textContent = 'Wallet connection failed: ' + e.message;
    });
  }
});

// Re-check when wallet connects or access level changes
// main.js fires 'msp:walletConnected' and 'msp:accessChanged' custom events
window.addEventListener('msp:walletConnected', checkStudioAccess);
window.addEventListener('msp:accessChanged',   checkStudioAccess);

// Initial check on load
checkStudioAccess();


var previewEl   = document.getElementById('studio-video-preview');
var noCamEl     = document.getElementById('studio-no-camera');
var hlsUrlEl    = document.getElementById('studio-hls-url');
var statusBadge = document.getElementById('studio-status-badge');
var statusText  = document.getElementById('studio-status-text');
var statsEl     = document.getElementById('studio-live-stats');
var setupEl     = document.getElementById('studio-setup');
var chatEl      = document.getElementById('studio-live-chat');
var chatLog     = document.getElementById('live-chat-log');
var chatInput   = document.getElementById('live-chat-input');
var chatSend    = document.getElementById('live-chat-send');
var endRow      = document.getElementById('studio-end-row');
var previewArea = document.getElementById('studio-preview-area');
var bitrateBar  = document.getElementById('studio-bitrate-bar');

// FIX #9: setStatus with optional spinner state
function setStatus(msg, live, spinning) {
```

```
  if (statusText)  statusText.textContent = msg;
  var cls = 'studio-status';
  if (live)    cls += ' live';
  if (spinning) cls += ' spinning';
  if (statusBadge) statusBadge.className = cls;
}

function appendChat(name, text, type) {
  if (!chatLog) return;
  var row  = document.createElement('div');
  row.className = 'chat-row' + (type === 'tip' ? ' chat-tip' : '') + (type === 'system' ? ' chat-sys
  var nm   = document.createElement('span'); nm.className = 'chat-name'; nm.textContent = name + ' '
  var tx   = document.createElement('span'); tx.className = 'chat-text'; tx.textContent = text;
  row.appendChild(nm); row.appendChild(tx);
  chatLog.appendChild(row);
  chatLog.scrollTop = chatLog.scrollHeight;
}

// FIX #14: Poll WebRTC stats for bitrate / resolution / fps
function startBitratePolling() {
  if (!broadcaster || !broadcaster._pc) return;
  bitrateBar.classList.add('visible');
  var prevBytes = 0;
  var prevTs    = performance.now();

  bitrateInterval = setInterval(async function () {
    if (!broadcaster || !broadcaster._pc) return stopBitratePolling();
    try {
      var stats = await broadcaster._pc.getStats();
      stats.forEach(function (report) {
        if (report.type === 'outbound-rtp' && report.kind === 'video') {
          var now      = performance.now();
          var elapsed  = (now - prevTs) / 1000;
          var bytes    = report.bytesSent || 0;
          var kbps     = Math.round(((bytes - prevBytes) * 8) / elapsed / 1000);
          prevBytes    = bytes;
          prevTs       = now;

          var resStat   = document.getElementById('stat-resolution');
          var bitStat   = document.getElementById('stat-bitrate');
          var fpsStat   = document.getElementById('stat-fps');

          if (resStat && report.frameWidth)  resStat.textContent  = report.frameWidth + '×' + report
          if (bitStat && kbps > 0)           bitStat.textContent  = kbps + ' kbps';
          if (fpsStat && report.framesPerSecond) fpsStat.textContent = Math.round(report.framesPerSe
        }
      });
```

```
    } catch (e) { /* silently ignore getStats errors */ }
  }, 2000);
}

function stopBitratePolling() {
  if (bitrateInterval) clearInterval(bitrateInterval);
  bitrateInterval = null;
  if (bitrateBar) bitrateBar.classList.remove('visible');
}

// Source selection
var srcCamera = document.getElementById('src-camera');
var srcScreen = document.getElementById('src-screen');

if (srcCamera) srcCamera.addEventListener('click', async function () {
  try {
    if (!broadcaster) broadcaster = new MSPLive.Broadcaster({
      previewEl: previewEl, statusEl: statusText, statsEl: statsEl,
      quality: (document.getElementById('live-quality') || {}).value || '720p',
    });
    await broadcaster.getCamera();
    selectedSource = 'camera';
    srcCamera.classList.add('active');
    if (srcScreen) srcScreen.classList.remove('active');
    previewEl.style.display = 'block';
    noCamEl.style.display   = 'none';
    document.getElementById('btn-go-live').disabled = false;
    setStatus('Camera ready');
  } catch (e) {
    setStatus('Camera denied');
    alert('Camera access denied: ' + e.message);
  }
});

if (srcScreen) srcScreen.addEventListener('click', async function () {
  try {
    if (!broadcaster) broadcaster = new MSPLive.Broadcaster({
      previewEl: previewEl, statusEl: statusText, statsEl: statsEl,
      quality: (document.getElementById('live-quality') || {}).value || '720p',
    });
    await broadcaster.getScreenShare();
    selectedSource = 'screen';
    srcScreen.classList.add('active');
    if (srcCamera) srcCamera.classList.remove('active');
    previewEl.style.display = 'block';
    noCamEl.style.display   = 'none';
    document.getElementById('btn-go-live').disabled = false;
```

```
    setStatus('Screen capture ready');
  } catch (e) {
    setStatus('Screen share denied');
    alert('Screen share denied: ' + e.message);
  }
});

// Go Live
var goLiveBtn = document.getElementById('btn-go-live');
if (goLiveBtn) goLiveBtn.addEventListener('click', async function () {
  var title      = (document.getElementById('live-title')  || {}).value || '';
  var artistName = (document.getElementById('live-artist') || {}).value || '';
  if (!title || !artistName) { alert('Please enter a stream title and artist name.'); return; }
  if (!broadcaster)          { alert('Start camera or screen share first.'); return; }

  goLiveBtn.disabled = true;
  // FIX #9: Show spinner while connecting
  setStatus('Connecting…', false, true);

  try {
    broadcaster._opts.quality = (document.getElementById('live-quality') || {}).value || '720p';
    var data = await broadcaster.startSession(title, artistName);

    // Switch UI to live mode
    setupEl.classList.add('hidden');
    statsEl.classList.add('visible');
    chatEl.classList.add('visible');
    endRow.classList.add('visible');
    previewArea.classList.add('live-active');

    hlsUrlEl.textContent = 'HLS: ' + data.hlsUrl;
    hlsUrlEl.classList.add('visible');
    setStatus('● LIVE', true, false);

    appendChat('MSP', 'You are now live! Share your stream link.', 'system');

    // FIX #14: Start polling WebRTC stats
    startBitratePolling();

  } catch (e) {
    goLiveBtn.disabled = false;
    // If the server returns 403, the client-side gate missed it —
    // re-run the access check so the gate UI reflects the real state.
    if (e.status === 403 || (e.message && e.message.includes('403'))) {
      setStatus('Access denied');
      checkStudioAccess();
      // checkStudioAccess() will replace the body with the gate UI automatically
```

```
    } else {
      setStatus('Error — ' + e.message);
      alert('Failed to go live: ' + e.message);
    }
  }
});

// Mute / Camera toggles
var ovMute = document.getElementById('ov-mute');
var ovCam  = document.getElementById('ov-cam');

if (ovMute) ovMute.addEventListener('click', function () {
  if (!broadcaster) return;
  var muted = broadcaster.toggleMute();
  ovMute.classList.toggle('muted', muted);
  ovMute.textContent = muted ? '🔇' : '🎤';
  ovMute.setAttribute('aria-label', muted ? 'Unmute Microphone' : 'Mute Microphone');
});

if (ovCam) ovCam.addEventListener('click', function () {
  if (!broadcaster) return;
  var off = broadcaster.toggleCamera();
  ovCam.classList.toggle('cam-off', off);
  ovCam.textContent = off ? '🚫' : '📷';
  ovCam.setAttribute('aria-label', off ? 'Enable Camera' : 'Disable Camera');
});

// Chat send
function sendChat() {
  var text = (chatInput && chatInput.value.trim()) || '';
  if (!text || !broadcaster) return;
  if (broadcaster._ws && broadcaster._ws.send) {
    broadcaster._ws.send(JSON.stringify({ type: 'chat', text: text }));
  }
  appendChat('You', text);
  chatInput.value = '';
}

if (chatSend)  chatSend.addEventListener('click', sendChat);
if (chatInput) chatInput.addEventListener('keydown', function (e) { if (e.key === 'Enter') sendChat(

// End Stream — FIX #4: wrap endStream in try/catch
var endBtn = document.getElementById('btn-end-stream');
if (endBtn) endBtn.addEventListener('click', async function () {
  if (!broadcaster || !confirm('End your live stream?')) return;
  endBtn.disabled = true;
  // FIX #9: spinner while ending
```

```
  setStatus('Ending stream…', false, true);

  var summary;
  try {
    summary = await broadcaster.endStream();
  } catch (e) {
    setStatus('Error ending stream — ' + e.message);
    endBtn.disabled = false;
    alert('Could not end stream cleanly: ' + e.message);
    return;
  }

  // Stop bitrate polling
  stopBitratePolling();

  // Reset UI
  statsEl.classList.remove('visible');
  chatEl.classList.remove('visible');
  endRow.classList.remove('visible');
  setupEl.classList.remove('hidden');
  hlsUrlEl.classList.remove('visible');
  previewArea.classList.remove('live-active');
  goLiveBtn.disabled = false;
  endBtn.disabled    = false;
  setStatus('Offline');
  broadcaster = null;

  // Open post-stream modal
  var modal = new MSPLive.PostStreamModal(summary || {}, {
    onArchive: function (data, mintNft) {
      console.log('Archived:', data.archiveCid, mintNft ? '— minting NFT' : '— catalog only');
    },
    onRetry:   function () { goLiveBtn.click(); },
    onDiscard: function () { console.log('Recording discarded'); },
  });
  modal.open();
});

// ── Studio mode tab switching ──────────────────────────────
document.querySelectorAll('.studio-mode-tab').forEach(function (tab) {
  tab.addEventListener('click', function () {
    document.querySelectorAll('.studio-mode-tab').forEach(function (t) { t.classList.remove('active'
    document.querySelectorAll('.studio-tab-panel').forEach(function (p) { p.classList.remove('active
    tab.classList.add('active');
    var panel = document.getElementById('tab-' + tab.dataset.mode);
    if (panel) panel.classList.add('active');
```

```
      // FIX #8: Load RTMP key panel when switching to that tab
      if (tab.dataset.mode === 'rtmp') {
        var keyPanel = document.getElementById('rtmp-key-panel');
        if (window.walletAddress && window.MSPLive) {
          if (!broadcaster) broadcaster = new MSPLive.Broadcaster({ previewEl: previewEl, statusEl: st
          broadcaster.renderStreamKeyPanel('rtmp-key-panel').catch(function (e) {
            if (keyPanel) keyPanel.innerHTML = '<p style="color:var(--ember);font-family:var(--font-m)
          });
        } else {
          // Restore the skeleton / wallet-prompt if not connected
          var walletPrompt = document.getElementById('rtmp-wallet-prompt');
          if (walletPrompt) walletPrompt.style.display = 'block';
        }
      }
    });
  });

})();


// ────────────────────────────────────────────────────────────────
// VIEWER UI WIRING
// ────────────────────────────────────────────────────────────────

(function () {

  var currentViewer = null;
  var overlay       = document.getElementById('viewer-overlay');
  var videoEl       = document.getElementById('viewer-video');
  var chatLog       = document.getElementById('engage-chat-log');
  var chatInput     = document.getElementById('engage-chat-input');
  var chatSend      = document.getElementById('engage-send');
  var closeBtn      = document.getElementById('viewer-close');
  var reactLayer    = document.getElementById('reactions-layer');
  var titleEl       = document.getElementById('viewer-stream-title');

  function openViewer(sessionId) {
    if (currentViewer) currentViewer.leave();

    currentViewer = new MSPLive.Viewer({
      videoEl:     videoEl,
      chatEl:      chatLog,
      reactionsEl: reactLayer,
      statsEl:     overlay,
      onEnded: function (msg) {
        var p = document.createElement('div');
        p.style.cssText = 'position:absolute;inset:0;background:rgba(8,8,10,0.8);display:flex;align-it
```

```
      p.innerHTML = '<div style="text-align:center;">' +
        '<div style="font-family:var(--font-d);font-size:48px;font-weight:300;color:var(--text-1)">S
        '<div style="font-family:var(--font-m);font-size:11px;color:var(--text-3);margin-top:12px;le
        '</div>';
      // FIX #1 + #2: Use the correct element reference — id is now on the element
      var videoCol = document.getElementById('viewer-video-col');
      if (videoCol) videoCol.appendChild(p);
    },
  });
  currentViewer.join(sessionId);
  overlay.classList.add('open');
}

// Load sessions and wire join buttons
if (window.MSPLive && MSPLive.renderSessions) {
  MSPLive.renderSessions('live-sessions-grid', function (sessionId) {
    openViewer(sessionId);
  });
}

// Close viewer
if (closeBtn) closeBtn.addEventListener('click', function () {
  if (currentViewer) { currentViewer.leave(); currentViewer = null; }
  overlay.classList.remove('open');
});

// FIX #13: Fullscreen button
var fsBtn = document.getElementById('viewer-fullscreen-btn');
if (fsBtn && videoEl) {
  fsBtn.addEventListener('click', function () {
    if (!document.fullscreenElement) {
      (videoEl.requestFullscreen ? videoEl.requestFullscreen() :
       videoEl.webkitRequestFullscreen ? videoEl.webkitRequestFullscreen() : null);
      fsBtn.textContent = '⊡';
      fsBtn.setAttribute('aria-label', 'Exit fullscreen');
    } else {
      document.exitFullscreen && document.exitFullscreen();
      fsBtn.textContent = '⛶';
      fsBtn.setAttribute('aria-label', 'Toggle fullscreen');
    }
  });
}

// Chat
function sendViewerChat() {
  var text = chatInput && chatInput.value.trim();
  if (!text || !currentViewer) return;
```

```
  currentViewer.sendChat(text);
  chatInput.value = '';
}

if (chatSend)  chatSend.addEventListener('click', sendViewerChat);
if (chatInput) chatInput.addEventListener('keydown', function (e) { if (e.key === 'Enter') sendViewe

// FIX #7: Reactions with organic random x-offset and rotation
document.querySelectorAll('.react-btn').forEach(function (btn) {
  btn.addEventListener('click', function () {
    if (!currentViewer) return;
    currentViewer.sendReaction(btn.dataset.emoji);
    spawnFloatingReaction(btn.dataset.emoji);
  });
});

function spawnFloatingReaction(emoji) {
  if (!reactLayer) return;
  var el = document.createElement('span');
  el.className = 'floating-reaction';
  el.textContent = emoji;
  // Random x-offset: -40px to +40px; random rotation: -12deg to +12deg
  var rx = (Math.random() * 80 - 40).toFixed(1) + 'px';
  var rr = (Math.random() * 24 - 12).toFixed(1) + 'deg';
  el.style.setProperty('--rx', rx);
  el.style.setProperty('--rr', rr);
  // Spread them horizontally across the lower portion of the video
  el.style.left = (20 + Math.random() * 60).toFixed(1) + '%';
  reactLayer.appendChild(el);
  el.addEventListener('animationend', function () { el.remove(); });
}

// FIX #5: Tip modal wiring (ethers.js integration)
var tipBtn        = document.getElementById('viewer-tip-btn');
var tipOverlay    = document.getElementById('tip-modal-overlay');
var tipCancelBtn  = document.getElementById('tip-cancel-btn');
var tipSendBtn    = document.getElementById('tip-send-btn');
var tipCustomInput = document.getElementById('tip-custom-input');
var tipStatus     = document.getElementById('tip-status');
var tipPresets    = document.getElementById('tip-presets');

// Preset selection
if (tipPresets) {
  tipPresets.querySelectorAll('.tip-preset-btn').forEach(function (btn) {
    btn.addEventListener('click', function () {
      tipPresets.querySelectorAll('.tip-preset-btn').forEach(function (b) { b.classList.remove('sele
      btn.classList.add('selected');
```

```
      if (tipCustomInput) tipCustomInput.value = btn.dataset.amount;
    });
  });
}

if (tipCustomInput) {
  tipCustomInput.addEventListener('input', function () {
    if (tipPresets) {
      tipPresets.querySelectorAll('.tip-preset-btn').forEach(function (b) { b.classList.remove('sele
      var matched = tipPresets.querySelector('[data-amount="' + tipCustomInput.value + '"]');
      if (matched) matched.classList.add('selected');
    }
  });
}

if (tipBtn) tipBtn.addEventListener('click', function () {
  if (!window.walletAddress || !currentViewer) {
    alert('Connect wallet to tip');
    return;
  }
  if (tipStatus) tipStatus.textContent = '';
  if (tipOverlay) tipOverlay.classList.add('open');
});

if (tipCancelBtn) tipCancelBtn.addEventListener('click', function () {
  if (tipOverlay) tipOverlay.classList.remove('open');
});

if (tipOverlay) tipOverlay.addEventListener('click', function (e) {
  if (e.target === tipOverlay) tipOverlay.classList.remove('open');
});

if (tipSendBtn) tipSendBtn.addEventListener('click', async function () {
  var amtRaw = tipCustomInput ? parseFloat(tipCustomInput.value) : NaN;
  if (!amtRaw || isNaN(amtRaw) || amtRaw <= 0) {
    if (tipStatus) { tipStatus.textContent = 'Enter a valid ETH amount.'; tipStatus.className = 'tip
    return;
  }

  tipSendBtn.disabled = true;
  if (tipStatus) { tipStatus.textContent = 'Confirm in wallet…'; tipStatus.className = 'tip-status';

  try {
    // Use ethers.js v5 (available as window.ethers from main.js)
    if (!window.ethers) throw new Error('ethers.js not loaded');
    var provider = new ethers.providers.Web3Provider(window.ethereum);
    var signer   = provider.getSigner();
```

```
    // Resolve recipient: prefer currentViewer's creator address if available
    var recipient = (currentViewer && currentViewer._sessionData && currentViewer._sessionData.creat
      || window.MSPLive.config.tipRecipient
      || null;

    if (!recipient) throw new Error('No recipient address found for this stream.');

    var tx = await signer.sendTransaction({
      to:    recipient,
      value: ethers.utils.parseEther(String(amtRaw)),
    });

    if (tipStatus) { tipStatus.textContent = 'Pending… ' + tx.hash.slice(0, 10) + '…'; tipStatus.cla
    await tx.wait(1);

    if (tipStatus) { tipStatus.textContent = '✓ Tip sent! Thank you.'; tipStatus.className = 'tip-st

    // Broadcast the tip via WebSocket for chat display
    if (currentViewer && currentViewer._ws && currentViewer._ws.send) {
      currentViewer._ws.send(JSON.stringify({ type: 'tip', amount: amtRaw, txHash: tx.hash }));
    }

    setTimeout(function () {
      if (tipOverlay) tipOverlay.classList.remove('open');
      tipSendBtn.disabled = false;
      if (tipStatus) tipStatus.textContent = '';
    }, 2500);

  } catch (e) {
    if (tipStatus) { tipStatus.textContent = e.message || 'Transaction failed.'; tipStatus.className
    tipSendBtn.disabled = false;
  }
});

// FIX #3: WebSocket with reconnect logic and configurable endpoint
(function initNotifWs() {
  var endpoint = (window.MSPLive && window.MSPLive.config && window.MSPLive.config.wsEndpoint)
    || ((location.protocol === 'https:' ? 'wss' : 'ws') + '://' + location.host + '/ws');

  var ws;
  var retryDelay = 1000;
  var maxDelay   = 30000;
  var destroyed  = false;

  function connect() {
    if (destroyed) return;
```

```
      ws = new WebSocket(endpoint);

      ws.onopen = function () {
        retryDelay = 1000; // reset backoff on successful connect
      };

      ws.onmessage = function (e) {
        var msg;
        try { msg = JSON.parse(e.data); } catch (err) { return; }
        if (msg.type === 'session_started') {
          if (window.MSPLive && MSPLive.renderSessions) {
            MSPLive.renderSessions('live-sessions-grid', function (sessionId) { openViewer(sessionId);
          }
        }
        if (msg.type === 'reaction' && msg.emoji) {
          spawnFloatingReaction(msg.emoji);
        }
      };

      ws.onerror = function () { /* errors are handled by onclose */ };

      ws.onclose = function () {
        if (destroyed) return;
        // Exponential backoff reconnect
        setTimeout(function () {
          retryDelay = Math.min(retryDelay * 2, maxDelay);
          connect();
        }, retryDelay);
      };
    }

    connect();

    // Expose teardown for page unload
    window.addEventListener('unload', function () {
      destroyed = true;
      if (ws) ws.close();
    });
  })();

})();
</script>

</body>
</html>
```

`asset-manager.html` (31.0 KB)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Asset Manager — Michie Stream Platform</title>
  <link rel="icon" type="image/svg+xml" href="assets/msp-vinyl.svg">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#F5F000">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet
  <link rel="stylesheet" href="styles/styles.css">
<style>
body { padding-bottom: 40px; }

/* ── Stats bar ──────────────────────────────────────────────────────── */
.am-stats-bar {
  background: var(--bg-surface);
  border: 1px solid var(--border-subtle);
  border-radius: 10px;
  display: flex;
  flex-wrap: wrap;
  margin-bottom: 24px;
  overflow: hidden;
}
.am-stat {
  border-right: 1px solid var(--border-subtle);
  flex: 1;
  min-width: 120px;
  padding: 14px 18px;
  text-align: center;
}
.am-stat:last-child { border-right: none; }
.am-stat-val { font-family: var(--font-mono); font-size: 20px; font-weight: 700; }
.am-stat-val.teal   { color: var(--teal); }
.am-stat-val.gold   { color: var(--gold); }
.am-stat-val.violet { color: var(--violet); }
.am-stat-label { color: var(--text-muted); font-size: 10px; letter-spacing: .08em; margin-top: 2px; te

/* ── Content type tabs ──────────────────────────────────────────────── */
.am-tabs { border-bottom: 1px solid var(--border-subtle); display: flex; gap: 4px; margin-bottom: 20px
.am-tab {
  background: none; border: none; border-bottom: 2px solid transparent;
  color: var(--text-secondary); cursor: pointer;
  font-family: var(--font-ui); font-size: 12px; font-weight: 700;
```

```
  letter-spacing: .06em; padding: 10px 16px; text-transform: uppercase;
  transition: color .15s, border-color .15s; white-space: nowrap;
}
.am-tab:hover  { color: var(--text-primary); }
.am-tab.active { color: var(--teal); border-bottom-color: var(--teal); }
.am-tab-count  { background: var(--bg-raised); border-radius: 8px; font-family: var(--font-mono); font

/* ── Asset rows ──────────────────────────────────────────────────────── */
.am-list { display: flex; flex-direction: column; }
.am-row {
  align-items: center;
  border-bottom: 1px solid var(--border-subtle);
  display: grid;
  gap: 0 12px;
  grid-template-columns: 44px 1fr 70px 80px 110px 160px;
  padding: 10px 4px;
  transition: background .12s;
}
.am-row:hover      { background: var(--bg-raised); border-radius: 6px; }
.am-row:last-child { border-bottom: none; }
@media (max-width: 900px) {
  .am-row { grid-template-columns: 44px 1fr 110px 160px; }
  .am-col-plays, .am-col-royalties { display: none; }
}

.am-cover    { border-radius: 4px; height: 44px; object-fit: cover; width: 44px; }
.am-cover-ph { align-items: center; background: var(--bg-raised); border-radius: 4px; color: var(--tex
.am-info     { min-width: 0; }
.am-title-text {
  align-items: center; color: var(--text-primary);
  display: flex; font-size: 13px; font-weight: 600;
  gap: 5px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap;
}
.am-meta { color: var(--text-secondary); font-size: 11px; }

.am-col-plays, .am-col-royalties { color: var(--text-muted); font-family: var(--font-mono); font-size:
.am-col-royalties { color: var(--gold); }

/* ── Supporter royalty toggle ────────────────────────────────────────── */
.sr-wrap { align-items: center; display: flex; flex-direction: column; gap: 2px; }
.sr-label { color: var(--text-muted); font-size: 9px; letter-spacing: .05em; text-align: center; text-
.sr-label.on { color: var(--gold); }
.form-check-input:checked { background-color: var(--gold) !important; border-color: var(--gold) !impor
.sr-cooldown { background: rgba(232,93,58,.08); border: 1px solid rgba(232,93,58,.2); border-radius: 5

/* ── Action buttons ──────────────────────────────────────────────────── */
.am-actions { align-items: center; display: flex; gap: 5px; justify-content: flex-end; flex-wrap: wrap
```

```
.am-btn { background: none; border: 1px solid var(--border-subtle); border-radius: 5px; color: var(--t
.am-btn:hover { border-color: var(--border-mid); color: var(--text-primary); }
.am-btn.play    { border-color: rgba(0,212,187,.3);   color: var(--teal); }
.am-btn.splits  { border-color: rgba(212,168,83,.3);  color: var(--gold); }
.am-btn.privacy { border-color: rgba(139,92,246,.3);  color: var(--violet); }

/* ── Privacy badge ──────────────────────────────────────────────────── */
.priv-badge { border-radius: 4px; font-size: 9px; font-weight: 700; letter-spacing: .05em; padding: 2p
.priv-badge.pub  { background: rgba(0,212,187,.12);  color: var(--teal); }
.priv-badge.priv { background: rgba(139,92,246,.12); color: var(--violet); }

/* ── Splits panel ───────────────────────────────────────────────────── */
.splits-panel { background: var(--bg-raised); border: 1px solid var(--border-subtle); border-radius: 8
.splits-panel.open { display: block; }
.splits-grid { display: grid; gap: 10px; grid-template-columns: repeat(auto-fill, minmax(120px, 1fr));
.split-field label { color: var(--text-muted); font-size: 10px; letter-spacing: .06em; text-transform:
.split-field input { background: var(--bg-surface); border: 1px solid var(--border-mid); border-radius
.split-field input:focus { border-color: var(--gold); }

/* ── Empty / loading ────────────────────────────────────────────────── */
.am-empty { color: var(--text-muted); font-size: 13px; padding: 40px 0; text-align: center; }
.am-empty a { color: var(--teal); }
.am-loading { align-items: center; color: var(--text-muted); display: flex; font-size: 13px; gap: 10px
.am-spinner { animation: am-spin .8s linear infinite; border: 2px solid var(--border-subtle); border-r
@keyframes am-spin { to { transform: rotate(360deg); } }

/* ── Toast ──────────────────────────────────────────────────────────── */
#am-toast { background: var(--bg-raised); border: 1px solid var(--border-mid); border-left: 3px solid
#am-toast.show { display: block; }
#am-toast.warn { border-left-color: var(--ember); }
</style>
</head>
<body class="page-asset-manager">

<!-- ═══ NAVBAR ═══════════════════════════════════════════════════════ -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark fixed-top">
  <div class="container-fluid">
    <a class="navbar-brand" href="index.html">Michie Stream Platform</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav"
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav me-auto">
        <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="listen.html">Just Listen</a></li>
        <li class="nav-item"><a class="nav-link" href="creators.html">Creators Corner</a></li>
        <li class="nav-item"><a class="nav-link" href="marketplace.html">NFT Marketplace</a></li>
```

```
        <li class="nav-item"><a class="nav-link" href="profile.html">Profile</a></li>
        <li class="nav-item"><a class="nav-link active" href="asset-manager.html">My Assets</a></li>
        <li class="nav-item" data-requires="hostConcert" style="display:none;"><a class="nav-link" hre
      </ul>
      <div class="d-flex align-items-center gap-2">
        <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
        <button id="btn-disconnect" class="btn btn-outline-light btn-sm d-none">Disconnect</button>
        <span id="walletAddress" class="small text-light wallet-address-display"></span>
        <span class="small text-warning fw-semibold user-name-display"></span>
      </div>
    </div>
  </div>
</nav>

<!-- ═══ PAGE CONTENT ═══════════════════════════════════════════════════ -->
<div class="container py-4">

  <div class="d-flex align-items-center justify-content-between flex-wrap gap-2 mb-4" style="border-bo
    <h1 style="font-family:var(--font-display);font-size:clamp(22px,3.5vw,32px);font-weight:300;">My A
    <a href="creators.html" class="btn btn-primary btn-sm">+ Upload New</a>
  </div>

  <!-- Wallet prompt -->
  <div id="am-wallet-prompt" style="display:none;">
    <div class="am-empty">
      <p class="mb-3">Connect your wallet to manage your assets.</p>
      <button class="btn btn-primary btn-sm" data-connect-wallet>Connect Wallet</button>
    </div>
  </div>

  <!-- Stats -->
  <div class="am-stats-bar" id="am-stats" style="display:none;">
    <div class="am-stat"><div class="am-stat-val teal" id="stat-total">0</div><div class="am-stat-labe
    <div class="am-stat"><div class="am-stat-val" id="stat-plays">0</div><div class="am-stat-label">To
    <div class="am-stat"><div class="am-stat-val gold" id="stat-royalties">0.0000</div><div class="am-
    <div class="am-stat"><div class="am-stat-val violet" id="stat-sr">0</div><div class="am-stat-label
  </div>

  <!-- Tabs -->
  <div class="am-tabs" id="am-tabs" style="display:none;" role="tablist">
    <button class="am-tab active" data-am-tab="all">All <span class="am-tab-count" id="tc-all">0</span
    <button class="am-tab" data-am-tab="music">Music <span class="am-tab-count" id="tc-music">0</span>
    <button class="am-tab" data-am-tab="video">Videos <span class="am-tab-count" id="tc-video">0</span
    <button class="am-tab" data-am-tab="podcast">Podcasts <span class="am-tab-count" id="tc-podcast">0
    <button class="am-tab" data-am-tab="art">Art <span class="am-tab-count" id="tc-art">0</span></butt
  </div>
```

```
  <!-- Asset list -->
  <div id="am-list-container">
    <div class="am-loading"><div class="am-spinner"></div>Loading your assets…</div>
  </div>

</div>

<div id="am-toast"></div>

<!-- ═══ WALLET MODAL ═══════════════════════════════════════════════════ -->
<div class="modal fade" id="walletModal" tabindex="-1" aria-labelledby="walletModalLabel" aria-hidden=
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content bg-dark text-light">
      <div class="modal-header border-0">
        <h5 class="modal-title" id="walletModalLabel">Connect a Wallet</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal" aria-label="Cl
      </div>
      <div class="modal-body pt-0">
        <div class="list-group">
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
          <button class="list-group-item list-group-item-action d-flex align-items-center gap-3" id="b
        </div>
        <div class="small text-secondary mt-3" id="wallet-help"></div>
      </div>
    </div>
  </div>
</div>

<!-- ═══ SCRIPTS ═══════════════════════════════════════════════════════ -->
<script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
<script src="vendor/ethers/ethers.umd.min.js"></script>
<script src="vendor/hls/hls.min.js"></script>
<script src="vendor/ipfs/index.min.js"></script>
<script src="Scripts/wallets.js"></script>
<script src="Scripts/common.js"></script>
<script src="Scripts/favorites.js"></script>
<script src="Scripts/main.js"></script>

<script>
(function () {
  'use strict';

  var allAssets  = [];
  var currentTab = 'all';
```

```
var openSplits = null;

function esc(s) {
  return String(s||'').replace(/&/g,'&amp;').replace(/"/g,'&quot;').replace(/</g,'&lt;').replace(/>/
}

// ── Toast ──────────────────────────────────────────────────────────
var toastTimer;
function toast(msg, warn) {
  var el = document.getElementById('am-toast');
  if (!el) return;
  el.textContent = msg;
  el.className = 'show' + (warn ? ' warn' : '');
  clearTimeout(toastTimer);
  toastTimer = setTimeout(function () { el.className = ''; }, 3500);
}

// ── 30-day cooldown (localStorage) ────────────────────────────────
function lockKey(cid) { return 'am_sr_lock:' + (window.walletAddress||'anon').toLowerCase() + ':' +
function getLock(cid) { var v = localStorage.getItem(lockKey(cid)); return v ? parseInt(v,10) : 0; }
function setLock(cid) { localStorage.setItem(lockKey(cid), String(Date.now() + 30*24*60*60*1000)); }
function isLocked(cid) { return getLock(cid) > Date.now(); }
function daysLeft(cid) { return Math.ceil((getLock(cid) - Date.now()) / (24*60*60*1000)); }

// ── Tab wiring ─────────────────────────────────────────────────────
document.querySelectorAll('.am-tab').forEach(function (btn) {
  btn.addEventListener('click', function () {
    document.querySelectorAll('.am-tab').forEach(function (b) { b.classList.remove('active'); });
    btn.classList.add('active');
    currentTab = btn.dataset.amTab;
    renderList();
  });
});

function filtered() {
  if (currentTab === 'all') return allAssets;
  if (currentTab === 'art') return allAssets.filter(function (a) { return a.contentType==='art_still
  return allAssets.filter(function (a) { return (a.contentType||'music') === currentTab; });
}

// ── Render ─────────────────────────────────────────────────────────
function renderList() {
  var c = document.getElementById('am-list-container');
  if (!c) return;
  var items = filtered();
  if (!items.length) {
    c.innerHTML = '<div class="am-empty">No assets in this category. <a href="creators.html">Upload
```

```
  return;
}
var icons = { music:'🎵', podcast:'🎙', video:'🎬', art_still:'🖼', art_animated:'🎨' };
c.innerHTML = '<div class="am-list">' + items.map(function (item) {
  var ct      = item.contentType || 'music';
  var icon    = icons[ct] || '🎵';
  var cover   = item.coverUrl
    ? '<img class="am-cover" src="' + esc(item.coverUrl) + '" alt="">'
    : '<div class="am-cover-ph">' + icon + '</div>';
  var srOn    = !!item.supporterRoyaltyEnabled;
  var locked  = isLocked(item.contentId);
  var isPriv  = !!item.isPrivate;
  var vinyl   = srOn ? '<img src="assets/msp-vinyl.svg" width="13" height="13" title="Supporter ro
  var privBadge = '<span class="priv-badge ' + (isPriv?'priv':'pub') + '">' + (isPriv?'🔒 Private'
  var plays   = item.plays !== undefined ? item.plays : '—';
  var earned  = item.royaltiesEarned ? parseFloat(item.royaltiesEarned).toFixed(4)+' ETH' : '0.000
  var coolNote = locked ? '<div class="sr-cooldown">' + daysLeft(item.contentId) + ' days locked</
  var srBlock =
    '<div class="sr-wrap">' +
      '<div class="form-check form-switch mb-0">' +
        '<input class="form-check-input" type="checkbox" role="switch"' +
          ' id="sr-' + esc(item.contentId) + '"' +
          (srOn?' checked':'') + (locked?' disabled':'') +
          ' data-contentid="' + esc(item.contentId) + '">' +
      '</div>' +
      '<label class="sr-label' + (srOn?' on':'') + '" for="sr-' + esc(item.contentId) + '">' + (sr
      coolNote +
    '</div>';
  return (
    '<div class="am-row" id="row-' + esc(item.contentId) + '">' +
      cover +
      '<div class="am-info">' +
        '<div class="am-title-text">' + esc(item.title||'Untitled') + vinyl + '</div>' +
        '<div class="am-meta">' + esc(item.artistName||'—') + ' · ' + ct.toUpperCase() + ' · ' + p
      '</div>' +
      '<div class="am-col-plays">' + plays + '<br><span style="font-size:9px;color:var(--text-mute
      '<div class="am-col-royalties">' + earned + '<br><span style="font-size:9px">EARNED</span></
      srBlock +
      '<div class="am-actions">' +
        '<button class="am-btn play" data-action="play"' +
          ' data-hlsurl="' + esc(item.hlsUrl||'') + '"' +
          ' data-title="' + esc(item.title||'') + '"' +
          ' data-artist="' + esc(item.artistName||'') + '"' +
          ' data-cover="' + esc(item.coverUrl||'') + '">▶ Play</button>' +
        '<button class="am-btn splits" data-action="splits" data-contentid="' + esc(item.contentId
        '<button class="am-btn privacy" data-action="privacy" data-contentid="' + esc(item.content
          (isPriv?'🌐 Make Public':'🔒 Make Private') + '</button>' +
```

```
        '</div>' +
      '</div>' +
      // Splits panel
      '<div class="splits-panel" id="splits-' + esc(item.contentId) + '">' +
        '<h6 class="mb-3" style="color:var(--gold)">💸 Royalty Splits — ' + esc(item.title||'Untitle
        '<p class="text-muted small mb-3">All splits must total 100%. Supporter royalties flow to cu
        '<div class="splits-grid">' +
          '<div class="split-field"><label>Artist %</label><input type="number" min="0" max="100" va
          '<div class="split-field"><label>NFT Holders %</label><input type="number" min="0" max="10
          '<div class="split-field"><label>Activity Pool %</label><input type="number" min="0" max="
          '<div class="split-field"><label>Supporter %</label><input type="number" min="0" max="100"
        '</div>' +
        '<div class="d-flex gap-2 mt-3">' +
          '<button class="btn btn-sm btn-warning text-dark" data-action="save-splits" data-contentid
          '<button class="btn btn-sm btn-outline-secondary" data-action="close-splits" data-contenti
        '</div>' +
      '</div>'
    );
  }).join('') + '</div>';
  wireActions();
}

// ── Event delegation ──────────────────────────────────────────────────
function wireActions() {
  var c = document.getElementById('am-list-container');
  if (!c) return;

  // SR toggles
  c.querySelectorAll('.form-check-input[data-contentid]').forEach(function (chk) {
    chk.addEventListener('change', function () {
      var cid = chk.dataset.contentid;
      var on  = chk.checked;
      if (isLocked(cid)) { chk.checked = !on; toast('Asset is in 30-day cooldown — cannot change.',
      if (!on) {
        if (!confirm('Disabling Supporter Royalties notifies all supporters who have this track in p
        setLock(cid);
      }
      // Update label + vinyl badge
      var label = c.querySelector('label[for="sr-' + cid + '"]');
      if (label) { label.textContent = on ? '★ Supporter' : 'Supporter'; label.className = 'sr-label
      var title = c.querySelector('#row-' + cid + ' .am-title-text');
      if (title) {
        var existing = title.querySelector('img');
        if (on && !existing) {
          var img = document.createElement('img'); img.src='assets/msp-vinyl.svg'; img.width=13; img
          title.appendChild(img);
        } else if (!on && existing) { existing.remove(); }
```

```
    }
    var item = allAssets.find(function (a) { return a.contentId===cid; });
    if (item) item.supporterRoyaltyEnabled = on;
    updateStats();
    apiSR(cid, on);
  });
});

c.addEventListener('click', function (e) {
  var btn = e.target.closest('[data-action]');
  if (!btn) return;
  var action = btn.dataset.action;
  var cid    = btn.dataset.contentid;

  if (action === 'play') {
    if (typeof window.playHls === 'function') {
      window.playHls(btn.dataset.hlsurl||'', '');
      var tn = document.getElementById('track-name'); if (tn) { tn.textContent=btn.dataset.title||
      var an = document.getElementById('player-artist-name'); if (an) an.textContent = btn.dataset
    }
    return;
  }
  if (action === 'splits') {
    var p = document.getElementById('splits-' + cid); if (!p) return;
    if (openSplits && openSplits !== cid) { var prev = document.getElementById('splits-' + openSpl
    var wasOpen = p.classList.contains('open');
    p.classList.toggle('open', !wasOpen);
    openSplits = wasOpen ? null : cid;
    return;
  }
  if (action === 'close-splits') {
    var p2 = document.getElementById('splits-' + cid); if (p2) p2.classList.remove('open'); openSp
  }
  if (action === 'save-splits') {
    var inputs = c.querySelectorAll('[data-split][data-contentid="' + cid + '"]');
    var splits = {}; var total = 0;
    inputs.forEach(function (inp) { splits[inp.dataset.split] = parseFloat(inp.value)||0; total +=
    if (Math.abs(total-100) > 0.01) { toast('Splits must total 100%. Currently: ' + total.toFixed(
    var p3 = document.getElementById('splits-' + cid); if (p3) p3.classList.remove('open'); openSp
    var item = allAssets.find(function (a) { return a.contentId===cid; }); if (item) item.splits=s
    apiSplits(cid, splits);
    toast('✓ Royalty splits saved.');
    return;
  }
  if (action === 'privacy') {
    var isP    = btn.dataset.private === '1';
    var newP   = !isP;
```

```
      btn.dataset.private = newP ? '1' : '0';
      btn.textContent = newP ? '🌐 Make Public' : '🔒 Make Private';
      var badge = c.querySelector('#row-' + cid + ' .priv-badge');
      if (badge) { badge.className='priv-badge '+(newP?'priv':'pub'); badge.textContent=newP?'🔒 Pri
      var item2 = allAssets.find(function (a) { return a.contentId===cid; }); if (item2) item2.isPri
      apiPrivacy(cid, newP);
      toast(newP ? '🔒 Set to Private.' : '🌐 Set to Public.');
      return;
    }
  });
}

// ── API stubs (server routes to implement) ──────────────────────────────
async function apiSR(cid, enabled) {
  try {
    await fetch('/api/catalog/' + cid + '/supporter-royalty', { method:'POST', headers:{'Content-Typ
  } catch (_) {}
}
async function apiSplits(cid, splits) {
  try {
    await fetch('/api/royalty-splits', { method:'POST', headers:{'Content-Type':'application/json'},
  } catch (_) {}
}
async function apiPrivacy(cid, isPrivate) {
  try {
    await fetch('/api/catalog/' + cid + '/privacy', { method:'POST', headers:{'Content-Type':'applic
  } catch (_) {}
}

// ── Stats + tab counts ──────────────────────────────────────────────────
function updateStats() {
  var totalP = allAssets.reduce(function(s,a){return s+(a.plays||0);},0);
  var totalR = allAssets.reduce(function(s,a){return s+(parseFloat(a.royaltiesEarned)||0);},0);
  var srCnt  = allAssets.filter(function(a){return a.supporterRoyaltyEnabled;}).length;
  var el; ['stat-total','stat-plays','stat-royalties','stat-sr'].forEach(function(id,i){
    el=document.getElementById(id); if(!el) return;
    el.textContent=[allAssets.length,totalP,totalR.toFixed(4),srCnt][i];
  });
}
function updateTabCounts() {
  var counts={all:allAssets.length,music:0,video:0,podcast:0,art:0};
  allAssets.forEach(function(a){
    var ct=a.contentType||'music';
    if(ct==='art_still'||ct==='art_animated') counts.art++;
    else if(counts[ct]!==undefined) counts[ct]++;
  });
  Object.keys(counts).forEach(function(k){var el=document.getElementById('tc-'+k);if(el)el.textConte
```

```
  }

  // ── Boot ──────────────────────────────────────────────────────────────
  async function boot() {
    if (!window.walletAddress) {
      var prompt = document.getElementById('am-wallet-prompt');
      var loader = document.getElementById('am-list-container');
      if (prompt) prompt.style.display = '';
      if (loader) loader.innerHTML = '';
      document.addEventListener('walletConnected', function(){ if(prompt)prompt.style.display='none';
      return;
    }
    try {
      var r = await fetch('/api/catalog');
      if (!r.ok) throw new Error('HTTP ' + r.status);
      var all = await r.json();
      allAssets = all.filter(function(item){ return (item.wallet||'').toLowerCase()===window.walletAdd

      var statsEl=document.getElementById('am-stats'); if(statsEl) statsEl.style.display=allAssets.len
      var tabsEl=document.getElementById('am-tabs');   if(tabsEl)  tabsEl.style.display=allAssets.leng
      updateTabCounts(); updateStats();

      // Handle deep-link anchor (#contentId from profile.html)
      var anchor = window.location.hash.slice(1);
      if (anchor) {
        var found = allAssets.find(function(a){return a.contentId===anchor;});
        if (found) currentTab = found.contentType||'music';
        // Activate matching tab button
        document.querySelectorAll('.am-tab').forEach(function(b){
          b.classList.toggle('active', b.dataset.amTab===currentTab||(!found&&b.dataset.amTab==='all')
        });
      }
      renderList();
      if (anchor) setTimeout(function(){ var el=document.getElementById('row-'+anchor); if(el)el.scrol
    } catch(err) {
      var c=document.getElementById('am-list-container'); if(c) c.innerHTML='<div class="am-empty">Fai
    }
  }

  document.addEventListener('walletConnected', function(){ boot(); });
  if (window.walletAddress) boot();

})();
</script>

</body>
</html>
```

```

## FRONTEND — JAVASCRIPT
---

### `main.js` (78.6 KB)

```javascript
// public/Scripts/main.js
'use strict';

/**
 * Michie Stream Platform — main browser script
 *
 * Globals expected (set by vendor scripts + wallets.js):
 *   window.ethers           — ethers v5 UMD
 *   window.MSP_CONFIG       — { contentCAAddress, royaltyPayoutAddress, escrowAddress, mspAdminAddres
 *   window.MSP_NFT_BYTECODE — bytecode for user NFT contract (optional)
 *   window.IPFS_GATEWAY     — e.g. 'https://ipfs.io/ipfs/' (common.js sets default)
 *   window.playHls          — function(url, metaUrl) from common.js
 *   window.openWalletModal  — from wallets.js
 *   window.walletAddress    — set by wallets.js after connect
 *   window.ethersSigner     — set by wallets.js after connect
 *   window.ethersProvider   — set by wallets.js after connect
 */

(function () {
  'use strict';

  var $ = function (id) { return document.getElementById(id); };
  var GATEWAY = function () { return window.IPFS_GATEWAY || 'https://ipfs.io/ipfs/'; };

  // Resolve a CID or URL to a playable HTTP URL.
  // Handles ipfs://, local: (DEV_MODE local catalog), and plain HTTP.
  function resolveUrl(cid) {
    if (!cid) return '';
    if (cid.startsWith('local:')) {
      var id = cid.replace('local:', '');
      return '/catalog/' + id + '/hls/master.m3u8';
    }
    if (cid.startsWith('/')) return cid;  // already a local path
    if (cid.startsWith('http')) return cid;
    return cid.replace('ipfs://', GATEWAY());
  }
  function resolveMetaUrl(cid) {
    if (!cid) return null;
    if (cid.startsWith('local:')) {
```
```

```
    var id = cid.replace('local:', '');
    return '/catalog/' + id + '/metadata.json';
  }
  return GATEWAY() + cid;
}

// ═══════════════════════════════════════════════════════════
//  CONFIG
// ═══════════════════════════════════════════════════════════
var CFG                  = window.MSP_CONFIG || {};
var ABIS                 = CFG.abis           || {};
var contentCAAddress     = CFG.contentCAAddress;
var royaltyPayoutAddress = CFG.royaltyPayoutAddress;
var escrowAddress        = CFG.escrowAddress;
var mspAdminAddress      = CFG.mspAdminAddress;

if (!window.ethers) { console.warn('MSP: ethers not found — load ethers UMD before main.js'); }

// ═══════════════════════════════════════════════════════════
//  SESSION STATE
// ═══════════════════════════════════════════════════════════
var _access    = null;
var _profile   = null;
var _favorites = null;   // Set of CIDs the current user has favorited

function getAccess()    { return _access  || {}; }
function getProfile()   { return _profile || JSON.parse(localStorage.getItem('profile') || '{}'); }
function getFavorites() { return _favorites || new Set(); }

// ── Favorites API helpers ──────────────────────────────────
async function loadFavorites(address) {
  if (!address) return;
  try {
    var res = await fetch('/api/favorites/' + address);
    if (res.ok) {
      var data = await res.json();
      _favorites = new Set(data.favorites || []);
      _syncHeartButtons();
    }
  } catch (e) { console.debug('loadFavorites failed:', e.message); }
}

async function toggleFavorite(cid) {
  var addr = window.walletAddress;
  if (!addr) throw new Error('Connect your wallet first.');
  var isFav = getFavorites().has(cid);
  var endpoint = isFav ? '/api/favorites/remove' : '/api/favorites/add';
```

```
  var res = await fetch(endpoint, {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ wallet: addr, cid: cid }),
  });
  if (!res.ok) { throw new Error(await safeApiError(res, 'Failed to update favorites')); }
  var data = await res.json();
  _favorites = new Set(data.favorites || []);
  _syncHeartButtons();
  return !isFav;  // returns new state: true = now favorited
}

function _syncHeartButtons() {
  document.querySelectorAll('[data-fav-cid]').forEach(function (btn) {
    var cid = btn.dataset.favCid;
    var active = getFavorites().has(cid);
    btn.classList.toggle('fav-active', active);
    btn.setAttribute('aria-label', active ? 'Remove from Favorites' : 'Add to Favorites');
    btn.textContent = active ? '♥' : '♡';
  });
}

function makeFavButton(cid) {
  var btn = document.createElement('button');
  btn.className    = 'fav-btn';
  btn.dataset.favCid = cid;
  btn.textContent  = getFavorites().has(cid) ? '♥' : '♡';
  btn.setAttribute('aria-label', getFavorites().has(cid) ? 'Remove from Favorites' : 'Add to Favorit
  btn.addEventListener('click', async function (e) {
    e.stopPropagation();
    if (!window.walletAddress) {
      if (typeof window.openWalletModal === 'function') window.openWalletModal();
      return;
    }
    btn.disabled = true;
    try {
      var nowFav = await toggleFavorite(cid);
      btn.classList.toggle('fav-active', nowFav);
      btn.textContent = nowFav ? '♥' : '♡';
    } catch (err) {
      console.warn('toggleFavorite:', err.message);
    } finally {
      btn.disabled = false;
    }
  });
  return btn;
}
```

```
// ═══════════════════════════════════════════════════════════
//  CAPABILITY CHECKS
// ═══════════════════════════════════════════════════════════
var CAN = {
  stream:         function () { return ['listener_1','listener_2','listener_3','creator_active','nft
  browseNFTs:     function () { return !!getAccess().level && getAccess().level !== 'none'; },
  buyNFTs:        function () { return CAN.browseNFTs(); },
  watchConcerts:  function () { return true; },
  concertChat:    function () { return ['listener_2','listener_3','creator_active','nft_creator_acti
  tipAnonymous:   function () { return true; },
  tipRecognized:  function () { return CAN.stream(); },
  createPlaylist: function () { return ['listener_2','listener_3','creator_active','nft_creator_acti
  hostDjSet:      function () { return ['listener_2','listener_3','creator_active','nft_creator_acti
  upload:         function () { return ['creator_active','nft_creator_active','nft_creator_passive']
  mintNFT:        function () { return CAN.upload(); },
  hostConcert:    function () { return ['creator_active','nft_creator_active'].indexOf(getAccess().l
  createAds:      function () { return ['creator_active','nft_creator_active'].indexOf(getAccess().l
  setSplits:      function () { return ['creator_active','nft_creator_active','nft_creator_passive']
  earnPassive:    function () { return ['listener_3','creator_active','nft_creator_active'].indexOf(
  earnActivity:   function () { return ['listener_2','listener_3','creator_active','nft_creator_acti
  downloads:      function () { return ['listener_3','creator_active','nft_creator_active'].indexOf(
  favorite:       function () { return true; },  // All user roles — no subscription gate (MSP spec)
  supporterSub:   function () { return CAN.upload(); }
};

// ═══════════════════════════════════════════════════════════
//  WALLET HELPERS
// ═══════════════════════════════════════════════════════════
function requireSigner() {
  var signer = window.ethersSigner;
  var addr   = window.walletAddress;
  if (!signer || !addr) { throw new Error('Wallet not connected. Please connect your wallet first.')
  return { signer: signer, address: addr };
}

function requireProvider() {
  var provider = window.ethersProvider;
  if (!provider) { throw new Error('Wallet provider not available. Connect your wallet first.'); }
  return provider;
}

function getContract(address, abi, signerOrProvider) {
  if (!window.ethers) { throw new Error('ethers not loaded'); }
  return new window.ethers.Contract(address, abi, signerOrProvider);
}

// ═══════════════════════════════════════════════════════════
```

```
//  PROFILE CACHE  (address-scoped localStorage)
// ═══════════════════════════════════════════════════════════

function profileCacheKey(address) {
  return 'msp_profile_' + address.toLowerCase();
}

function saveProfileCache(address, data) {
  _profile = data;
  localStorage.setItem(profileCacheKey(address), JSON.stringify(data));
  localStorage.setItem('profile', JSON.stringify(data)); // keep legacy key in sync
}

function loadProfileCache(address) {
  try {
    var raw = localStorage.getItem(profileCacheKey(address));
    if (raw) { return JSON.parse(raw); }
  } catch (_) {}
  return null;
}

// ═══════════════════════════════════════════════════════════
//  DISPLAY NAME  — propagate to all .user-name-display elements
// ═══════════════════════════════════════════════════════════
function applyUserName(name) {
  if (!name) { return; }
  document.querySelectorAll('.user-name-display').forEach(function (el) {
    el.textContent = name;
  });
  // Pre-fill artist name on creators page if the field is still blank
  var artistInput = $('artist-name');
  if (artistInput && !artistInput.value) { artistInput.value = name; }
}

// ═══════════════════════════════════════════════════════════
//  PROFILE + ACCESS
// ═══════════════════════════════════════════════════════════
async function fetchAccess(address) {
  try {
    var res = await fetch('/api/access/' + address);
    if (res.ok) { _access = await res.json(); return _access; }
  } catch (e) { console.warn('fetchAccess failed:', e.message); }
  _access = { level: 'none', tier: 0, active: false };
  return _access;
}

/**
```

```
 * fetchOrCreateProfile — the only place a name prompt ever appears.
 *
 * Order:
 *   1. Hit localStorage first (address-scoped key).  Instant, no network, no prompt.
 *      A background fetch silently refreshes the cache.
 *   2. Cache miss → fetch /api/profile/:address.
 *      Success → save to cache, return.
 *      404     → this is a brand-new user; show the name prompt exactly once.
 *      Other error / network fail → use legacy 'profile' key if present, else throw.
 *   3. On prompt: create profile via POST, save to cache, never prompt again.
 */
async function fetchOrCreateProfile(address) {

  // ── Step 1: local cache hit ──────────────────────────────────────────────
  var cached = loadProfileCache(address);
  if (cached && cached.user_id) {
    _profile = cached;
    applyUserName(cached.name);

    if (cached._pending) {
      // Profile was saved locally when the server was unreachable.
      // Retry creation in the background — once it succeeds the real profile
      // replaces the pending one and the _pending flag is gone.
      fetch('/api/create-profile', {
        method:  'POST',
        headers: { 'Content-Type': 'application/json' },
        body:    JSON.stringify({ wallet: address, name: cached.name, account_type: cached.account_t
      }).then(function (r) { return r.ok ? r.json() : null; })
        .then(function (data) {
          if (data && data.user_id) {
            saveProfileCache(address, data);
            applyUserName(data.name);
          }
        }).catch(function () {});
    } else {
      // Regular background refresh to pick up server-side changes
      fetch('/api/profile/' + address)
        .then(function (r) { return r.ok ? r.json() : null; })
        .then(function (data) {
          if (data && data.user_id) {
            saveProfileCache(address, data);
            applyUserName(data.name);
          }
        }).catch(function () {});
    }
    return _profile;
  }
```

```
// ── Step 2: cache miss — ask the server ──────────────────────────────
var serverStatus = 0;
var serverData   = null;
var serverReachable = true;
try {
  var res = await fetch('/api/profile/' + address);
  serverStatus = res.status;
  if (res.ok) {
    var data = await res.json();
    if (data && data.user_id) { serverData = data; }
  }
} catch (e) {
  // fetch() threw — backend is not running (e.g. VS Code Live Server).
  // Fall back to the legacy 'profile' key if it exists, otherwise treat
  // this wallet as having no profile yet but don't prompt — wait until
  // the backend is reachable.
  serverReachable = false;
  var legacyRaw = localStorage.getItem('profile');
  if (legacyRaw) {
    try {
      var legacy = JSON.parse(legacyRaw);
      if (legacy && legacy.user_id) {
        saveProfileCache(address, legacy);
        applyUserName(legacy.name);
        return _profile;
      }
    } catch (_) {}
  }
  // No legacy data either — treat as new user but still show the prompt
  // so they can interact with the UI. serverReachable=false means the
  // profile will be saved locally and creation retried when server is up.
}

if (serverData) {
  saveProfileCache(address, serverData);
  applyUserName(serverData.name);
  return _profile;
}

// Non-404 server error — don't prompt, surface the error
if (serverReachable && serverStatus !== 0 && serverStatus !== 404) {
  throw new Error('Profile server error: ' + serverStatus);
}

// If server is unreachable AND there's no legacy data, the code falls
// through to Step 3 which will save a pending profile locally.
```

```
// ── Step 3: true 404 — brand-new user, prompt exactly once ───────────────
//
// IMPORTANT: We save a local "pending" profile to localStorage BEFORE the
// server call. This means even if the backend is unavailable (e.g. VS Code
// Live Server with no running Node server), the user is only ever asked for
// their name once. The pending profile is replaced by the real server
// profile as soon as the backend becomes reachable.

var name = prompt('Welcome to Michie Stream!\n\nEnter your display name:');
if (!name || !name.trim()) {
  throw new Error('A display name is required to create your profile.');
}

var isCreator    = confirm('Signing up as a Creator or Artist?\n\nOK = Creator  |  Cancel = Listen
var account_type = isCreator ? 'creator' : 'listener';

// Persist immediately — guarantees the cache hit on every subsequent load
var pendingProfile = {
  user_id:       'pending_' + address.toLowerCase(),
  wallet_address: address,
  name:          name.trim(),
  account_type:  account_type,
  _pending:      true   // flag so background refresh knows to retry creation
};
saveProfileCache(address, pendingProfile);
applyUserName(pendingProfile.name);

// Best-effort server creation — failure is silent, pending profile stays in cache
try {
  var createRes = await fetch('/api/create-profile', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({ wallet: address, name: name.trim(), account_type: account_type })
  });
  if (createRes.ok) {
    var created = await createRes.json();
    saveProfileCache(address, created);  // overwrite pending with real profile
    applyUserName(created.name);
  } else {
    console.warn('Profile creation returned', createRes.status, '— using local pending profile unt
  }
} catch (netErr) {
  console.warn('Profile creation network error — using local pending profile:', netErr.message);
}

return _profile;
```

```
}

// ═══════════════════════════════════════════════════════════
//  UI GATES
// ═══════════════════════════════════════════════════════════
function applyCapabilityGates() {
  var level  = getAccess().level || 'none';
  var active = getAccess().active;

  document.querySelectorAll('[data-requires]').forEach(function (el) {
    var cap = el.dataset.requires;
    el.style.display = (typeof CAN[cap] === 'function' && CAN[cap]()) ? '' : 'none';
  });

  var chatInput = $('concert-chat-input');
  if (chatInput) { chatInput.style.display = CAN.concertChat() ? '' : 'none'; }

  var expiryEl = $('subscription-expiry');
  if (expiryEl && getAccess().subscription_expiry) {
    var days = Math.max(0, Math.floor((getAccess().subscription_expiry - Date.now()) / 86400000));
    expiryEl.textContent = active
      ? ('Access expires in ' + days + ' day' + (days !== 1 ? 's' : ''))
      : 'Subscription expired';
    expiryEl.className = active ? 'text-success small' : 'text-danger small';
  }

  var nftBadge = $('platform-nft-badge');
  if (nftBadge) {
    var isPlatformNft = ['nft_creator_active','nft_creator_passive'].indexOf(level) !== -1;
    nftBadge.style.display = isPlatformNft ? '' : 'none';
    if (isPlatformNft) {
      nftBadge.textContent = 'Platform NFT \u00b7 ' + (getAccess().royalty_fee_rate * 100).toFixed(1
      nftBadge.className   = 'badge bg-warning text-dark';
    }
  }

  var feeEl = $('royalty-fee-rate');
  if (feeEl && getAccess().royalty_fee_rate != null) {
    feeEl.textContent = (getAccess().royalty_fee_rate * 100).toFixed(1) + '%';
  }

  var uploadBtn = $('upload-btn');
  if (uploadBtn) { uploadBtn.textContent = CAN.mintNFT() ? 'Upload & Mint' : 'Upload'; }

  var mktSection = $('marketplace-section');
  if (mktSection && !CAN.browseNFTs()) {
    mktSection.innerHTML =
```

```
      '<div class="text-center py-5">' +
      '<h4>NFT Marketplace</h4>' +
      '<p class="text-muted">Subscribe to browse and purchase NFTs.</p>' +
      '<a href="listen.html#subscribe" class="btn btn-primary">Subscribe Now</a>' +
      '</div>';
  }

  // Show/hide the royalty fee row on creators page
  var feeRow = $('fee-row');
  if (feeRow) { feeRow.style.display = (feeEl && feeEl.textContent.trim()) ? '' : 'none'; }
}

// ═══════════════════════════════════════════════════════════
//  API ERROR HELPER
// ═══════════════════════════════════════════════════════════
async function safeApiError(res, fallback) {
  try {
    var ct = res.headers.get('content-type') || '';
    if (ct.includes('application/json')) {
      var j = await res.json();
      return j.error || j.message || fallback || ('Server error ' + res.status);
    }
  } catch (_) {}
  return fallback || ('Server error ' + res.status);
}

// ═══════════════════════════════════════════════════════════
//  SUBSCRIPTION
// ═══════════════════════════════════════════════════════════
async function subscribePlan(plan) {
  var addr = requireSigner().address;
  var profile = getProfile();
  if (!profile || !profile.user_id) { throw new Error('Profile not found. Reconnect your wallet.');

  try {
    var feesData = await fetch('/api/fees').then(function (r) { return r.ok ? r.json() : {}; });
    var planDef  = feesData.subscription_plans && feesData.subscription_plans[plan];
    if (planDef && planDef.price_eth) {
      var signer = requireSigner().signer;
      var esc    = getContract(escrowAddress, ABIS.escrow, signer);
      var tx     = await esc.subscribe({ value: window.ethers.utils.parseEther(String(planDef.price_
      await tx.wait();
    }
  } catch (e) { console.warn('On-chain subscription payment skipped (ok in dev):', e.message); }

  var res = await fetch('/api/subscribe', {
    method:  'POST',
```

```
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({ wallet: addr, plan: plan })
  });
  if (!res.ok) { throw new Error(await safeApiError(res, 'Subscription failed. Please try again.'));
  var data = await res.json();

  _access = await fetchAccess(addr);
  _profile = await fetchOrCreateProfile(addr);
  applyCapabilityGates();
  // Notify live_studio.html that access level changed (e.g. after subscribing)
  window.dispatchEvent(new CustomEvent('msp:accessChanged'));
  return data;
}

// ═══════════════════════════════════════════════════════════
//  TIPS
// ═══════════════════════════════════════════════════════════
async function sendTip(params) {
  var toWallet    = params.toWallet;
  var tipType     = params.tipType;
  var amountEth   = params.amountEth;
  var djSetId     = params.djSetId;
  var artistSplits = params.artistSplits;
  var djPercent   = params.djPercent;
  var s           = requireSigner();
  var tx = await s.signer.sendTransaction({
    to:    toWallet,
    value: window.ethers.utils.parseEther(String(amountEth))
  });
  await tx.wait();
  var res = await fetch('/api/tip', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({
      from_wallet:   s.address,
      to_wallet:     toWallet,
      tip_type:      tipType,
      amount_eth:    amountEth,
      dj_set_id:     djSetId       || null,
      artist_splits: artistSplits  || [],
      dj_percent:    (djPercent != null) ? djPercent : 100
    })
  });
  if (!res.ok) { throw new Error(await safeApiError(res)); }
  return res.json();
}
```

```
function renderTipButton(container, opts) {
  if (!container || !opts.toWallet) { return; }
  var btn = document.createElement('button');
  btn.className   = 'btn btn-sm btn-outline-warning ms-2 tip-btn';
  btn.textContent = '\uD83D\uDCB0 ' + (opts.label || 'Tip');
  btn.addEventListener('click', async function () {
    var amt = prompt('Enter tip amount in ETH (e.g. 0.001):');
    if (!amt || isNaN(parseFloat(amt))) { return; }
    btn.disabled    = true;
    btn.textContent = 'Sending\u2026';
    try {
      var result = await sendTip({ toWallet: opts.toWallet, tipType: opts.tipType, amountEth: parseF
      btn.textContent = result.recognized ? '\u2714 Tipped!' : '\u2714 Tipped (anonymous)';
      btn.className   = 'btn btn-sm btn-success ms-2';
    } catch (err) {
      btn.disabled    = false;
      btn.textContent = opts.label || 'Tip';
      alert('Tip failed: ' + err.message);
    }
  });
  container.appendChild(btn);
}

// ═══════════════════════════════════════════════════════════
//  CONTENT PLAYBACK
// ═══════════════════════════════════════════════════════════
async function playContent(metadataCid, nftContractAddress, tokenId, opts) {
  opts = opts || {};
  var live       = opts.live       || false;
  var playlistId = opts.playlistId || null;
  var addr       = window.walletAddress;

  var isOwner = false;
  if (addr && nftContractAddress && tokenId != null) {
    try {
      var nft   = getContract(nftContractAddress, ABIS.nftMetadata, requireProvider());
      var owner = await nft.ownerOf(tokenId);
      isOwner   = owner.toLowerCase() === addr.toLowerCase();
    } catch (e) { console.debug('ownerOf check skipped:', e.message); }
  }

  var profile = getProfile();
  if (profile && profile.supporter_subaccount && profile.supporter_subaccount.enabled && nftContract
    var creatorContract = profile.nft_contract_address || profile.nftContractAddress;
    if (creatorContract && nftContractAddress.toLowerCase() === creatorContract.toLowerCase()) {
      throw new Error('Your supporter account cannot stream your own creator content.');
    }
```

```
  }

  var tokenRes  = await fetch('/api/request-play-token', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({ cid: metadataCid, listener: addr, live: live, playlistId: playlistId }
  });
  var tokenData = await tokenRes.json();

  if (!tokenRes.ok) {
    if (tokenData.pay_per_play) {
      var doPPP = confirm('You need a subscription to stream.\n\nOK = Pay 0.001 ETH per play  |  Can
      if (!doPPP) { window.location.href = 'listen.html#subscribe'; return; }
      var signer2 = requireSigner().signer;
      var esc2 = getContract(escrowAddress, ABIS.escrow, signer2);
      var tx2  = await esc2.depositForPlay(metadataCid, { value: window.ethers.utils.parseEther('0.0
      await tx2.wait();
    } else {
      throw new Error(tokenData.error || 'Could not get play token.');
    }
  }

  var playToken = tokenData.playToken;
  var metaUrl   = resolveMetaUrl(metadataCid);
  var metadata  = metaUrl ? await (await fetch(metaUrl)).json() : null;
  var rawUrl    = metadata ? ((isOwner || CAN.stream()) ? metadata.ipfs_audio_url : metadata.files &
  var url       = rawUrl;
  if (!url) { throw new Error('No playable URL found in metadata.'); }
  if (typeof window.playHls !== 'function') { throw new Error('playHls not available. Load common.js
  window.playHls(resolveUrl(url), metaUrl);

  if (playToken) {
    var audioEl = $('audio-player');
    if (audioEl) {
      audioEl.addEventListener('ended', function () {
        fetch('/api/submit-play-proof', {
          method:  'POST',
          headers: { 'Content-Type': 'application/json' },
          body:    JSON.stringify({ playToken: playToken })
        }).catch(function (e) { console.warn('Play proof failed:', e.message); });
      }, { once: true });
    }
  }
}

// ═══════════════════════════════════════════════════════
//  NFT MINTING
```

```
// ═══════════════════════════════════════════════════════════
async function ensureUserNftContract(profile) {
  var existing = profile.nft_contract_address || profile.nftContractAddress;
  if (existing) { return existing; }
  var bytecode = window.MSP_NFT_BYTECODE;
  if (!bytecode) { throw new Error('NFT bytecode (MSP_NFT_BYTECODE) not provided.'); }
  var signer  = requireSigner().signer;
  var factory = new window.ethers.ContractFactory(ABIS.nftMetadata, bytecode, signer);
  var contract = await factory.deploy(profile.name + "'s NFTs", 'MNFT', mspAdminAddress);
  await contract.deployed();
  var addr = contract.address;
  await fetch('/api/update-profile', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({ wallet: window.walletAddress, nftContractAddress: addr })
  });
  if (_profile) {
    _profile.nft_contract_address = addr;
    saveProfileCache(window.walletAddress, _profile);
  }
  return addr;
}

async function registerCertificate(metadataCid, contentType, caSignature) {
  var signer = requireSigner().signer;
  var ca = getContract(contentCAAddress, ABIS.contentCA, signer);
  var tx = await ca.registerCertificate(metadataCid, contentType, caSignature);
  await tx.wait();
}

async function mintNftEth(profile, params) {
  var contractAddr = await ensureUserNftContract(profile);
  var signer       = requireSigner().signer;
  var nft          = getContract(contractAddr, ABIS.nftMetadata, signer);
  var est          = await nft.mintNFT.estimateGas(params.songTitle, params.artistName, params.year,
  var gas          = est ? est.mul(120).div(100) : undefined;
  var tx           = await nft.mintNFT(params.songTitle, params.artistName, params.year, params.meta
  await tx.wait();
  return contractAddr;
}

// ═══════════════════════════════════════════════════════════
//  NFT MARKETPLACE
// ═══════════════════════════════════════════════════════════
async function loadNFTs(containerId) {
  var container = $(containerId);
  if (!container) { return; }
```

```
if (!CAN.browseNFTs()) {
  container.innerHTML =
    '<div class="col-12 text-center py-4">' +
    '<p class="text-muted">Subscribe to browse and purchase NFTs.</p>' +
    '<a href="listen.html#subscribe" class="btn btn-primary btn-sm">Subscribe</a>' +
    '</div>';
  return;
}

try {
  var nfts = await fetch('/api/nfts').then(function (r) { return r.ok ? r.json() : []; });
  if (!Array.isArray(nfts) || !nfts.length) {
    container.innerHTML = '<p class="text-muted text-center py-4">No NFTs available yet.</p>';
    return;
  }

  container.innerHTML = nfts.map(function (nft) {
    var imgSrc = (nft.cover_image || '').replace('ipfs://', GATEWAY());
    return '<div class="col-md-4 mb-3">' +
      '<div class="nft-card h-100">' +
      '<img src="' + imgSrc + '" alt="' + (nft.title || '') + '" style="width:100%;aspect-ratio:1/
      '<div class="p-2">' +
      '<h5 class="mb-1">' + (nft.title || 'Untitled') + '</h5>' +
      '<p class="mb-1 small text-muted">Artist: ' + (nft.artist || 'Unknown') + '</p>' +
      (nft.price_eth ? '<p class="mb-2 small">Price: ' + nft.price_eth + ' ETH</p>' : '') +
      '<div class="d-flex gap-1 flex-wrap">' +
      '<button class="btn btn-primary btn-sm play-nft"' +
        ' data-cid="' + (nft.metadataCid || '') + '"' +
        ' data-contract="' + (nft.contractAddress || '') + '"' +
        ' data-tokenid="' + (nft.tokenId || '') + '"' +
        ' data-artist="' + (nft.artistWallet || '') + '">\u25B6 Play</button>' +
      '<button class="btn btn-outline-primary btn-sm buy-nft"' +
        ' data-price="' + (nft.price_eth || '0') + '"' +
        ' data-contract="' + (nft.contractAddress || '') + '"' +
        ' data-tokenid="' + (nft.tokenId || '') + '">Buy NFT</button>' +
      '</div></div></div></div>';
  }).join('');

  container.querySelectorAll('.play-nft').forEach(function (btn) {
    btn.addEventListener('click', async function () {
      btn.disabled = true;
      try {
        await playContent(btn.dataset.cid, btn.dataset.contract || null, btn.dataset.tokenid || nu
        if (btn.dataset.artist) {
          renderTipButton(btn.closest('.d-flex'), { toWallet: btn.dataset.artist, tipType: 'artist
        }
```

```
        } catch (e) { alert(e.message); }
        finally { btn.disabled = false; }
      });
    });

    container.querySelectorAll('.buy-nft').forEach(function (btn) {
      btn.addEventListener('click', async function () {
        if (!CAN.buyNFTs()) { return alert('Subscribe to purchase NFTs.'); }
        var price = btn.dataset.price;
        if (!price || price === '0') { return alert('This NFT has no listing price.'); }
        btn.disabled = true;
        try {
          var feeRes  = await fetch('/api/nft-sale-fee', {
            method:  'POST',
            headers: { 'Content-Type': 'application/json' },
            body:    JSON.stringify({ sale_price_eth: price, nft_type: 'music', seller_wallet: btn.d
          });
          var feeData = feeRes.ok ? await feeRes.json() : {};
          var ok = confirm(
            'Buy NFT for ' + price + ' ETH?\n' +
            'Platform fee: ' + (feeData.platform_fee_eth || '?') + ' ETH\n' +
            'Seller receives: ' + (feeData.seller_gets_eth || '?') + ' ETH'
          );
          if (!ok) { btn.disabled = false; return; }
          var signer      = requireSigner().signer;
          var nftContract = getContract(btn.dataset.contract, ABIS.nftMetadata, signer);
          var tx          = await nftContract.purchase(btn.dataset.tokenid, { value: window.ethers.u
          await tx.wait();
          btn.textContent = '\u2714 Purchased';
          btn.className   = 'btn btn-sm btn-success';
        } catch (e) { alert('Purchase failed: ' + e.message); btn.disabled = false; }
      });
    });
  } catch (e) {
    console.error('loadNFTs failed:', e);
    container.innerHTML = '<p class="text-danger text-center">Failed to load NFTs. Try again later.<
  }
}

// ═══════════════════════════════════════════════════════════
//  ROYALTY SPLITS
// ═══════════════════════════════════════════════════════════
async function submitRoyaltySplits(cid, splits) {
  var addr = requireSigner().address;
  var res = await fetch('/api/set-royalty-splits', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
```

```
    body:    JSON.stringify({ wallet: addr, cid: cid, splits: splits })
  });
  if (!res.ok) { throw new Error(await safeApiError(res)); }
  return res.json();
}

// ============================================================
//  DJ SET
// ============================================================
async function startDjSet(params) {
  var addr = requireSigner().address;
  var res = await fetch('/api/start-dj-set', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({
      wallet:        addr,
      set_name:      params.setName,
      tips_enabled:  params.tipsEnabled,
      dj_percent:    params.djPercent,
      artist_splits: params.artistSplits || []
    })
  });
  if (!res.ok) { throw new Error(await safeApiError(res)); }
  return res.json();
}

// ============================================================
//  LIVE ENCODE
// ============================================================
async function startLiveEncode(wallet, eventTitle, artistName) {
  var res = await fetch('/api/start-live-encode', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({ wallet: wallet, eventTitle: eventTitle, artistName: artistName })
  });
  if (!res.ok) { throw new Error(await safeApiError(res)); }
  return res.json();
}

// ============================================================
//  WALLET CONNECTED HANDLER
//  Fires on: explicit connect AND session restore (wallets.js now calls notifyConnected both ways)
// ============================================================
document.addEventListener('walletConnected', async function (e) {
  var address = (e.detail && e.detail.address) || window.walletAddress;
  if (!address) { return; }
```

```
  // Update abbreviated address displays in navbar
  document.querySelectorAll('.wallet-address-display').forEach(function (el) {
    el.textContent = address.slice(0, 6) + '\u2026' + address.slice(-4);
  });

  try {
    var results = await Promise.all([
      fetchOrCreateProfile(address),
      fetchAccess(address)
    ]);
    _profile = results[0];
    _access  = results[1];
    applyCapabilityGates();
    applyUserName(_profile && _profile.name);

    // Notify live_studio.html (and any other panels) that wallet + access are ready
    window.dispatchEvent(new CustomEvent('msp:walletConnected', { detail: { address: address } }));
    fetch('/api/check-platform-nft', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ wallet: address })
    }).then(function (r) { return r.json(); })
      .then(function (data) {
        if (_access && data.account_type !== _access.account_type) {
          fetchAccess(address).then(applyCapabilityGates);
        }
      }).catch(function () {});

    // Fix aria-hidden warning
    var walletModal = $('walletModal');
    if (walletModal) {
      walletModal.addEventListener('hide.bs.modal', function () {
        if (document.activeElement instanceof HTMLElement) { document.activeElement.blur(); }
      }, { once: true });
    }
  } catch (err) {
    console.error('Profile/access setup failed:', err.message);
  }
});

// ═══════════════════════════════════════════════════════════
//  PAGE ROUTERS
// ═══════════════════════════════════════════════════════════
document.addEventListener('DOMContentLoaded', async function () {

  // walletConnected fires on session restore now, but restore is async.
  // As a safety net: if wallet address is already set by the time DOMContentLoaded
```

```
// runs, load from cache immediately (no server call).
if (window.walletAddress) {
  var cached0 = loadProfileCache(window.walletAddress);
  if (cached0 && cached0.user_id) {
    _profile = cached0;
    applyUserName(cached0.name);
  }
}

// ── LISTEN PAGE ──────────────────────────────────────────
if (location.pathname.endsWith('listen.html')) {

  function showSubscribeError(btn, msg) {
    var existing = btn.parentNode.querySelector('.subscribe-error');
    if (existing) existing.remove();
    var el = document.createElement('p');
    el.className = 'subscribe-error text-danger small mt-2 mb-0';
    el.textContent = msg;
    btn.insertAdjacentElement('afterend', el);
  }
  function clearSubscribeError(btn) {
    var existing = btn.parentNode.querySelector('.subscribe-error');
    if (existing) existing.remove();
  }

  document.querySelectorAll('[data-subscribe-plan]').forEach(function (btn) {
    btn.addEventListener('click', async function () {
      if (!window.walletAddress) {
        if (typeof window.openWalletModal === 'function') { window.openWalletModal(); }
        showSubscribeError(btn, 'Connect your wallet first, then try again.');
        return;
      }
      var plan = btn.dataset.subscribePlan;
      clearSubscribeError(btn);
      btn.disabled    = true;
      btn.textContent = 'Processing\u2026';
      try {
        var data = await subscribePlan(plan);
        btn.textContent = '\u2714 Subscribed!';
        btn.className   = btn.className.replace('btn-primary','btn-success').replace('btn-warning'
        var statusEl = $('subscription-status');
        if (statusEl) { statusEl.textContent = 'Active \u2014 expires ' + new Date(data.expiry).to
      } catch (err) {
        btn.disabled    = false;
        btn.textContent = 'Subscribe';
        showSubscribeError(btn, err.message);
      }
```

```
  });
});

// ── Favorites section ──────────────────────────────────────────
async function renderFavoritesSection() {
  var favSection = $('favorites-section');
  var favList    = $('favorites-list');
  if (!favSection || !favList) return;
  if (!CAN.favorite() || !window.walletAddress) {
    favSection.style.display = 'none';
    return;
  }
  favSection.style.display = '';
  var cids = Array.from(getFavorites());
  if (!cids.length) {
    favList.innerHTML = '<p class="text-muted small py-2">No favorites yet. Heart a track to sav
    return;
  }
  favList.innerHTML = cids.map(function (cid) {
    var short = cid.slice(0, 8);
    var isFav = true;
    return '<div class="fav-list-row d-flex align-items-center gap-2 py-1">' +
      '<button class="fav-btn fav-active" data-fav-cid="' + cid + '" aria-label="Remove from Fav
      '<button class="btn btn-sm btn-link p-0 play-fav" data-cid="' + cid + '">▶ ' + short + '…
      '<span class="ms-auto fav-convert-check"><input type="checkbox" class="fav-select" data-ci
    '</div>';
  }).join('');

  // Wire play buttons
  favList.querySelectorAll('.play-fav').forEach(function (btn) {
    btn.addEventListener('click', function () {
      playContent(btn.dataset.cid, null, null, { live: false })
        .catch(function (e) { console.warn(e.message); });
    });
  });

  // Wire fav (remove) buttons — reuse makeFavButton logic via _syncHeartButtons
  _syncHeartButtons();

  // Convert-to-playlist button
  var convertBtn = $('fav-convert-btn');
  if (convertBtn) {
    convertBtn.onclick = async function () {
      var selected = Array.from(favList.querySelectorAll('.fav-select:checked')).map(function (c
      if (!selected.length) { alert('Check at least one track to add to a playlist.'); return; }
      var name = prompt('Name your playlist:');
      if (!name || !name.trim()) return;
```

```
      try {
        var res = await fetch('/api/favorites/convert-to-playlist', {
          method: 'POST', headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ wallet: window.walletAddress, name: name.trim(), cids: selected
        });
        var data = await res.json();
        if (!res.ok) throw new Error(data.error);
        alert('✓ Playlist “' + name + '” created!');
      } catch (e) { alert('Failed: ' + e.message); }
    };
  }
}
renderFavoritesSection();

(async function () {
  try {
    // Load local DEV catalog first
    var libraryEl = document.getElementById('library-list');
    if (libraryEl) {
      try {
        var catalog = await fetch('/api/catalog').then(function (r) { return r.ok ? r.json() : [
        if (catalog.length) {
          libraryEl.innerHTML = catalog.map(function (item) {
            return '<div class="col-md-4 mb-3"><div class="nft-card p-2">' +
              (item.coverUrl ? '<img src="' + item.coverUrl + '" alt="' + item.title + '" style=
              '<h5 class="mb-1">' + item.title + '</h5>' +
              '<p class="small text-muted mb-2">' + item.artistName + '</p>' +
              '<button class="btn btn-teal btn-sm play-catalog-item" data-contentid="' + item.co
              '</div></div>';
          }).join('');
          document.querySelectorAll('.play-catalog-item').forEach(function (btn) {
            btn.addEventListener('click', async function () {
              try {
                // For local catalog items play directly via HLS URL
                var hlsUrl = btn.dataset.hlsurl;
                if (typeof window.playHls === 'function') {
                  window.playHls(hlsUrl, btn.dataset.metaurl);
                }
              } catch (e) { alert(e.message); }
            });
          });
        } else {
          libraryEl.innerHTML = '<p class="text-muted small">No tracks uploaded yet. Upload your
        }
      } catch (e) {
        await loadNFTs('library-list');
      }
```

```
}

var trendingEl = $('trending-playlists');
var newEl      = $('new-playlists');
if (trendingEl || newEl) {
  var playlists = await fetch('/api/playlists').then(function (r) { return r.ok ? r.json() :
  var markup = playlists.map(function (p) {
    return '<div class="col-md-4 mb-3"><div class="nft-card">' +
      '<img src="' + (p.cover_image || '').replace('ipfs://', GATEWAY()) + '" alt="' + p.nam
      '<div class="p-2"><h5>' + p.name + '</h5>' +
      '<p class="small text-muted">Curator: ' + (p.curator || 'User') + '</p>' +
      '<button class="btn btn-primary btn-sm play-playlist" data-playlistid="' + p.id + '">\
      '</div></div></div>';
  }).join('');
  if (trendingEl) { trendingEl.innerHTML = markup; }
  if (newEl)      { newEl.innerHTML      = markup; }
  document.querySelectorAll('.play-playlist').forEach(function (btn) {
    btn.addEventListener('click', function () {
      playContent('mockcid', null, null, { live: false, playlistId: btn.dataset.playlistid }
    });
  });
}

var liveConcertsEl = $('live-concerts');
if (liveConcertsEl) {
  var concerts = await fetch('/api/live-concerts').then(function (r) { return r.ok ? r.json(
  if (!concerts.length) {
    liveConcertsEl.innerHTML = '<p class="text-muted small">No live concerts right now. Chec
  } else {
    liveConcertsEl.innerHTML = concerts.map(function (c) {
      return '<div class="col-md-4 mb-3"><div class="nft-card">' +
        '<div class="p-2" style="background:#1a0a06;border-radius:6px;">' +
        '<h5>\uD83D\uDD34 Live: ' + (c.title || c.artist) + '</h5>' +
        '<p class="small text-muted mb-1">' + c.artist + ' &middot; ' + (c.viewerCount || 0)
        '<div class="d-flex gap-1">' +
        '<button class="btn btn-danger btn-sm join-concert"' +
          ' data-sessionid="' + c.sessionId + '"' +
          ' data-hlsurl="' + (c.hlsUrl || '') + '"' +
          ' data-artist="' + (c.artistWallet || '') + '"' +
          ' data-title="' + (c.title || 'Live Stream') + '">' +
          'Join Live</button>' +
        '</div></div></div></div>';
    }).join('');
  }
  document.querySelectorAll('.join-concert').forEach(function (btn) {
    btn.addEventListener('click', async function () {
      try {
```

```
            var hlsUrl = btn.dataset.hlsurl;
            if (!hlsUrl) { alert('No stream URL available.'); return; }

            // Live streams: play HLS directly — no IPFS metadata fetch needed
            if (typeof window.playHls !== 'function') {
              alert('Player not ready. Please refresh the page.');
              return;
            }
            window.playHls(hlsUrl, null);

            if (btn.dataset.artist) {
              renderTipButton(btn.parentElement, { toWallet: btn.dataset.artist, tipType: 'artis
            }
          } catch (e) { alert(e.message); }
        });
      });
    }
  } catch (e) { console.debug('Listen page load failed:', e.message); }
})();
}

// ─── CREATORS PAGE ──────────────────────────────────────
if (location.pathname.endsWith('creators.html')) {

  // ── Content type tab switcher ──────────────────────────────
  var typeReqs = {
    music:        'Audio ≥ 128 kbps · 44.1/48 kHz · MP3, WAV, OGG, FLAC, AAC, M4A · max 500 MB',
    podcast:      'Audio ≥ 128 kbps · MP3, WAV, OGG, AAC, M4A · max 500 MB',
    video:        'MP4, MOV, MKV, WebM · min 500 kbps video · audio required · max 500 MB',
    art_still:    'Any supported media · no encoding — uploaded directly to IPFS · max 500 MB',
    art_animated: 'MP4, WebM · video required · max 500 MB',
  };
  var typeAccept = {
    music:        'audio/mpeg,audio/wav,audio/ogg,audio/flac,audio/aac,audio/mp4,audio/x-m4a',
    podcast:      'audio/mpeg,audio/wav,audio/ogg,audio/aac,audio/mp4,audio/x-m4a',
    video:        'video/mp4,video/quicktime,video/x-matroska,video/webm',
    art_still:    'audio/mpeg,audio/wav,video/mp4,video/webm,image/png,image/jpeg',
    art_animated: 'video/mp4,video/webm',
  };
  var typeHint = {
    music:        'MP3 · WAV · OGG · FLAC · AAC · M4A',
    podcast:      'MP3 · WAV · OGG · AAC · M4A',
    video:        'MP4 · MOV · MKV · WebM',
    art_still:    'MP3 · WAV · MP4 · WebM · PNG · JPG',
    art_animated: 'MP4 · WebM',
  };
  var typeDropIcon = {
```

```
  music: '♪', podcast: '🎙', video: '▶', art_still: '◈', art_animated: '◉',
};

var currentType  = 'music';
var audioFileInput   = $('audio-file');
var contentTypeField = $('content-type-field');
var reqsEl           = $('upload-reqs');
var dropHintEl       = $('drop-hint');
var dropIconEl       = $('drop-icon');

document.querySelectorAll('.utype-btn').forEach(function (btn) {
  btn.addEventListener('click', function () {
    currentType = btn.dataset.type;
    document.querySelectorAll('.utype-btn').forEach(function (b) { b.classList.remove('active');
    btn.classList.add('active');

    if (contentTypeField) contentTypeField.value = currentType;
    if (reqsEl)           reqsEl.textContent     = typeReqs[currentType];
    if (dropHintEl)       dropHintEl.textContent = typeHint[currentType];
    if (dropIconEl)       dropIconEl.textContent = typeDropIcon[currentType] || '↑';
    if (audioFileInput)   audioFileInput.accept  = typeAccept[currentType];

    // Show/hide type-specific field groups
    ['music', 'podcast'].forEach(function (t) {
      var el = $('fields-' + t);
      if (el) el.style.display = (currentType === t) ? '' : 'none';
    });
    var rightsEl = $('fields-rights');
    if (rightsEl) rightsEl.style.display =
      (currentType === 'music' || currentType === 'podcast') ? '' : 'none';

    // Update submit button text
    var btnText = $('upload-btn-text');
    if (btnText) {
      var labels = {
        music: '↑ Upload & Mint', podcast: '↑ Upload Podcast',
        video: '↑ Upload Video', art_still: '↑ Upload Art',
        art_animated: '↑ Upload Animated Art',
      };
      btnText.textContent = (CAN.mintNFT() && currentType !== 'podcast')
        ? (labels[currentType] || '↑ Upload & Mint')
        : '↑ Upload';
    }
  });
});

// ── Tags live preview ──────────────────────────────────────────
```

```
var tagsInput   = $('tags');
var tagsPreview = $('tags-preview');
if (tagsInput && tagsPreview) {
  tagsInput.addEventListener('input', function () {
    var chips = tagsInput.value.split(',')
      .map(function (t) { return t.trim(); })
      .filter(Boolean);
    tagsPreview.innerHTML = chips.map(function (t) {
      return '<span class="tag-chip">' + t.replace(/[<>&"]/g, '') + '</span>';
    }).join('');
  });
}

// ── Cover image preview ──────────────────────────────────────────
var coverInput    = $('cover-image');
var coverPreview  = $('cover-preview');
var coverImg      = $('cover-preview-img');
var coverPH       = $('cover-placeholder');
var coverClearBtn = $('cover-clear');
var coverDropEl   = $('cover-drop');
var coverBrowse   = $('cover-browse-btn');

function showCoverPreview(file) {
  if (!file || !coverImg) return;
  var reader = new FileReader();
  reader.onload = function (e) {
    coverImg.src = e.target.result;
    if (coverPreview) coverPreview.style.display = 'block';
    if (coverPH)      coverPH.style.display      = 'none';
  };
  reader.readAsDataURL(file);
}

if (coverInput) {
  coverInput.addEventListener('change', function () {
    if (coverInput.files && coverInput.files[0]) showCoverPreview(coverInput.files[0]);
  });
}
if (coverBrowse) coverBrowse.addEventListener('click', function () { if (coverInput) coverInput.
if (coverDropEl) coverDropEl.addEventListener('click', function (e) {
  if (e.target === coverClearBtn) return;
  if (coverInput) coverInput.click();
});
if (coverClearBtn) coverClearBtn.addEventListener('click', function (e) {
  e.stopPropagation();
  if (coverInput) coverInput.value = '';
  if (coverPreview) coverPreview.style.display = 'none';
```

```
  if (coverPH)      coverPH.style.display      = '';
  if (coverImg)     coverImg.src                = '';
});

// ── Audio drop zone ──────────────────────────────────────────────
var dragDropArea = $('drag-drop-area');
var dropInner    = $('drop-zone-inner');
var browseBtn    = $('browse-btn');
var releaseDateInput = $('release-date');
var dateCreatedInput = $('date-created');

function showFileName(name) {
  if (!dragDropArea) return;
  dragDropArea.classList.add('has-file');
  if (dropIconEl) dropIconEl.textContent = '✓';
  var primary = dragDropArea.querySelector('.drop-primary');
  if (primary) primary.textContent = name;
  var secondary = dragDropArea.querySelector('.drop-secondary');
  if (secondary) secondary.innerHTML =
    '<button type="button" id="browse-btn-change" class="drop-browse">Change file</button>';
  var changeBtn = $('browse-btn-change');
  if (changeBtn) changeBtn.addEventListener('click', function () { if (audioFileInput) audioFile
}

function handleFileSelection() {
  if (!audioFileInput || !audioFileInput.files || !audioFileInput.files.length) return;
  var file     = audioFileInput.files[0];
  var fileDate = new Date(file.lastModified);
  if (dateCreatedInput) dateCreatedInput.value = fileDate.toISOString().split('T')[0];
  if (releaseDateInput && !releaseDateInput.value) releaseDateInput.value = new Date().toISOStri
  showFileName(file.name);
}

if (browseBtn && audioFileInput) {
  browseBtn.addEventListener('click', function (e) { e.preventDefault(); audioFileInput.click();
}
if (audioFileInput) {
  audioFileInput.addEventListener('change', handleFileSelection);
}
if (dragDropArea && audioFileInput) {
  dragDropArea.addEventListener('dragover', function (e) { e.preventDefault(); dragDropArea.clas
  dragDropArea.addEventListener('dragleave', function () { dragDropArea.classList.remove('drag-o
  dragDropArea.addEventListener('drop', function (e) {
    e.preventDefault();
    dragDropArea.classList.remove('drag-over');
    var files = e.dataTransfer.files;
    if (!files || !files.length) return;
```

```
    try { var dt = new DataTransfer(); dt.items.add(files[0]); audioFileInput.files = dt.files;
    handleFileSelection();
  });
}

// ── Mint NFT toggle ──────────────────────────────────────────────
var mintNftCheckbox = $('mint-nft');
var mintChainWrap   = $('mint-chain-wrap');
if (mintNftCheckbox && mintChainWrap) {
  mintChainWrap.style.display = mintNftCheckbox.checked ? '' : 'none';
  mintNftCheckbox.addEventListener('change', function () {
    mintChainWrap.style.display = mintNftCheckbox.checked ? '' : 'none';
  });
}

// ── Live encode form (unchanged) ─────────────────────────────────
var liveForm = $('liveEncodeForm');
if (liveForm) {
  liveForm.addEventListener('submit', async function (e) {
    e.preventDefault();
    if (!CAN.hostConcert()) { return alert('An active Creator subscription is required to host l
    var eventTitle = $('eventTitle') ? $('eventTitle').value.trim() : '';
    var artistName = $('artistName') ? $('artistName').value.trim() : '';
    var statusEl   = $('liveStatus');
    if (!eventTitle || !artistName) {
      if (statusEl) { statusEl.textContent = 'Please fill in both fields.'; statusEl.className =
      return;
    }
    if (statusEl) { statusEl.textContent = 'Starting live…'; statusEl.className = ''; }
    try {
      var addr2 = requireSigner().address;
      var data2 = await startLiveEncode(addr2, eventTitle, artistName);
      if (statusEl) {
        statusEl.innerHTML = '✓ Live started! <a href="' + data2.hlsUrl + '" target="_blank">Wat
        statusEl.className = 'text-success';
      }
    } catch (err) {
      if (statusEl) { statusEl.textContent = 'Error: ' + err.message; statusEl.className = 'text
    }
  });
}

// ── DJ set form (unchanged) ──────────────────────────────────────
var djSetForm = $('dj-set-form');
if (djSetForm) {
  djSetForm.addEventListener('submit', async function (e) {
    e.preventDefault();
```

```
    if (!CAN.hostDjSet()) { return alert('A Tier 2 or higher subscription is required to host DJ
    var setName    = $('dj-set-name')    ? $('dj-set-name').value.trim()    : '';
    var tipsToggle = $('dj-tips-enabled');
    var djPctInput = $('dj-tip-percent');
    var statusEl2  = $('dj-set-status');
    if (!setName) { return alert('Please enter a set name.'); }
    try {
      var result = await startDjSet({
        setName:      setName,
        tipsEnabled:  tipsToggle ? tipsToggle.checked : undefined,
        djPercent:    djPctInput ? parseFloat(djPctInput.value) : 100,
        artistSplits: []
      });
      if (statusEl2) { statusEl2.textContent = '✓ DJ Set started! ID: ' + result.set_id; statusE
    } catch (err) {
      if (statusEl2) { statusEl2.textContent = 'Error: ' + err.message; statusEl2.className = 't
    }
  });
}

// ── Upload form submit ──────────────────────────────────────────
var uploadForm   = $('upload-form');
var uploadBtn    = $('upload-btn');
var uploadStatus = $('upload-status');
var progressEl   = $('upload-progress');

function setLoading(on) {
  if (uploadBtn) uploadBtn.disabled = on;
  var btnText = $('upload-btn-text');
  if (btnText) btnText.textContent = on ? 'Processing…' : (
    CAN.mintNFT() ? '↑ Upload & Mint' : '↑ Upload'
  );
  if (progressEl) progressEl.style.display = on ? 'block' : 'none';
  if (uploadStatus && !on) uploadStatus.style.display = 'none';
}

function setUploadStatus(msg, type) {
  if (!uploadStatus) return;
  uploadStatus.textContent   = msg;
  uploadStatus.className     = type || '';
  uploadStatus.style.display = msg ? 'block' : 'none';
}

if (uploadForm) {
  uploadForm.addEventListener('submit', async function (e) {
    e.preventDefault();
    console.log('[UPLOAD] Step 1 — form submit intercepted');
```

```
var address;
try {
  address = requireSigner().address;
  console.log('[UPLOAD] Step 2 — wallet address:', address);
}
catch (err) {
  console.error('[UPLOAD] Step 2 FAILED — no signer:', err.message);
  setUploadStatus(err.message, 'error'); return;
}

if (!CAN.upload()) {
  console.warn('[UPLOAD] Step 3 FAILED — CAN.upload() returned false');
  setUploadStatus('A Creator account is required to upload. Subscribe on the listen page.',
  return;
}
console.log('[UPLOAD] Step 3 — CAN.upload() passed');

if (!audioFileInput || !audioFileInput.files || !audioFileInput.files.length) {
  console.warn('[UPLOAD] Step 4 FAILED — no audio file selected. audioFileInput:', audioFile
  setUploadStatus('Please select a media file.', 'error'); return;
}
console.log('[UPLOAD] Step 4 — audio file:', audioFileInput.files[0].name, audioFileInput.fi

var coverEl = $('cover-image');
if (!coverEl || !coverEl.files || !coverEl.files.length) {
  console.warn('[UPLOAD] Step 5 FAILED — no cover image selected. coverEl:', coverEl, 'files
  setUploadStatus('Please select a cover image.', 'error'); return;
}
console.log('[UPLOAD] Step 5 — cover image:', coverEl.files[0].name, coverEl.files[0].size,

var tagsVal = ($('tags') || {}).value || '';
var parsedTags = tagsVal.split(',').map(function(t){return t.trim();}).filter(Boolean);
console.log('[UPLOAD] Step 6 — tags:', parsedTags);
if (parsedTags.length < 1) {
  console.warn('[UPLOAD] Step 6 FAILED — not enough tags');
  setUploadStatus('Please add at least 1 tag.', 'error'); return;
}

setLoading(true);
setUploadStatus('Uploading…');

try {
  var profile2 = getProfile();
  console.log('[UPLOAD] Step 7 — profile:', profile2 ? { user_id: profile2.user_id, account_
  if (!profile2 || !profile2.user_id) throw new Error('Profile missing. Reconnect your walle
```

```
var formData = new FormData(uploadForm);
formData.set('userId', profile2.user_id);
formData.set('wallet', address);
formData.set('contentType', currentType);

// Log everything going into the FormData
console.log('[UPLOAD] Step 8 — FormData contents:');
for (var pair of formData.entries()) {
  if (pair[1] instanceof File) {
    console.log('  ', pair[0], '→ File:', pair[1].name, pair[1].size, 'bytes', pair[1].typ
  } else {
    console.log('  ', pair[0], '→', pair[1]);
  }
}

var xhr = new XMLHttpRequest();
xhr.open('POST', '/api/upload');
console.log('[UPLOAD] Step 9 — XHR opened, sending...');

xhr.upload.onprogress = function (evt) {
  if (!progressEl || !evt.lengthComputable) return;
  var pct = Math.round((evt.loaded / evt.total) * 95);
  progressEl.value = pct;
  console.log('[UPLOAD] Progress:', pct + '%', '(' + evt.loaded + '/' + evt.total + ' byte
};

xhr.onload = async function () {
  console.log('[UPLOAD] Step 10 — XHR response received. Status:', xhr.status);
  console.log('[UPLOAD] Response body:', xhr.responseText);
  try {
    var ct = xhr.getResponseHeader('content-type') || '';
    if (xhr.status !== 200) {
      var errMsg = 'Server error ' + xhr.status;
      if (ct.includes('application/json')) {
        try { errMsg = JSON.parse(xhr.responseText).error || errMsg; } catch (_) {}
      }
      console.error('[UPLOAD] Step 10 FAILED — server rejected:', errMsg);
      throw new Error(errMsg);
    }
    var data3 = JSON.parse(xhr.responseText);
    console.log('[UPLOAD] Step 11 — success! Response:', data3);
    if (progressEl) progressEl.value = 100;

    // Certificate registration (non-blocking if chain not ready)
    if (data3.caSignature) {
      await registerCertificate(data3.metadataCid, currentType, data3.caSignature).catch(f
        console.warn('Certificate registration skipped:', e.message);
```

```
        });
      }

      // NFT mint
      var shouldMint = data3.mint_pending && CAN.mintNFT() && currentType !== 'podcast';
      if (shouldMint) {
        setUploadStatus('Minting NFT on-chain…');
        var songTitle  = ($('song-title')  || {}).value || 'Untitled';
        var artistNm   = ($('artist-name') || {}).value || 'Unknown';
        var rdInput    = $('release-date');
        var year       = rdInput && rdInput.value ? new Date(rdInput.value).getFullYear() :
        await mintNftEth(profile2, { songTitle: songTitle, artistName: artistNm, year: year,
        setUploadStatus('✓ Uploaded, certified, and NFT minted!', 'success');
      } else {
        setUploadStatus('✓ Upload complete! Content saved to your catalog.', 'success');
      }

      // Auto-play if audio/video
      if ((currentType === 'music' || currentType === 'podcast') && typeof window.playHls ==
        window.playHls(data3.hlsUrl.replace('ipfs://', GATEWAY()), data3.metadataUrl);
      }

      // Reset form
      uploadForm.reset();
      if (dragDropArea) dragDropArea.classList.remove('has-file');
      if (coverPreview) { coverPreview.style.display = 'none'; }
      if (coverPH)      { coverPH.style.display = ''; }
      if (tagsPreview)  tagsPreview.innerHTML = '';

    } catch (err) {
      console.error('[UPLOAD] Handler error:', err.message);
      setUploadStatus('Failed: ' + err.message, 'error');
    }
    setLoading(false);
  };
  xhr.onerror = function () {
    console.error('[UPLOAD] XHR network error — server unreachable');
    setUploadStatus('Network error — check your connection and try again.', 'error');
    setLoading(false);
  };
  xhr.send(formData);
  console.log('[UPLOAD] Step 9 — XHR sent');
} catch (err) {
  console.error('[UPLOAD] Outer catch:', err.message);
  setUploadStatus(err.message, 'error');
  setLoading(false);
}
```

```
    });
  }
} // end creators.html

// ── MARKETPLACE PAGE ──────────────────────────────────
if (location.pathname.endsWith('marketplace.html')) {
  await loadNFTs('marketplace-list');
}

// ── PROFILE PAGE ──────────────────────────────────────
if (location.pathname.endsWith('profile.html')) {

  if (window.walletAddress) {
    var addrEl = $('profile-address');
    if (addrEl) { addrEl.textContent = window.walletAddress.slice(0,6) + '\u2026' + window.walletA
  }

  var accountTypeEl = $('account-type-display');
  if (accountTypeEl) {
    var labels = {
      'none':               'No active subscription',
      'listener_1':         'Listener \u2014 Tier 1',
      'listener_2':         'Listener \u2014 Tier 2',
      'listener_3':         'Listener \u2014 Tier 3',
      'creator_active':     'Creator (Active)',
      'creator_inactive':   'Creator (Inactive \u2014 renew subscription)',
      'nft_creator_active': 'Platform NFT Creator (Active)',
      'nft_creator_passive':'Platform NFT Creator (Passive \u2014 subscribe for full tools)'
    };
    accountTypeEl.textContent = labels[getAccess().level || 'none'] || (getAccess().level || 'none
  }

  await loadNFTs('user-nfts');

  // ── Profile favorites panel ──────────────────────────
  var profFavList  = $('profile-favorites-list');
  var profFavCount = $('profile-favorites-count');
  if (profFavList && window.walletAddress && CAN.favorite()) {
    await loadFavorites(window.walletAddress);
    var favCids = Array.from(getFavorites());
    if (profFavCount) profFavCount.textContent = favCids.length;
    if (!favCids.length) {
      profFavList.innerHTML = '<p class="text-muted small">No favorites yet.</p>';
    } else {
      profFavList.innerHTML = favCids.map(function (cid) {
        return '<div class="d-flex align-items-center gap-2 py-1 border-bottom border-dark">' +
          '<button class="fav-btn fav-active" data-fav-cid="' + cid + '">♥</button>' +
```

```
        '<span class="font-monospace small text-muted">' + cid.slice(0, 12) + '…</span>' +
      '</div>';
    }).join('');
    _syncHeartButtons();
  }
}

var splitsForm = $('splits-form');
if (splitsForm) {
  splitsForm.addEventListener('submit', async function (e) {
    e.preventDefault();
    if (!CAN.setSplits()) { return alert('Creator account required to set royalty splits.'); }
    var cid          = $('splits-cid')           ? $('splits-cid').value.trim()           : '';
    var artist       = parseFloat($('split-artist')        ? $('split-artist').value        : '0
    var nftHolders   = parseFloat($('split-nft-holders')   ? $('split-nft-holders').value   : '0
    var activityPool = parseFloat($('split-activity-pool') ? $('split-activity-pool').value : '0
    var passWallet   = $('split-passive-wallet')  ? $('split-passive-wallet').value.trim()  : ''
    var passPct      = parseFloat($('split-passive-pct')   ? $('split-passive-pct').value   : '0
    if (!cid) { return alert('Please enter a content CID.'); }
    var passive = (passWallet && passPct > 0) ? [{ wallet: passWallet, percent: passPct }] : [];
    var total   = artist + nftHolders + activityPool + passive.reduce(function (s, p) { return s
    if (Math.abs(total - 100) > 0.01) { return alert('Splits must total 100%. Currently: ' + tot
    try {
      await submitRoyaltySplits(cid, { artist: artist, nft_holders: nftHolders, activity_pool: a
      alert('\u2714 Royalty splits saved!');
    } catch (err) { alert('Failed: ' + err.message); }
  });
}

var claimNftBtn = $('claim-platform-nft-btn');
if (claimNftBtn) {
  claimNftBtn.addEventListener('click', async function () {
    if (!window.walletAddress) { return alert('Connect wallet first.'); }
    claimNftBtn.disabled    = true;
    claimNftBtn.textContent = 'Verifying\u2026';
    try {
      var res2  = await fetch('/api/claim-platform-nft', {
        method:  'POST',
        headers: { 'Content-Type': 'application/json' },
        body:    JSON.stringify({ wallet: window.walletAddress })
      });
      var data4 = await res2.json();
      if (!res2.ok) { throw new Error(data4.error); }
      _access = await fetchAccess(window.walletAddress);
      applyCapabilityGates();
      claimNftBtn.textContent = '\u2714 Platform NFT Activated!';
      claimNftBtn.className   = claimNftBtn.className.replace('btn-warning','btn-success');
```

```
      alert(data4.message);
    } catch (err) {
      claimNftBtn.disabled    = false;
      claimNftBtn.textContent = 'Claim Platform NFT';
      alert('Claim failed: ' + err.message);
    }
  });
}

var supporterToggle = $('supporter-subaccount-toggle');
if (supporterToggle) {
  supporterToggle.checked = getAccess().supporter_enabled || false;
  supporterToggle.addEventListener('change', async function () {
    if (!CAN.supporterSub()) { supporterToggle.checked = false; return alert('Only creator accou
    var enabled  = supporterToggle.checked;
    var endpoint = enabled ? '/api/add-supporter-subaccount' : '/api/toggle-supporter-subaccount
    var body     = enabled ? { wallet: window.walletAddress } : { wallet: window.walletAddress,
    try {
      await fetch(endpoint, { method: 'POST', headers: { 'Content-Type': 'application/json' }, b
      var st = $('supporter-subaccount-status');
      if (st) { st.textContent = enabled ? 'Supporter sub-account enabled.' : 'Supporter sub-acc
    } catch (err) { supporterToggle.checked = !enabled; alert('Toggle failed: ' + err.message);
  });
}

var djTipToggle = $('dj-tips-default-toggle');
if (djTipToggle) {
  djTipToggle.checked = getAccess().dj_tips_default !== false;
  djTipToggle.addEventListener('change', async function () {
    await fetch('/api/update-profile', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ wallet: window.walletAddress, djTipsDefault: djTipToggle.checked
    });
  });
}

var playlistForm = $('playlist-form');
if (playlistForm) {
  playlistForm.addEventListener('submit', async function (e) {
    e.preventDefault();
    if (!CAN.createPlaylist()) { return alert('Tier 2 or higher subscription required to create
    var tracks = ($('playlist-tracks') ? $('playlist-tracks').value : '').split('\n').map(functi
    if (!tracks.length) { return alert('Add at least one CID per line.'); }
    var playlist = { id: crypto.randomUUID(), curator: window.walletAddress, tracks: tracks, sha
    try {
      var res3 = await fetch('/api/create-playlist', { method: 'POST', headers: { 'Content-Type'
```

```
            if (!res3.ok) { throw new Error(await res3.text()); }
            alert('\u2714 Playlist created!');
          } catch (err) { alert('Failed: ' + err.message); }
        });
      }
    } // end profile.html

  }); // end DOMContentLoaded

})();
```

`common.js` (7.5 KB)

```javascript
// public/Scripts/common.js
'use strict';

// ============================
// Config
// ============================
const IPFS_GATEWAY = (window.IPFS_GATEWAY || 'https://ipfs.io/ipfs/').replace(/\/+$/, '') + '/';

function ipfsToHttp(url) {
  if (typeof url !== 'string') return url;
  if (!url.startsWith('ipfs://')) return url;
  return url.replace(/^ipfs:\/\//, IPFS_GATEWAY);
}

// ============================
// Audio Player elements (all optional)
// ============================
const audio           = document.getElementById('audio-player');
const playBtn         = document.getElementById('play-btn');
const pauseBtn        = document.getElementById('pause-btn');
const stopBtn         = document.getElementById('stop-btn');
const progressBar     = document.getElementById('progress-bar');
const volumeBar       = document.getElementById('volume-bar');
const timeDisplay     = document.getElementById('time-display');
const durationDisplay = document.getElementById('duration-display');
const totalPlaysEl    = document.getElementById('total-plays');
const vinylIcon       = document.getElementById('vinyl-icon');

let totalPlays = 0;
let hls = null;

// ============================
```

```
// HLS playback
// ============================
function ensureHlsAvailable() {
  return (typeof window !== 'undefined' && window.Hls && typeof window.Hls.isSupported === 'function')
}

async function playHls(url, metadataUrl) {
  if (!audio) return;

  const httpUrl  = ipfsToHttp(url);
  const httpMeta = metadataUrl ? ipfsToHttp(metadataUrl) : null;

  if (hls) {
    try { hls.destroy(); } catch (_) {}
    hls = null;
  }

  let candidate = httpUrl;
  if (httpMeta) {
    try {
      const res      = await fetch(httpMeta);
      const metadata = await res.json();
      if (metadata && metadata.availability_type === 'live' &&
          metadata.rollup && metadata.rollup.presentations &&
          metadata.rollup.presentations[0] &&
          metadata.rollup.presentations[0].playlist &&
          metadata.rollup.presentations[0].playlist.streams &&
          metadata.rollup.presentations[0].playlist.streams[0]) {
        candidate = metadata.rollup.presentations[0].playlist.streams[0].url;
      }
    } catch (err) {
      console.warn('Failed to load metadata; falling back to provided URL', err);
    }
  }

  const isM3U8   = /\.m3u8($|\?)/i.test(candidate);
  const nativeHls = audio.canPlayType('application/vnd.apple.mpegurl');

  if (isM3U8 && ensureHlsAvailable() && window.Hls.isSupported() && !nativeHls) {
    hls = new window.Hls({ enableWorker: false });
    hls.loadSource(candidate);
    hls.attachMedia(audio);
    hls.on(window.Hls.Events.MANIFEST_PARSED, function () {
      audio.play().catch(function () {});
    });
    hls.on(window.Hls.Events.LEVEL_SWITCHED, function (_, data) {
      const level = hls.levels && hls.levels[data.level];
```

```
      if (level && level.bitrate) {
        console.log('Switched to bitrate: ' + Math.round(level.bitrate / 1000) + ' kbps');
      }
    });
  } else if (isM3U8 && nativeHls) {
    audio.src = candidate;
    audio.addEventListener('loadedmetadata', function () {
      audio.play().catch(function () {});
    }, { once: true });
  } else {
    audio.src = candidate;
    audio.play().catch(function () {});
  }
}

window.playHls = playHls;

// ============================
// Player controls (guarded)
// ============================
if (audio) {
  // Wire play button — vinylIcon is optional, do not gate on it
  if (playBtn) {
    playBtn.addEventListener('click', function () {
      audio.play().catch(function () {});
      playBtn.style.display  = 'none';
      if (pauseBtn) pauseBtn.style.display = 'inline';
      if (vinylIcon) {
        vinylIcon.style.display = 'block';
        const glowDuration = Math.max(0.5, 4 - audio.volume * 3);
        vinylIcon.style.animation = 'spin 2s linear infinite, glow ' + glowDuration + 's ease-in-out i
      }
    });
  }

  if (pauseBtn) {
    pauseBtn.addEventListener('click', function () {
      audio.pause();
      pauseBtn.style.display = 'none';
      if (playBtn) playBtn.style.display = 'inline';
      if (vinylIcon) {
        vinylIcon.style.animation = 'none';
        vinylIcon.style.display   = 'none';
      }
    });
  }
```

```
if (stopBtn) {
  stopBtn.addEventListener('click', function () {
    audio.pause();
    audio.currentTime = 0;
    if (hls) { try { hls.destroy(); } catch (_) {} hls = null; }
    if (pauseBtn) pauseBtn.style.display = 'none';
    if (playBtn)  playBtn.style.display  = 'inline';
    if (vinylIcon) {
      vinylIcon.style.animation = 'none';
      vinylIcon.style.display   = 'none';
    }
  });
}

audio.addEventListener('play', function () {
  document.body.classList.add('audio-playing');
  if (totalPlaysEl) totalPlaysEl.textContent = String(++totalPlays);
});

audio.addEventListener('pause', function () {
  document.body.classList.remove('audio-playing');
});

audio.addEventListener('ended', function () {
  document.body.classList.remove('audio-playing');
  if (vinylIcon) {
    vinylIcon.style.animation = 'none';
    vinylIcon.style.display   = 'none';
  }
  if (pauseBtn && playBtn) {
    pauseBtn.style.display = 'none';
    playBtn.style.display  = 'inline';
  }
  if (hls) { try { hls.destroy(); } catch (_) {} hls = null; }
});

audio.addEventListener('timeupdate', function () {
  if (!audio.duration || !isFinite(audio.duration)) return;
  const pct = (audio.currentTime / audio.duration) * 100 || 0;
  if (progressBar) {
    progressBar.value = String(pct);
    progressBar.style.setProperty('--val', pct + '%');
  }
  if (timeDisplay) {
    const m = Math.floor(audio.currentTime / 60);
    const s = Math.floor(audio.currentTime % 60).toString().padStart(2, '0');
    timeDisplay.textContent = m + ':' + s;
```

```
    }
  });

  audio.addEventListener('loadedmetadata', function () {
    if (durationDisplay && audio.duration && isFinite(audio.duration)) {
      const m = Math.floor(audio.duration / 60);
      const s = Math.floor(audio.duration % 60).toString().padStart(2, '0');
      durationDisplay.textContent = 'Duration: ' + m + ':' + s;
    }
    if (progressBar) progressBar.max = 100;
  });

  if (progressBar) {
    progressBar.addEventListener('input', function () {
      if (!audio.duration || !isFinite(audio.duration)) return;
      const pct = Number(progressBar.value || 0);
      audio.currentTime = (pct / 100) * audio.duration;
      progressBar.style.setProperty('--val', pct + '%');
    });
    progressBar.style.setProperty('--val', '0%');
  }

  if (volumeBar) {
    volumeBar.addEventListener('input', function () {
      const v = Math.max(0, Math.min(1, parseFloat(volumeBar.value)));
      audio.volume = isFinite(v) ? v : 1;
      const pct = audio.volume * 100;
      volumeBar.style.setProperty('--val', pct + '%');
      if (!audio.paused && vinylIcon) {
        const glowDuration = Math.max(0.5, 4 - audio.volume * 3);
        vinylIcon.style.animation = 'spin 2s linear infinite, glow ' + glowDuration + 's ease-in-out i
      }
    });
    volumeBar.style.setProperty('--val', '100%');
  }
}

// ============================
// Wallet button — delegates to wallets.js
// ============================
const connectWalletBtn = document.getElementById('connectWallet');
if (connectWalletBtn) {
  connectWalletBtn.addEventListener('click', function () {
    if (typeof window.openWalletModal === 'function') {
      window.openWalletModal();
    } else {
      console.warn('openWalletModal() not found. Did you load wallets.js?');
```

```
    }
  });
}
```

`wallets.js` (18.9 KB)

```javascript
// public/Scripts/wallets.js
// Drop-in replacement — no wallet-boot.js needed.
// Handles: modal open, MetaMask/Coinbase/Phantom/Solflare/Zcash connect,
//          global state (window.walletAddress / ethersProvider / ethersSigner),
//          UI updates, disconnect, session restore, account-change listeners,
//          mobile deep links, ethers v5 + v6 compatibility.
'use strict';

(function () {

  // ──────────────────────────────────────────────
  // 1.  Constants & tiny helpers
  // ──────────────────────────────────────────────
  const SESSION_KEY = 'msp_wallet';   // localStorage key  { address, type }
  const isMobile    = /Android|iPhone|iPad|iPod/i.test(navigator.userAgent);
  const origin      = location.origin;

  /** Lazy DOM lookups — called only after DOMContentLoaded */
  const el = {
    connectBtns  : () => document.querySelectorAll('[data-connect-wallet]'),
    disconnect   : () => document.getElementById('btn-disconnect'),
    addrDisplay  : () => document.getElementById('walletAddress'),
    help         : () => document.getElementById('wallet-help'),
    modalEl      : () => document.getElementById('walletModal'),
  };

  function setHelp(msg, isError = false) {
    const h = el.help();
    if (!h) return;
    h.textContent = msg || '';
    h.className   = isError
      ? 'small text-danger mt-3'
      : 'small text-secondary mt-3';
  }

  /**
   * Show a help message with an optional install link rendered inline.
   * Never force-opens a new tab — the user clicks if they want to install.
   */
```

```
function setHelpNotInstalled(walletName, installUrl, deepLinkUrl) {
  const h = el.help();
  if (!h) return;
  h.className = 'small text-warning mt-3';
  h.innerHTML = '';
  const msg  = document.createTextNode(walletName + ' extension not detected. ');
  const link = document.createElement('a');
  link.href        = installUrl;
  link.target      = '_blank';
  link.rel         = 'noopener noreferrer';
  link.textContent = 'Install ' + walletName;
  link.className   = 'text-warning';
  const msg2 = document.createTextNode(', then refresh this page.');
  h.appendChild(msg);
  h.appendChild(link);
  h.appendChild(msg2);
  // Mobile deep-link: only redirect inside the wallet browser, never to the app store
  if (isMobile && deepLinkUrl) {
    location.href = deepLinkUrl;
  }
}

function openNew(url) {
  window.open(url, '_blank', 'noopener,noreferrer');
}

/** openNew kept for any future utility use */
// (tryDeepLink removed — mobile deep-links handled inside setHelpNotInstalled)

// ────────────────────────────────────────────
// 2.  Ethers v5 / v6 compatibility
// ────────────────────────────────────────────
function ethersVersion() {
  // v6 exposes BrowserProvider; v5 exposes providers.Web3Provider
  if (typeof window.ethers?.BrowserProvider === 'function') return 6;
  if (typeof window.ethers?.providers?.Web3Provider === 'function') return 5;
  return null;
}

async function buildProviderAndSigner(rawProvider) {
  const ver = ethersVersion();
  if (ver === 6) {
    const provider = new window.ethers.BrowserProvider(rawProvider);
    const signer   = await provider.getSigner();
    return { provider, signer };
  }
  if (ver === 5) {
```

```
    const provider = new window.ethers.providers.Web3Provider(rawProvider);
    await provider.send('eth_requestAccounts', []);
    const signer = provider.getSigner();
    return { provider, signer };
  }
  // ethers not loaded — store raw address only, no signer
  console.warn('[wallets] ethers not found — provider/signer unavailable');
  return { provider: null, signer: null };
}

// ────────────────────────────────────────────
// 3.  Global state writers
// ────────────────────────────────────────────
function setGlobals(address, provider, signer) {
  window.walletAddress   = address  || null;
  window.ethersProvider  = provider || null;
  window.ethersSigner    = signer   || null;
}

function clearGlobals() {
  setGlobals(null, null, null);
}

// ────────────────────────────────────────────
// 4.  UI helpers
// ────────────────────────────────────────────
function formatAddr(address) {
  if (!address || address.length < 10) return address || '';
  return `${address.slice(0, 6)}…${address.slice(-4)}`;
}

function setConnectedUI(address) {
  const display = formatAddr(address);

  // All "Connect Wallet" buttons on the page
  el.connectBtns().forEach(btn => {
    btn.textContent = 'Connected';
    btn.disabled    = true;
  });

  // Address display
  const addrEl = el.addrDisplay();
  if (addrEl) addrEl.textContent = display;

  // Disconnect button
  const dc = el.disconnect();
  if (dc) dc.classList.remove('d-none');
```

```
  // Persist for session restore
  const saved = JSON.parse(localStorage.getItem(SESSION_KEY) || '{}');
  localStorage.setItem(SESSION_KEY, JSON.stringify({ ...saved, address }));
}

function setDisconnectedUI() {
  el.connectBtns().forEach(btn => {
    btn.textContent = 'Connect Wallet';
    btn.disabled    = false;
  });

  const addrEl = el.addrDisplay();
  if (addrEl) addrEl.textContent = '';

  const dc = el.disconnect();
  if (dc) dc.classList.add('d-none');

  localStorage.removeItem(SESSION_KEY);
  setHelp('');
}

/** Close the Bootstrap modal if it's open */
function closeModal() {
  const modalEl = el.modalEl();
  if (!modalEl || !window.bootstrap) return;
  try {
    bootstrap.Modal.getInstance(modalEl)?.hide();
  } catch (_) {}
}

/** Open the Bootstrap modal — exposed as window.openWalletModal for main.js */
function openWalletModal() {
  const modalEl = el.modalEl();
  if (!modalEl || !window.bootstrap) {
    console.warn('[wallets] Bootstrap modal not available');
    return;
  }
  try {
    const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
    modal.show();
  } catch (e) {
    console.error('[wallets] Could not open wallet modal:', e);
  }
}
window.openWalletModal = openWalletModal;
```

```
// ────────────────────────────────────────────
// 5.  EVM provider utilities
// ────────────────────────────────────────────
function getInjectedProviders() {
  const eth = window.ethereum;
  if (!eth) return [];
  if (Array.isArray(eth.providers) && eth.providers.length) return eth.providers;
  return [eth];
}

function findProvider(predicate) {
  for (const p of getInjectedProviders()) {
    try { if (predicate(p)) return p; } catch (_) {}
  }
  return null;
}

const getMetaMaskProvider = () => findProvider(p => p.isMetaMask && !p.isCoinbaseWallet);
const getCoinbaseProvider = () => findProvider(p => p.isCoinbaseWallet);
const getAnyEvmProvider   = () => findProvider(() => true);

/** Request accounts and build ethers objects for an EVM provider */
async function connectEvmProvider(raw) {
  const accounts = await raw.request({ method: 'eth_requestAccounts' });
  if (!accounts || !accounts[0]) throw new Error('No accounts returned from wallet.');
  const address = accounts[0];
  const { provider, signer } = await buildProviderAndSigner(raw);
  return { address, provider, signer, raw };
}

/** Wire accountsChanged / disconnect events so the UI stays in sync */
function wireEvmEvents(raw) {
  const onAccountsChanged = async (accounts) => {
    if (accounts && accounts[0]) {
      const { provider, signer } = await buildProviderAndSigner(raw).catch(() => ({}));
      setGlobals(accounts[0], provider, signer);
      setConnectedUI(accounts[0]);
    } else {
      clearGlobals();
      setDisconnectedUI();
    }
  };
  const onDisconnect = () => { clearGlobals(); setDisconnectedUI(); };

  // Remove stale listeners before adding (prevents duplicates on re-connect)
  try { raw.removeListener('accountsChanged', onAccountsChanged); } catch (_) {}
  try { raw.removeListener('disconnect',       onDisconnect);      } catch (_) {}
```

```
  raw.on?.('accountsChanged', onAccountsChanged);
  raw.on?.('disconnect',      onDisconnect);
}

// ────────────────────────────────────────────
// 6.  Connect handlers
// ────────────────────────────────────────────
async function connectMetaMask() {
  setHelp('Connecting to MetaMask…');
  try {
    const raw = getMetaMaskProvider();
    if (!raw) {
      setHelpNotInstalled(
        'MetaMask',
        'https://metamask.io/download/',
        `https://metamask.app.link/dapp/${location.host}`
      );
      return;
    }
    const { address, provider, signer } = await connectEvmProvider(raw);
    setGlobals(address, provider, signer);
    setConnectedUI(address);
    wireEvmEvents(raw);
    setHelp(`Connected: ${formatAddr(address)}`);
    closeModal();
    notifyConnected(address);
  } catch (e) {
    setHelp(`MetaMask: ${e.message || e}`, true);
  }
}

async function connectCoinbase() {
  setHelp('Connecting to Coinbase Wallet…');
  try {
    const raw = getCoinbaseProvider();
    if (!raw) {
      setHelpNotInstalled(
        'Coinbase Wallet',
        'https://www.coinbase.com/wallet/downloads',
        `https://go.cb-w.com/dapp?cb_url=${encodeURIComponent(origin)}`
      );
      return;
    }
    const { address, provider, signer } = await connectEvmProvider(raw);
    setGlobals(address, provider, signer);
    setConnectedUI(address);
    wireEvmEvents(raw);
```

```
    setHelp(`Connected: ${formatAddr(address)}`);
    closeModal();
    notifyConnected(address);
  } catch (e) {
    setHelp(`Coinbase Wallet: ${e.message || e}`, true);
  }
}

async function connectPhantom() {
  setHelp('Connecting to Phantom…');
  try {
    const provider = window.solana;
    if (!provider?.isPhantom) {
      setHelpNotInstalled(
        'Phantom',
        'https://phantom.app/download',
        `https://phantom.app/ul/browse/${encodeURIComponent(origin)}`
      );
      return;
    }
    const resp    = await provider.connect({ onlyIfTrusted: false });
    const address = resp.publicKey?.toBase58?.() || String(resp.publicKey);
    setGlobals(address, null, null);
    setConnectedUI(address);
    setHelp(`Connected: ${formatAddr(address)}`);
    closeModal();

    provider.off?.('disconnect');
    provider.off?.('accountChanged');
    provider.on?.('disconnect', () => { clearGlobals(); setDisconnectedUI(); });
    provider.on?.('accountChanged', (pk) => {
      if (pk) { const a = pk.toBase58?.() || String(pk); setGlobals(a, null, null); setConnectedUI(a
      else     { clearGlobals(); setDisconnectedUI(); }
    });

    notifyConnected(address);
  } catch (e) {
    setHelp(`Phantom: ${e.message || e}`, true);
  }
}

async function connectSolflare() {
  setHelp('Connecting to Solflare…');
  try {
    const provider = window.solflare || (window.solana?.isSolflare ? window.solana : null);
    if (!provider) {
      setHelpNotInstalled(
```

```
        'Solflare',
        'https://solflare.com/download',
        `https://solflare.com/ul/v1/browse/${encodeURIComponent(origin)}`
      );
      return;
    }
    await provider.connect();
    const address = provider.publicKey?.toBase58?.() || String(provider.publicKey);
    setGlobals(address, null, null);
    setConnectedUI(address);
    setHelp(`Connected: ${formatAddr(address)}`);
    closeModal();

    provider.off?.('disconnect');
    provider.off?.('accountChanged');
    provider.on?.('disconnect', () => { clearGlobals(); setDisconnectedUI(); });
    provider.on?.('accountChanged', (pk) => {
      if (pk) { const a = pk.toBase58?.() || String(pk); setGlobals(a, null, null); setConnectedUI(a
      else     { clearGlobals(); setDisconnectedUI(); }
    });

    notifyConnected(address);
  } catch (e) {
    setHelp(`Solflare: ${e.message || e}`, true);
  }
}

function connectZcash() {
  setHelpNotInstalled('Zcash Wallet', 'https://z.cash/wallets/', null);
}

// ────────────────────────────────────────────
// 7.  Disconnect
// ────────────────────────────────────────────
async function disconnect() {
  // Solana wallets support real disconnect()
  try { if (window.solana?.isConnected)   await window.solana.disconnect();   } catch (_) {}
  try { if (window.solflare?.isConnected) await window.solflare.disconnect(); } catch (_) {}
  // EVM wallets: no programmatic disconnect — clear app state only
  clearGlobals();
  setDisconnectedUI();
  setHelp('Disconnected. To fully revoke access, use your wallet\'s "Connected Sites" settings.');
}

// ────────────────────────────────────────────
// 8.  Post-connect hook (lets main.js react)
//     main.js looks for window.onWalletConnected
```

```
// ──────────────────────────────────────────────
function notifyConnected(address) {
  if (typeof window.onWalletConnected === 'function') {
    try { window.onWalletConnected(address); } catch (e) { console.error('[wallets] onWalletConnecte
  }
  // Also dispatch a DOM event for flexibility
  document.dispatchEvent(new CustomEvent('walletConnected', { detail: { address } }));
}

// ──────────────────────────────────────────────
// 9.  Session restore
// ──────────────────────────────────────────────
async function tryRestoreSession() {
  try {
    const saved = JSON.parse(localStorage.getItem(SESSION_KEY) || 'null');
    if (!saved?.address) return;

    // EVM: check if provider still has the account authorised (no prompt)
    const raw = getMetaMaskProvider() || getCoinbaseProvider() || getAnyEvmProvider();
    if (raw) {
      const accounts = await raw.request({ method: 'eth_accounts' }); // no popup
      const match = accounts?.find(a => a.toLowerCase() === saved.address.toLowerCase());
      if (match) {
        const { provider, signer } = await buildProviderAndSigner(raw);
        setGlobals(match, provider, signer);
        setConnectedUI(match);
        wireEvmEvents(raw);
        notifyConnected(match);   // let main.js display the cached name
        return;
      }
    }

    // Solana: check Phantom
    if (window.solana?.isPhantom && window.solana.isConnected) {
      const address = window.solana.publicKey?.toBase58?.();
      if (address && address === saved.address) {
        setGlobals(address, null, null);
        setConnectedUI(address);
        notifyConnected(address); // let main.js display the cached name
        return;
      }
    }

    // Solflare
    if ((window.solflare || window.solana?.isSolflare) && (window.solflare || window.solana)?.isConn
      const prov    = window.solflare || window.solana;
      const address = prov.publicKey?.toBase58?.();
```

```
      if (address && address === saved.address) {
        setGlobals(address, null, null);
        setConnectedUI(address);
        notifyConnected(address); // let main.js display the cached name
        return;
      }
    }

    // Could not verify — clear stale session silently
    localStorage.removeItem(SESSION_KEY);
  } catch (e) {
    console.debug('[wallets] Session restore failed (ok):', e.message);
    localStorage.removeItem(SESSION_KEY);
  }
}

// ────────────────────────────────────────────
// 10.  DOM wiring (after DOMContentLoaded)
// ────────────────────────────────────────────
function wireDom() {
  // "Connect Wallet" buttons → open modal
  el.connectBtns().forEach(btn => {
    btn.addEventListener('click', (e) => {
      e.preventDefault();
      openWalletModal();
    });
  });

  // Wallet choice buttons inside the modal (delegated, safe for any page)
  document.addEventListener('click', (e) => {
    const btn = e.target.closest('button');
    if (!btn) return;
    switch (btn.id) {
      case 'btn-metamask': connectMetaMask();  break;
      case 'btn-coinbase': connectCoinbase();  break;
      case 'btn-phantom':  connectPhantom();   break;
      case 'btn-solflare': connectSolflare();  break;
      case 'btn-zcash':    connectZcash();     break;
    }
  });

  // Disconnect button
  const dc = el.disconnect();
  if (dc) {
    dc.addEventListener('click', async (e) => {
      e.preventDefault();
      await disconnect();
```

```
      });
    }
  }

  // ────────────────────────────────────────────
  // 11.  Boot
  // ────────────────────────────────────────────
  function boot() {
    wireDom();
    tryRestoreSession();
  }

  if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', boot, { once: true });
  } else {
    boot();
  }

  // ────────────────────────────────────────────
  // 12.  Public API
  // ────────────────────────────────────────────
  window.mspWallets = {
    openWalletModal,
    connectMetaMask,
    connectCoinbase,
    connectPhantom,
    connectSolflare,
    connectZcash,
    disconnect,
  };

})();
```

`listen.js` (36.3 KB)

```javascript
// Scripts/listen.js
// All page logic for listen.html.
// Depends on: common.js, favorites.js, main.js (all loaded before this file).
'use strict';

(function () {

  // ── Audio element — declared first to prevent hoisting bug ───────────────
  var audio = document.getElementById('audio-player');
```

```
// ── MSPFavorites — always read lazily so load-order never causes stale null
function F()                      { return window.MSPFavorites || null; }
function isLocalFav(type, id)     { var m = F(); return m ? m.isFav(type, id)    : false; }
function toggleLocalFav(type, id) { var m = F(); return m ? m.toggle(type, id)   : false; }

// ── Vinyl badge HTML ──────────────────────────────────────────────────────
var VINYL_BADGE_COVER =
  '<img src="assets/msp-vinyl.svg" width="10" height="10"' +
  ' class="vinyl-cover-badge" title="Supporter royalties enabled" alt="">';
var VINYL_BADGE_INLINE =
  '<img src="assets/msp-vinyl.svg" width="11" height="11"' +
  ' class="vinyl-inline-badge" title="Supporter royalties enabled" alt=""' +
  ' style="display:inline-block;vertical-align:middle;flex-shrink:0;margin-left:3px;">';

// ── State ─────────────────────────────────────────────────────────────────
var catalogData   = [];
var currentFilter = 'all';
var currentView   = 'list';
var activeMenuCid = null;

// ═══════════════════════════════════════════════════════════════════════════
//  CONTENT TYPE TABS
// ═══════════════════════════════════════════════════════════════════════════
document.querySelectorAll('.ct-tab').forEach(function (btn) {
  btn.addEventListener('click', function () {
    document.querySelectorAll('.ct-tab').forEach(function (t) {
      t.classList.remove('active'); t.setAttribute('aria-selected', 'false');
    });
    btn.classList.add('active'); btn.setAttribute('aria-selected', 'true');
    document.querySelectorAll('.tab-pane').forEach(function (p) { p.style.display = 'none'; });
    var pane = document.getElementById('tab-' + btn.dataset.ctTab);
    if (pane) pane.style.display = '';
  });
});

// ═══════════════════════════════════════════════════════════════════════════
//  MUSIC SUB-FILTER PILLS
// ═══════════════════════════════════════════════════════════════════════════
document.querySelectorAll('[data-music-filter]').forEach(function (pill) {
  pill.addEventListener('click', function () {
    document.querySelectorAll('[data-music-filter]').forEach(function (p) { p.classList.remove('acti
    pill.classList.add('active');
    currentFilter = pill.dataset.musicFilter;
    var catalogEl  = document.getElementById('msp-catalog');
    var favSection = document.getElementById('favorites-section');
    if (currentFilter === 'favorites') {
      if (catalogEl)  catalogEl.style.display  = 'none';
```

```
      if (favSection) favSection.style.display = '';
      renderFavoritesPanel();
    } else {
      if (catalogEl)  catalogEl.style.display  = '';
      if (favSection) favSection.style.display = 'none';
      renderCatalog(currentFilter);
    }
  });
});

// ═══════════════════════════════════════════════════════════════════════
//  VIEW TOGGLE (list / grid)
// ═══════════════════════════════════════════════════════════════════════
document.querySelectorAll('[data-view]').forEach(function (btn) {
  btn.addEventListener('click', function () {
    document.querySelectorAll('[data-view]').forEach(function (b) { b.classList.remove('active'); })
    btn.classList.add('active');
    currentView = btn.dataset.view;
    renderCatalog(currentFilter);
  });
});

// ═══════════════════════════════════════════════════════════════════════
//  CATALOG FETCH
// ═══════════════════════════════════════════════════════════════════════
async function fetchCatalog() {
  try { var r = await fetch('/api/catalog'); return r.ok ? await r.json() : []; }
  catch (_) { return []; }
}

// ═══════════════════════════════════════════════════════════════════════
//  RENDER CATALOG
// ═══════════════════════════════════════════════════════════════════════
function renderCatalog(filter) {
  var el = document.getElementById('msp-catalog');
  if (!el) return;
  var items = filterCatalog(catalogData, filter);
  if (!items.length) {
    el.innerHTML = '<div class="catalog-empty">No tracks yet. <a href="creators.html">Upload your fi
    return;
  }
  el.innerHTML = currentView === 'list'
    ? '<div class="track-list">' + items.map(function (item, idx) { return buildTrackRow(item, idx +
    : '<div class="track-grid">'  + items.map(function (item) { return buildTrackTile(item); }).join
  wireInteractions(el);
  loadDurations(items);
}
```

```
function filterCatalog(data, filter) {
  if (filter === 'videos') return data.filter(function (i) { return i.contentType === 'video' || i.c
  return data.filter(function (i) { return i.contentType === 'music' || !filter || filter === 'all';
}

// ═══════════════════════════════════════════════════════════════════════
//  BUILD LIST ROW
//  Grid columns: num | cover-wrap | info | tags | duration+vinyl | heart | ⋮
// ═══════════════════════════════════════════════════════════════════════
function buildTrackRow(item, num) {
  var ct          = item.contentType || 'music';
  var primaryType = ct === 'video' ? 'video' : 'track';
  var isRoyalty   = !!item.supporterRoyaltyEnabled;

  var cover = '<div class="track-cover-wrap">' +
    (item.coverUrl
      ? '<img class="track-cover" src="' + esc(item.coverUrl) + '" alt="' + esc(item.title) + '">'
      : '<div class="track-cover-placeholder">🎵</div>') +
    (isRoyalty ? VINYL_BADGE_COVER : '') +
  '</div>';

  var tags = (item.tags || []).slice(0, 3).map(function (t) {
    return '<span class="track-tag">' + esc(t) + '</span>';
  }).join('');

  var heartClass = computeHeartClass(primaryType, item);
  var typeBadge  = buildTypeBadge(ct);
  var durCell    = '<span class="track-duration" data-dur-id="' + esc(item.contentId) + '">—</span>'
                   (isRoyalty ? VINYL_BADGE_INLINE : '');

  return '<div class="track-row"' + dataAttrs(item, ct) + '>' +
    '<span class="track-num">' + num + '</span>' +
    '<button class="track-play-inline" title="Play">▶</button>' +
    cover +
    '<div class="track-info">' +
      '<div class="track-title">'  + esc(item.title || 'Untitled') + '</div>' +
      '<div class="track-artist">' + esc(item.artistName || '—') + ' ' + typeBadge + '</div>' +
    '</div>' +
    '<div class="track-tags">' + tags + '</div>' +
    durCell +
    '<button class="fav-heart-btn ' + heartClass + '" data-contentid="' + esc(item.contentId) + '" t
    '<button class="track-options-btn" data-contentid="' + esc(item.contentId) + '" title="More opti
  '</div>';
}

// ═══════════════════════════════════════════════════════════════════════
```

```
//  BUILD GRID TILE
//  Structure: cover-wrap (full-width square) | title/artist | footer (heart · dur · ⋮)
// ═══════════════════════════════════════════════════════════════════════════════
function buildTrackTile(item) {
  var ct          = item.contentType || 'music';
  var primaryType = ct === 'video' ? 'video' : 'track';
  var isRoyalty   = !!item.supporterRoyaltyEnabled;
  var heartClass  = computeHeartClass(primaryType, item);

  var cover = '<div class="track-cover-wrap">' +
    (item.coverUrl
      ? '<img src="' + esc(item.coverUrl) + '" alt="' + esc(item.title) + '">'
      : '<div class="track-tile-placeholder">🎵</div>') +
    (isRoyalty ? VINYL_BADGE_COVER : '') +
    '<button class="track-tile-play" title="Play">▶</button>' +
  '</div>';

  return '<div class="track-tile"' + dataAttrs(item, ct) + '>' +
    cover +
    '<div class="track-tile-info">' +
      '<div class="track-tile-title">'  + esc(item.title || 'Untitled') + '</div>' +
      '<div class="track-tile-artist">' + esc(item.artistName || '—') + '</div>' +
    '</div>' +
    '<div class="track-tile-footer">' +
      '<button class="fav-heart-btn ' + heartClass + '" data-contentid="' + esc(item.contentId) + '"
      '<span class="track-duration" data-dur-id="' + esc(item.contentId) + '">—</span>' +
      '<button class="track-options-btn" data-contentid="' + esc(item.contentId) + '" title="More op
    '</div>' +
  '</div>';
}

// ── Shared builders ──────────────────────────────────────────────────────
function dataAttrs(item, ct) {
  return ' data-contentid="' + esc(item.contentId)                  + '"' +
         ' data-hlsurl="'    + esc(item.hlsUrl || '')               + '"' +
         ' data-metaurl="'   + esc(item.metadataUrl || '')          + '"' +
         ' data-title="'     + esc(item.title || '')                + '"' +
         ' data-artist="'    + esc(item.artistName || '')           + '"' +
         ' data-cover="'     + esc(item.coverUrl || '')             + '"' +
         ' data-album="'     + esc(item.album || item.title || '')  + '"' +
         ' data-type="'      + esc(ct)                              + '"';
}

function computeHeartClass(primaryType, item) {
  if (isLocalFav(primaryType, item.contentId))              return 'fav-' + primaryType;
  if (isLocalFav('artist', item.artistName || ''))          return 'fav-artist';
  if (isLocalFav('album',  item.album || item.title || '')) return 'fav-album';
```

```
  return '';
}

function buildTypeBadge(ct) {
  var typeMap   = { music:'MUSIC', podcast:'PODCAST', video:'VIDEO', art_still:'ART', art_animated:'
  var typeClass = { music:'type-music', podcast:'type-podcast', video:'type-video', art_still:'type-
  return '<span class="track-type-badge ' + (typeClass[ct] || 'type-music') + '">' + (typeMap[ct] ||
}

// ═══════════════════════════════════════════════════════════════════════
//  EVENT DELEGATION — single listener on container handles everything
// ═══════════════════════════════════════════════════════════════════════
function wireInteractions(container) {
  container.addEventListener('click', function (e) {

    // Play inline button (list)
    if (e.target.closest('.track-play-inline')) {
      e.stopPropagation();
      playItem(e.target.closest('.track-row'));
      return;
    }
    // Play button (tile)
    if (e.target.closest('.track-tile-play')) {
      e.stopPropagation();
      playItem(e.target.closest('.track-tile'));
      return;
    }
    // Click info or cover → play
    if (e.target.closest('.track-info') || e.target.closest('.track-cover')) {
      playItem(e.target.closest('.track-row'));
      return;
    }

    // Heart — quick-favorite the track
    if (e.target.closest('.fav-heart-btn')) {
      e.stopPropagation();
      closeOptionsMenu();
      var heartBtn = e.target.closest('.fav-heart-btn');
      var cid      = heartBtn.dataset.contentid;
      var row      = container.querySelector('[data-contentid="' + cid + '"]');
      var ct       = row ? (row.dataset.type || 'music') : 'music';
      var favType  = ct === 'video' ? 'video' : 'track';
      var nowFav   = toggleLocalFav(favType, cid);
      // Update every heart for this cid on the whole page
      document.querySelectorAll('.fav-heart-btn[data-contentid="' + cid + '"]').forEach(function (h)
        h.classList.remove('fav-track', 'fav-video', 'fav-artist', 'fav-album');
        if (nowFav) h.classList.add('fav-' + favType);
```

```
      });
      return;
    }

    // Three-dot menu button
    if (e.target.closest('.track-options-btn')) {
      e.stopPropagation();
      var optBtn = e.target.closest('.track-options-btn');
      var cid2   = optBtn.dataset.contentid;
      if (activeMenuCid === cid2) { closeOptionsMenu(); return; }
      openOptionsMenu(optBtn, cid2);
      return;
    }
  });

  // Double-click to play
  container.addEventListener('dblclick', function (e) {
    var row  = e.target.closest('.track-row');
    var tile = e.target.closest('.track-tile');
    if (row)  playItem(row);
    if (tile) playItem(tile);
  });
}

// ═══════════════════════════════════════════════════════════════════════
//  OPTIONS MENU
// ═══════════════════════════════════════════════════════════════════════
var optionsMenu  = document.getElementById('track-options-menu');
var _menuWired   = false;

function openOptionsMenu(triggerBtn, cid) {
  if (!optionsMenu) return;
  activeMenuCid = cid;

  var item      = catalogData.find(function (i) { return i.contentId === cid; }) || {};
  var ct        = item.contentType || 'music';
  var favType   = ct === 'video' ? 'video' : 'track';
  var isFavNow  = isLocalFav(favType, cid);
  var isFavArt  = isLocalFav('artist', item.artistName || '');

  // Update fav button labels
  var favTBtn = optionsMenu.querySelector('[data-opt="fav-track"]');
  var favABtn = optionsMenu.querySelector('[data-opt="fav-artist"]');
  if (favTBtn) {
    favTBtn.innerHTML = '<i class="opt-icon">♥</i> ' + (isFavNow ? '✓ ' : '') + (ct === 'video' ? 'F
    favTBtn.classList.toggle('fav-active-track', isFavNow);
  }
```

```
  if (favABtn) {
    favABtn.innerHTML = '<i class="opt-icon">★</i> ' + (isFavArt ? '✓ ' : '') + 'Favorite Artist';
    favABtn.classList.toggle('fav-active-artist', isFavArt);
  }

  // Populate playlist submenu
  var plListEl = optionsMenu.querySelector('#opt-playlist-list');
  if (plListEl) plListEl.innerHTML = buildPlaylistSubmenu(item);

  // Download gating
  var canDl   = window.CAN && window.CAN.download && window.CAN.download();
  var dlBtn   = optionsMenu.querySelector('[data-opt="download"]');
  var dlBadge = optionsMenu.querySelector('#opt-download-badge');
  if (dlBtn)   dlBtn.classList.toggle('opt-disabled', !canDl);
  if (dlBadge) dlBadge.style.display = canDl ? 'none' : '';

  // Store context on menu element for the handler
  optionsMenu.dataset.cid    = cid;
  optionsMenu.dataset.ct     = ct;
  optionsMenu.dataset.hlsurl = item.hlsUrl || '';
  optionsMenu.dataset.title  = item.title  || '';
  optionsMenu.dataset.artist = item.artistName || '';

  // Position near trigger
  var rect  = triggerBtn.getBoundingClientRect();
  var left  = Math.max(8, rect.right - 220);
  var top   = rect.bottom + 4;
  if (top + 340 > window.innerHeight) top = Math.max(8, rect.top - 340);
  optionsMenu.style.left = left + 'px';
  optionsMenu.style.top  = top  + 'px';
  optionsMenu.classList.add('open');

  if (!_menuWired) wireMenuHandlers();
}

function closeOptionsMenu() {
  if (optionsMenu) optionsMenu.classList.remove('open');
  activeMenuCid = null;
}

function buildPlaylistSubmenu(item) {
  var canPlaylist = window.CAN && window.CAN.createPlaylist && window.CAN.createPlaylist();
  if (!canPlaylist) return '<span class="opt-pl-empty">Tier 2+ required to use playlists</span>';

  var profile   = window.userProfile || null;
  var playlists = (profile && profile.playlists) || [];
  var myWallet  = (window.walletAddress || '').toLowerCase();
```

```
  var itemOwner = (item.wallet || '').toLowerCase();

  // Creators cannot add their own assets to their own playlists
  var eligible = playlists.filter(function () {
    return itemOwner !== myWallet;
  });

  var rows = eligible.slice(0, 8).map(function (pl) {
    return '<button class="opt-pl-item" data-opt="add-to-playlist" data-playlist-id="' + esc(pl.id)
      '<i class="opt-icon">🎵</i> ' + esc(pl.name || 'Playlist') + '</button>';
  }).join('');

  rows += '<button class="opt-pl-item" data-opt="new-playlist"><i class="opt-icon">+</i> New Playli
  return rows;
}

function wireMenuHandlers() {
  _menuWired = true;
  optionsMenu.addEventListener('click', function (e) {
    var btn = e.target.closest('[data-opt]');
    if (!btn) return;
    e.stopPropagation();

    var opt     = btn.dataset.opt;
    var cid     = optionsMenu.dataset.cid;
    var ct      = optionsMenu.dataset.ct     || 'music';
    var hlsUrl  = optionsMenu.dataset.hlsurl || '';
    var title   = optionsMenu.dataset.title  || '';
    var artist  = optionsMenu.dataset.artist || '';
    var favType = ct === 'video' ? 'video' : 'track';
    var item    = catalogData.find(function (i) { return i.contentId === cid; }) || {};

    if (opt === 'fav-track') {
      var nowFav = toggleLocalFav(favType, cid);
      document.querySelectorAll('.fav-heart-btn[data-contentid="' + cid + '"]').forEach(function (h)
        h.classList.remove('fav-track', 'fav-video', 'fav-artist', 'fav-album');
        if (nowFav) h.classList.add('fav-' + favType);
      });
      btn.innerHTML = '<i class="opt-icon">♥</i> ' + (nowFav ? '✓ ' : '') + (ct === 'video' ? 'Favor
      btn.classList.toggle('fav-active-track', nowFav);
      return;
    }

    if (opt === 'fav-artist') {
      var nowFavA = toggleLocalFav('artist', item.artistName || '');
      btn.innerHTML = '<i class="opt-icon">★</i> ' + (nowFavA ? '✓ ' : '') + 'Favorite Artist';
      btn.classList.toggle('fav-active-artist', nowFavA);
```

```
      closeOptionsMenu();
      return;
    }

    if (opt === 'share') {
      var url = window.location.origin + '/listen.html?cid=' + cid;
      if (navigator.share) {
        navigator.share({ title: title, text: title + ' by ' + artist, url: url }).catch(function ()
      } else {
        navigator.clipboard.writeText(url)
          .then(function () { showToast('Link copied to clipboard'); })
          .catch(function () { showToast('Could not copy link'); });
      }
      closeOptionsMenu(); return;
    }

    if (opt === 'download') {
      if (btn.classList.contains('opt-disabled')) { showToast('Tier 2+ subscription required to down
      if (hlsUrl) window.open(hlsUrl, '_blank');
      closeOptionsMenu(); return;
    }

    if (opt === 'lyrics') {
      showToast('Lyrics not yet available for this track.');
      closeOptionsMenu(); return;
    }

    if (opt === 'info') {
      showTrackInfo(item);
      closeOptionsMenu(); return;
    }

    if (opt === 'add-to-playlist') {
      addToPlaylist(cid, btn.dataset.playlistId, title);
      closeOptionsMenu(); return;
    }

    if (opt === 'new-playlist') {
      // Playlist creation lives on profile page
      window.location.href = 'profile.html';
      closeOptionsMenu(); return;
    }
  });
}

async function addToPlaylist(cid, playlistId, title) {
  if (!window.walletAddress) { showToast('Connect wallet first'); return; }
```

```
  try {
    var r = await fetch('/api/playlists/' + playlistId + '/add', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ wallet: window.walletAddress, cid: cid }),
    });
    showToast(r.ok ? '✓ Added to playlist' : 'Could not add to playlist');
  } catch (_) {
    showToast('Could not add to playlist');
  }
}

// ── Toast ──────────────────────────────────────────────────────────────
var _toastEl, _toastTimer;
function showToast(msg) {
  if (!_toastEl) {
    _toastEl = document.createElement('div');
    _toastEl.style.cssText =
      'background:var(--bg-raised);border:1px solid var(--border-mid);border-left:3px solid var(--te
      'border-radius:8px;bottom:96px;box-shadow:0 8px 32px rgba(0,0,0,.6);color:var(--text-primary);
      'font-size:13px;padding:10px 16px;position:fixed;right:20px;z-index:4000;transition:opacity .2
    document.body.appendChild(_toastEl);
  }
  _toastEl.textContent = msg;
  _toastEl.style.opacity = '1';
  clearTimeout(_toastTimer);
  _toastTimer = setTimeout(function () { _toastEl.style.opacity = '0'; }, 3000);
}

function showTrackInfo(item) {
  var lines = [
    'Title: '     + (item.title || '—'),
    'Artist: '    + (item.artistName || '—'),
    'Type: '      + (item.contentType || '—'),
    'Content ID: '+ (item.contentId || '—'),
  ];
  if (item.supporterRoyaltyEnabled) lines.push('★ Supporter royalties enabled');
  alert(lines.join('\n'));
}

// ═══════════════════════════════════════════════════════════════════════
//  PLAY AN ITEM
// ═══════════════════════════════════════════════════════════════════════
function playItem(el) {
  if (!el) return;
  var hlsUrl   = el.dataset.hlsurl  || '';
  var metaUrl  = el.dataset.metaurl || '';
```

```
  var title    = el.dataset.title   || 'Unknown Track';
  var artist   = el.dataset.artist  || '—';
  var coverUrl = el.dataset.cover   || '';
  if (!hlsUrl) return;

  var nameEl   = document.getElementById('track-name');
  var artistEl = document.getElementById('player-artist-name');
  var vinylEl  = document.getElementById('vinyl-icon');
  if (nameEl)   { nameEl.textContent = title; nameEl.style.color = ''; nameEl.style.fontStyle = '';
  if (artistEl)   artistEl.textContent = artist;
  if (vinylEl && coverUrl) vinylEl.src = coverUrl;

  document.querySelectorAll('.track-row.is-playing, .track-tile.is-playing').forEach(function (r) {
  el.classList.add('is-playing');

  if (typeof window.playHls === 'function') window.playHls(hlsUrl, metaUrl);
}

// ═══════════════════════════════════════════════════════════════════════
//  PLAY/PAUSE TOGGLE
// ═══════════════════════════════════════════════════════════════════════
var mspToggle = document.getElementById('msp-play-toggle');
var iconPlay  = document.getElementById('icon-play');
var iconPause = document.getElementById('icon-pause');

function setPlayingState(playing) {
  if (!mspToggle) return;
  mspToggle.classList.toggle('is-playing', playing);
  if (iconPlay)  iconPlay.style.display  = playing ? 'none' : '';
  if (iconPause) iconPause.style.display = playing ? '' : 'none';
  mspToggle.setAttribute('aria-label', playing ? 'Pause' : 'Play');
}

if (mspToggle && audio) {
  mspToggle.addEventListener('click', function () {
    if (audio.paused || audio.ended) audio.play().catch(function () {});
    else audio.pause();
  });
  audio.addEventListener('play',  function () { setPlayingState(true);  });
  audio.addEventListener('pause', function () { setPlayingState(false); });
  audio.addEventListener('ended', function () { setPlayingState(false); });
}

// ═══════════════════════════════════════════════════════════════════════
//  SKIP ±15 s
// ═══════════════════════════════════════════════════════════════════════
var skipBack = document.getElementById('btn-skip-back');
```

```
var skipFwd  = document.getElementById('btn-skip-fwd');
if (skipBack) skipBack.addEventListener('click', function () {
  if (audio) audio.currentTime = Math.max(0, audio.currentTime - 15);
});
if (skipFwd) skipFwd.addEventListener('click', function () {
  if (audio && audio.duration) audio.currentTime = Math.min(audio.duration, audio.currentTime + 15);
});

// ═══════════════════════════════════════════════════════════════════════════
//  CLOSE MENU on outside click / Escape
// ═══════════════════════════════════════════════════════════════════════════
document.addEventListener('click', function () { closeOptionsMenu(); });
document.addEventListener('keydown', function (e) { if (e.key === 'Escape') closeOptionsMenu(); });

// ═══════════════════════════════════════════════════════════════════════════
//  BILLING PERIOD TOGGLE
// ═══════════════════════════════════════════════════════════════════════════
document.querySelectorAll('input[name="billing-period"]').forEach(function (radio) {
  radio.addEventListener('change', function () {
    var period = this.value;
    var prices = { monthly:['$10.99','$19.99','$34.99'], annual:['$131.88/yr','$239.88/yr','$419.88/
    var labels = { monthly:'/mo', annual:'/yr', rolling:'/3 days' };
    var p = prices[period];
    var t1 = document.querySelector('.price-t1'); if (t1) t1.textContent = p[0];
    var t2 = document.querySelector('.price-t2'); if (t2) t2.textContent = p[1];
    var t3 = document.querySelector('.price-t3'); if (t3) t3.textContent = p[2];
    document.querySelectorAll('.period-label').forEach(function (el) { el.textContent = labels[perio
    document.querySelectorAll('[data-plan-monthly]').forEach(function (btn) {
      var key = 'plan' + period.charAt(0).toUpperCase() + period.slice(1);
      btn.dataset.subscribePlan = btn.dataset[key];
    });
  });
});

// ═══════════════════════════════════════════════════════════════════════════
//  DURATION HELPERS
// ═══════════════════════════════════════════════════════════════════════════
function fmtDuration(secs) {
  if (!secs || !isFinite(secs)) return '—';
  var s = Math.round(secs);
  var h = Math.floor(s / 3600);
  var m = Math.floor((s % 3600) / 60);
  var ss = String(s % 60).padStart(2, '0');
  return h > 0 ? h + ':' + String(m).padStart(2, '0') + ':' + ss : m + ':' + ss;
}

function loadDurations(items) {
```

```
  items.forEach(function (item) {
    if (!item.metadataUrl) return;
    fetch(item.metadataUrl)
      .then(function (r) { return r.ok ? r.json() : null; })
      .then(function (meta) {
        if (!meta) return;
        var dur = meta.duration
          || (meta.files && meta.files.duration)
          || (meta.video && meta.video.duration)
          || (meta.audio && meta.audio.duration);
        if (!dur) return;
        var formatted = fmtDuration(parseFloat(dur));
        // Update all spans for this contentId (list + tile both rendered)
        document.querySelectorAll('[data-dur-id="' + item.contentId + '"]').forEach(function (span)
          span.textContent = formatted;
        });
      })
      .catch(function () {});
  });
}

// ═══════════════════════════════════════════════════════════════════════
//  FAVORITES PANEL (compact, listen.html only — management is on profile.html)
// ═══════════════════════════════════════════════════════════════════════
function renderFavoritesPanel() {
  var chipsEl = document.getElementById('fav-summary-chips');
  var listEl  = document.getElementById('favorites-list');
  var fav     = F();

  if (chipsEl && fav) {
    var chips = Object.keys(fav.FAV_TYPES).map(function (t) {
      var count = fav.countOf(t);
      if (!count) return '';
      var meta = fav.FAV_TYPES[t];
      return '<a href="favorites.html" style="align-items:center;background:rgba(0,0,0,.3);border:1p
        'border-radius:16px;color:' + meta.cssColor + ';display:inline-flex;font-size:11px;font-weig
        meta.icon + ' ' + meta.label + ' <strong>' + count + '</strong></a>';
    }).join('');
    chipsEl.innerHTML = chips || '<span class="text-muted small">No favorites yet.</span>';
  }

  if (!listEl) return;
  var fav2     = F();
  var playable = [];
  ['track', 'video', 'nft_music', 'nft_video'].forEach(function (t) {
    (fav2 ? fav2.getAll(t) : []).forEach(function (cid) {
      var item = catalogData.find(function (c) { return c.contentId === cid; });
```

```
      if (item) playable.push(item);
    });
  });

  if (!playable.length) {
    listEl.innerHTML = '<p class="text-muted small py-2">No favorited tracks yet. Tap ♥ on any track
    return;
  }

  listEl.innerHTML = '<div class="track-list">' +
    playable.map(function (item, idx) { return buildTrackRow(item, idx + 1); }).join('') +
    '</div>';
  wireInteractions(listEl);
  loadDurations(playable);
}

// ═══════════════════════════════════════════════════════════════════════════
//  HTML ESCAPE
// ═══════════════════════════════════════════════════════════════════════════
function esc(str) {
  return String(str || '')
    .replace(/&/g, '&amp;').replace(/"/g, '&quot;')
    .replace(/</g, '&lt;').replace(/>/g, '&gt;');
}

// ═══════════════════════════════════════════════════════════════════════════
//  BOOT
// ═══════════════════════════════════════════════════════════════════════════
(async function boot() {
  var catalogEl = document.getElementById('msp-catalog');
  catalogData   = await fetchCatalog();

  if (catalogData.length) {
    renderCatalog('all');
  } else {
    if (catalogEl) {
      catalogEl.innerHTML = '<div class="catalog-empty">No tracks yet.<br>' +
        '<a href="creators.html">Upload your first track on Creators Corner</a>.</div>';
    }
  }

  // Podcasts tab
  var podcastItems = catalogData.filter(function (i) { return i.contentType === 'podcast'; });
  var podList      = document.getElementById('podcast-list');
  if (podList && podcastItems.length) {
    podList.innerHTML = podcastItems.map(function (item) {
      var isRoyalty = !!item.supporterRoyaltyEnabled;
```

```
        return '<div class="podcast-tile">' +
          (item.coverUrl
            ? '<img src="' + esc(item.coverUrl) + '" alt="' + esc(item.title) + '">'
            : '<div style="aspect-ratio:1;background:var(--bg-raised);display:flex;align-items:center;
          '<div class="podcast-tile-info">' +
            '<div class="podcast-tile-title">' + esc(item.title) + (isRoyalty ? VINYL_BADGE_INLINE : '
            '<div class="podcast-tile-meta">'  + esc(item.artistName || '—') + '</div>' +
            '<button class="podcast-tile-play"' +
              ' data-hlsurl="'  + esc(item.hlsUrl || '')     + '"' +
              ' data-metaurl="' + esc(item.metadataUrl || '') + '"' +
              ' data-title="'   + esc(item.title || '')      + '"' +
              ' data-artist="'  + esc(item.artistName || '') + '"' +
              ' data-cover="'   + esc(item.coverUrl || '')   + '">▶ Play Episode</button>' +
          '</div></div>';
      }).join('');
      podList.querySelectorAll('.podcast-tile-play').forEach(function (btn) {
        btn.addEventListener('click', function () { playItem(btn); });
      });
    }
  })();

})(); // end listen.js
```

`favorites.js` (3.5 KB)

```javascript
'use strict';

const express        = require('express');
const { v4: uuidv4 } = require('uuid');
const logger         = require('../config/logger');
const { PLAYLIST_CAPABLE_LEVELS } = require('../config/constants');
const profileService = require('../services/profileService');

const router = express.Router();

// SPEC: Favorites are available to ALL user roles — no subscription gate.
// Favoriting alone generates NO royalty event.
// Any complete playthrough of a favorited track IS royalty-eligible (same as any play).

// GET /api/favorites/:wallet
router.get('/:wallet', async (req, res, next) => {
  try {
    const profiles = await profileService.loadProfiles();
    const profile  = profiles[req.params.wallet];
    if (!profile) return res.status(404).json({ error: 'Profile not found' });
```

```
    res.json({ favorites: profile.favorites || [] });
  } catch (err) { next(err); }
});

// POST /api/favorites/add
router.post('/add', async (req, res, next) => {
  try {
    const { wallet, cid } = req.body || {};
    if (!wallet || !cid) return res.status(400).json({ error: 'Missing wallet or cid' });

    const profiles = await profileService.loadProfiles();
    if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

    if (!profiles[wallet].favorites) profiles[wallet].favorites = [];
    if (!profiles[wallet].favorites.includes(cid)) {
      profiles[wallet].favorites.push(cid);
      await profileService.saveProfiles(profiles);
    }

    res.json({ success: true, favorites: profiles[wallet].favorites });
  } catch (err) { next(err); }
});

// POST /api/favorites/remove
router.post('/remove', async (req, res, next) => {
  try {
    const { wallet, cid } = req.body || {};
    if (!wallet || !cid) return res.status(400).json({ error: 'Missing wallet or cid' });

    const profiles = await profileService.loadProfiles();
    if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

    profiles[wallet].favorites = (profiles[wallet].favorites || []).filter((c) => c !== cid);
    await profileService.saveProfiles(profiles);
    res.json({ success: true, favorites: profiles[wallet].favorites });
  } catch (err) { next(err); }
});

// POST /api/favorites/convert-to-playlist
router.post('/convert-to-playlist', async (req, res, next) => {
  try {
    const { wallet, name, cids, isPublic = true } = req.body || {};
    if (!wallet || !name || !Array.isArray(cids) || !cids.length) {
      return res.status(400).json({ error: 'Missing wallet, name, or cids' });
    }

    const profiles = await profileService.loadProfiles();
```

```
    if (!profiles[wallet]) return res.status(404).json({ error: 'Profile not found' });

    const level = profileService.getCapabilityLevel(profiles[wallet]);
    if (!PLAYLIST_CAPABLE_LEVELS.has(level)) {
      return res.status(403).json({ error: 'Tier 2 or higher required to create playlists from Favorit
    }

    const playlistId = uuidv4();
    const playlist   = {
      id:           playlistId,
      name,
      cids,
      wallet,
      sharePercent:    8,
      isPublic:        !!isPublic,
      royaltyEligible: !!isPublic,
      fromFavorites:   true,
      createdAt:       Date.now(),
    };

    if (!profiles[wallet].playlists) profiles[wallet].playlists = [];
    profiles[wallet].playlists.push(playlist);
    await profileService.saveProfiles(profiles);

    logger.info({ wallet, playlistId, name, cidsCount: cids.length, isPublic }, 'Favorites converted t
    res.status(201).json({ success: true, playlist });
  } catch (err) { next(err); }
});

module.exports = router;
```

`dashboard.js` (18.3 KB)

```javascript
// Scripts/dashboard.js
// Dashboard page logic for dashboard.html.
// Depends on: common.js, favorites.js, main.js (all loaded before this).
'use strict';

(function () {

  var F = window.MSPFavorites;

  function esc(str) {
    return String(str || '')
      .replace(/&/g, '&amp;')
```

```
    .replace(/"/g, '&quot;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;');
}

function relativeTime(ts) {
  if (!ts) return '';
  var diff  = Date.now() - ts;
  var mins  = Math.floor(diff / 60000);
  var hours = Math.floor(diff / 3600000);
  var days  = Math.floor(diff / 86400000);
  if (mins  < 1)   return 'just now';
  if (mins  < 60)  return mins + 'm ago';
  if (hours < 24)  return hours + 'h ago';
  if (days  < 30)  return days + 'd ago';
  return new Date(ts).toLocaleDateString();
}

// ═══════════════════════════════════════════════════════════════════════
//  RENDER STAT CARDS
// ═══════════════════════════════════════════════════════════════════════
function renderStats(profile, myAssets) {
  var statsEl = document.getElementById('dash-stats');
  if (!statsEl) return;

  var totalPlays    = myAssets.reduce(function (s, a) { return s + (a.plays || 0); }, 0);
  var totalEarned   = myAssets.reduce(function (s, a) { return s + (parseFloat(a.royaltiesEarned) ||
  var srEnabled     = myAssets.filter(function (a) { return a.supporterRoyaltyEnabled; }).length;
  var favTotal      = F ? F.totalCount() : 0;
  var isCreator     = profile && ['creator', 'platform_nft_creator'].includes(profile.account_type);
  var tier          = profile ? (profile.listener_tier || null) : null;
  var subActive     = profile && profile.subscription_expiry && Date.now() < profile.subscription_ex

  var cards = [
    {
      label: 'Subscription',
      value: subActive
        ? (isCreator ? 'Creator' : 'Tier ' + (tier || '—'))
        : 'None',
      colorClass: subActive ? 'teal' : '',
      sub: subActive
        ? 'Active · expires ' + new Date(profile.subscription_expiry).toLocaleDateString()
        : 'No active subscription',
      link: 'listen.html#subscribe',
      linkLabel: 'Subscribe →',
      always: true,
    },
```

```
    {
      label: 'Favorites Saved',
      value: favTotal,
      colorClass: favTotal > 0 ? 'ember' : '',
      sub: favTotal > 0 ? F.summary().map(function (s) { return s.meta.icon + ' ' + s.count; }).join
      link: 'favorites.html',
      linkLabel: 'Manage →',
      always: true,
    },
  ];

  if (isCreator) {
    cards.push(
      {
        label: 'Total Plays',
        value: totalPlays.toLocaleString(),
        colorClass: 'teal',
        sub: myAssets.length + ' asset' + (myAssets.length !== 1 ? 's' : '') + ' uploaded',
        link: 'asset-manager.html',
        linkLabel: 'View Assets →',
        always: false,
      },
      {
        label: 'ETH Royalties Earned',
        value: totalEarned.toFixed(4),
        colorClass: 'gold',
        sub: srEnabled + ' asset' + (srEnabled !== 1 ? 's' : '') + ' with supporter royalties',
        link: 'asset-manager.html',
        linkLabel: 'Manage Royalties →',
        always: false,
      }
    );
  }

  statsEl.innerHTML = cards.map(function (c) {
    return '<div class="stat-card">' +
      '<div class="stat-card-label">' + esc(c.label) + '</div>' +
      '<div class="stat-card-value ' + c.colorClass + '">' + esc(String(c.value)) + '</div>' +
      '<div class="stat-card-sub">' + esc(c.sub) + '</div>' +
      (c.link ? '<a href="' + esc(c.link) + '" class="stat-card-link">' + esc(c.linkLabel) + '</a>'
    '</div>';
  }).join('');
}

// ═══════════════════════════════════════════════════════════════════════
//  RENDER EARNINGS PANEL
// ═══════════════════════════════════════════════════════════════════════
```

```
function renderEarnings(myAssets) {
  var bodyEl = document.getElementById('dash-earnings-body');
  if (!bodyEl) return;

  // Show manage button only for creators
  var manageBtn = document.querySelector('#dash-earnings-panel [data-requires="upload"]');
  if (manageBtn) manageBtn.style.display = '';

  var earners = myAssets
    .filter(function (a) { return (parseFloat(a.royaltiesEarned) || 0) > 0 || a.supporterRoyaltyEnab
    .sort(function (a, b) { return (parseFloat(b.royaltiesEarned) || 0) - (parseFloat(a.royaltiesEar
    .slice(0, 8);

  if (!earners.length) {
    bodyEl.innerHTML = '<p class="text-muted small py-3">No royalties earned yet. Enable supporter r
    return;
  }

  var maxEarned = earners.reduce(function (m, a) { return Math.max(m, parseFloat(a.royaltiesEarned)

  bodyEl.innerHTML = '<div style="padding-top:10px;">' +
    earners.map(function (a) {
      var earned  = parseFloat(a.royaltiesEarned) || 0;
      var pct     = Math.max(4, (earned / maxEarned) * 100);
      var vinyl   = a.supporterRoyaltyEnabled
        ? '<img src="assets/msp-vinyl.svg" width="11" height="11" style="vertical-align:middle;margi
        : '';
      return '<div class="earnings-bar-wrap">' +
        '<span class="earnings-bar-label">' + esc((a.title || 'Untitled').slice(0, 14)) + vinyl + '<
        '<div class="earnings-bar-track"><div class="earnings-bar-fill" style="width:' + pct.toFixed
        '<span class="earnings-bar-val">' + earned.toFixed(4) + ' Ξ</span>' +
      '</div>';
    }).join('') +
  '</div>';
}

// ═══════════════════════════════════════════════════════════════════════
//  RENDER FAVORITES SUMMARY
// ═══════════════════════════════════════════════════════════════════════
function renderFavSummary() {
  var bodyEl = document.getElementById('dash-fav-body');
  if (!bodyEl || !F) return;

  var summary = F.summary();
  if (!summary.length) {
    bodyEl.innerHTML = '<p class="text-muted small py-3">No favorites yet. Browse <a href="listen.ht
    return;
```

```
  }

  bodyEl.innerHTML = '<div style="display:flex;flex-wrap:wrap;gap:8px;padding-top:12px;">' +
    summary.map(function (s) {
      return '<a href="favorites.html" style="' +
        'align-items:center;background:rgba(0,0,0,.3);border:1px solid ' + s.meta.cssColor + ';' +
        'border-radius:16px;color:' + s.meta.cssColor + ';display:inline-flex;' +
        'font-size:11px;font-weight:600;gap:5px;padding:4px 12px;text-decoration:none;">' +
        s.meta.icon + ' ' + s.meta.label + ' <strong>' + s.count + '</strong>' +
      '</a>';
    }).join('') +
  '</div>';
}

// ═══════════════════════════════════════════════════════════════════════
//  RENDER PLAYLISTS
// ═══════════════════════════════════════════════════════════════════════
function renderPlaylists(profile) {
  var panel  = document.getElementById('dash-playlists-panel');
  var bodyEl = document.getElementById('dash-playlists-body');
  if (!panel || !bodyEl) return;

  var playlists = profile ? (profile.playlists || []) : [];
  if (!playlists.length) {
    panel.style.display = '';
    bodyEl.innerHTML = '<p class="text-muted small py-3">No playlists yet. <a href="favorites.html"
    return;
  }

  panel.style.display = '';
  bodyEl.innerHTML = playlists.slice(0, 6).map(function (pl) {
    var isPublic = pl.isPublic !== false;
    return '<div class="pl-row">' +
      '<div>' +
        '<div class="pl-name">' + esc(pl.name || 'Untitled Playlist') + '</div>' +
        '<div class="pl-meta">' + (pl.cids ? pl.cids.length : 0) + ' tracks · created ' + relativeTi
      '</div>' +
      '<span class="pl-badge ' + (isPublic ? 'pub' : 'priv') + '">' + (isPublic ? '🌐 Public' : '🔒
    '</div>';
  }).join('');
}

// ═══════════════════════════════════════════════════════════════════════
//  RENDER RECENT UPLOADS
// ═══════════════════════════════════════════════════════════════════════
function renderUploads(myAssets) {
  var panel  = document.getElementById('dash-uploads-panel');
```

```
  var bodyEl = document.getElementById('dash-uploads-body');
  if (!panel || !bodyEl) return;

  panel.style.display = '';

  if (!myAssets.length) {
    bodyEl.innerHTML = '<p class="text-muted small py-3">No uploads yet. <a href="creators.html" sty
    return;
  }

  var typeIcon = { music: '🎵', podcast: '🎙', video: '🎬', art_still: '🖼', art_animated: '🎨' };

  bodyEl.innerHTML = myAssets.slice(0, 5).map(function (item) {
    var icon    = typeIcon[item.contentType] || '🎵';
    var cover   = item.coverUrl
      ? '<img style="border-radius:4px;flex-shrink:0;height:36px;object-fit:cover;width:36px;" src="
      : '<div style="align-items:center;background:var(--bg-raised);border-radius:4px;color:var(--te
    var vinyl   = item.supporterRoyaltyEnabled
      ? '<img src="assets/msp-vinyl.svg" width="11" height="11" style="vertical-align:middle;margin-
      : '';
    return '<div style="align-items:center;border-bottom:1px solid var(--border-subtle);display:flex
      cover +
      '<div style="flex:1;min-width:0;">' +
        '<div style="color:var(--text-primary);font-size:12px;font-weight:600;overflow:hidden;text-o
          esc(item.title || 'Untitled') + vinyl +
        '</div>' +
        '<div style="color:var(--text-secondary);font-size:10px;">' +
          esc(item.artistName || '—') + ' · ' + (item.contentType || 'music').toUpperCase() +
          ' · ' + (item.plays || 0) + ' plays' +
        '</div>' +
      '</div>' +
      '<a href="asset-manager.html#' + esc(item.contentId) + '" style="color:var(--text-muted);font-
    '</div>';
  }).join('');
}

// ═══════════════════════════════════════════════════════════════════════
//  RENDER SUBSCRIPTION PANEL
// ═══════════════════════════════════════════════════════════════════════
function renderSubscription(profile) {
  var bodyEl = document.getElementById('dash-sub-body');
  if (!bodyEl) return;

  if (!profile) {
    bodyEl.innerHTML = '<p class="text-muted small">No profile found.</p>';
    return;
  }
```

```
  var subActive = profile.subscription_expiry && Date.now() < profile.subscription_expiry;
  var typeLabel = {
    listener:            'Listener Tier ' + (profile.listener_tier || 1),
    creator:             'Creator',
    platform_nft_creator:'Creator (Platform NFT)',
    admin:               'Admin',
  };
  var label = typeLabel[profile.account_type] || profile.account_type || 'Unknown';
  var color = subActive ? 'var(--teal)' : 'var(--text-muted)';

  bodyEl.innerHTML =
    '<div style="display:flex;flex-wrap:wrap;gap:16px;align-items:center;">' +
      '<div>' +
        '<div style="color:' + color + ';font-size:15px;font-weight:700;">' + esc(label) + '</div>'
        '<div class="text-muted small">' +
          (subActive
            ? 'Active · expires ' + new Date(profile.subscription_expiry).toLocaleDateString()
            : 'No active subscription') +
        '</div>' +
      '</div>' +
      '<a href="listen.html#subscribe" class="btn btn-sm btn-outline-primary ms-auto">Manage Plan →<
    '</div>';
}

// ═══════════════════════════════════════════════════════════════════════════
//  SUPPORTER ROYALTY DISABLE NOTIFICATIONS
//  Show a notice for any asset where the 30-day cooldown is active
//  (meaning supporter royalties were recently disabled).
// ═══════════════════════════════════════════════════════════════════════════
function renderNotices(myAssets) {
  var noticesEl = document.getElementById('dash-notices');
  if (!noticesEl || !window.walletAddress) return;

  var notices = [];
  myAssets.forEach(function (item) {
    var lockKey = 'am_sr_lock:' + window.walletAddress.toLowerCase() + ':' + item.contentId;
    var lockUntil = parseInt(localStorage.getItem(lockKey) || '0', 10);
    if (lockUntil > Date.now()) {
      var daysLeft = Math.ceil((lockUntil - Date.now()) / (24 * 60 * 60 * 1000));
      notices.push(
        '<div class="dash-notice">' +
          '<span class="dash-notice-icon">⚠</span>' +
          '<div class="dash-notice-text">' +
            '<strong>' + esc(item.title || 'Untitled') + '</strong> — ' +
            'Supporter royalties disabled. Supporters have been notified. ' +
            'You can re-enable in <strong>' + daysLeft + ' day' + (daysLeft !== 1 ? 's' : '') + '</s
```

```
            '<a href="asset-manager.html#' + esc(item.contentId) + '" style="color:var(--teal);">Man
          '</div>' +
          '<button class="dash-notice-dismiss" onclick="this.closest(\'.dash-notice\').remove()" tit
        '</div>'
      );
    }
  });

  noticesEl.innerHTML = notices.join('');
}

// ═══════════════════════════════════════════════════════════════════════
//  BOOT
// ═══════════════════════════════════════════════════════════════════════
async function boot() {
  if (!window.walletAddress) {
    var prompt = document.getElementById('dash-wallet-prompt');
    if (prompt) prompt.style.display = '';
    document.addEventListener('walletConnected', function () {
      if (prompt) prompt.style.display = 'none';
      boot();
    });
    return;
  }

  var main = document.getElementById('dash-main');
  if (main) main.style.display = '';

  // Fetch profile and catalog in parallel
  var profileData = null;
  var catalogData = [];

  try {
    var results = await Promise.all([
      fetch('/api/profile/' + window.walletAddress).then(function (r) { return r.ok ? r.json() : nul
      fetch('/api/catalog').then(function (r) { return r.ok ? r.json() : []; }),
    ]);
    profileData = results[0];
    catalogData = results[1];
  } catch (_) {}

  // Filter catalog to this wallet's assets
  var myAssets = catalogData.filter(function (item) {
    return (item.wallet || '').toLowerCase() === window.walletAddress.toLowerCase();
  });

  var isCreator = profileData && ['creator', 'platform_nft_creator'].includes(profileData.account_ty
```

```
    // Page subtitle
    var subtitle = document.getElementById('dash-subtitle');
    if (subtitle) {
      subtitle.textContent = profileData
        ? (profileData.name || 'Welcome') + ' · ' + (isCreator ? 'Creator' : 'Listener')
        : window.walletAddress.slice(0, 10) + '…';
    }

    // Creator-only panels
    if (isCreator) {
      var uploadsPanel = document.getElementById('dash-uploads-panel');
      var earningsCol  = document.getElementById('dash-earnings-col');
      var manageBtn    = document.querySelector('#dash-earnings-panel [data-requires="upload"]');
      if (uploadsPanel) uploadsPanel.style.removeProperty('display');
      if (earningsCol)  earningsCol.style.removeProperty('display');
      if (manageBtn)    manageBtn.style.display = '';
    }

    // Render all panels
    renderStats(profileData, myAssets);
    renderEarnings(myAssets);
    renderFavSummary();
    renderPlaylists(profileData);
    if (isCreator) renderUploads(myAssets);
    renderSubscription(profileData);
    renderNotices(myAssets);
  }

  // Wait for main.js capability gates before booting
  document.addEventListener('walletConnected', function () {
    setTimeout(boot, 300);
  });
  if (window.walletAddress) setTimeout(boot, 300);

})(); // end dashboard.js IIFE
```

`live_broadcast.js` (35.7 KB)

```javascript
/**
 * MSP Live Broadcast Module
 * public/Scripts/live_broadcast.js
 *
 * Handles both the BROADCASTER (creator going live) and
 * the VIEWER (watching + engaging with a live stream).
```

```
 *
 * Globals expected:
 *   window.walletAddress, window.ethersSigner (from wallets.js)
 *   window.Hls (from vendor/hls/hls.min.js)
 */

(function (global) {
  'use strict';

  // ═══════════════════════════════════════════════════════════
  //  CONSTANTS
  // ═══════════════════════════════════════════════════════════
  var CHUNK_INTERVAL_MS  = 2000;   // Send a chunk every 2 seconds
  var WS_RECONNECT_DELAY = 3000;   // WebSocket reconnect delay
  var MAX_RECONNECTS     = 5;

  var REACTIONS = ['🔥', '❤️', '👏', '🎵', '💎', '🚀'];

  // ═══════════════════════════════════════════════════════════
  //  BROADCASTER
  // ═══════════════════════════════════════════════════════════

  function LiveBroadcaster(opts) {
    /**
     * opts: {
     *   previewEl:    <video> element for camera preview
     *   statusEl:     element to write status messages
     *   statsEl:      element to display viewer count / duration / tips
     *   quality:      '720p' | '480p' | '360p' | '1080p'
     *   onSessionStart(sessionData),  ← called with { sessionId, hlsUrl }
     *   onStreamEnd(summary),
     *   onError(err),
     * }
     */
    this._opts       = opts || {};
    this._stream     = null;       // MediaStream from getUserMedia
    this._recorder   = null;       // MediaRecorder
    this._sessionId  = null;
    this._ws         = null;
    this._ping       = null;
    this._alive      = false;
    this._uploadQueue = [];
    this._uploading  = false;
    this._startTime  = null;
    this._timer      = null;
    this._reconnects = 0;
  }
```

```
LiveBroadcaster.prototype.getCamera = async function (constraints) {
  var c = constraints || {
    video: { width: { ideal: 1280 }, height: { ideal: 720 }, frameRate: { ideal: 30 } },
    audio: { echoCancellation: true, noiseSuppression: true, sampleRate: 44100 },
  };
  this._stream = await navigator.mediaDevices.getUserMedia(c);
  if (this._opts.previewEl) {
    this._opts.previewEl.srcObject = this._stream;
    this._opts.previewEl.muted     = true;
    this._opts.previewEl.play().catch(function () {});
  }
  return this._stream;
};

LiveBroadcaster.prototype.getScreenShare = async function () {
  this._stream = await navigator.mediaDevices.getDisplayMedia({
    video: true,
    audio: { echoCancellation: true },
  });
  if (this._opts.previewEl) {
    this._opts.previewEl.srcObject = this._stream;
    this._opts.previewEl.muted     = true;
    this._opts.previewEl.play().catch(function () {});
  }
  return this._stream;
};

// ── RTMP stream key (for OBS / mobile apps) ──────────────────────────
/**
 * Fetch or display the creator's RTMP stream key.
 * Returns { stream_key, rtmp_url, rtmp_server, instructions }
 */
LiveBroadcaster.prototype.getStreamKey = async function () {
  var wallet = global.walletAddress;
  if (!wallet) throw new Error('Connect your wallet first.');
  var res = await fetch('/api/stream-key/' + wallet);
  if (!res.ok) {
    var err = await res.json().catch(function () { return {}; });
    throw new Error(err.error || 'Failed to get stream key');
  }
  return res.json();
};

LiveBroadcaster.prototype.regenerateStreamKey = async function () {
  var wallet = global.walletAddress;
  if (!wallet) throw new Error('Connect your wallet first.');
```

```
  if (!confirm('Generate a new stream key? Your old key will stop working immediately.')) return nul
  var res = await fetch('/api/stream-key/regenerate', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ wallet: wallet }),
  });
  if (!res.ok) throw new Error('Failed to regenerate key');
  return res.json();
};

/**
 * Render the RTMP stream key panel into a container element.
 * Shows key, server URL, copy buttons, and OBS/mobile instructions.
 */
LiveBroadcaster.prototype.renderStreamKeyPanel = async function (containerId) {
  var container = document.getElementById(containerId);
  if (!container) return;

  container.innerHTML = '<p style="color:var(--text-3);font-family:var(--font-m);font-size:11px;">Lo

  var data;
  try {
    data = await this.getStreamKey();
  } catch (e) {
    container.innerHTML = '<p style="color:var(--ember);font-size:12px;">' + _esc(e.message) + '</p>
    return;
  }

  container.innerHTML =
    '<div class="rtmp-panel">' +
      '<div class="rtmp-eyebrow">External Streaming (OBS / Mobile Apps)</div>' +
      '<div class="rtmp-row">' +
        '<label class="rtmp-label">RTMP Server</label>' +
        '<div class="rtmp-value-row">' +
          '<code class="rtmp-code" id="rtmp-server-val">' + _esc(data.rtmp_server) + '</code>' +
          '<button class="rtmp-copy-btn" data-copy="rtmp-server-val">Copy</button>' +
        '</div>' +
      '</div>' +
      '<div class="rtmp-row">' +
        '<label class="rtmp-label">Stream Key <span class="rtmp-private">— keep private</span></labe
        '<div class="rtmp-value-row">' +
          '<code class="rtmp-code rtmp-key-masked" id="rtmp-key-val">' + _esc(data.stream_key) + '</
          '<button class="rtmp-reveal-btn" id="rtmp-reveal">Show</button>' +
          '<button class="rtmp-copy-btn" data-copy="rtmp-key-val">Copy</button>' +
        '</div>' +
      '</div>' +
      '<details class="rtmp-instructions">' +
        '<summary>Setup instructions</summary>' +
```

```
      '<div class="rtmp-inst-body">' +
        '<strong>OBS Studio</strong><br>' +
        'Settings → Stream → Service: Custom RTMP<br>' +
        'Server: <code>' + _esc(data.rtmp_server) + '</code><br>' +
        'Stream Key: <em>your key above</em><br><br>' +
        '<strong>Larix Broadcaster / Streamlabs Mobile</strong><br>' +
        'Connections → Add → URL: <code>' + _esc(data.rtmp_url) + '</code>' +
      '</div>' +
    '</details>' +
    '<button class="rtmp-regen-btn" id="rtmp-regen">↺ Regenerate Key</button>' +
  '</div>';

// Copy buttons
container.querySelectorAll('.rtmp-copy-btn').forEach(function (btn) {
  btn.addEventListener('click', function () {
    var src = document.getElementById(btn.dataset.copy);
    if (!src) return;
    navigator.clipboard.writeText(src.textContent).then(function () {
      btn.textContent = '✓ Copied';
      setTimeout(function () { btn.textContent = 'Copy'; }, 2000);
    });
  });
});

// Reveal/hide key
var revealBtn = document.getElementById('rtmp-reveal');
var keyEl     = document.getElementById('rtmp-key-val');
if (revealBtn && keyEl) {
  revealBtn.addEventListener('click', function () {
    keyEl.classList.toggle('rtmp-key-masked');
    revealBtn.textContent = keyEl.classList.contains('rtmp-key-masked') ? 'Show' : 'Hide';
  });
}

// Regenerate
var regenBtn = document.getElementById('rtmp-regen');
if (regenBtn) {
  var self2 = this;
  regenBtn.addEventListener('click', async function () {
    regenBtn.disabled = true;
    try {
      await self2.regenerateStreamKey();
      self2.renderStreamKeyPanel(containerId);
    } catch (e) {
      alert(e.message);
      regenBtn.disabled = false;
    }
```

```
    });
  }
};

LiveBroadcaster.prototype.startSession = async function (title, artistName) {
  if (!this._stream) throw new Error('Call getCamera() first');
  var wallet = global.walletAddress;
  if (!wallet) throw new Error('Wallet not connected');

  this._setStatus('Starting session…');

  var res = await fetch('/api/live-start', {
    method:  'POST',
    headers: { 'Content-Type': 'application/json' },
    body:    JSON.stringify({
      wallet, title, artistName,
      quality: this._opts.quality || '720p',
    }),
  });
  if (!res.ok) {
    var err = await res.json().catch(function () { return {}; });
    throw new Error(err.error || 'Failed to start session');
  }
  var data = await res.json();
  this._sessionId = data.sessionId;
  this._alive     = true;
  this._startTime = Date.now();

  if (typeof this._opts.onSessionStart === 'function') {
    this._opts.onSessionStart(data);
  }

  this._startRecorder();
  this._startWebSocket();
  this._startTimer();
  this._setStatus('🔴 LIVE');

  return data;
};

LiveBroadcaster.prototype._startRecorder = function () {
  var self = this;

  // Pick best supported codec
  var mimeTypes = [
    'video/webm;codecs=h264,opus',
    'video/webm;codecs=vp8,opus',
```

```
    'video/webm;codecs=vp9,opus',
    'video/webm',
  ];
  var mimeType = mimeTypes.find(function (t) { return MediaRecorder.isTypeSupported(t); });
  if (!mimeType) throw new Error('No supported video codec found in this browser');

  this._recorder = new MediaRecorder(this._stream, {
    mimeType:       mimeType,
    videoBitsPerSecond: 2500000,
    audioBitsPerSecond: 128000,
  });

  this._recorder.ondataavailable = function (e) {
    if (!e.data || e.data.size < 100) return;
    self._uploadQueue.push(e.data);
    self._drainQueue();
  };

  this._recorder.onerror = function (e) {
    self._setStatus('⚠ Recorder error: ' + e.error);
    if (typeof self._opts.onError === 'function') self._opts.onError(e.error);
  };

  this._recorder.onstop = function () {
    // Drain any remaining chunks
    self._drainQueue();
  };

  this._recorder.start(CHUNK_INTERVAL_MS);
};

LiveBroadcaster.prototype._drainQueue = async function () {
  if (this._uploading || !this._uploadQueue.length) return;
  this._uploading = true;
  while (this._uploadQueue.length && this._alive) {
    var blob    = this._uploadQueue.shift();
    var attempt = 0;
    var sent    = false;
    while (attempt < 3 && !sent) {
      try {
        var res = await fetch('/api/live-ingest/' + this._sessionId, {
          method:  'POST',
          headers: { 'Content-Type': 'application/octet-stream' },
          body:    blob,
        });
        if (res.ok) { sent = true; }
        else {
```

```
          var data = await res.json().catch(function () { return {}; });
          if (data.status === 'ended_unexpectedly') { this._alive = false; break; }
          if (data.status === 'duration_cap_reached') {
            // Server enforced the stream duration limit — end gracefully
            this._alive = false;
            this._setStatus('⏱ Stream limit reached — ending stream…');
            if (data.error) alert(data.error);
            // Trigger the normal end-stream flow
            setTimeout(function () {
              if (typeof self.endStream === 'function') self.endStream();
            }, 1000);
            break;
          }
          attempt++;
        }
      } catch (e) {
        attempt++;
        await _sleep(500 * attempt);
      }
    }
    if (!sent) this._setStatus('⚠ Upload issue — retrying…');
  }
  this._uploading = false;
};

LiveBroadcaster.prototype._startWebSocket = function () {
  var self    = this;
  var proto   = location.protocol === 'https:' ? 'wss' : 'ws';
  var url     = proto + '://' + location.host + '/ws';

  function connect() {
    self._ws = new WebSocket(url);

    self._ws.onopen = function () {
      self._reconnects = 0;
      // Broadcaster joins as "host" — no chat history needed
      self._ws.send(JSON.stringify({
        type:      'join_session',
        sessionId: self._sessionId,
        wallet:    global.walletAddress,
        name:      'HOST',
      }));
      // Keepalive ping every 15s
      self._ping = setInterval(function () {
        if (self._ws.readyState === WebSocket.OPEN) {
          self._ws.send(JSON.stringify({ type: 'ping' }));
        }
```

```
      }, 15000);
    };

    self._ws.onmessage = function (e) {
      var msg;
      try { msg = JSON.parse(e.data); } catch { return; }
      if (msg.type === 'tip_alert' && self._opts.statsEl) {
        _appendToStats(self._opts.statsEl, '💸 Tip: ' + msg.amountEth + ' ETH from ' + msg.name);
      }
      if (msg.type === 'viewer_count' && self._opts.statsEl) {
        var vEl = self._opts.statsEl.querySelector('.viewer-count');
        if (vEl) vEl.textContent = msg.viewerCount + ' watching';
      }
    };

    self._ws.onclose = function () {
      clearInterval(self._ping);
      if (self._alive && self._reconnects < MAX_RECONNECTS) {
        self._reconnects++;
        setTimeout(connect, WS_RECONNECT_DELAY);
      }
    };

    self._ws.onerror = function () { self._ws.close(); };
  }
  connect();
};

LiveBroadcaster.prototype._startTimer = function () {
  var self    = this;
  var elapsed = self._opts.statsEl && self._opts.statsEl.querySelector('.duration');
  if (!elapsed) return;
  this._timer = setInterval(function () {
    if (!self._startTime) return;
    var s = Math.floor((Date.now() - self._startTime) / 1000);
    var h = Math.floor(s / 3600);
    var m = Math.floor((s % 3600) / 60);
    var sec = s % 60;
    elapsed.textContent = (h ? _pad(h) + ':' : '') + _pad(m) + ':' + _pad(sec);
  }, 1000);
};

LiveBroadcaster.prototype.toggleMute = function () {
  if (!this._stream) return false;
  var audio = this._stream.getAudioTracks()[0];
  if (!audio) return false;
  audio.enabled = !audio.enabled;
```

```
  return !audio.enabled; // returns true if NOW muted
};

LiveBroadcaster.prototype.toggleCamera = function () {
  if (!this._stream) return false;
  var video = this._stream.getVideoTracks()[0];
  if (!video) return false;
  video.enabled = !video.enabled;
  return !video.enabled; // returns true if NOW off
};

LiveBroadcaster.prototype.endStream = async function () {
  this._alive = false;

  // Stop recorder — flushes final chunk
  if (this._recorder && this._recorder.state !== 'inactive') {
    this._recorder.stop();
  }
  // Drain remaining chunks
  await _sleep(500);
  await this._drainQueue();

  // Notify server
  var summary = null;
  if (this._sessionId && global.walletAddress) {
    var res = await fetch('/api/live-end/' + this._sessionId, {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ wallet: global.walletAddress }),
    }).catch(function () { return null; });
    if (res && res.ok) summary = await res.json();
  }

  // Cleanup
  clearInterval(this._timer);
  clearInterval(this._ping);
  if (this._ws && this._ws.readyState === WebSocket.OPEN) this._ws.close();
  if (this._stream) this._stream.getTracks().forEach(function (t) { t.stop(); });
  if (this._opts.previewEl) this._opts.previewEl.srcObject = null;

  this._setStatus('Stream ended');
  if (typeof this._opts.onStreamEnd === 'function') {
    this._opts.onStreamEnd(summary || {
      sessionId:   this._sessionId,
      duration:    Math.floor((Date.now() - this._startTime) / 1000),
      chunkCount:  0,
      tipsTotal:   0,
```

```
      peakViewers: 0,
    });
  }
  return summary;
};

LiveBroadcaster.prototype._setStatus = function (msg) {
  if (this._opts.statusEl) this._opts.statusEl.textContent = msg;
  console.log('[LiveBroadcaster]', msg);
};

// ═══════════════════════════════════════════════════════════
//  VIEWER
// ═══════════════════════════════════════════════════════════

function LiveViewer(opts) {
  /**
   * opts: {
   *   videoEl:      <video> for HLS playback
   *   chatEl:       <div> to append chat messages
   *   reactionsEl:  <div> for floating emoji reactions
   *   statsEl:      <div> for viewer count / duration / tips
   *   sessionId:    (optional) auto-join this session
   *   onEnded(),    ← called when stream ends
   * }
   */
  this._opts      = opts || {};
  this._ws        = null;
  this._hls       = null;
  this._sessionId = opts.sessionId || null;
  this._reconnects = 0;
  this._alive     = false;
}

LiveViewer.prototype.join = function (sessionId) {
  this._sessionId = sessionId || this._sessionId;
  if (!this._sessionId) throw new Error('sessionId required');
  this._alive = true;
  this._connectWs();
};

LiveViewer.prototype._connectWs = function () {
  var self  = this;
  var proto = location.protocol === 'https:' ? 'wss' : 'ws';
  var url   = proto + '://' + location.host + '/ws';

  function connect() {
```

```
    self._ws = new WebSocket(url);

    self._ws.onopen = function () {
      self._reconnects = 0;
      self._ws.send(JSON.stringify({
        type:      'join_session',
        sessionId: self._sessionId,
        wallet:    global.walletAddress || null,
        name:      (global.mspProfile && global.mspProfile.name) || 'Listener',
      }));
    };

    self._ws.onmessage = function (e) {
      var msg;
      try { msg = JSON.parse(e.data); } catch { return; }
      self._handleMessage(msg);
    };

    self._ws.onclose = function () {
      if (self._alive && self._reconnects < MAX_RECONNECTS) {
        self._reconnects++;
        setTimeout(connect, WS_RECONNECT_DELAY * self._reconnects);
      }
    };

    self._ws.onerror = function () { self._ws.close(); };
  }
  connect();
};

LiveViewer.prototype._handleMessage = function (msg) {
  switch (msg.type) {

    case 'session_state':
      this._startHls(msg.hlsUrl);
      this._renderChatHistory(msg.chatHistory);
      this._updateStats({ viewerCount: msg.viewerCount, duration: msg.duration, tipsTotal: msg.tipsT
      break;

    case 'chat':
      this._appendChat(msg);
      break;

    case 'reaction':
      this._floatReaction(msg.emoji);
      break;
```

```
    case 'viewer_count':
      this._updateStats({ viewerCount: msg.viewerCount });
      break;

    case 'stats':
      this._updateStats(msg);
      break;

    case 'tip_alert':
      this._appendChat({ name: '💸 ' + msg.name, text: 'sent ' + msg.amountEth.toFixed(4) + ' ETH!',
      this._updateStats({ tipsTotal: msg.tipsTotal });
      this._floatReaction('💎');
      break;

    case 'stream_ended':
      this._alive = false;
      this._appendChat({ name: 'MSP', text: '🎬 Stream ended.', system: true });
      if (this._hls) { this._hls.stopLoad(); }
      if (typeof this._opts.onEnded === 'function') this._opts.onEnded(msg);
      break;
  }
};

LiveViewer.prototype._startHls = function (hlsUrl) {
  var videoEl = this._opts.videoEl;
  if (!videoEl) return;

  if (global.Hls && global.Hls.isSupported()) {
    if (this._hls) { this._hls.destroy(); }
    this._hls = new global.Hls({ lowLatencyMode: true, liveSyncDurationCount: 2 });
    this._hls.loadSource(hlsUrl);
    this._hls.attachMedia(videoEl);
    this._hls.on(global.Hls.Events.MANIFEST_PARSED, function () { videoEl.play().catch(function () {
  } else if (videoEl.canPlayType('application/vnd.apple.mpegurl')) {
    videoEl.src = hlsUrl;
    videoEl.play().catch(function () {});
  }
};

LiveViewer.prototype._renderChatHistory = function (history) {
  if (!this._opts.chatEl || !history) return;
  var self = this;
  history.forEach(function (m) { self._appendChat(m); });
};

LiveViewer.prototype._appendChat = function (msg) {
  var el = this._opts.chatEl;
```

```
  if (!el) return;
  var row   = document.createElement('div');
  row.className = 'chat-row' + (msg.tip ? ' chat-tip' : '') + (msg.system ? ' chat-system' : '');
  var name  = document.createElement('span');
  name.className   = 'chat-name';
  name.textContent = msg.name + ' ';
  var text  = document.createElement('span');
  text.className   = 'chat-text';
  text.textContent = msg.text;
  row.appendChild(name);
  row.appendChild(text);
  el.appendChild(row);
  el.scrollTop = el.scrollHeight;
  // Keep max 100 messages
  while (el.children.length > 100) el.removeChild(el.firstChild);
};

LiveViewer.prototype._floatReaction = function (emoji) {
  var el = this._opts.reactionsEl;
  if (!el) return;
  var span      = document.createElement('span');
  span.className    = 'floating-reaction';
  span.textContent  = emoji;
  // Random horizontal position
  span.style.left   = (10 + Math.random() * 80) + '%';
  el.appendChild(span);
  setTimeout(function () { if (span.parentNode) span.parentNode.removeChild(span); }, 2500);
};

LiveViewer.prototype._updateStats = function (data) {
  var el = this._opts.statsEl;
  if (!el) return;
  if (data.viewerCount != null) {
    var vc = el.querySelector('.viewer-count');
    if (vc) vc.textContent = data.viewerCount + ' watching';
  }
  if (data.duration != null) {
    var d  = el.querySelector('.duration');
    if (d) {
      var s = data.duration;
      var h = Math.floor(s / 3600);
      var m = Math.floor((s % 3600) / 60);
      var sec = s % 60;
      d.textContent = (h ? _pad(h) + ':' : '') + _pad(m) + ':' + _pad(sec);
    }
  }
  if (data.tipsTotal != null) {
```

```
    var tt = el.querySelector('.tips-total');
    if (tt) tt.textContent = parseFloat(data.tipsTotal).toFixed(4) + ' ETH tipped';
  }
};

LiveViewer.prototype.sendChat = function (text) {
  if (!this._ws || this._ws.readyState !== WebSocket.OPEN) return;
  this._ws.send(JSON.stringify({ type: 'chat', text: text }));
};

LiveViewer.prototype.sendReaction = function (emoji) {
  if (!this._ws || this._ws.readyState !== WebSocket.OPEN) return;
  this._ws.send(JSON.stringify({ type: 'reaction', emoji: emoji }));
  this._floatReaction(emoji); // Optimistic local display
};

LiveViewer.prototype.sendTipAlert = function (amountEth) {
  if (!this._ws || this._ws.readyState !== WebSocket.OPEN) return;
  this._ws.send(JSON.stringify({ type: 'tip_alert', amountEth: amountEth }));
};

LiveViewer.prototype.leave = function () {
  this._alive = false;
  if (this._ws) {
    this._ws.send(JSON.stringify({ type: 'leave_session' }));
    this._ws.close();
  }
  if (this._hls) this._hls.destroy();
};

// ═══════════════════════════════════════════════════════════
//  POST-STREAM MODAL
// ═══════════════════════════════════════════════════════════

function PostStreamModal(summary, opts) {
  /**
   * summary: { sessionId, duration, tipsTotal, peakViewers, chunkCount }
   * opts:    { onArchive, onDiscard, onRetry }
   */
  this._summary = summary;
  this._opts    = opts || {};
  this._el      = null;
}

PostStreamModal.prototype.open = function () {
  var self  = this;
  var s     = this._summary;
```

```
var dur   = _formatDuration(s.duration || 0);

// Keep a local reference to the recorded chunks for download
// These are passed in summary.chunks if available
var recordedChunks = s.chunks || null;

var overlay = document.createElement('div');
overlay.id        = 'post-stream-overlay';
overlay.className = 'psm-overlay';
overlay.innerHTML =
  '<div class="psm-modal">' +
    '<div class="psm-header">' +
      '<div class="psm-eyebrow">Stream Ended</div>' +
      '<h2 class="psm-title">What would you like to do?</h2>' +
    '</div>' +
    '<div class="psm-stats">' +
      '<div class="psm-stat"><span class="psm-stat-val">' + dur + '</span><span class="psm-stat-lb
      '<div class="psm-stat"><span class="psm-stat-val">' + (s.peakViewers || 0) + '</span><span c
      '<div class="psm-stat"><span class="psm-stat-val">' + parseFloat(s.tipsTotal || 0).toFixed(4
    '</div>' +
    '<div class="psm-fields">' +
      '<label class="psm-label">Title</label>' +
      '<input class="psm-input" id="psm-title" type="text" placeholder="Recording title…" value="'
      '<label class="psm-label">Description</label>' +
      '<textarea class="psm-input" id="psm-desc" rows="2" placeholder="What happened in this strea
      '<label class="psm-label">Tags (comma-separated)</label>' +
      '<input class="psm-input" id="psm-tags" type="text" placeholder="live, dj, techno…">' +
    '</div>' +
    '<div class="psm-status" id="psm-status"></div>' +
    '<div class="psm-actions">' +
      '<button class="psm-btn psm-btn-primary" id="psm-mint">◈ Mint NFT + Save to Catalog</button>
      '<button class="psm-btn psm-btn-secondary" id="psm-save">Save to Catalog Only</button>' +
      '<button class="psm-btn psm-btn-download" id="psm-download" style="background:transparent;bo
      '<button class="psm-btn psm-btn-warn" id="psm-retry">↺ Retry Stream</button>' +
      '<button class="psm-btn psm-btn-ghost" id="psm-discard">✕ Discard Recording</button>' +
    '</div>' +
  '</div>';

document.body.appendChild(overlay);
this._el = overlay;

function getFields() {
  return {
    title:       (document.getElementById('psm-title')  || {}).value || s.title || '',
    description: (document.getElementById('psm-desc')   || {}).value || '',
    tags:        (document.getElementById('psm-tags')   || {}).value || '',
  };
```

```
}

function setStatus(msg, color) {
  var el = document.getElementById('psm-status');
  if (el) { el.textContent = msg; el.style.color = color || '#eeeae4'; }
}

function disableAll() {
  ['psm-mint','psm-save','psm-download','psm-retry','psm-discard'].forEach(function (id) {
    var b = document.getElementById(id); if (b) b.disabled = true;
  });
}

// Resolve wallet — use global or fall back to summary
function resolveWallet() {
  return (global.walletAddress) || s.wallet || '';
}

// ── Mint NFT + Save ──────────────────────────────────────────────────
document.getElementById('psm-mint').onclick = async function () {
  disableAll();
  setStatus('Archiving to IPFS…');
  var f = getFields();
  try {
    var res = await fetch('/api/live-archive', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({
        sessionId:   s.sessionId,
        wallet:      resolveWallet(),
        title:       f.title,
        description: f.description,
        tags:        f.tags,
        mintNft:     true,
      }),
    });
    var data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Archive failed');
    setStatus('✓ Archived! Minting NFT…', '#00d4bb');
    if (typeof self._opts.onArchive === 'function') self._opts.onArchive(data, true);
    setTimeout(function () { self.close(); }, 2000);
  } catch (e) { setStatus('Error: ' + e.message, '#e85d3a'); disableAll(); }
};

// ── Save Only ────────────────────────────────────────────────────────
document.getElementById('psm-save').onclick = async function () {
  disableAll();
```

```
  setStatus('Saving to catalog…');
  var f = getFields();
  try {
    var res = await fetch('/api/live-archive', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({
        sessionId:   s.sessionId,
        wallet:      resolveWallet(),
        title:       f.title,
        description: f.description,
        tags:        f.tags,
        mintNft:     false,
      }),
    });
    var data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Archive failed');
    setStatus('✓ Saved to catalog!', '#00d4bb');
    if (typeof self._opts.onArchive === 'function') self._opts.onArchive(data, false);
    setTimeout(function () { self.close(); }, 1500);
  } catch (e) { setStatus('Error: ' + e.message, '#e85d3a'); disableAll(); }
};

// ── Download to Device ──────────────────────────────────────────────
document.getElementById('psm-download').onclick = async function () {
  setStatus('Preparing download…');
  try {
    // Request the recorded chunks from the server
    var res = await fetch('/api/live-download/' + s.sessionId, {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ wallet: resolveWallet() }),
    });

    if (!res.ok) {
      var err = await res.json().catch(function () { return {}; });
      throw new Error(err.error || 'Download not available');
    }

    // Server streams the WebM blob back
    var blob = await res.blob();
    var filename = _esc(getFields().title || s.title || 'stream') + '_' +
      new Date().toISOString().slice(0, 10) + '.webm';

    // Trigger browser download
    var url = URL.createObjectURL(blob);
    var a   = document.createElement('a');
```

```
      a.href     = url;
      a.download = filename;
      document.body.appendChild(a);
      a.click();
      setTimeout(function () { URL.revokeObjectURL(url); a.remove(); }, 2000);

      setStatus('✓ Download started — check your Downloads folder.', '#00d4bb');
    } catch (e) { setStatus('Error: ' + e.message, '#e85d3a'); }
  };

  // ── Retry ──────────────────────────────────────────────────────────
  document.getElementById('psm-retry').onclick = function () {
    self.close();
    if (typeof self._opts.onRetry === 'function') self._opts.onRetry(s);
  };

  // ── Discard ────────────────────────────────────────────────────────
  document.getElementById('psm-discard').onclick = async function () {
    if (!confirm('Permanently delete this recording? This cannot be undone.')) return;
    disableAll();
    setStatus('Discarding…');
    await fetch('/api/live-discard', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ sessionId: s.sessionId, wallet: resolveWallet() }),
    }).catch(function () {});
    if (typeof self._opts.onDiscard === 'function') self._opts.onDiscard();
    self.close();
  };
};

PostStreamModal.prototype.close = function () {
  if (this._el && this._el.parentNode) this._el.parentNode.removeChild(this._el);
};

// ═══════════════════════════════════════════════════════════════
//  LIVE SESSIONS BROWSER
//  Renders the grid of active live sessions on listen.html
// ═══════════════════════════════════════════════════════════════

async function renderLiveSessions(containerId, onJoin) {
  var el = document.getElementById(containerId);
  if (!el) return;

  var sessions = [];
  try {
    var res = await fetch('/api/live-sessions');
```

```
    if (res.ok) sessions = await res.json();
  } catch (e) { console.debug('live-sessions fetch failed:', e.message); }

  if (!sessions.length) {
    el.innerHTML = '<p class="live-empty">No live streams right now. Check back soon.</p>';
    return;
  }

  el.innerHTML = sessions.map(function (s) {
    var dur = _formatDuration(s.duration || 0);
    return (
      '<div class="live-card" data-session="' + _esc(s.sessionId) + '">' +
        '<div class="live-thumb">' +
          '<img src="' + _esc(s.thumbnailUrl) + '" onerror="this.style.display=\'none\'">' +
          '<div class="live-badge">● LIVE</div>' +
          '<div class="live-viewers">' + (s.viewerCount || 0) + ' watching</div>' +
        '</div>' +
        '<div class="live-info">' +
          '<div class="live-title">' + _esc(s.title) + '</div>' +
          '<div class="live-artist">' + _esc(s.artistName) + '</div>' +
          '<div class="live-dur">' + dur + '</div>' +
        '</div>' +
        '<button class="live-join-btn">Join Live</button>' +
      '</div>'
    );
  }).join('');

  el.querySelectorAll('.live-join-btn').forEach(function (btn) {
    btn.addEventListener('click', function () {
      var sessionId = btn.closest('.live-card').dataset.session;
      if (typeof onJoin === 'function') onJoin(sessionId);
    });
  });
}

// ═══════════════════════════════════════════════════════════
//  UTILS
// ═══════════════════════════════════════════════════════════
function _pad(n)            { return String(n).padStart(2, '0'); }
function _sleep(ms)         { return new Promise(function (r) { setTimeout(r, ms); }); }
function _esc(s)            { return String(s || '').replace(/[&<>"']/g, function (c) { return ({'&'
function _appendToStats(el, msg) {
  var p = document.createElement('p');
  p.className = 'stat-alert';
  p.textContent = msg;
  el.appendChild(p);
  setTimeout(function () { if (p.parentNode) p.parentNode.removeChild(p); }, 5000);
```

```
  }
  function _formatDuration(s) {
    var h = Math.floor(s / 3600), m = Math.floor((s % 3600) / 60), sec = s % 60;
    return (h ? _pad(h) + ':' : '') + _pad(m) + ':' + _pad(sec);
  }

  // ═══════════════════════════════════════════════════════════
  //  EXPORTS
  // ═══════════════════════════════════════════════════════════
  global.MSPLive = {
    Broadcaster:      LiveBroadcaster,
    Viewer:           LiveViewer,
    PostStreamModal:  PostStreamModal,
    renderSessions:   renderLiveSessions,
    REACTIONS:        REACTIONS,
  };

})(window);
```

`gst_pipeline.js` (41.7 KB)

```javascript
'use strict';
/**
 * ═══════════════════════════════════════════════════════════════════════
 *  MSP GStreamer Pipeline Manager  —  gst_pipeline.js
 *  src/gst_pipeline.js
 *
 *  Two operating modes — fundamentally different pipeline architectures:
 *
 *  PRODUCTION  ("The Monster" — like the flowchart)
 *  ────────────────────────────────────────────────
 *  Single gst-launch-1.0 process, true tee-based parallel fan-out.
 *  Mirrors a professional broadcast transcoder.
 *
 *  filesrc/rtmpsrc/fdsrc
 *    → demux (qtdemux / flvdemux / matroskademux)
 *    → video decode (avdec_h264 / vp8dec)
 *    → videoconvert → videorate(30fps)
 *    → vtee ──► queue → scale 1920×1080 → [HW/SW enc] → h264parse → mux1080 → hlssink2
 *           ──► queue → scale 1280×720  → [HW/SW enc] → h264parse → mux720  → hlssink2
 *           ──► queue → scale 854×480   → [HW/SW enc] → h264parse → mux480  → hlssink2
 *           ──► queue → scale 640×360   → [HW/SW enc] → h264parse → mux360  → hlssink2
 *           ──► queue → scale 320×180   → jpegenc     → multifilesink (thumb)
 *    → audio decode (avdec_aac / vorbisdec)
 *    → audioconvert → audioresample(44100,stereo)
```

```
 *    → level (loudness analysis, peak metering)
 *    → atee ──► avenc_aac(192k) → mux1080.
 *           ──► avenc_aac(128k) → mux720.
 *           ──► avenc_aac(128k) → mux480.
 *           ──► avenc_aac(96k)  → mux360.
 *
 *  Use for: catalog assets, royalty-eligible streams, VOD transcode
 *  Archive: YES → output lands in STREAMS_ROOT/{cid}/ → served by Nginx
 *
 *  SOCIAL  ("Bare Bones" — fast, minimal, ephemeral)
 *  ────────────────────────────────────────────────
 *  Single source → decode → 480p encode → 1s HLS fragments
 *  No tee, no multi-bitrate, no analysis, no thumbnail.
 *
 *  Use for: fan interaction streams, casual DJ sets
 *  Archive: NO by default (creator can opt-in post-stream)
 *
 *  Hardware encoder priority (auto-detected at startup):
 *    1. NVIDIA NVENC  (nvh264enc)
 *    2. Intel/AMD VA-API  (vaapih264enc)
 *    3. Apple VideoToolbox  (vtenc_h264)
 *    4. Software x264enc  ← always available if gst-plugins-ugly installed
 *
 *  Every path has an FFmpeg fallback — GStreamer not required to run MSP.
 * ═══════════════════════════════════════════════════════════════════════════
 */

const { spawn }    = require('child_process');
const path         = require('path');
const fs           = require('fs-extra');
const EventEmitter = require('events');
const crypto       = require('crypto');

// ──────────────────────────────────────────────────────────────────────────
//  Config (all overridable via environment)
// ──────────────────────────────────────────────────────────────────────────

const GST_LAUNCH   = process.env.GST_LAUNCH_PATH  || 'gst-launch-1.0';
const GST_INSPECT  = process.env.GST_INSPECT_PATH || 'gst-inspect-1.0';
const FFMPEG_PATH  = process.env.FFMPEG_PATH       || 'ffmpeg';

/** Where Nginx serves catalog HLS from (VOD assets) */
const STREAMS_ROOT = process.env.STREAMS_ROOT || '/var/www/msp/streams';
/** Where Nginx serves live HLS from */
const HLS_ROOT     = process.env.HLS_ROOT     || '/var/www/msp/live';

// ──────────────────────────────────────────────────────────────────────────
```

```
//  Modes
// ────────────────────────────────────────────────────────────────────────────

const MODES = {
  PRODUCTION: 'production',   // heavy, multi-bitrate, archived, royalty-eligible
  SOCIAL:     'social',       // bare-bones, single quality, ephemeral
};

// ────────────────────────────────────────────────────────────────────────────
//  Ladders
// ────────────────────────────────────────────────────────────────────────────

/** PRODUCTION — all four video rungs. Rungs above source height are skipped. */
const PROD_VIDEO_LADDER = [
  { name: '1080p', w: 1920, h: 1080, vbr: 4500, abr: 192, seg: 10 },
  { name: '720p',  w: 1280, h: 720,  vbr: 2800, abr: 128, seg: 10 },
  { name: '480p',  w: 854,  h: 480,  vbr: 1400, abr: 128, seg: 10 },
  { name: '360p',  w: 640,  h: 360,  vbr: 700,  abr: 96,  seg: 10 },
];

/** PRODUCTION audio-only ladder (music / podcast VOD) */
const PROD_AUDIO_LADDER = [
  { name: 'hi',  bps: 320000 },
  { name: 'mid', bps: 256000 },
  { name: 'lo',  bps: 128000 },
];

/** SOCIAL — single rung, lowest latency */
const SOCIAL_RUNG = { w: 854, h: 480, vbr: 1200, abr: 128 };

// ────────────────────────────────────────────────────────────────────────────
//  Capability detection
// ────────────────────────────────────────────────────────────────────────────

async function detectCapabilities() {
  const caps = {
    gstreamer: false, gstVersion: null,
    nvenc: false, vaapi: false, videotoolbox: false, x264: false,
    hlssink2: false, rtmpsrc: false, level: false, jpegenc: false,
    matroskademux: false,
    ffmpeg: false, ffmpegVersion: null,
  };

  try {
    const out = await _cmd(GST_LAUNCH, ['--version']);
    caps.gstreamer = true;
    caps.gstVersion = (out.match(/GStreamer\s+([\d.]+)/) || [])[1] || 'unknown';
```

```
  } catch (_) {}

  if (caps.gstreamer) {
    await Promise.all([
      ['nvh264enc',    'nvenc'],
      ['vaapih264enc', 'vaapi'],
      ['vtenc_h264',   'videotoolbox'],
      ['x264enc',      'x264'],
      ['hlssink2',     'hlssink2'],
      ['rtmpsrc',      'rtmpsrc'],
      ['level',        'level'],
      ['jpegenc',      'jpegenc'],
      ['matroskademux','matroskademux'],
    ].map(async ([el, key]) => {
      try { await _cmd(GST_INSPECT, [el]); caps[key] = true; } catch (_) {}
    }));
  }

  try {
    const out = await _cmd(FFMPEG_PATH, ['-version']);
    caps.ffmpeg = true;
    caps.ffmpegVersion = (out.match(/ffmpeg version (\S+)/) || [])[1] || 'unknown';
  } catch (_) {}

  return caps;
}

/**
 * Returns an object with:
 *   el     — GStreamer element name
 *   hw     — hardware backend label
 *   enc(kbps) — function that returns the full encoder element string
 */
function pickVideoEncoder(caps) {
  if (caps.nvenc)        return {
    el: 'nvh264enc', hw: 'nvidia',
    enc: kbps => `nvh264enc bitrate=${kbps} rc-mode=cbr preset=low-latency ! h264parse`,
  };
  if (caps.vaapi)        return {
    el: 'vaapih264enc', hw: 'vaapi',
    enc: kbps => `vaapivpp ! vaapih264enc rate-control=cbr bitrate=${kbps} ! h264parse`,
  };
  if (caps.videotoolbox) return {
    el: 'vtenc_h264', hw: 'videotoolbox',
    enc: kbps => `vtenc_h264 bitrate=${kbps} realtime=true ! h264parse`,
  };
  return {
```

```
    el: 'x264enc', hw: 'software',
    enc: kbps => `x264enc bitrate=${kbps} speed-preset=fast tune=film key-int-max=60 ! h264parse`,
  };
}

function pickSocialEncoder(caps) {
  // Social always prefers low-latency over quality
  if (caps.nvenc)
    return kbps => `nvh264enc bitrate=${kbps} rc-mode=cbr preset=low-latency ! h264parse`;
  if (caps.vaapi)
    return kbps => `vaapivpp ! vaapih264enc rate-control=cbr bitrate=${kbps} ! h264parse`;
  return kbps => `x264enc bitrate=${kbps} speed-preset=ultrafast tune=zerolatency key-int-max=30 ! h26
}

// ────────────────────────────────────────────────────────────────────────────
//  GstPipeline
// ────────────────────────────────────────────────────────────────────────────

class GstPipeline extends EventEmitter {
  /**
   * @param {object} opts
   * @param {string}  opts.id          Content or session ID (used in logs)
   * @param {string}  opts.mode        MODES.PRODUCTION | MODES.SOCIAL
   * @param {string}  opts.hlsDir      Output directory for HLS segments
   * @param {object}  opts.caps        From detectCapabilities()
   * @param {object}  opts.logger      Pino or console logger
   */
  constructor(opts) {
    super();
    this.id       = opts.id || opts.sessionId || crypto.randomUUID();
    this.mode     = opts.mode   || MODES.PRODUCTION;
    this.hlsDir   = opts.hlsDir || path.join(STREAMS_ROOT, this.id);
    this.caps     = opts.caps   || {};
    this.logger   = opts.logger || { info: console.log, warn: console.warn, error: console.error, debu
    this._procs   = [];
    this._stopped = false;
    this._healthT = null;
  }

  // ═══════════════════════════════════════════════════════════════════════════
  //  PRIMARY API
  // ═══════════════════════════════════════════════════════════════════════════

  /**
   * VOD transcode  (file on disk → HLS)
   * Runs to completion, resolves when done.
   */
```

```
async transcodeFile(opts) {
  const { inputPath, contentType = 'music', sourceHeight = 0 } = opts;
  await fs.ensureDir(this.hlsDir);

  const isAudio = contentType === 'music' || contentType === 'podcast';
  this.logger.info({ id: this.id, mode: this.mode, contentType }, 'transcodeFile start');

  // PRODUCTION mode -> full tee-based pipeline
  if (this.mode === MODES.PRODUCTION) {
    if (isAudio) {
      return this.caps.gstreamer
        ? this._gstAudioProduction(inputPath)
        : this._ffmpegAudioProduction(inputPath);
    } else {
      const ladder = PROD_VIDEO_LADDER.filter(r => r.h <= (sourceHeight || 720) + 120);
      if (!ladder.length) ladder.push(PROD_VIDEO_LADDER[1]);
      return this.caps.gstreamer
        ? this._gstVideoProduction(inputPath, ladder)
        : this._ffmpegVideoProduction(inputPath, ladder);
    }
  }

  // SOCIAL mode -> single quality, fast
  if (isAudio) {
    return this.caps.gstreamer
      ? this._gstAudioSocial(inputPath)
      : this._ffmpegAudioSocial(inputPath);
  } else {
    return this.caps.gstreamer
      ? this._gstVideoSocial(inputPath)
      : this._ffmpegVideoSocial(inputPath);
  }
}

/**
 * Live RTMP ingest  (OBS / Larix / hardware encoder → HLS)
 * Returns the spawned child process(es).
 */
startRtmpLive(opts) {
  const { rtmpUrl, audioOnly = false, qualities = ['720p', '480p'] } = opts;
  if (!fs.existsSync(this.hlsDir)) fs.ensureDirSync(this.hlsDir);

  this.logger.info({ id: this.id, mode: this.mode, rtmpUrl, audioOnly }, 'startRtmpLive');

  if (this.mode === MODES.SOCIAL) {
    return this.caps.gstreamer && this.caps.rtmpsrc
      ? this._gstRtmpSocial({ rtmpUrl, audioOnly })
```

```
      : this._ffmpegRtmpSocial({ rtmpUrl, audioOnly });
  }

  const ladder = audioOnly ? [] : PROD_VIDEO_LADDER.filter(r => qualities.includes(r.name));
  if (!ladder.length && !audioOnly) ladder.push(PROD_VIDEO_LADDER[1]);

  return this.caps.gstreamer && this.caps.rtmpsrc
    ? this._gstRtmpProduction({ rtmpUrl, audioOnly, ladder })
    : this._ffmpegRtmpProduction({ rtmpUrl, audioOnly, ladder });
}

/**
 * Live browser pipe  (MediaRecorder WebM chunks → stdin → HLS)
 */
startBrowserLive(opts) {
  const { passThrough, audioOnly = false, qualities = ['720p', '480p'] } = opts;
  if (!fs.existsSync(this.hlsDir)) fs.ensureDirSync(this.hlsDir);

  this.logger.info({ id: this.id, mode: this.mode, audioOnly }, 'startBrowserLive');

  if (this.mode === MODES.SOCIAL) {
    return this.caps.gstreamer
      ? this._gstBrowserSocial({ passThrough, audioOnly })
      : this._ffmpegBrowserSocial({ passThrough, audioOnly });
  }

  const ladder = audioOnly ? [] : PROD_VIDEO_LADDER.filter(r => qualities.includes(r.name));
  if (!ladder.length && !audioOnly) ladder.push(PROD_VIDEO_LADDER[1]);

  return this.caps.gstreamer
    ? this._gstBrowserProduction({ passThrough, audioOnly, ladder })
    : this._ffmpegBrowserProduction({ passThrough, audioOnly });
}

// ═══════════════════════════════════════════════════════════════════════
//  PRODUCTION GST — audio VOD
//
//  filesrc → decodebin → audioconvert → audioresample → level → atee
//    atee → avenc_aac(320k) → hlssink2(hi)
//    atee → avenc_aac(256k) → hlssink2(mid)
//    atee → avenc_aac(128k) → hlssink2(lo)
// ═══════════════════════════════════════════════════════════════════════

async _gstAudioProduction(inputPath) {
  const { hlsDir } = this;
  const levelPart = this.caps.level
    ? `! level name=lvl message=true interval=500000000 peak-ttl=300000000 `
```

```
    : '';

  const lines = [
    `filesrc location="${inputPath}"`,
    `! decodebin name=dec`,
    `dec.`,
    `! queue max-size-time=0 max-size-bytes=0`,
    `! audioconvert`,
    `! audioresample`,
    `! audio/x-raw,rate=44100,channels=2`,
    levelPart,
    `! tee name=atee`,
  ];

  for (const r of PROD_AUDIO_LADDER) {
    lines.push(
      `atee.`,
      `! queue max-size-time=0 max-size-bytes=0`,
      `! avenc_aac bitrate=${r.bps} compliance=-2`,
      `! aacparse`,
      `! hlssink2`,
      `    location="${hlsDir}/${r.name}_%05d.ts"`,
      `    playlist-location="${hlsDir}/${r.name}.m3u8"`,
      `    target-duration=10 max-files=0`,
    );
  }

  await this._gstRun(lines.join(' '));
  await this._writeAudioMaster();
  this.logger.info({ id: this.id }, 'Audio PRODUCTION complete');
}

// ═══════════════════════════════════════════════════════════════════════
//  PRODUCTION GST — video VOD  (THE MONSTER)
//
//  filesrc → demux → video decode → videoconvert → videorate(30fps) → vtee
//    vtee → scale(1080p) → enc → h264parse → mux1080 → hlssink2
//    vtee → scale(720p)  → enc → h264parse → mux720  → hlssink2
//    vtee → scale(480p)  → enc → h264parse → mux480  → hlssink2
//    vtee → scale(360p)  → enc → h264parse → mux360  → hlssink2
//    vtee → scale(320x180) → jpegenc → multifilesink (thumbnail every ~30s)
//
//    demux → audio decode → audioconvert → audioresample → level → atee
//    atee → avenc_aac(192k) → mux1080.
//    atee → avenc_aac(128k) → mux720.
//    atee → avenc_aac(128k) → mux480.
//    atee → avenc_aac(96k)  → mux360.
```

```
// ═══════════════════════════════════════════════════════════════════════

async _gstVideoProduction(inputPath, ladder) {
  const { hlsDir } = this;
  const enc        = pickVideoEncoder(this.caps);
  const levelPart  = this.caps.level
    ? `! level name=lvl message=true interval=500000000 `
    : '';
  const ext   = path.extname(inputPath).toLowerCase();
  const demux = ext === '.mp4' || ext === '.m4v'  ? 'qtdemux name=dmx'
              : ext === '.mkv' || ext === '.webm' ? 'matroskademux name=dmx'
              : ext === '.flv'                    ? 'flvdemux name=dmx'
              : 'qtdemux name=dmx';

  const lines = [];

  // ── Source + demux ──────────────────────────────────────────────────
  lines.push(
    `filesrc location="${inputPath}"`,
    `! ${demux}`,
  );

  // ── Video decode → vtee ─────────────────────────────────────────────
  lines.push(
    `dmx.`,
    `! queue max-size-bytes=0 max-size-time=0`,
    `! h264parse ! avdec_h264`,
    `! videoconvert`,
    `! videorate`,
    `! video/x-raw,framerate=30/1`,
    `! tee name=vtee`,
  );

  // ── One video branch per rung ───────────────────────────────────────
  for (const r of ladder) {
    lines.push(
      `vtee.`,
      `! queue max-size-bytes=0 max-size-time=0 leaky=downstream`,
      `! videoscale method=bilinear add-borders=true`,
      `! video/x-raw,width=${r.w},height=${r.h}`,
      `! ${enc.enc(r.vbr)}`,
      `! mpegtsmux name=mux${r.name}`,
      `! hlssink2`,
      `    location="${hlsDir}/${r.name}_%05d.ts"`,
      `    playlist-location="${hlsDir}/${r.name}.m3u8"`,
      `    target-duration=${r.seg} max-files=0`,
    );
```

```
  }

  // ── Thumbnail branch (JPEG) ──────────────────────────────────────────
  if (this.caps.jpegenc) {
    lines.push(
      `vtee.`,
      `! queue leaky=downstream`,
      `! videoscale ! video/x-raw,width=320,height=180`,
      `! jpegenc quality=82`,
      `! multifilesink location="${hlsDir}/thumb_%05d.jpg" max-files=3`,
    );
  }

  // ── Audio decode → atee ──────────────────────────────────────────────
  lines.push(
    `dmx.`,
    `! queue max-size-bytes=0 max-size-time=0`,
    `! aacparse ! avdec_aac`,
    `! audioconvert ! audioresample`,
    `! audio/x-raw,rate=44100,channels=2`,
    levelPart,
    `! tee name=atee`,
  );

  // ── One audio branch per rung → feed into corresponding video mux ───────
  const abrMap = { '1080p': 196608, '720p': 131072, '480p': 131072, '360p': 98304 };
  for (const r of ladder) {
    lines.push(
      `atee.`,
      `! queue max-size-time=0`,
      `! avenc_aac bitrate=${abrMap[r.name] || 131072} compliance=-2`,
      `! aacparse`,
      `! mux${r.name}.`,
    );
  }

  const pipeline = lines.join(' ');
  this.logger.info(
    { id: this.id, encoder: enc.hw, rungs: ladder.map(r => r.name) },
    `GST VIDEO PRODUCTION: ${pipeline.slice(0, 160)}…`,
  );

  await this._gstRun(pipeline);
  await this._writeVideoMaster(ladder);
  await this._resolveThumb();
  this.logger.info({ id: this.id }, 'Video PRODUCTION complete');
}
```

```
// ═══════════════════════════════════════════════════════════════════════
//  SOCIAL GST — audio and video VOD
// ═══════════════════════════════════════════════════════════════════════

async _gstAudioSocial(inputPath) {
  const { hlsDir } = this;
  const pl = [
    `filesrc location="${inputPath}"`,
    `! decodebin`,
    `! audioconvert ! audioresample`,
    `! audio/x-raw,rate=44100,channels=2`,
    `! avenc_aac bitrate=131072 compliance=-2 ! aacparse`,
    `! hlssink2 location="${hlsDir}/stream_%05d.ts"`,
    `           playlist-location="${hlsDir}/stream.m3u8"`,
    `           target-duration=10 max-files=0`,
  ].join(' ');
  await this._gstRun(pl);
  await fs.writeFile(path.join(hlsDir, 'master.m3u8'),
    '#EXTM3U\n#EXT-X-VERSION:3\n#EXT-X-STREAM-INF:BANDWIDTH=131000,CODECS="mp4a.40.2"\nstream.m3u8\n
}

async _gstVideoSocial(inputPath) {
  const { hlsDir } = this;
  const encFn = pickSocialEncoder(this.caps);
  const ext   = path.extname(inputPath).toLowerCase();
  const demux = ext === '.mp4' ? 'qtdemux name=d' : 'matroskademux name=d';
  const pl = [
    `filesrc location="${inputPath}" ! ${demux}`,
    `d. ! queue ! h264parse ! avdec_h264 ! videoconvert`,
    `! videoscale ! video/x-raw,width=${SOCIAL_RUNG.w},height=${SOCIAL_RUNG.h}`,
    `! ${encFn(SOCIAL_RUNG.vbr)}`,
    `! mpegtsmux name=mux`,
    `! hlssink2 location="${hlsDir}/stream_%05d.ts"`,
    `           playlist-location="${hlsDir}/stream.m3u8"`,
    `           target-duration=10 max-files=0`,
    `d. ! queue ! aacparse ! avdec_aac ! audioconvert ! audioresample`,
    `! avenc_aac bitrate=131072 compliance=-2 ! aacparse ! mux.`,
  ].join(' ');
  await this._gstRun(pl);
  await fs.writeFile(path.join(hlsDir, 'master.m3u8'),
    `#EXTM3U\n#EXT-X-VERSION:3\n#EXT-X-STREAM-INF:BANDWIDTH=1350000,RESOLUTION=${SOCIAL_RUNG.w}x${SO
}

// ═══════════════════════════════════════════════════════════════════════
//  PRODUCTION GST LIVE  —  RTMP
//  Same tee architecture as VOD but uses rtmpsrc and 2s HLS fragments
```

```
// ═══════════════════════════════════════════════════════════════════

_gstRtmpProduction({ rtmpUrl, audioOnly, ladder }) {
  const { hlsDir } = this;

  if (audioOnly) {
    const pl = [
      `rtmpsrc location="${rtmpUrl} live=1"`,
      `! flvdemux name=d`,
      `d. ! queue ! aacparse ! avdec_aac ! audioconvert ! audioresample`,
      `! audio/x-raw,rate=44100,channels=2 ! tee name=atee`,
      ...PROD_AUDIO_LADDER.flatMap(r => [
        `atee. ! queue ! avenc_aac bitrate=${r.bps} compliance=-2 ! aacparse`,
        `! hlssink2 location="${hlsDir}/${r.name}_%05d.ts"`,
        `           playlist-location="${hlsDir}/${r.name}.m3u8"`,
        `           target-duration=2 max-files=16`,
      ]),
    ].join(' ');
    const proc = this._gstLive(pl, 'rtmp-prod-audio');
    this._startHealthWatch();
    return proc;
  }

  const enc   = pickVideoEncoder(this.caps);
  const lines = [
    `rtmpsrc location="${rtmpUrl} live=1"`,
    `! flvdemux name=d`,
    `d. ! queue max-size-time=2000000000 max-size-bytes=0`,
    `! h264parse ! avdec_h264 ! videoconvert ! videorate`,
    `! video/x-raw,framerate=30/1 ! tee name=vtee`,
    `d. ! queue max-size-time=2000000000`,
    `! aacparse ! avdec_aac ! audioconvert ! audioresample`,
    `! audio/x-raw,rate=44100,channels=2 ! tee name=atee`,
  ];

  for (const r of ladder) {
    lines.push(
      `vtee. ! queue max-size-time=0 leaky=downstream`,
      `! videoscale ! video/x-raw,width=${r.w},height=${r.h}`,
      `! ${enc.enc(r.vbr)} ! mpegtsmux name=mux${r.name}`,
      `! hlssink2 location="${hlsDir}/${r.name}_%05d.ts"`,
      `           playlist-location="${hlsDir}/${r.name}.m3u8"`,
      `           target-duration=2 max-files=16`,
      `atee. ! queue ! avenc_aac bitrate=${r.abr * 1000} compliance=-2 ! aacparse ! mux${r.name}.`,
    );
  }
```

```
  this._writeVideoMasterSync(hlsDir, ladder, 2);
  const proc = this._gstLive(lines.join(' '), 'rtmp-prod-video');
  this._startHealthWatch();
  return proc;
}

_gstRtmpSocial({ rtmpUrl, audioOnly }) {
  const { hlsDir } = this;
  if (audioOnly) {
    const pl = [
      `rtmpsrc location="${rtmpUrl} live=1" ! flvdemux name=d`,
      `d. ! queue ! aacparse ! avdec_aac ! audioconvert ! audioresample`,
      `! avenc_aac bitrate=131072 compliance=-2 ! aacparse`,
      `! hlssink2 location="${hlsDir}/stream_%05d.ts"`,
      `           playlist-location="${hlsDir}/stream.m3u8"`,
      `           target-duration=1 max-files=10`,
    ].join(' ');
    return this._gstLive(pl, 'rtmp-social-audio');
  }
  const encFn = pickSocialEncoder(this.caps);
  const pl = [
    `rtmpsrc location="${rtmpUrl} live=1" ! flvdemux name=d`,
    `d. ! queue max-size-time=2000000000 ! h264parse ! avdec_h264 ! videoconvert`,
    `! videoscale ! video/x-raw,width=${SOCIAL_RUNG.w},height=${SOCIAL_RUNG.h}`,
    `! ${encFn(SOCIAL_RUNG.vbr)} ! mpegtsmux name=mux`,
    `! hlssink2 location="${hlsDir}/stream_%05d.ts"`,
    `           playlist-location="${hlsDir}/stream.m3u8"`,
    `           target-duration=1 max-files=10`,
    `d. ! queue ! aacparse ! avdec_aac ! audioconvert ! audioresample`,
    `! avenc_aac bitrate=131072 compliance=-2 ! aacparse ! mux.`,
  ].join(' ');
  return this._gstLive(pl, 'rtmp-social-video');
}

// ═══════════════════════════════════════════════════════════════════════
//  BROWSER LIVE  (MediaRecorder WebM → fdsrc stdin → GStreamer)
// ═══════════════════════════════════════════════════════════════════════

_gstBrowserProduction({ passThrough, audioOnly, ladder }) {
  const { hlsDir } = this;
  const enc = pickVideoEncoder(this.caps);

  if (audioOnly || !ladder.length) {
    return this._gstBrowserSocial({ passThrough, audioOnly: true });
  }

  const lines = [
```

```
    `fdsrc fd=0`,
    `! matroskademux name=d`,
    `d.video_0 ! queue ! vp8dec ! videoconvert ! videorate`,
    `! video/x-raw,framerate=30/1 ! tee name=vtee`,
    `d.audio_0 ! queue ! vorbisdec ! audioconvert ! audioresample`,
    `! audio/x-raw,rate=44100,channels=2 ! tee name=atee`,
  ];

  for (const r of ladder) {
    lines.push(
      `vtee. ! queue leaky=downstream ! videoscale`,
      `! video/x-raw,width=${r.w},height=${r.h}`,
      `! ${enc.enc(r.vbr)} ! mpegtsmux name=mux${r.name}`,
      `! hlssink2 location="${hlsDir}/${r.name}_%05d.ts"`,
      `           playlist-location="${hlsDir}/${r.name}.m3u8"`,
      `           target-duration=2 max-files=16`,
      `atee. ! queue ! avenc_aac bitrate=${r.abr * 1000} compliance=-2 ! aacparse ! mux${r.name}.`,
    );
  }

  this._writeVideoMasterSync(hlsDir, ladder, 2);
  const proc = this._gstLive(lines.join(' '), 'browser-prod', passThrough);
  this._startHealthWatch();
  return proc;
}

_gstBrowserSocial({ passThrough, audioOnly }) {
  const { hlsDir } = this;
  if (audioOnly) {
    const pl = [
      `fdsrc fd=0 ! matroskademux`,
      `! vorbisdec ! audioconvert ! audioresample`,
      `! avenc_aac bitrate=131072 compliance=-2 ! aacparse`,
      `! hlssink2 location="${hlsDir}/stream_%05d.ts"`,
      `           playlist-location="${hlsDir}/stream.m3u8"`,
      `           target-duration=1 max-files=10`,
    ].join(' ');
    return this._gstLive(pl, 'browser-social-audio', passThrough);
  }
  const encFn = pickSocialEncoder(this.caps);
  const pl = [
    `fdsrc fd=0 ! matroskademux name=d`,
    `d.video_0 ! queue ! vp8dec ! videoconvert`,
    `! videoscale ! video/x-raw,width=${SOCIAL_RUNG.w},height=${SOCIAL_RUNG.h}`,
    `! ${encFn(SOCIAL_RUNG.vbr)} ! mpegtsmux name=mux`,
    `! hlssink2 location="${hlsDir}/stream_%05d.ts"`,
    `           playlist-location="${hlsDir}/stream.m3u8"`,
```

```
    `           target-duration=1 max-files=10`,
    `d.audio_0 ! queue ! vorbisdec ! audioconvert ! audioresample`,
    `! avenc_aac bitrate=131072 compliance=-2 ! aacparse ! mux.`,
  ].join(' ');
  return this._gstLive(pl, 'browser-social-video', passThrough);
}

// ═══════════════════════════════════════════════════════════════════════
//  FFMPEG FALLBACKS  (mirrors every GStreamer path)
// ═══════════════════════════════════════════════════════════════════════

async _ffmpegAudioProduction(inputPath) {
  const { hlsDir } = this;
  for (const r of PROD_AUDIO_LADDER) {
    await _runProc(FFMPEG_PATH, [
      '-i', inputPath, '-vn',
      '-c:a', 'aac', '-b:a', `${Math.round(r.bps/1000)}k`, '-ar', '44100', '-ac', '2',
      '-f', 'hls', '-hls_time', '10', '-hls_list_size', '0',
      '-hls_segment_filename', `${hlsDir}/${r.name}_%05d.ts`,
      `${hlsDir}/${r.name}.m3u8`,
    ]);
  }
  await this._writeAudioMaster();
}

async _ffmpegAudioSocial(inputPath) {
  const { hlsDir } = this;
  await _runProc(FFMPEG_PATH, [
    '-i', inputPath, '-vn',
    '-c:a', 'aac', '-b:a', '128k', '-ar', '44100', '-ac', '2',
    '-f', 'hls', '-hls_time', '10', '-hls_list_size', '0',
    '-hls_segment_filename', `${hlsDir}/stream_%05d.ts`,
    `${hlsDir}/stream.m3u8`,
  ]);
  await fs.writeFile(`${hlsDir}/master.m3u8`,
    '#EXTM3U\n#EXT-X-VERSION:3\n#EXT-X-STREAM-INF:BANDWIDTH=131000,CODECS="mp4a.40.2"\nstream.m3u8\n
}

async _ffmpegVideoProduction(inputPath, ladder) {
  const { hlsDir } = this;
  for (const r of ladder) {
    await _runProc(FFMPEG_PATH, [
      '-i', inputPath,
      '-c:v', 'libx264', '-preset', 'fast', '-crf', '20',
      '-maxrate', `${r.vbr}k`, '-bufsize', `${r.vbr * 2}k`,
      '-vf', `scale=${r.w}:${r.h}:force_original_aspect_ratio=decrease,pad=${r.w}:${r.h}:(ow-iw)/2:(
      '-g', '60', '-keyint_min', '60', '-sc_threshold', '0',
```

```
      '-c:a', 'aac', '-b:a', `${r.abr}k`, '-ar', '44100', '-ac', '2',
      '-f', 'hls', '-hls_time', `${r.seg}`, '-hls_list_size', '0',
      '-hls_segment_filename', `${hlsDir}/${r.name}_%05d.ts`,
      `${hlsDir}/${r.name}.m3u8`,
    ]);
  }
  await _runProc(FFMPEG_PATH, ['-i', inputPath, '-ss', '5', '-frames:v', '1',
    '-vf', 'scale=320:-1', `${hlsDir}/thumb.jpg`]).catch(() => {});
  await this._writeVideoMaster(ladder);
}

async _ffmpegVideoSocial(inputPath) {
  const { hlsDir } = this;
  await _runProc(FFMPEG_PATH, [
    '-i', inputPath,
    '-c:v', 'libx264', '-preset', 'veryfast', '-crf', '23',
    '-vf', `scale=${SOCIAL_RUNG.w}:${SOCIAL_RUNG.h}:force_original_aspect_ratio=decrease`,
    '-c:a', 'aac', '-b:a', `${SOCIAL_RUNG.abr}k`, '-ar', '44100',
    '-f', 'hls', '-hls_time', '10', '-hls_list_size', '0',
    '-hls_segment_filename', `${hlsDir}/stream_%05d.ts`,
    `${hlsDir}/stream.m3u8`,
  ]);
  await fs.writeFile(`${hlsDir}/master.m3u8`,
    `#EXTM3U\n#EXT-X-VERSION:3\n#EXT-X-STREAM-INF:BANDWIDTH=1350000,RESOLUTION=${SOCIAL_RUNG.w}x${SO
}

_ffmpegRtmpProduction({ rtmpUrl, audioOnly, ladder }) {
  const { hlsDir } = this;
  const r = ladder[0] || { w: 1280, h: 720, vbr: 2800, abr: 128 };
  const args = audioOnly ? [
    '-i', rtmpUrl, '-vn', '-c:a', 'aac', '-b:a', '128k',
    '-f', 'hls', '-hls_time', '2', '-hls_list_size', '16',
    '-hls_flags', 'delete_segments',
    '-hls_segment_filename', `${hlsDir}/hi_%05d.ts`, `${hlsDir}/hi.m3u8`,
  ] : [
    '-i', rtmpUrl,
    '-c:v', 'libx264', '-preset', 'veryfast', '-tune', 'zerolatency', `-b:v`, `${r.vbr}k`,
    '-c:a', 'aac', `-b:a`, `${r.abr}k`, '-ar', '44100',
    '-f', 'hls', '-hls_time', '2', '-hls_list_size', '16',
    '-hls_flags', 'delete_segments+append_list',
    '-hls_segment_filename', `${hlsDir}/${r.name || '720p'}_%05d.ts`,
    `${hlsDir}/${r.name || '720p'}.m3u8`,
  ];
  this._writeVideoMasterSync(hlsDir, ladder.slice(0,1), 2);
  return this._ffmpegLive(args, 'rtmp-prod-ffmpeg');
}
```

```
_ffmpegRtmpSocial({ rtmpUrl, audioOnly }) {
  const { hlsDir } = this;
  const args = audioOnly ? [
    '-i', rtmpUrl, '-vn', '-c:a', 'aac', '-b:a', '128k',
    '-f', 'hls', '-hls_time', '1', '-hls_list_size', '10', '-hls_flags', 'delete_segments',
    '-hls_segment_filename', `${hlsDir}/stream_%05d.ts`, `${hlsDir}/stream.m3u8`,
  ] : [
    '-i', rtmpUrl,
    '-c:v', 'libx264', '-preset', 'ultrafast', '-tune', 'zerolatency', '-b:v', `${SOCIAL_RUNG.vbr}k`
    '-c:a', 'aac', '-b:a', `${SOCIAL_RUNG.abr}k`, '-ar', '44100',
    '-f', 'hls', '-hls_time', '1', '-hls_list_size', '10', '-hls_flags', 'delete_segments',
    '-hls_segment_filename', `${hlsDir}/stream_%05d.ts`, `${hlsDir}/stream.m3u8`,
  ];
  return this._ffmpegLive(args, 'rtmp-social-ffmpeg');
}

_ffmpegBrowserProduction({ passThrough, audioOnly }) {
  return this._ffmpegBrowserLive({ passThrough, audioOnly, vbr: 2800, abr: 128, hlsSeg: 2, maxFiles:
}

_ffmpegBrowserSocial({ passThrough, audioOnly }) {
  return this._ffmpegBrowserLive({ passThrough, audioOnly, vbr: SOCIAL_RUNG.vbr, abr: SOCIAL_RUNG.ab
}

_ffmpegBrowserLive({ passThrough, audioOnly, vbr, abr, hlsSeg, maxFiles }) {
  const { hlsDir } = this;
  const args = [
    '-fflags', 'nobuffer', '-flags', 'low_delay',
    '-i', 'pipe:0',
    ...(audioOnly ? ['-vn'] : ['-c:v', 'libx264', '-preset', 'ultrafast', '-tune', 'zerolatency', '-
    '-c:a', 'aac', '-b:a', `${abr}k`, '-ar', '44100',
    '-f', 'hls', '-hls_time', `${hlsSeg}`, '-hls_list_size', `${maxFiles}`,
    '-hls_flags', 'delete_segments+append_list',
    '-hls_segment_filename', `${hlsDir}/stream_%05d.ts`,
    `${hlsDir}/stream.m3u8`,
  ];
  const proc = spawn(FFMPEG_PATH, args, { stdio: ['pipe', 'pipe', 'pipe'] });
  passThrough.pipe(proc.stdin);
  proc.stderr?.on('data', d => this.logger.debug({ id: this.id }, `FF: ${d}`));
  proc.on('error', err => this.emit('error', err));
  proc.on('close', (c, s) => this._onClose(c, 'ff-browser', s));
  this._procs.push(proc);
  return proc;
}

// ═══════════════════════════════════════════════════════════════════════
//  STOP + HEALTH
```

```
// ═══════════════════════════════════════════════════════════════════════

async stop() {
  this._stopped = true;
  clearInterval(this._healthT);
  await Promise.all(this._procs.map(p => new Promise(res => {
    if (!p || p.exitCode !== null) return res();
    const t = setTimeout(() => { try { p.kill('SIGKILL'); } catch (_) {} res(); }, 8000);
    p.on('close', () => { clearTimeout(t); res(); });
    try { p.kill('SIGINT'); } catch (_) {}
  })));
  this._procs = [];
  this.emit('stopped', { id: this.id });
  this.logger.info({ id: this.id }, 'Pipeline stopped');
}

_startHealthWatch() {
  this._healthT = setInterval(() => {
    const alive = this._procs.filter(p => p.exitCode === null).length;
    this.emit('health', { id: this.id, alive, total: this._procs.length });
    if (!this._stopped && alive === 0)
      this.emit('all_dead', { id: this.id });
  }, 15000);
}

// ═══════════════════════════════════════════════════════════════════════
//  INTERNALS
// ═══════════════════════════════════════════════════════════════════════

async _gstRun(pipelineStr) {
  return _runProc(GST_LAUNCH, ['-e', pipelineStr]);
}

_gstLive(pipelineStr, label, stdinStream) {
  const stdio = stdinStream ? ['pipe', 'pipe', 'pipe'] : ['ignore', 'pipe', 'pipe'];
  const proc  = spawn(GST_LAUNCH, ['-e', pipelineStr], { stdio });
  if (stdinStream) stdinStream.pipe(proc.stdin);
  proc.stdout?.on('data', d => this.logger.debug({ id: this.id, label }, `GST: ${d}`));
  proc.stderr?.on('data', d => this.logger.debug({ id: this.id, label }, `GST: ${d}`));
  proc.on('error', err => { this.logger.error({ id: this.id, label, err }); this.emit('error', { lab
  proc.on('close', (c, s) => this._onClose(c, label, s));
  this._procs.push(proc);
  return proc;
}

_ffmpegLive(args, label) {
  const proc = spawn(FFMPEG_PATH, args, { stdio: ['ignore', 'pipe', 'pipe'] });
```

```
  proc.stderr?.on('data', d => this.logger.debug({ id: this.id, label }, `FF: ${d}`));
  proc.on('error', err => { this.logger.error({ id: this.id, label, err }); this.emit('error', { lab
  proc.on('close', (c, s) => this._onClose(c, label, s));
  this._procs.push(proc);
  return proc;
}

_onClose(code, label, signal) {
  this.logger.info({ id: this.id, label, code, signal }, 'process closed');
  if (!this._stopped) this.emit('pipeline_closed', { id: this.id, label, code, signal });
}

async _writeAudioMaster() {
  const bwMap = { hi: 320000, mid: 256000, lo: 128000 };
  const lines = ['#EXTM3U', '#EXT-X-VERSION:3'];
  for (const r of PROD_AUDIO_LADDER) {
    lines.push(`#EXT-X-STREAM-INF:BANDWIDTH=${bwMap[r.name]},CODECS="mp4a.40.2"\n${r.name}.m3u8`);
  }
  await fs.writeFile(path.join(this.hlsDir, 'master.m3u8'), lines.join('\n') + '\n');
}

async _writeVideoMaster(ladder) {
  const lines = ['#EXTM3U', '#EXT-X-VERSION:3'];
  for (const r of ladder) {
    const bw = (r.vbr + r.abr) * 1000;
    lines.push(`#EXT-X-STREAM-INF:BANDWIDTH=${bw},RESOLUTION=${r.w}x${r.h},CODECS="avc1.42e01e,mp4a.
  }
  await fs.writeFile(path.join(this.hlsDir, 'master.m3u8'), lines.join('\n') + '\n');
}

_writeVideoMasterSync(hlsDir, ladder, seg) {
  const lines = ['#EXTM3U', '#EXT-X-VERSION:3'];
  for (const r of ladder) {
    const bw = ((r.vbr || 2800) + (r.abr || 128)) * 1000;
    const name = r.name || '720p';
    lines.push(`#EXT-X-STREAM-INF:BANDWIDTH=${bw},RESOLUTION=${r.w || 1280}x${r.h || 720},CODECS="av
  }
  fs.writeFileSync(path.join(hlsDir, 'master.m3u8'), lines.join('\n') + '\n');
}

async _resolveThumb() {
  try {
    const files = (await fs.readdir(this.hlsDir))
      .filter(f => /^thumb_\d+\.jpg$/.test(f)).sort().reverse();
    if (files.length)
      await fs.copy(path.join(this.hlsDir, files[0]), path.join(this.hlsDir, 'thumb.jpg'));
  } catch (_) {}
```

```
  }
}

// ────────────────────────────────────────────────────────────────────────────
//  Utilities
// ────────────────────────────────────────────────────────────────────────────

function _cmd(cmd, args) {
  return new Promise((res, rej) => {
    const chunks = [];
    const p = spawn(cmd, args, { stdio: ['ignore', 'pipe', 'pipe'] });
    p.stdout.on('data', d => chunks.push(d));
    p.on('error', rej);
    p.on('close', c => c === 0 ? res(Buffer.concat(chunks).toString()) : rej(new Error(`${cmd} exit ${
  });
}

function _runProc(cmd, args) {
  return new Promise((res, rej) => {
    const errs = [];
    const p = spawn(cmd, args, { stdio: ['ignore', 'pipe', 'pipe'] });
    p.stderr?.on('data', d => errs.push(d));
    p.on('error', rej);
    p.on('close', c => c === 0 ? res() : rej(new Error(`${cmd} exit ${c}: ${Buffer.concat(errs).toStri
  });
}

// ────────────────────────────────────────────────────────────────────────────
//  Exports
// ────────────────────────────────────────────────────────────────────────────

module.exports = {
  GstPipeline,
  detectCapabilities,
  pickVideoEncoder,
  pickSocialEncoder,
  MODES,
  STREAMS_ROOT,
  HLS_ROOT,
  PROD_VIDEO_LADDER,
  PROD_AUDIO_LADDER,
  SOCIAL_RUNG,
};
```

FRONTEND — CSS

`styles.css` (21.3 KB)

```css
/* ═══════════════════════════════════════════════════════════════════════════
   MSP Global Styles  —  SIGNAL Design System
   public/styles/styles.css
   ═══════════════════════════════════════════════════════════════════════════ */


/* ─── Design tokens ─────────────────────────────────────────────────────── */
:root {
  /* Backgrounds */
  --bg-void:       #08080A;
  --bg-base:       #0F0F12;
  --bg-surface:    #151518;
  --bg-raised:     #1C1C21;
  --bg-hover:      #222228;

  /* Borders */
  --border-subtle: #222228;
  --border-mid:    #2E2E36;
  --border-strong: #3E3E4A;

  /* Brand colors */
  --gold:          #D4A853;   /* royalties, premium, earnings  */
  --gold-bright:   #F0C060;
  --gold-dim:      #8A6A2A;
  --teal:          #00D4BB;   /* streaming, on-chain, active   */
  --teal-bright:   #20F0D8;
  --teal-dim:      #007A6A;
  --ember:         #E85D3A;   /* live, alerts, energy          */
  --ember-dim:     #6A2A1A;
  --violet:        #8B5CF6;   /* NFT, ownership, minting       */
  --violet-dim:    #3D2080;

  /* Text */
  --text-primary:  #EEEAE4;
  --text-secondary:#8A8A98;
  --text-muted:    #484852;
  --text-inverse:  #0F0F12;

  /* Typography */
  --font-display:  'Cormorant', Georgia, serif;
  --font-ui:       'Syne', sans-serif;
  --font-mono:     'Space Mono', monospace;
```

```
  /* Motion */
  --ease-out:      cubic-bezier(0.16, 1, 0.3, 1);
  --ease-in-out:   cubic-bezier(0.65, 0, 0.35, 1);
}


/* ─── Reset ──────────────────────────────────────────────────────────── */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}


/* ─── Base body ──────────────────────────────────────────────────────── */
body {
  background:    var(--bg-void);
  color:         var(--text-primary);
  font-family:   var(--font-ui);
  font-size:     14px;
  line-height:   1.6;
  min-height:    100vh;
  overflow-x:    hidden;
  padding-top:   70px;   /* navbar clearance */
  position:      relative;
  isolation:     isolate;
}

/* ── Per-page body padding (accounts for fixed player bar where present) ── */
.page-listen        { padding-bottom: 90px; }  /* fixed player bar */
.page-dashboard     { padding-bottom: 40px; }
.page-favorites     { padding-bottom: 40px; }
.page-profile       { padding-bottom: 40px; }
.page-asset-manager { padding-bottom: 40px; }
.page-creators      { padding-bottom: 40px; }
.page-marketplace   { padding-bottom: 40px; }
.page-index         { padding-bottom: 0; }
.page-live-studio   { padding-bottom: 0; }


/* ═══════════════════════════════════════════════════════════════════════
   VINYL RECORD BACKGROUND  —  MSP signature effect
   Two pseudo-elements stacked behind all content:

   ::before  — fine groove texture  (micro radial rings)
   ::after   — vinyl color rings    (SIGNAL palette, pulsing)
```

```
   Your original CSS mapped to SIGNAL tokens:
     pink   → --ember  (live / energy)
     purple → --violet (NFT / ownership)
     teal   → --teal   (streaming / on-chain)
     dark   → --bg-void
   ═══════════════════════════════════════════════════════════════════════ */

/* Layer 1 — fine groove texture (fades in after vinyl boot) */
body::before {
  content: '';
  position: fixed;
  inset: 0;
  background: repeating-radial-gradient(
    circle,
    transparent,
    transparent     4px,
    rgba(244, 238, 238, 0.06) 3px,
    transparent     8px
  );
  pointer-events: none;
  z-index: -1;
  opacity: 0;
  animation: groove-in 1.2s ease-out 2s forwards;
}

@keyframes groove-in {
  0%   { opacity: 0; }
  100% { opacity: 1; }
}

/* Layer 2 — vinyl color rings (boot flicker then steady pulse) */
body::after {
  content: '';
  position: fixed;
  inset: 0;
  background:
    radial-gradient(
      circle,
      var(--ember)       0%,
      var(--ember)       6%,
      var(--violet-dim)  6%,
      var(--violet)      22%,
      var(--teal-dim)    22%,
      #004a40            42%,
      #0a0a0e            42%,
      var(--bg-void)     70%,
      var(--bg-void)     100%
```

```
    ),
    repeating-radial-gradient(
      circle at center,
      rgba(255, 255, 255, 0.06) 2px,
      rgba(255, 255, 255, 0.06) 3px,
      transparent               4px,
      transparent               8px
    );
  background-blend-mode:   overlay;
  background-attachment:   fixed;
  pointer-events:          none;
  z-index:                 -2;
  animation:
    vinyl-boot  2.8s ease-out forwards,
    vinyl-pulse 3s   ease-in-out infinite 2.8s;
}

@keyframes vinyl-boot {
  0%   { opacity: 0;    transform: scale(0.96); }
  6%   { opacity: .45;  transform: scale(1.01); }
  8%   { opacity: 0;    transform: scale(0.97); }
  13%  { opacity: .6;   transform: scale(1.02); }
  15%  { opacity: .05;  transform: scale(0.98); }
  21%  { opacity: .55;  transform: scale(1);    }
  23%  { opacity: .1;   transform: scale(0.99); }
  29%  { opacity: .65;  transform: scale(1.01); }
  31%  { opacity: .3;   transform: scale(1);    }
  38%  { opacity: .6;   transform: scale(1.015);}
  100% { opacity: .55;  transform: scale(1);    }
}

@keyframes vinyl-pulse {
  0%,  100% { opacity: .55; transform: scale(1);     }
  50%        { opacity: .70; transform: scale(1.015); }
}

/* Neon text boot then steady flicker */
.neon-title {
  animation:
    neon-boot    2.4s ease-out forwards,
    neon-flicker 5s   ease-in-out infinite 2.4s;
}

@keyframes neon-boot {
  0%   { opacity: 0;    text-shadow: none; }
  8%   { opacity: 1;    text-shadow: 0 0 4px #F5F000, 0 0 12px #F5F000, 0 0 28px rgba(245,240,0,.8); }
  10%  { opacity: .1;   text-shadow: none; }
```

```
  14%  { opacity: 1;    text-shadow: 0 0 4px #F5F000, 0 0 20px #F5F000, 0 0 40px rgba(245,240,0,.9); }
  16%  { opacity: .3;   text-shadow: none; }
  20%  { opacity: 1;    text-shadow: 0 0 4px #F5F000, 0 0 12px #F5F000; }
  22%  { opacity: .05;  text-shadow: none; }
  28%  { opacity: 1;    text-shadow: 0 0 4px #F5F000, 0 0 28px rgba(245,240,0,.8), 0 0 60px rgba(245,2
  30%  { opacity: .6;   text-shadow: none; }
  36%  { opacity: 1;    text-shadow: 0 0 4px #F5F000, 0 0 28px rgba(245,240,0,.8), 0 0 60px rgba(245,2
  100% { opacity: 1;    text-shadow: 0 0 4px #F5F000, 0 0 28px rgba(245,240,0,.8), 0 0 60px rgba(245,2
}

@keyframes neon-flicker {
  0%,18%,20%,22%,24%,53%,55%,100% {
    text-shadow: 0 0 4px #F5F000, 0 0 12px #F5F000, 0 0 28px rgba(245,240,0,.8), 0 0 60px rgba(245,240
    opacity: 1;
  }
  19%, 23%, 54% { text-shadow: none; opacity: .82; }
}


/* ── Navbar ──────────────────────────────────────────────────────── */
.navbar {
  background:     rgba(8, 8, 10, 0.85) !important;
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border-bottom:  1px solid var(--border-subtle);
  position:       fixed;
  top: 0; left: 0; right: 0;
  z-index:        100;
}

.navbar-brand {
  font-family:    var(--font-display);
  font-size:      22px;
  font-weight:    400;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color:          var(--text-primary) !important;
}

.nav-link {
  font-family:  var(--font-ui);
  font-size:    13px;
  font-weight:  600;
  color:        var(--text-secondary) !important;
  letter-spacing: 0.03em;
  transition:   color 0.15s;
}
```

```
.nav-link:hover,
.nav-link.active {
  color: var(--gold) !important;
}

.user-name-display {
  font-family:  var(--font-mono);
  font-size:    11px;
  color:        var(--teal);
  letter-spacing: 0.06em;
}


/* ── Cards ──────────────────────────────────────────────────────────── */
.card,
.nft-card {
  background:   var(--bg-raised);
  border:       1px solid var(--border-subtle) !important;
  border-radius: 10px;
  color:        var(--text-primary);
  transition:   all 0.25s var(--ease-out);
}

.card:hover,
.nft-card:hover {
  border-color: var(--border-mid) !important;
  transform:    translateY(-3px);
  box-shadow:   0 16px 40px rgba(0, 0, 0, 0.5);
}

.card-header {
  background:   var(--bg-surface) !important;
  border-bottom: 1px solid var(--border-subtle) !important;
  font-family:  var(--font-ui);
  font-weight:  600;
}


/* ── Buttons ────────────────────────────────────────────────────────── */
.btn-primary {
  background:    var(--gold);
  border-color:  var(--gold);
  color:         var(--text-inverse);
  font-family:   var(--font-ui);
  font-weight:   700;
  letter-spacing: 0.04em;
```

```
  transition:    all 0.2s var(--ease-out);
}
.btn-primary:hover {
  background:   var(--gold-bright);
  border-color: var(--gold-bright);
  transform:    translateY(-1px);
  box-shadow:   0 8px 24px rgba(212, 168, 83, 0.3);
}

.btn-success  { background: var(--teal);   border-color: var(--teal);   color: var(--text-inverse); }
.btn-danger   { background: var(--ember);  border-color: var(--ember);  color: #fff; }
.btn-warning  { background: var(--gold);   border-color: var(--gold);   color: var(--text-inverse); }
.btn-secondary { background: var(--bg-raised); border-color: var(--border-mid); color: var(--text-seco

.btn-outline-primary { border-color: var(--gold);   color: var(--gold); }
.btn-outline-primary:hover { background: var(--gold); color: var(--text-inverse); }

.btn-outline-warning { border-color: var(--gold);   color: var(--gold); }
.btn-outline-warning:hover { background: var(--gold); color: var(--text-inverse); }


/* ─── Forms ──────────────────────────────────────────────────────────── */
.form-control,
.form-select {
  background:   var(--bg-raised);
  border:       1px solid var(--border-mid);
  border-radius: 5px;
  color:        var(--text-primary);
  font-family:  var(--font-ui);
  transition:   border-color 0.15s;
}

.form-control:focus,
.form-select:focus {
  background:  var(--bg-raised);
  border-color: var(--gold);
  box-shadow:  0 0 0 3px rgba(212, 168, 83, 0.12);
  color:       var(--text-primary);
}

.form-control::placeholder { color: var(--text-muted); }
.form-label {
  font-family:   var(--font-mono);
  font-size:     10px;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color:         var(--text-muted);
```

```
}


/* ── Typography ──────────────────────────────────────────────────────── */
h1, h2, h3, h4, h5, h6 {
  font-family: var(--font-display);
  font-weight: 300;
  color:       var(--text-primary);
  line-height: 1.1;
}

.font-mono,
code,
.badge-mono {
  font-family: var(--font-mono);
  font-size:   0.85em;
}

.text-gold    { color: var(--gold) !important; }
.text-teal    { color: var(--teal) !important; }
.text-ember   { color: var(--ember) !important; }
.text-violet  { color: var(--violet) !important; }
.text-muted   { color: var(--text-muted) !important; }


/* ── Badges ──────────────────────────────────────────────────────────── */
.badge {
  font-family:   var(--font-mono);
  font-size:     9px;
  letter-spacing: 0.1em;
  font-weight:   700;
  text-transform: uppercase;
}

.badge.bg-warning { background: var(--gold)   !important; color: var(--text-inverse); }
.badge.bg-danger  { background: var(--ember)  !important; }
.badge.bg-success { background: var(--teal)   !important; color: var(--text-inverse); }
.badge.bg-primary { background: var(--violet) !important; }


/* ── Alerts ──────────────────────────────────────────────────────────── */
.alert {
  background:   var(--bg-surface);
  border:       1px solid var(--border-mid);
  color:        var(--text-secondary);
  border-radius: 6px;
}
```

```
.alert-dark { background: var(--bg-raised); border-color: var(--border-subtle); }


/* ─── Tables ──────────────────────────────────────────────────────────── */
.table {
  color:        var(--text-primary);
  border-color: var(--border-subtle);
}
.table th {
  font-family:   var(--font-mono);
  font-size:     10px;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color:         var(--text-muted);
  border-color:  var(--border-mid);
  background:    var(--bg-surface);
}
.table td { border-color: var(--border-subtle); }
.table-striped > tbody > tr:nth-of-type(odd) > * {
  background: rgba(255, 255, 255, 0.02);
}


/* ─── Modals ──────────────────────────────────────────────────────────── */
.modal-content {
  background:   var(--bg-surface);
  border:       1px solid var(--border-mid);
  border-radius: 12px;
}
.modal-header {
  border-bottom: 1px solid var(--border-subtle);
}
.modal-footer {
  border-top: 1px solid var(--border-subtle);
}
.btn-close-white { filter: invert(1); }


/* ─── Scrollbar ───────────────────────────────────────────────────────── */
::-webkit-scrollbar       { width: 4px; height: 4px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: var(--border-mid); border-radius: 2px; }
::-webkit-scrollbar-thumb:hover { background: var(--border-strong); }


/* ─── Subscription status bar ─────────────────────────────────────────── */
#subscription-status-bar {
```

```
  background:    var(--bg-surface);
  border:        1px solid var(--border-subtle);
  border-radius: 8px;
  padding:       10px 16px;
  font-family:   var(--font-mono);
  font-size:     11px;
  color:         var(--text-secondary);
}

.text-success { color: var(--teal)  !important; }
.text-danger  { color: var(--ember) !important; }


/* ── Drag-drop upload zone ──────────────────────────────────────────── */
.drag-over {
  border-color: var(--ember) !important;
  background:   rgba(232, 93, 58, 0.04) !important;
}


/* ── Live indicator ─────────────────────────────────────────────────── */
.live-dot {
  display:       inline-block;
  width:         8px; height: 8px;
  border-radius: 50%;
  background:    var(--ember);
  box-shadow:    0 0 0 3px rgba(232, 93, 58, 0.25);
  animation:     live-pulse 1.4s ease-in-out infinite;
}

@keyframes live-pulse {
  0%,  100% { box-shadow: 0 0 0 3px rgba(232, 93, 58, 0.25); }
  50%        { box-shadow: 0 0 0 7px rgba(232, 93, 58, 0.05); }
}


/* ── Favorites heart button ─────────────────────────────────────────── */
.fav-btn {
  background:    none;
  border:        none;
  cursor:        pointer;
  font-size:     18px;
  line-height:   1;
  padding:       2px 5px;
  color:         var(--text-muted);
  transition:    color 0.15s, transform 0.15s;
}
```

```
.fav-btn:hover        { color: var(--ember); transform: scale(1.2); }
.fav-btn.fav-active   { color: var(--ember); }
.fav-btn:disabled     { opacity: 0.4; }


/* ── Progress bar ──────────────────────────────────────────────────── */
#upload-progress {
  width:        100%;
  height:       4px;
  appearance:   none;
  border:       none;
  border-radius: 2px;
}
#upload-progress::-webkit-progress-bar   { background: var(--border-mid);  border-radius: 2px; }
#upload-progress::-webkit-progress-value { background: var(--gold);        border-radius: 2px; transit


/* ═══════════════════════════════════════════════════════════════════════
   RESPONSIVE NAVBAR  —  MSP Signal Design System
   Problem solved: between 992px (navbar-expand-lg threshold) and ~1280px
   the nav is expanded but cramped. Text wraps, buttons are oversized,
   the wallet address overflows. These rules fix all of that proportionally.
   ═══════════════════════════════════════════════════════════════════════ */

/* ── Prevent nav link text from ever wrapping ──────────────────────── */
.navbar-nav .nav-link {
  white-space: nowrap;
}

/* ── Brand — scale down gracefully instead of pushing nav items ────── */
.navbar-brand {
  font-size:   clamp(13px, 1.6vw, 22px);
  white-space: nowrap;
  flex-shrink: 0;
}

/* ── Wallet address — truncate, never overflow ────────────────────── */
.wallet-address-display {
  display:       inline-block;
  font-family:   var(--font-mono);
  font-size:     10px;
  max-width:     80px;
  overflow:      hidden;
  text-overflow: ellipsis;
  vertical-align: middle;
  white-space:   nowrap;
}
```

```
/* ── Wallet action buttons — compact at all times ──────────────────────── */
.navbar [data-connect-wallet],
#btn-disconnect {
  font-size:     11px !important;
  padding:       3px 10px !important;
  white-space:   nowrap;
}

/* ── Wallet + address wrapper — never grow, never push nav items ─────────── */
.navbar .d-flex.align-items-center {
  flex-shrink: 1;
  gap:         4px !important;
  min-width:   0;
}

/* ── Nav items row — allow shrink, keep items on one line ────────────────── */
.navbar-collapse .navbar-nav {
  flex-wrap:  nowrap;
  align-items: center;
}

/* ═══ Medium screens 992–1199px — scale everything down proportionally ═══ */
@media (min-width: 992px) and (max-width: 1199px) {

  .navbar-brand {
    font-size:    clamp(11px, 1.2vw, 16px);
    letter-spacing: 0.04em;
  }

  .navbar-nav .nav-link {
    font-size:  11px;
    padding:    0.4rem 0.5rem;
  }

  .navbar [data-connect-wallet],
  #btn-disconnect {
    font-size: 10px !important;
    padding:   2px 8px !important;
  }

  .wallet-address-display {
    max-width: 60px;
    font-size: 9px;
  }

  .user-name-display {
```

```
    font-size: 9px;
    max-width: 60px;
    overflow:  hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
  }
}

/* ═══ Large screens 1200–1399px — mild scaling ═══════════════════════ */
@media (min-width: 1200px) and (max-width: 1399px) {

  .navbar-brand {
    font-size: clamp(14px, 1.4vw, 20px);
  }

  .navbar-nav .nav-link {
    font-size: 12px;
    padding:   0.4rem 0.65rem;
  }

  .wallet-address-display {
    max-width: 72px;
  }
}

/* ═══ When collapsed (< 992px) — tidy up the dropdown menu ═══════════ */
@media (max-width: 991px) {

  body {
    padding-top: 60px;
  }

  .navbar-brand {
    font-size: 14px;
    letter-spacing: 0.05em;
  }

  /* Stack wallet controls neatly inside the collapsed menu */
  .navbar-collapse .d-flex.align-items-center {
    flex-wrap:  wrap;
    padding:    8px 0 4px;
    gap:        6px !important;
  }

  .wallet-address-display {
    max-width:  none;
    font-size:  10px;
```

```
    width:      100%;
  }

  .user-name-display {
    width: 100%;
  }

  .navbar-nav .nav-link {
    font-size:  13px;
    padding:    0.5rem 0;
  }
}
```

SMART CONTRACTS (Solidity)

`ContentCA.sol` (2.2 KB)

```solidity
// ContentCA.sol (updated for verifiable signatures)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
import "@openzeppelin/contracts/utils/cryptography/EIP712.sol";

contract ContentCA is Ownable, EIP712 {
    using ECDSA for bytes32;

    address public immutable caSigner;

    // Record per content CID
    struct Certificate {
        string cid; // ipfs cid (metadata json)
        address signer; // address that signed (CA)
        uint256 timestamp;
        string contentType; // "music"|"podcast"|"art"
    }

    mapping(bytes32 => Certificate) public certificates; // key = keccak256(cid, signer)

    event CertificateRegistered(bytes32 indexed key, string cid, address indexed signer, string conten

    constructor(address caSigner_) Ownable(msg.sender) EIP712("ContentCA", "1") {
        caSigner = caSigner_;
```

```
    }

    function registerCertificate(string calldata cid, string calldata contentType, bytes calldata sign
        bytes32 structHash = keccak256(abi.encode(
            keccak256("Certificate(string cid,string contentType)"),
            keccak256(bytes(cid)),
            keccak256(bytes(contentType))
        ));
        bytes32 hash = _hashTypedDataV4(structHash);
        address recovered = hash.recover(signature);
        require(recovered == caSigner, "Invalid CA signature");

        bytes32 key = keccak256(abi.encodePacked(cid, recovered));
        require(certificates[key].timestamp == 0, "Certificate already exists");

        certificates[key] = Certificate({cid: cid, signer: recovered, timestamp: block.timestamp, cont
        emit CertificateRegistered(key, cid, recovered, contentType, block.timestamp);
    }

    function getCertificateKey(string calldata cid, address signer) external pure returns (bytes32) {
        return keccak256(abi.encodePacked(cid, signer));
    }

    function certificateExists(string calldata cid, address signer) external view returns (bool) {
        bytes32 key = keccak256(abi.encodePacked(cid, signer));
        return certificates[key].timestamp != 0;
    }
}
```

`Escrow.sol` (2.7 KB)

```solidity
// Escrow.sol (for payments, updated for DOGE/SOL events)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract Escrow is Ownable {
    address public royaltyPayoutAddress;

    mapping(address => uint256) public subscriptionExpiryByUser;

    event DepositForPlay(address indexed user, string cid, uint256 amount);
    event Subscription(address indexed user, uint256 expiry);
```

```
    event DogePaymentLogged(address indexed user, string cid, uint256 amountEthEquivalent);
    event SolPaymentLogged(address indexed user, string cid, uint256 amountEthEquivalent);
    event DogeSubscriptionLogged(address indexed user, uint256 amountEthEquivalent);
    event SolSubscriptionLogged(address indexed user, uint256 amountEthEquivalent);

    constructor(address royaltyPayoutAddress_) Ownable(msg.sender) {
        royaltyPayoutAddress = royaltyPayoutAddress_;
    }

    function depositForPlay(string calldata cid) external payable {
        require(msg.value > 0, "Amount must be greater than 0");
        payable(royaltyPayoutAddress).transfer(msg.value);
        emit DepositForPlay(msg.sender, cid, msg.value);
    }

    function subscribe() external payable {
        require(msg.value > 0, "Amount must be greater than 0");
        subscriptionExpiryByUser[msg.sender] = block.timestamp + 30 days;
        payable(royaltyPayoutAddress).transfer(msg.value);
        emit Subscription(msg.sender, subscriptionExpiryByUser[msg.sender]);
    }

    function isSubscribed(address user) external view returns (bool) {
        return subscriptionExpiryByUser[user] > block.timestamp;
    }

    function logDogePayment(string calldata cid, address user, uint256 amountEthEquivalent) external o
        emit DogePaymentLogged(user, cid, amountEthEquivalent);
    }

    function logSolPayment(string calldata cid, address user, uint256 amountEthEquivalent) external on
        emit SolPaymentLogged(user, cid, amountEthEquivalent);
    }

    function logDogeSubscription(address user, uint256 amountEthEquivalent) external onlyOwner {
        subscriptionExpiryByUser[user] = block.timestamp + 30 days;
        emit DogeSubscriptionLogged(user, amountEthEquivalent);
        emit Subscription(user, subscriptionExpiryByUser[user]);
    }

    function logSolSubscription(address user, uint256 amountEthEquivalent) external onlyOwner {
        subscriptionExpiryByUser[user] = block.timestamp + 30 days;
        emit SolSubscriptionLogged(user, amountEthEquivalent);
        emit Subscription(user, subscriptionExpiryByUser[user]);
    }
}
```

`MusicNFT.sol` (1.6 KB)

```solidity
// MusicNFT.sol (per-user contract)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract MusicNFT is ERC721, Ownable {
    struct NFTData {
        string title;
        string artist;
        uint256 year;
        string metadataUrl;
    }

    mapping(uint256 => NFTData) public musicNFTs;
    uint256 public totalSupply;
    address public immutable mspAdmin;

    constructor(string memory name_, string memory symbol_, address mspAdmin_) ERC721(name_, symbol_)
        mspAdmin = mspAdmin_;
    }

    function mintNFT(string memory title, string memory artist, uint256 year, string memory metadataUr
        totalSupply++;
        _safeMint(owner(), totalSupply); // Mint to owner (user wallet after transfer)
        musicNFTs[totalSupply] = NFTData(title, artist, year, metadataUrl);
        return totalSupply;
    }

    function emergencyTransferOwnership(address newOwner) public {
        require(msg.sender == mspAdmin, "Only MSP admin can call this");
        _transferOwnership(newOwner);
    }

    // Additional ERC721 overrides if needed (e.g., tokenURI can derive from metadataUrl)
    function tokenURI(uint256 tokenId) public view virtual override returns (string memory) {
        _requireOwned(tokenId); // Updated to _requireOwned in 0.8.20+
        return musicNFTs[tokenId].metadataUrl; // Assumes metadataUrl is the full IPFS URI for JSON
    }
}
```

`NFTMetadataContract.sol` (1.4 KB)

```solidity
// NFTMetadataContract.sol (primary NFT contract)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract NFTMetadataContract is ERC721, Ownable {
    struct NFTData {
        string title;
        string artist;
        uint256 year;
        string metadataUrl;
    }

    mapping(uint256 => NFTData) public musicNFTs;
    uint256 public totalSupply;
    address public immutable mspAdmin;

    constructor(string memory name_, string memory symbol_, address mspAdmin_) ERC721(name_, symbol_)
        mspAdmin = mspAdmin_;
    }

    function mintNFT(string memory title, string memory artist, uint256 year, string memory metadataUr
        totalSupply++;
        _safeMint(owner(), totalSupply);
        musicNFTs[totalSupply] = NFTData(title, artist, year, metadataUrl);
        return totalSupply;
    }

    function emergencyTransferOwnership(address newOwner) public {
        require(msg.sender == mspAdmin, "Only MSP admin can call this");
        _transferOwnership(newOwner);
    }

    function tokenURI(uint256 tokenId) public view virtual override returns (string memory) {
        _requireOwned(tokenId);
        return musicNFTs[tokenId].metadataUrl;
    }
}
```

`RoyaltyPayout.sol` (3.9 KB)

```solidity
// RoyaltyPayout.sol (extended with configurable splits)
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC721/IERC721.sol";
import "@openzeppelin/contracts/token/ERC721/extensions/IERC721Enumerable.sol"; // Updated import for

contract RoyaltyPayout is Ownable {
    struct SplitConfig {
        uint256 artistShare; // bp, default 8000
        uint256 mspShare; // bp, default 200
        uint256 holderShare; // bp, default 1000 (pro-rata among holders)
        uint256 curatorShare; // bp, default 800 (to playlist DJ if applicable)
        address artistWallet;
        address mspWallet; // Fixed MSP wallet for its share
    }

    mapping(string => SplitConfig) public splitsByCid; // cid -> config
    mapping(string => address) public nftContractByCid; // cid -> NFTMetadataContract address for hold

    event SplitsUpdated(string cid, uint256 artistShare, uint256 mspShare, uint256 holderShare, uint25
    event PayoutExecuted(bytes32 indexed playId, uint256 amount, address token);

    constructor(address mspWallet_) Ownable(msg.sender) {}

    // Artist sets initial/configures splits (called during/after mint, verify caller owns NFT via NFT
    function setSplits(string calldata cid, uint256 artistShare, uint256 mspShare, uint256 holderShare
        require(artistShare + mspShare + holderShare + curatorShare == 10000, "Splits must sum to 1000
        // Verify caller is artist (e.g., owns the NFT or via metadata)
        IERC721 nft = IERC721(nftContract);
        require(nft.ownerOf(1) == msg.sender, "Only artist can set splits"); // Assume tokenId 1 for s
        splitsByCid[cid] = SplitConfig(artistShare, mspShare, holderShare, curatorShare, artistWallet,
        nftContractByCid[cid] = nftContract;
        emit SplitsUpdated(cid, artistShare, mspShare, holderShare, curatorShare);
    }

    // Off-chain indexer calls this with computed amounts (artist bulk, MSP fixed, holders pro-rata, c
    function executePayoutEther(bytes32 playId, address payable[] calldata wallets, uint256[] calldata
        require(wallets.length == amounts.length, "len");
        uint256 total;
        for (uint i = 0; i < amounts.length; i++) total += amounts[i];
        require(msg.value == total, "msg.value mismatch");
        for (uint i = 0; i < wallets.length; i++) wallets[i].transfer(amounts[i]);
        emit PayoutExecuted(playId, total, address(0));
    }
```

```
    function executePayoutERC20(bytes32 playId, address token, address[] calldata wallets, uint256[] c
        require(wallets.length == amounts.length, "len");
        IERC20 erc = IERC20(token);
        uint256 total;
        for (uint i = 0; i < amounts.length; i++) total += amounts[i];
        // Must have transferred `total` to this contract beforehand or have allowance flow
        for (uint i = 0; i < wallets.length; i++) {
            require(erc.transfer(wallets[i], amounts[i]));
        }
        emit PayoutExecuted(playId, total, token);
    }

    // Helper: Get pro-rata holder amounts (off-chain preferred, but on-chain for transparency if need
    function getHolderWallets(string calldata cid) public view returns (address[] memory) {
        IERC721Enumerable nft = IERC721Enumerable(nftContractByCid[cid]);
        uint256 supply = nft.totalSupply();
        address[] memory holders = new address[](supply);
        for (uint i = 0; i < supply; i++) holders[i] = nft.ownerOf(i + 1); // Assuming tokenIds start
        return holders;
    }
}
```

`StreamingRegistry.sol` (0.8 KB)

```solidity
// StreamingRegistry.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/Ownable.sol";
contract StreamingRegistry is Ownable {
    // A play event - minimal onchain
    event PlayLogged(
        bytes32 indexed playId,
        string indexed cid,
        address indexed listener,
        uint256 timestamp,
        bool live, // live vs on_demand
        bytes32 metadataHash // keccak of full play details stored offchain
    );
    constructor() Ownable(msg.sender) {}
    // owner can write if you want only relayer to log; or make public
    function logPlay(bytes32 playId, string calldata cid, address listener, bool live, bytes32 metadat
        emit PlayLogged(playId, cid, listener, block.timestamp, live, metadataHash);
    }
}
```

```

## SCHEMAS (JSON)
---

### `core_schema.json` (1.4 KB)

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "@context": "https://schema.org",
  "@type": "CreativeWork",
  "title": "CoreMetadata",
  "type": "object",
  "required": [
    "id",
    "title",
    "description",
    "creator",
    "content_type",
    "availability_type"
  ],
  "properties": {
    "id": {
      "type": "string"
    },
    "title": {
      "type": "string"
    },
    "description": {
      "type": "string"
    },
    "creator": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string"
        },
        "user_id": {
          "type": "string"
        },
        "wallet_address": {
          "type": "string"
        }
      },
      "required": [
        "name",
```
```

```
        "user_id"
      ]
    },
    "tags": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "release_date": {
      "type": "string",
      "format": "date"
    },
    "content_type": {
      "type": "string",
      "enum": [
        "music",
        "podcast",
        "art"
      ]
    },
    "availability_type": {
      "type": "string",
      "enum": [
        "on_demand",
        "live"
      ]
    },
    "files": {
      "type": "object",
      "properties": {
        "cover_image": {
          "type": "string"
        },
        "thumbnail": {
          "type": "string"
        }
      }
    }
  }
}
```

`music_schema.json` (3.5 KB)

```json
{
```

```
"$schema": "http://json-schema.org/draft-07/schema#",
"@context": "https://schema.org",
"@type": "CreativeWork",
"title": "MusicMetadata",
"type": "object",
"required": [
  "id",
  "title",
  "description",
  "creator",
  "content_type",
  "availability_type",
  "release_date",
  "duration",
  "tags",
  "mlc_metadata",
  "files"
],
"properties": {
  "id": { "type": "string" },
  "title": { "type": "string", "minLength": 1 },
  "description": { "type": "string", "minLength": 10 },
  "creator": {
    "type": "object",
    "properties": {
      "name": { "type": "string", "minLength": 1 },
      "user_id": { "type": "string" },
      "wallet_address": { "type": "string" }
    },
    "required": ["name", "user_id"]
  },
  "tags": {
    "type": "array",
    "items": { "type": "string", "minLength": 1 },
    "minItems": 5
  },
  "release_date": { "type": "string", "format": "date" },
  "content_type": { "type": "string", "enum": ["music"] },
  "availability_type": { "type": "string", "enum": ["on_demand", "live"] },
  "files": {
    "type": "object",
    "properties": {
      "cover_image": { "type": "string", "format": "uri" },
      "thumbnail": { "type": "string", "format": "uri" },
      "preview_url": { "type": "string", "format": "uri" }
    },
    "required": ["cover_image", "preview_url"]
```

```
},
"album": { "type": "string" },
"track_number": { "type": "integer" },
"bpm": { "type": "number" },
"key": { "type": "string" },
"duration": { "type": "number", "minimum": 60, "maximum": 900 },
"lyrics": { "type": "string" },
"record_label": { "type": "string" },
"ipfs_audio_url": { "type": "string", "format": "uri" },
"mlc_metadata": {
  "type": "object",
  "properties": {
    "work_title": { "type": "string", "minLength": 1 },
    "iswc": { "type": "string", "pattern": "^[A-Z]-[0-9]{3}\\.[0-9]{3}\\.[0-9]{3}-[0-9]$" },
    "isrc": { "type": "string", "pattern": "^[A-Z]{2}-[A-Z0-9]{3}-[0-9]{2}-[0-9]{5}$" },
    "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
    "ipi_base_number": { "type": "string" },
    "writers": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["name", "role", "ipi_name_number", "ownership_percent"],
        "properties": {
          "name": { "type": "string", "minLength": 1 },
          "role": { "type": "string" },
          "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
          "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
        }
      }
    },
    "publishers": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["name", "ipi_name_number", "ownership_percent"],
        "properties": {
          "name": { "type": "string", "minLength": 1 },
          "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
          "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
        }
      }
    }
  },
  "required": ["work_title", "writers"]
},
"integrityHashes": {
  "type": "object",
```

```
      "properties": {
        "sha256Audio": { "type": "string" },
        "sha256CoverImage": { "type": "string" },
        "sha256Metadata": { "type": "string" }
      },
      "required": ["sha256Audio", "sha256CoverImage", "sha256Metadata"]
    }
  }
}
```

`art_schema.json` (1.9 KB)

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "@context": "https://schema.org",
  "@type": "CreativeWork",
  "title": "ArtMetadata",
  "type": "object",
  "required": [
    "id",
    "title",
    "description",
    "creator",
    "content_type",
    "availability_type"
  ],
  "properties": {
    "id": {
      "type": "string"
    },
    "title": {
      "type": "string"
    },
    "description": {
      "type": "string"
    },
    "creator": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string"
        },
        "user_id": {
          "type": "string"
        },
```

```
    "wallet_address": {
      "type": "string"
    }
  },
  "required": [
    "name",
    "user_id"
  ]
},
"tags": {
  "type": "array",
  "items": {
    "type": "string"
  }
},
"release_date": {
  "type": "string",
  "format": "date"
},
"content_type": {
  "type": "string",
  "enum": [
    "music",
    "podcast",
    "art"
  ]
},
"availability_type": {
  "type": "string",
  "enum": [
    "on_demand",
    "live"
  ]
},
"files": {
  "type": "object",
  "properties": {
    "cover_image": {
      "type": "string"
    },
    "thumbnail": {
      "type": "string"
    }
  }
},
"medium": {
  "type": "string"
```

```
    },
    "style": {
      "type": "string"
    },
    "dimensions": {
      "type": "string"
    },
    "dpi": {
      "type": "integer"
    },
    "color_profile": {
      "type": "string"
    },
    "series_name": {
      "type": "string"
    },
    "edition_number": {
      "type": "integer"
    },
    "total_editions": {
      "type": "integer"
    },
    "signed": {
      "type": "boolean"
    },
    "exhibition_history": {
      "type": "string"
    },
    "license_rights": {
      "type": "string"
    },
    "inspiration": {
      "type": "string"
    }
  }
}
```

`art_still.json` (3.1 KB)

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "@context": "https://schema.org",
  "@type": "CreativeWork",
  "title": "StillArtMetadata",
  "type": "object",
```

```
"required": ["id", "title", "description", "creator", "content_type", "availability_type", "release_
"properties": {
  "id": { "type": "string" },
  "title": { "type": "string", "minLength": 1 },
  "description": { "type": "string", "minLength": 10 },
  "creator": {
    "type": "object",
    "properties": {
      "name": { "type": "string", "minLength": 1 },
      "user_id": { "type": "string" },
      "wallet_address": { "type": "string" }
    },
    "required": ["name", "user_id"]
  },
  "tags": {
    "type": "array",
    "items": { "type": "string", "minLength": 1 },
    "minItems": 5
  },
  "release_date": { "type": "string", "format": "date" },
  "content_type": { "type": "string", "enum": ["art_still"] },
  "availability_type": { "type": "string", "enum": ["on_demand"] },
  "files": {
    "type": "object",
    "properties": {
      "cover_image": { "type": "string", "format": "uri" },
      "thumbnail": { "type": "string", "format": "uri" },
      "preview_url": { "type": "string", "format": "uri" }
    },
    "required": ["cover_image", "preview_url"]
  },
  "ipfs_audio_url": { "type": "string", "format": "uri" },
  "mlc_metadata": {
    "type": "object",
    "properties": {
      "work_title": { "type": "string", "minLength": 1 },
      "iswc": { "type": "string", "pattern": "^[A-Z]-[0-9]{3}\\.[0-9]{3}\\.[0-9]{3}-[0-9]$" },
      "isrc": { "type": "string", "pattern": "^[A-Z]{2}-[A-Z0-9]{3}-[0-9]{2}-[0-9]{5}$" },
      "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
      "writers": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["name", "role", "ipi_name_number", "ownership_percent"],
          "properties": {
            "name": { "type": "string", "minLength": 1 },
            "role": { "type": "string" },
```

```
            "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
            "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
          }
        }
      },
      "publishers": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["name", "ipi_name_number", "ownership_percent"],
          "properties": {
            "name": { "type": "string", "minLength": 1 },
            "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
            "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
          }
        }
      }
    },
    "required": ["work_title", "writers"]
  },
  "integrityHashes": {
    "type": "object",
    "properties": {
      "sha256Audio": { "type": "string" },
      "sha256CoverImage": { "type": "string" },
      "sha256Metadata": { "type": "string" }
    },
    "required": ["sha256Audio", "sha256CoverImage", "sha256Metadata"]
  }
 }
}
```

`art_animated_schema.json` (3.2 KB)

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "@context": "https://schema.org",
  "@type": "CreativeWork",
  "title": "AnimatedArtMetadata",
  "type": "object",
  "required": ["id", "title", "description", "creator", "content_type", "availability_type", "release_
  "properties": {
    "id": { "type": "string" },
    "title": { "type": "string", "minLength": 1 },
    "description": { "type": "string", "minLength": 10 },
```

```
"creator": {
  "type": "object",
  "properties": {
    "name": { "type": "string", "minLength": 1 },
    "user_id": { "type": "string" },
    "wallet_address": { "type": "string" }
  },
  "required": ["name", "user_id"]
},
"tags": {
  "type": "array",
  "items": { "type": "string", "minLength": 1 },
  "minItems": 5
},
"release_date": { "type": "string", "format": "date" },
"content_type": { "type": "string", "enum": ["art_animated"] },
"availability_type": { "type": "string", "enum": ["on_demand", "live"] },
"files": {
  "type": "object",
  "properties": {
    "cover_image": { "type": "string", "format": "uri" },
    "thumbnail": { "type": "string", "format": "uri" },
    "preview_url": { "type": "string", "format": "uri" }
  },
  "required": ["cover_image", "preview_url"]
},
"duration": { "type": "number", "minimum": 60, "maximum": 900 },
"ipfs_audio_url": { "type": "string", "format": "uri" },
"mlc_metadata": {
  "type": "object",
  "properties": {
    "work_title": { "type": "string", "minLength": 1 },
    "iswc": { "type": "string", "pattern": "^[A-Z]-[0-9]{3}\\.[0-9]{3}\\.[0-9]{3}-[0-9]$" },
    "isrc": { "type": "string", "pattern": "^[A-Z]{2}-[A-Z0-9]{3}-[0-9]{2}-[0-9]{5}$" },
    "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
    "writers": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["name", "role", "ipi_name_number", "ownership_percent"],
        "properties": {
          "name": { "type": "string", "minLength": 1 },
          "role": { "type": "string" },
          "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
          "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
        }
      }
```

```
        },
        "publishers": {
          "type": "array",
          "items": {
            "type": "object",
            "required": ["name", "ipi_name_number", "ownership_percent"],
            "properties": {
              "name": { "type": "string", "minLength": 1 },
              "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
              "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
            }
          }
        }
      },
      "required": ["work_title", "writers"]
    },
    "integrityHashes": {
      "type": "object",
      "properties": {
        "sha256Audio": { "type": "string" },
        "sha256CoverImage": { "type": "string" },
        "sha256Metadata": { "type": "string" }
      },
      "required": ["sha256Audio", "sha256CoverImage", "sha256Metadata"]
    }
  }
}
```

`podcast_schema.json` (3.2 KB)

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "@context": "https://schema.org",
  "@type": "CreativeWork",
  "title": "PodcastMetadata",
  "type": "object",
  "required": ["id", "title", "description", "creator", "content_type", "availability_type", "release_
  "properties": {
    "id": { "type": "string" },
    "title": { "type": "string", "minLength": 1 },
    "description": { "type": "string", "minLength": 10 },
    "creator": {
      "type": "object",
      "properties": {
        "name": { "type": "string", "minLength": 1 },
```

```
    "user_id": { "type": "string" },
    "wallet_address": { "type": "string" }
  },
  "required": ["name", "user_id"]
},
"tags": {
  "type": "array",
  "items": { "type": "string", "minLength": 1 },
  "minItems": 5
},
"release_date": { "type": "string", "format": "date" },
"content_type": { "type": "string", "enum": ["podcast"] },
"availability_type": { "type": "string", "enum": ["on_demand", "live"] },
"files": {
  "type": "object",
  "properties": {
    "cover_image": { "type": "string", "format": "uri" },
    "thumbnail": { "type": "string", "format": "uri" },
    "preview_url": { "type": "string", "format": "uri" }
  },
  "required": ["cover_image", "preview_url"]
},
"duration": { "type": "number", "minimum": 60, "maximum": 900 },
"ipfs_audio_url": { "type": "string", "format": "uri" },
"mlc_metadata": {
  "type": "object",
  "properties": {
    "work_title": { "type": "string", "minLength": 1 },
    "iswc": { "type": "string", "pattern": "^[A-Z]-[0-9]{3}\\.[0-9]{3}\\.[0-9]{3}-[0-9]$" },
    "isrc": { "type": "string", "pattern": "^[A-Z]{2}-[A-Z0-9]{3}-[0-9]{2}-[0-9]{5}$" },
    "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
    "writers": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["name", "role", "ipi_name_number", "ownership_percent"],
        "properties": {
          "name": { "type": "string", "minLength": 1 },
          "role": { "type": "string" },
          "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
          "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
        }
      }
    },
    "publishers": {
      "type": "array",
      "items": {
```

```
            "type": "object",
            "required": ["name", "ipi_name_number", "ownership_percent"],
            "properties": {
              "name": { "type": "string", "minLength": 1 },
              "ipi_name_number": { "type": "string", "pattern": "^[0-9]{9,11}$" },
              "ownership_percent": { "type": "number", "minimum": 0, "maximum": 100 }
            }
          }
        }
      },
      "required": ["work_title", "writers"]
    },
    "integrityHashes": {
      "type": "object",
      "properties": {
        "sha256Audio": { "type": "string" },
        "sha256CoverImage": { "type": "string" },
        "sha256Metadata": { "type": "string" }
      },
      "required": ["sha256Audio", "sha256CoverImage", "sha256Metadata"]
    }
  }
}
```

`music_media_rollup.json` (90.5 KB)

```json
{
    "mediaId": "279e0635-a783-4933-b478-400db725b289",
    "productType": "MAIN",
    "initialStreamingType": "LIVE",
    "presentations": [
        {
            "id": "9e389269-58d2-3841-b1b1-6b5d52fb9b06",
            "facet": {
                "label": "default"
            },
            "audio": {
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ],
                "audioGroups": [
```

```
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-cbcs-all-sliding",
        "streamingFormat": "HLS",
        "playlistType": "SLIDING",
        "bandwidth": {
            "max": 9798000,
            "min": 448000,
            "default": 1200000,
            "averageMin": 302000,
            "averageMax": 8426000
        },
        "iframeSupported": false,
        "streams": [
            {
                "bandwidth": 448000,
```

```
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
```

```
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 7548000,
            "averageBandwidth": 5526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
```

```
            {
                "bandwidth": 8398000,
                "averageBandwidth": 6926000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 9798000,
                "averageBandwidth": 8426000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "6b4defd1-f993-37ca-a431-ffca6e44f3ba",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
```

```
                    "code": "en",
                    "name": "English",
                    "role": "PRIMARY"
                }
            ]
        }
    ],
    "audioTypes": [
        "STEREO"
    ],
    "containerFormats": [
        "FMP4"
    ],
    "codecFormats": [
        "AAC"
    ],
    "muxed": false
},
"closedCaptions": {
    "languages": [
        {
            "code": "en",
            "name": "English [Lyrics]",
            "instreamId": "CC1"
        }
    ]
},
"streamingType": "LIVE",
"playlist": {
    "description": "audio-cbcs-minimum-sliding",
    "streamingFormat": "HLS",
    "playlistType": "SLIDING",
    "bandwidth": {
        "max": 3648000,
        "min": 448000,
        "default": 800000,
        "averageMin": 302000,
        "averageMax": 2526000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
```

```
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 653000,
        "averageBandwidth": 482000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1188000,
        "averageBandwidth": 842000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1728000,
        "averageBandwidth": 1202000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 2588000,
        "averageBandwidth": 1812000,
        "audioType": "STEREO",
        "audioChannels": "2",
```

```
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 3648000,
                "averageBandwidth": 2526000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "2f5e137a-d94f-3613-b9e8-e1f43f502c02",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
```

```
        }
    ],
    "audioTypes": [
        "STEREO"
    ],
    "containerFormats": [
        "FMP4"
    ],
    "codecFormats": [
        "AAC"
    ],
    "muxed": false
},
"closedCaptions": {
    "languages": [
        {
            "code": "en",
            "name": "English [Lyrics]",
            "instreamId": "CC1"
        }
    ]
},
"streamingType": "LIVE",
"playlist": {
    "description": "audio-cbcs-all-complete",
    "streamingFormat": "HLS",
    "playlistType": "COMPLETE",
    "bandwidth": {
        "max": 9798000,
        "min": 448000,
        "default": 1200000,
        "averageMin": 302000,
        "averageMax": 8426000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
```

```
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
```

```
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 7548000,
            "averageBandwidth": 5526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 8398000,
            "averageBandwidth": 6926000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
```

```
                    "AAC"
                ]
            },
            {
                "bandwidth": 9798000,
                "averageBandwidth": 8426000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "3a1e46f6-2789-3fcb-a038-0b248f6957e7",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
```

```
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-cbcs-minimum-complete",
        "streamingFormat": "HLS",
        "playlistType": "COMPLETE",
        "bandwidth": {
            "max": 3648000,
            "min": 448000,
            "default": 800000,
            "averageMin": 302000,
            "averageMax": 2526000
        },
        "iframeSupported": false,
        "streams": [
            {
                "bandwidth": 448000,
                "averageBandwidth": 302000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 653000,
                "averageBandwidth": 482000,
```

```
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
```

```
                "averageBandwidth": 2526000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "b5266599-ee44-31de-979d-1c13f217706d",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
```

```
        "AAC"
    ],
    "muxed": false
},
"closedCaptions": {
    "languages": [
        {
            "code": "en",
            "name": "English [Lyrics]",
            "instreamId": "CC1"
        }
    ]
},
"streamingType": "LIVE",
"playlist": {
    "description": "audio-cbcs-all-sliding",
    "streamingFormat": "HLS",
    "playlistType": "SLIDING",
    "bandwidth": {
        "max": 9798000,
        "min": 448000,
        "default": 1200000,
        "averageMin": 302000,
        "averageMax": 8426000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
```

```
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
```

```
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 7548000,
            "averageBandwidth": 5526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 8398000,
            "averageBandwidth": 6926000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 9798000,
            "averageBandwidth": 8426000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
```

```
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "e7b54752-2971-33d9-971a-1e79f261c668",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
```

```
"closedCaptions": {
    "languages": [
        {
            "code": "en",
            "name": "English [Lyrics]",
            "instreamId": "CC1"
        }
    ]
},
"streamingType": "LIVE",
"playlist": {
    "description": "audio-cbcs-minimum-sliding",
    "streamingFormat": "HLS",
    "playlistType": "SLIDING",
    "bandwidth": {
        "max": 3648000,
        "min": 448000,
        "default": 800000,
        "averageMin": 302000,
        "averageMax": 2526000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
```

```
{
    "bandwidth": 1188000,
    "averageBandwidth": 842000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
},
{
    "bandwidth": 1728000,
    "averageBandwidth": 1202000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
},
{
    "bandwidth": 2588000,
    "averageBandwidth": 1812000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
},
{
    "bandwidth": 3648000,
    "averageBandwidth": 2526000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
```

```
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "d42b07a9-1470-382e-8bd4-c55ffc104c17",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
```

```
            "instreamId": "CC1"
        }
    ]
},
"streamingType": "LIVE",
"playlist": {
    "description": "audio-cbcs-all-complete",
    "streamingFormat": "HLS",
    "playlistType": "COMPLETE",
    "bandwidth": {
        "max": 9798000,
        "min": 448000,
        "default": 1200000,
        "averageMin": 302000,
        "averageMax": 8426000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
```

```
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
```

```
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 7548000,
            "averageBandwidth": 5526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 8398000,
            "averageBandwidth": 6926000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 9798000,
            "averageBandwidth": 8426000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        }
    ]
},
"state": "ON",
```

```
        "createdAt": "1970-01-01T00:00:00Z"
    },
    {
        "id": "799598e9-caea-3cfc-9126-32d5e58f4130",
        "facet": {
            "label": "default"
        },
        "audio": {
            "languages": [
                {
                    "code": "en",
                    "name": "English",
                    "role": "PRIMARY"
                }
            ],
            "audioGroups": [
                {
                    "audioType": "STEREO",
                    "languages": [
                        {
                            "code": "en",
                            "name": "English",
                            "role": "PRIMARY"
                        }
                    ]
                }
            ],
            "audioTypes": [
                "STEREO"
            ],
            "containerFormats": [
                "FMP4"
            ],
            "codecFormats": [
                "AAC"
            ],
            "muxed": false
        },
        "closedCaptions": {
            "languages": [
                {
                    "code": "en",
                    "name": "English [Lyrics]",
                    "instreamId": "CC1"
                }
            ]
        },
```

```
"streamingType": "LIVE",
"playlist": {
    "description": "audio-cbcs-minimum-complete",
    "streamingFormat": "HLS",
    "playlistType": "COMPLETE",
    "bandwidth": {
        "max": 3648000,
        "min": 448000,
        "default": 800000,
        "averageMin": 302000,
        "averageMax": 2526000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
```

```
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        }
    ]
},
"state": "ON",
"createdAt": "1970-01-01T00:00:00Z"
},
{
"id": "81e0d4b5-1298-34c9-97ab-ba744117ebff",
"facet": {
```

```
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-ctr-all-sliding",
        "streamingFormat": "HLS",
        "playlistType": "SLIDING",
```

```
"bandwidth": {
    "max": 9798000,
    "min": 448000,
    "default": 1200000,
    "averageMin": 302000,
    "averageMax": 8426000
},
"iframeSupported": false,
"streams": [
    {
        "bandwidth": 448000,
        "averageBandwidth": 302000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 653000,
        "averageBandwidth": 482000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1188000,
        "averageBandwidth": 842000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1728000,
```

```
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
```

```
                "bandwidth": 7548000,
                "averageBandwidth": 5526000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 8398000,
                "averageBandwidth": 6926000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 9798000,
                "averageBandwidth": 8426000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "50f035fc-1f29-3be2-9042-5fc87a0ed995",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
```

```
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-ctr-minimum-sliding",
        "streamingFormat": "HLS",
        "playlistType": "SLIDING",
        "bandwidth": {
            "max": 3648000,
            "min": 448000,
            "default": 800000,
```

```
        "averageMin": 302000,
        "averageMax": 2526000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
```

```
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 2588000,
                "averageBandwidth": 1812000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 3648000,
                "averageBandwidth": 2526000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "8c68e8e1-1fef-36eb-ba57-8bf0b83c71ba",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
```

```
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-ctr-all-complete",
        "streamingFormat": "HLS",
        "playlistType": "COMPLETE",
        "bandwidth": {
            "max": 9798000,
            "min": 448000,
            "default": 1200000,
            "averageMin": 302000,
            "averageMax": 8426000
        },
        "iframeSupported": false,
        "streams": [
```

```
{
    "bandwidth": 448000,
    "averageBandwidth": 302000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
},
{
    "bandwidth": 653000,
    "averageBandwidth": 482000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
},
{
    "bandwidth": 1188000,
    "averageBandwidth": 842000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
},
{
    "bandwidth": 1728000,
    "averageBandwidth": 1202000,
    "audioType": "STEREO",
    "audioChannels": "2",
    "audioCodecs": [
        "mp4a.40.2"
    ],
    "audioCodecFormats": [
        "AAC"
    ]
```

```
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 7548000,
            "averageBandwidth": 5526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
```

```
                ]
            },
            {
                "bandwidth": 8398000,
                "averageBandwidth": 6926000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 9798000,
                "averageBandwidth": 8426000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "306870ed-116d-3148-8e72-b2ad907bb925",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
```

```
            "languages": [
                {
                    "code": "en",
                    "name": "English",
                    "role": "PRIMARY"
                }
            ]
        }
    ],
    "audioTypes": [
        "STEREO"
    ],
    "containerFormats": [
        "FMP4"
    ],
    "codecFormats": [
        "AAC"
    ],
    "muxed": false
},
"closedCaptions": {
    "languages": [
        {
            "code": "en",
            "name": "English [Lyrics]",
            "instreamId": "CC1"
        }
    ]
},
"streamingType": "LIVE",
"playlist": {
    "description": "audio-ctr-minimum-complete",
    "streamingFormat": "HLS",
    "playlistType": "COMPLETE",
    "bandwidth": {
        "max": 3648000,
        "min": 448000,
        "default": 800000,
        "averageMin": 302000,
        "averageMax": 2526000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
```

```
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
```

```
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 3648000,
                "averageBandwidth": 2526000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "da110f04-906f-3f56-bfdf-6f7e8aa936fa",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
```

```
                }
            ]
        }
    ],
    "audioTypes": [
        "STEREO"
    ],
    "containerFormats": [
        "FMP4"
    ],
    "codecFormats": [
        "AAC"
    ],
    "muxed": false
},
"closedCaptions": {
    "languages": [
        {
            "code": "en",
            "name": "English [Lyrics]",
            "instreamId": "CC1"
        }
    ]
},
"streamingType": "LIVE",
"playlist": {
    "description": "audio-ctr-all-sliding",
    "streamingFormat": "HLS",
    "playlistType": "SLIDING",
    "bandwidth": {
        "max": 9798000,
        "min": 448000,
        "default": 1200000,
        "averageMin": 302000,
        "averageMax": 8426000
    },
    "iframeSupported": false,
    "streams": [
        {
            "bandwidth": 448000,
            "averageBandwidth": 302000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
```

```
            "AAC"
        ]
    },
    {
        "bandwidth": 653000,
        "averageBandwidth": 482000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1188000,
        "averageBandwidth": 842000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1728000,
        "averageBandwidth": 1202000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 2588000,
        "averageBandwidth": 1812000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
```

```
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 7548000,
            "averageBandwidth": 5526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 8398000,
            "averageBandwidth": 6926000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
```

```
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 9798000,
                "averageBandwidth": 8426000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "9686a5dd-2753-30d3-acab-26eb597897f5",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
```

```
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-ctr-minimum-sliding",
        "streamingFormat": "HLS",
        "playlistType": "SLIDING",
        "bandwidth": {
            "max": 3648000,
            "min": 448000,
            "default": 800000,
            "averageMin": 302000,
            "averageMax": 2526000
        },
        "iframeSupported": false,
        "streams": [
            {
                "bandwidth": 448000,
                "averageBandwidth": 302000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
```

```
            "bandwidth": 653000,
            "averageBandwidth": 482000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
```

```
            {
                "bandwidth": 3648000,
                "averageBandwidth": 2526000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "02dacb36-12a8-3fcc-9773-50eb5f4b766d",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
```

```
        ],
        "codecFormats": [
            "AAC"
        ],
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-ctr-all-complete",
        "streamingFormat": "HLS",
        "playlistType": "COMPLETE",
        "bandwidth": {
            "max": 9798000,
            "min": 448000,
            "default": 1200000,
            "averageMin": 302000,
            "averageMax": 8426000
        },
        "iframeSupported": false,
        "streams": [
            {
                "bandwidth": 448000,
                "averageBandwidth": 302000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 653000,
                "averageBandwidth": 482000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
```

```
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1188000,
        "averageBandwidth": 842000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 1728000,
        "averageBandwidth": 1202000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 2588000,
        "averageBandwidth": 1812000,
        "audioType": "STEREO",
        "audioChannels": "2",
        "audioCodecs": [
            "mp4a.40.2"
        ],
        "audioCodecFormats": [
            "AAC"
        ]
    },
    {
        "bandwidth": 3648000,
        "averageBandwidth": 2526000,
        "audioType": "STEREO",
        "audioChannels": "2",
```

```
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 4948000,
            "averageBandwidth": 3526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 7548000,
            "averageBandwidth": 5526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 8398000,
            "averageBandwidth": 6926000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 9798000,
            "averageBandwidth": 8426000,
            "audioType": "STEREO",
```

```
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            }
        ]
    },
    "state": "ON",
    "createdAt": "1970-01-01T00:00:00Z"
},
{
    "id": "ec003f90-dae4-35c7-8e4a-02007076bab8",
    "facet": {
        "label": "default"
    },
    "audio": {
        "languages": [
            {
                "code": "en",
                "name": "English",
                "role": "PRIMARY"
            }
        ],
        "audioGroups": [
            {
                "audioType": "STEREO",
                "languages": [
                    {
                        "code": "en",
                        "name": "English",
                        "role": "PRIMARY"
                    }
                ]
            }
        ],
        "audioTypes": [
            "STEREO"
        ],
        "containerFormats": [
            "FMP4"
        ],
        "codecFormats": [
            "AAC"
        ],
```

```
        "muxed": false
    },
    "closedCaptions": {
        "languages": [
            {
                "code": "en",
                "name": "English [Lyrics]",
                "instreamId": "CC1"
            }
        ]
    },
    "streamingType": "LIVE",
    "playlist": {
        "description": "audio-ctr-minimum-complete",
        "streamingFormat": "HLS",
        "playlistType": "COMPLETE",
        "bandwidth": {
            "max": 3648000,
            "min": 448000,
            "default": 800000,
            "averageMin": 302000,
            "averageMax": 2526000
        },
        "iframeSupported": false,
        "streams": [
            {
                "bandwidth": 448000,
                "averageBandwidth": 302000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
                ]
            },
            {
                "bandwidth": 653000,
                "averageBandwidth": 482000,
                "audioType": "STEREO",
                "audioChannels": "2",
                "audioCodecs": [
                    "mp4a.40.2"
                ],
                "audioCodecFormats": [
                    "AAC"
```

```
            ]
        },
        {
            "bandwidth": 1188000,
            "averageBandwidth": 842000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 1728000,
            "averageBandwidth": 1202000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 2588000,
            "averageBandwidth": 1812000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
                "AAC"
            ]
        },
        {
            "bandwidth": 3648000,
            "averageBandwidth": 2526000,
            "audioType": "STEREO",
            "audioChannels": "2",
            "audioCodecs": [
                "mp4a.40.2"
            ],
            "audioCodecFormats": [
```

```
                            "AAC"
                        ]
                    }
                ]
            },
            "state": "ON",
            "createdAt": "1970-01-01T00:00:00Z"
        }
    ],
    "state": "ON",
    "version": "5.0",
    "createdAt": "2024-09-07T09:55:07.512878969Z",
    "updatedAt": "2024-09-07T09:55:07.629404595Z"
}
```


`dj_sets.json` (0.0 KB)

```json

```

`manifest.json` (0.4 KB)

```json
{
  "name": "Michie Stream Platform",
  "short_name": "MSP",
  "description": "Blockchain music and NFT streaming platform",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#08080A",
  "theme_color": "#F5F000",
  "icons": [
    { "src": "assets/msp-vinyl-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "assets/msp-vinyl-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

INFRASTRUCTURE

`nginx.conf` (19.3 KB)

```nginx
# ═══════════════════════════════════════════════════════════════════════
```

```
#  MSP — Production Nginx Configuration
#
#  Domains:
#    michie.com / www.michie.com  →  main app (API + static files + live HLS)
#    stream.michie.com            →  catalog HLS only (pure Nginx, no Node touch)
#
#  Ports:
#    :80    HTTP → redirect HTTPS
#    :443   HTTPS main app
#    :1935  RTMP ingest (OBS / Larix / hardware encoders)
#
#  Path layout:
#    /var/www/msp/public/          ← static files
#    /var/www/msp/live/{id}/       ← live HLS segments (GStreamer writes here)
#    /var/www/msp/streams/{cid}/   ← catalog HLS (PRODUCTION transcode output)
#    /var/www/msp/certs/           ← TLS certificates
#    /var/www/msp/scripts/         ← gst-transcode.sh etc.
#
#  Install:
#    sudo apt install nginx libnginx-mod-rtmp
#    sudo cp nginx.conf /etc/nginx/nginx.conf
#    sudo nginx -t && sudo systemctl reload nginx
# ═══════════════════════════════════════════════════════════════════════

load_module modules/ngx_rtmp_module.so;

worker_processes  auto;
worker_rlimit_nofile 65535;
error_log  /var/log/nginx/error.log warn;
pid        /run/nginx.pid;

events {
    worker_connections  8192;
    multi_accept        on;
    use                 epoll;
}

# ───────────────────────────────────────────────────────────────────────
#  RTMP  —  Live stream ingest
#  OBS / Larix / Streamlabs push to rtmp://stream.michie.com:1935/live
# ───────────────────────────────────────────────────────────────────────
rtmp {
    server {
        listen 1935;
        listen [::]:1935 ipv6only=on;
        chunk_size  4096;
        max_connections 512;
```

```
        buflen 500ms;

        # ── PRODUCTION application ──────────────────────────────────────
        application live {
            live on;
            record off;

            # Validate stream key BEFORE accepting publish
            on_publish      http://127.0.0.1:3001/api/rtmp-auth;
            on_done         http://127.0.0.1:3001/api/rtmp-done;
            on_publish_done http://127.0.0.1:3001/api/rtmp-done;

            # Deny viewers from pulling RTMP directly — viewers use HLS
            deny play all;

            # exec_publish calls gst-transcode.sh which:
            #   1. Hits /api/rtmp-publish to get sessionId + mode
            #   2. Launches GStreamer with appropriate pipeline (PRODUCTION or SOCIAL)
            #   3. Writes HLS to /var/www/msp/live/{sessionId}/
            exec_publish      /var/www/msp/scripts/gst-transcode.sh $name;
            exec_publish_done /var/www/msp/scripts/gst-transcode-done.sh $name;
        }

        # ── SOCIAL application (opt-in low-latency path) ────────────────
        # Creators can push to rtmp://host:1935/social/key for bare-bones mode
        application social {
            live on;
            record off;
            on_publish      http://127.0.0.1:3001/api/rtmp-auth;
            on_done         http://127.0.0.1:3001/api/rtmp-done;
            on_publish_done http://127.0.0.1:3001/api/rtmp-done;
            deny play all;
            exec_publish      /var/www/msp/scripts/gst-transcode.sh $name social;
            exec_publish_done /var/www/msp/scripts/gst-transcode-done.sh $name;
        }
    }
}

http {
    include      /etc/nginx/mime.types;
    default_type application/octet-stream;

    # ── Logging ──────────────────────────────────────────────────────
    log_format msp  '$remote_addr "$request" $status $body_bytes_sent '
                    '"$http_referer" rt=$request_time ua="$http_user_agent"';
    access_log /var/log/nginx/msp_access.log msp buffer=16k flush=5s;
```

```
# ── Core performance ──────────────────────────────────────────
sendfile           on;
tcp_nopush         on;
tcp_nodelay        on;
keepalive_timeout  65;
keepalive_requests 2000;
server_tokens      off;

# ── Gzip ──────────────────────────────────────────────────────
gzip on;
gzip_comp_level 4;
gzip_min_length 512;
gzip_vary on;
gzip_types
    text/plain text/css text/javascript text/xml
    application/javascript application/json application/xml
    application/vnd.apple.mpegurl;
gzip_disable "msie6";

# ── Limits ────────────────────────────────────────────────────
client_max_body_size    600m;
client_body_buffer_size 128k;
client_body_timeout     300s;
client_header_timeout   30s;
send_timeout            300s;

# ── Proxy defaults ────────────────────────────────────────────
proxy_http_version  1.1;
proxy_set_header    Host              $host;
proxy_set_header    X-Real-IP         $remote_addr;
proxy_set_header    X-Forwarded-For   $proxy_add_x_forwarded_for;
proxy_set_header    X-Forwarded-Proto $scheme;
proxy_connect_timeout  10s;
proxy_send_timeout     300s;
proxy_read_timeout     300s;
proxy_buffering        off;

# ── Rate limit zones ──────────────────────────────────────────
limit_req_zone  $binary_remote_addr  zone=api:10m       rate=30r/s;
limit_req_zone  $binary_remote_addr  zone=upload:5m     rate=2r/s;
limit_req_zone  $binary_remote_addr  zone=rtmhook:5m    rate=10r/s;
limit_req_zone  $binary_remote_addr  zone=stream_dl:20m rate=120r/s;

# ── Upstream ──────────────────────────────────────────────────
upstream msp_node {
    server 127.0.0.1:3001;
    keepalive 64;
```

```
}

# ── HLS MIME type map (not always in default mime.types) ──────────────
types {
    application/vnd.apple.mpegurl  m3u8;
    video/mp2t                     ts;
}

# ────────────────────────────────────────────────────────────────────
#  HTTP → HTTPS redirect  (all domains)
# ────────────────────────────────────────────────────────────────────
server {
    listen      80 default_server;
    listen      [::]:80 default_server;
    server_name _;
    return 301  https://$host$request_uri;
}

# ────────────────────────────────────────────────────────────────────
#  stream.michie.com  —  CATALOG HLS SERVER
#
#  Pure Nginx. Serves /var/www/msp/streams/{cid}/ directly.
#  Node.js is never touched. This is the highest-throughput path.
#
#  URL pattern:
#    https://stream.michie.com/{cid}/master.m3u8
#    https://stream.michie.com/{cid}/720p.m3u8
#    https://stream.michie.com/{cid}/720p_00001.ts
#    https://stream.michie.com/{cid}/thumb.jpg
# ────────────────────────────────────────────────────────────────────
server {
    listen      443 ssl http2;
    listen      [::]:443 ssl http2;
    server_name stream.michie.com;

    ssl_certificate     /var/www/msp/certs/stream_fullchain.pem;
    ssl_certificate_key /var/www/msp/certs/stream_privkey.pem;
    ssl_session_cache   shared:MozSSL:10m;
    ssl_session_timeout 1d;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-CHAC
    ssl_prefer_server_ciphers off;

    add_header X-Content-Type-Options nosniff always;

    # ── CID-based HLS serving ──────────────────────────────────────
    location ~* ^/([A-Za-z0-9]+)/ {
```

```
        root /var/www/msp/streams;

        # CORS — HLS.js needs this from any origin
        add_header Access-Control-Allow-Origin  * always;
        add_header Access-Control-Allow-Methods "GET, OPTIONS" always;
        add_header Access-Control-Allow-Headers "Range" always;

        # Playlist — never cache (viewer must get fresh segment list)
        location ~* \.m3u8$ {
            add_header Cache-Control "no-cache, no-store, must-revalidate";
            add_header Pragma        no-cache;
            add_header Expires       0;
            types { application/vnd.apple.mpegurl m3u8; }
            limit_req zone=stream_dl burst=200 nodelay;
        }

        # Segments — immutable once written, cache 60s
        location ~* \.ts$ {
            add_header Cache-Control "public, max-age=60, immutable";
            types { video/mp2t ts; }
            limit_req zone=stream_dl burst=500 nodelay;
        }

        # Thumbnail
        location ~* thumb\.jpg$ {
            add_header Cache-Control "public, max-age=30";
        }

        # If CID directory doesn't exist yet, return 404 JSON (not HTML)
        try_files $uri =404;
    }

    # ── Ready check  —  fast HEAD check ──────────────────────────────
    # Nginx returns 200 if the master.m3u8 exists on disk,
    # 404 if transcode not yet complete. Avoids Node round-trip.
    location ~* ^/ready/([A-Za-z0-9]+)$ {
        set $cid $1;
        try_files /var/www/msp/streams/$cid/master.m3u8 =404;
        add_header Content-Type application/json;
        return 200 '{"ready":true}';
    }

    # No other routes on this subdomain
    location / { return 404; }
}

# ────────────────────────────────────────────────────────────────────
```

```
#  michie.com  —  MAIN APP SERVER
# ──────────────────────────────────────────────────────────────────────
server {
    listen      443 ssl http2 default_server;
    listen      [::]:443 ssl http2 default_server;
    server_name michie.com www.michie.com;

    ssl_certificate     /var/www/msp/certs/fullchain.pem;
    ssl_certificate_key /var/www/msp/certs/privkey.pem;
    ssl_session_cache   shared:MozSSL:10m;
    ssl_session_timeout 1d;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-CHAC
    ssl_prefer_server_ciphers off;

    add_header X-Frame-Options        SAMEORIGIN     always;
    add_header X-Content-Type-Options nosniff        always;
    add_header Referrer-Policy        "strict-origin-when-cross-origin" always;

    # ── LIVE HLS  (served directly — no Node) ──────────────────────────
    # Written by GStreamer to /var/www/msp/live/{sessionId}/
    # Viewers get 1-2s latency for SOCIAL mode, ~10s for PRODUCTION.
    location /live/ {
        root /var/www/msp;
        add_header Access-Control-Allow-Origin  * always;
        add_header Access-Control-Allow-Methods "GET, OPTIONS" always;

        location ~* \.m3u8$ {
            add_header Cache-Control "no-cache, no-store, must-revalidate";
            add_header Pragma no-cache;
            add_header Expires 0;
            types { application/vnd.apple.mpegurl m3u8; }
        }
        location ~* \.ts$ {
            add_header Cache-Control "public, max-age=5";
            types { video/mp2t ts; }
        }
        location ~* thumb\.jpg$ {
            add_header Cache-Control "public, max-age=5";
        }

        try_files $uri =404;
    }

    # ── CATALOG HLS  (alias to streams directory) ──────────────────────
    # Also accessible at michie.com/streams/{cid}/master.m3u8
    # (stream.michie.com is the preferred CDN-friendly URL)
```

```
location /streams/ {
    alias /var/www/msp/streams/;
    add_header Access-Control-Allow-Origin  * always;
    add_header Access-Control-Allow-Methods "GET, OPTIONS" always;

    location ~* \.m3u8$ {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
        types { application/vnd.apple.mpegurl m3u8; }
    }
    location ~* \.ts$ {
        add_header Cache-Control "public, max-age=60, immutable";
        types { video/mp2t ts; }
    }
}

# ── Static files ──────────────────────────────────────────────
location / {
    root /var/www/msp/public;
    try_files $uri $uri/ @node;

    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
        access_log off;
    }
}

# ── WebSocket ─────────────────────────────────────────────────
location /ws {
    proxy_pass         http://msp_node;
    proxy_set_header   Upgrade    $http_upgrade;
    proxy_set_header   Connection "upgrade";
    proxy_read_timeout 86400s;
    proxy_send_timeout 86400s;
}

# ── Upload (large body, low rate) ────────────────────────────
location /api/upload {
    proxy_pass              http://msp_node;
    client_max_body_size    600m;
    client_body_timeout     600s;
    limit_req               zone=upload burst=3 nodelay;
    proxy_read_timeout      600s;
}

# ── Live ingest chunks ────────────────────────────────────────
location ~ ^/api/live-ingest/ {
```

```
        proxy_pass            http://msp_node;
        client_max_body_size  15m;
        proxy_read_timeout    30s;
        proxy_buffering       off;
    }

    # ── RTMP auth callbacks (internal webhook, strict rate limit) ─────
    location ~ ^/api/rtmp {
        proxy_pass http://msp_node;
        limit_req  zone=rtmhook burst=30 nodelay;
        # Only allow from localhost and known CDN IPs
        allow 127.0.0.1;
        allow ::1;
        deny  all;
    }

    # ── General API ──────────────────────────────────────────────────
    location /api/ {
        proxy_pass http://msp_node;
        limit_req  zone=api burst=80 nodelay;
    }

    # ── Node fallback (SPA) ──────────────────────────────────────────
    location @node {
        proxy_pass http://msp_node;
    }

    # ── Nginx RTMP stats (localhost only) ────────────────────────────
    location /rtmp-stat {
        rtmp_stat all;
        allow 127.0.0.1; deny all;
    }
    location /rtmp-control {
        rtmp_control all;
        allow 127.0.0.1; deny all;
    }
}

# ────────────────────────────────────────────────────────────────────
#  Local dev server (no TLS — for development without certificates)
# ────────────────────────────────────────────────────────────────────
server {
    listen 8080;
    server_name localhost 127.0.0.1;

    location /nginx-health { return 200 "ok\n"; add_header Content-Type text/plain; access_log off
```

```
        location /live/ {
            root /var/www/msp;
            add_header Access-Control-Allow-Origin *;
            types { application/vnd.apple.mpegurl m3u8; video/mp2t ts; }
            location ~* \.m3u8$ { add_header Cache-Control no-cache; }
            location ~* \.ts$   { add_header Cache-Control "max-age=5"; }
        }

        location /streams/ {
            alias /var/www/msp/streams/;
            add_header Access-Control-Allow-Origin *;
            types { application/vnd.apple.mpegurl m3u8; video/mp2t ts; }
            location ~* \.m3u8$ { add_header Cache-Control no-cache; }
            location ~* \.ts$   { add_header Cache-Control "max-age=60"; }
        }

        location /api/upload { proxy_pass http://msp_node; client_max_body_size 600m; proxy_read_timeo
        location ~ ^/api/live-ingest/ { proxy_pass http://msp_node; client_max_body_size 15m; proxy_bu
        location /api/  { proxy_pass http://msp_node; }
        location /ws    {
            proxy_pass http://msp_node;
            proxy_set_header Upgrade    $http_upgrade;
            proxy_set_header Connection "upgrade";
            proxy_read_timeout 86400s;
        }
        location / {
            root /var/www/msp/public;
            try_files $uri $uri/ @node_dev;
        }
        location @node_dev { proxy_pass http://msp_node; }
    }
}
```

`install-media.sh` (5.6 KB)

```bash
#!/usr/bin/env bash
# ═══════════════════════════════════════════════════════════════════════
#  MSP — Media Infrastructure Install Script
#  sudo bash install-media.sh
#  Tested: Ubuntu 22.04 LTS, Ubuntu 24.04 LTS, Debian 12
# ═══════════════════════════════════════════════════════════════════════
set -euo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; BLUE='\033[0;34m'; NC='\033[0m'
info()  { echo -e "${BLUE}[INFO]${NC}  $*"; }
```

```
ok()    { echo -e "${GREEN}[OK]${NC}    $*"; }
warn()  { echo -e "${YELLOW}[WARN]${NC}  $*"; }
die()   { echo -e "${RED}[FAIL]${NC}  $*"; exit 1; }

[[ $EUID -ne 0 ]] && die "Run as root: sudo bash $0"
SRC="$(cd "$(dirname "$0")" && pwd)"

info "=== MSP Media Infrastructure Install ==="

# 1. Nginx + RTMP module
info "Installing Nginx + nginx-rtmp-module..."
apt-get update -qq
apt-get install -y nginx libnginx-mod-rtmp curl python3
ok "Nginx $(nginx -v 2>&1 | grep -oP '[\d.]+')"

# 2. GStreamer
info "Installing GStreamer (full stack)..."
apt-get install -y \
  gstreamer1.0-tools \
  gstreamer1.0-plugins-base \
  gstreamer1.0-plugins-good \
  gstreamer1.0-plugins-bad \
  gstreamer1.0-plugins-ugly \
  gstreamer1.0-libav \
  libgstreamer1.0-dev \
  libgstreamer-plugins-base1.0-dev
ok "GStreamer $(gst-launch-1.0 --version | head -1)"

# 3. Hardware acceleration (auto-detect, non-fatal)
info "Checking hardware acceleration..."
HW=0
if lspci 2>/dev/null | grep -qi 'intel\|amd'; then
  apt-get install -y gstreamer1.0-vaapi vainfo 2>/dev/null && {
    gst-inspect-1.0 vaapih264enc >/dev/null 2>&1 && { ok "VAAPI (vaapih264enc)"; HW=1; } \
      || warn "VAAPI driver installed but vaapih264enc not found"
  } || warn "VAAPI packages unavailable in current repo"
fi
if lspci 2>/dev/null | grep -qi nvidia; then
  gst-inspect-1.0 nvh264enc >/dev/null 2>&1 \
    && { ok "NVIDIA NVENC (nvh264enc)"; HW=1; } \
    || warn "NVIDIA GPU found but nvh264enc unavailable — install NVIDIA drivers first"
fi
[[ $HW -eq 0 ]] && warn "No HW accel — using software x264enc (fine for < 4 streams)"

# 4. FFmpeg (fallback)
info "Installing FFmpeg..."
apt-get install -y ffmpeg
```

```
ok "FFmpeg $(ffmpeg -version 2>&1 | head -1 | awk '{print $3}')"

# 5. Node.js ws + node-fetch
info "Installing Node.js dependencies..."
MSP_DIR="/var/www/msp"
[[ -f "./package.json" ]] && MSP_DIR="$(pwd)"
if [[ -f "${MSP_DIR}/package.json" ]]; then
  cd "$MSP_DIR"
  npm install --save ws node-fetch
  ok "ws + node-fetch installed"
else
  warn "package.json not found — run: npm install --save ws node-fetch"
fi

# 6. Directory layout
info "Creating directory layout..."
mkdir -p /var/www/msp/{public,live,streams,scripts,certs,vod_recordings}
mkdir -p /var/log/msp
chown -R www-data:www-data /var/www/msp/live /var/www/msp/streams /var/www/msp/vod_recordings
chmod 755 /var/www/msp/scripts
ok "Directories created"

# 7. Install scripts
for f in gst-transcode.sh gst-transcode-done.sh; do
  [[ -f "${SRC}/${f}" ]] && {
    cp "${SRC}/${f}" /var/www/msp/scripts/
    chmod +x /var/www/msp/scripts/${f}
    ok "Installed ${f}"
  } || warn "${f} not found — copy manually"
done

# 8. Install gst_pipeline.js
[[ -f "${SRC}/gst_pipeline.js" ]] && {
  cp "${SRC}/gst_pipeline.js" "${MSP_DIR}/src/"
  ok "gst_pipeline.js installed to src/"
} || warn "gst_pipeline.js not found — copy manually to src/"

# 9. Nginx config
if [[ -f "${SRC}/nginx.conf" ]]; then
  [[ -f /etc/nginx/nginx.conf ]] && cp /etc/nginx/nginx.conf "/etc/nginx/nginx.conf.bak.$(date +%Y%m%d
  cp "${SRC}/nginx.conf" /etc/nginx/nginx.conf
  nginx -t && { systemctl reload nginx; ok "Nginx reloaded"; } || die "nginx config test FAILED"
else
  warn "nginx.conf not found — copy manually"
fi

# 10. Firewall
```

```
command -v ufw &>/dev/null && {
  ufw allow 80/tcp  comment "HTTP"  >/dev/null
  ufw allow 443/tcp comment "HTTPS" >/dev/null
  ufw allow 1935/tcp comment "RTMP" >/dev/null
  ok "Firewall: 80, 443, 1935 open"
} || warn "ufw not found — open ports 80, 443, 1935 manually"

echo ""
echo "═══════════════════════════════════════════════════════"
echo -e "  ${GREEN}MSP Media Infrastructure — Complete${NC}"
echo "═══════════════════════════════════════════════════════"
echo ""
echo "  GStreamer: $(gst-launch-1.0 --version | head -1)"
echo "  FFmpeg:    $(ffmpeg -version 2>&1 | head -1 | awk '{print $1,$2,$3}')"
echo ""
echo "  Catalog HLS:  https://stream.michie.com/{cid}/master.m3u8"
echo "  Live HLS:     https://michie.com/live/{sessionId}/master.m3u8"
echo "  RTMP ingest:  rtmp://michie.com:1935/live/{stream_key}"
echo "  RTMP social:  rtmp://michie.com:1935/social/{stream_key}"
echo ""
echo "  Next steps:"
echo "  1. Add TLS: certbot --nginx -d michie.com -d stream.michie.com"
echo "  2. Add to .env:"
echo "       STREAMS_ROOT=/var/www/msp/streams"
echo "       HLS_ROOT=/var/www/msp/live"
echo "       STREAM_HOST=michie.com"
echo "  3. Merge server_gst_additions.js into server.cjs"
echo "  4. Add to server.cjs requires:"
echo "       const { GstPipeline, detectCapabilities, MODES, STREAMS_ROOT, HLS_ROOT }"
echo "             = require('./gst_pipeline');"
echo "  5. Restart: node src/server.cjs"
echo "  6. Verify:  curl localhost:3001/api/media-capabilities"
echo ""
```

`gst-transcode.sh` (10.9 KB)

```bash
#!/usr/bin/env bash
# ═══════════════════════════════════════════════════════════════════════
#  /var/www/msp/scripts/gst-transcode.sh
#
#  Called by nginx-rtmp exec_publish when a creator's stream is accepted.
#  Receives: $1 = stream key, $2 = mode (optional, default 'production')
#
#  Workflow:
#    1. POST /api/rtmp-publish → get sessionId, qualities, mode
```

```
#    2. Create HLS output directory
#    3. Detect best encoder (NVENC → VAAPI → software)
#    4. Launch appropriate GStreamer pipeline:
#       production → tee-based multi-bitrate (the monster)
#       social     → bare-bones single quality
#    5. Write master.m3u8
#    6. POST /api/rtmp-live-ready → notify Node.js
# ═══════════════════════════════════════════════════════════════════════════

set -euo pipefail

STREAM_KEY="${1:?Stream key required}"
FORCE_MODE="${2:-}"    # optional: 'social' if called from /social app
NODE_URL="http://127.0.0.1:3001"
HLS_ROOT="${HLS_ROOT:-/var/www/msp/live}"
LOG_DIR="/var/log/msp"

mkdir -p "$LOG_DIR"
LOG_FILE="${LOG_DIR}/gst-${STREAM_KEY}.log"
exec >> "$LOG_FILE" 2>&1
echo "[$(date -u +%FT%TZ)] gst-transcode.sh START key=${STREAM_KEY} force_mode=${FORCE_MODE}"

# ── 1. Register with Node.js ─────────────────────────────────────────────
PAYLOAD="{\"streamKey\":\"${STREAM_KEY}\"$([ -n \"$FORCE_MODE\" ] && echo ",\"mode\":\"$FORCE_MODE\"")
RESPONSE=$(curl -sf -X POST \
  -H "Content-Type: application/json" \
  -d "${PAYLOAD}" \
  "${NODE_URL}/api/rtmp-publish" 2>/dev/null || echo '{}')

SESSION_ID=$(echo "$RESPONSE" | python3 -c "import sys,json;d=json.load(sys.stdin);print(d.get('sessio
MODE=$(echo "$RESPONSE"       | python3 -c "import sys,json;d=json.load(sys.stdin);print(d.get('mode',
AUDIO_ONLY=$(echo "$RESPONSE" | python3 -c "import sys,json;d=json.load(sys.stdin);print('1' if d.get(
QUALS=$(echo "$RESPONSE"      | python3 -c "import sys,json;d=json.load(sys.stdin);print(','.join(d.ge

[ -z "$SESSION_ID" ] && { echo "ERROR: no sessionId from Node.js"; exit 1; }
echo "[$(date -u +%FT%TZ)] session=${SESSION_ID} mode=${MODE} audioOnly=${AUDIO_ONLY} quals=${QUALS}"

HLS_DIR="${HLS_ROOT}/${SESSION_ID}"
mkdir -p "$HLS_DIR"
PID_FILE="${HLS_DIR}/.gst.pid"
RTMP_URL="rtmp://127.0.0.1:1935/live/${STREAM_KEY}"
GST="${GST_LAUNCH_PATH:-gst-launch-1.0}"

# ── 2. Detect video encoder ──────────────────────────────────────────────
detect_encoder() {
  if gst-inspect-1.0 nvh264enc    >/dev/null 2>&1; then echo "nvh264enc";    return; fi
  if gst-inspect-1.0 vaapih264enc >/dev/null 2>&1; then echo "vaapih264enc"; return; fi
```

```
  if gst-inspect-1.0 vtenc_h264   >/dev/null 2>&1; then echo "vtenc_h264";   return; fi
  echo "x264enc"
}

VIDEO_ENC=$(detect_encoder)
echo "[$(date -u +%FT%TZ)] encoder: ${VIDEO_ENC}"

# Build encoder element string
enc_str() {
  local enc="$1" kbps="$2"
  case "$enc" in
    nvh264enc)    echo "nvh264enc bitrate=${kbps} rc-mode=cbr preset=low-latency ! h264parse" ;;
    vaapih264enc) echo "vaapivpp ! vaapih264enc rate-control=cbr bitrate=${kbps} ! h264parse" ;;
    vtenc_h264)   echo "vtenc_h264 bitrate=${kbps} realtime=true ! h264parse" ;;
    *)            echo "x264enc bitrate=${kbps} speed-preset=ultrafast tune=zerolatency key-int-max=30
  esac
}

PIDS=()

# ── 3a. SOCIAL mode pipeline  ──────────────────────────────────────────────
#  Single quality, 1s segments, ephemeral
launch_social() {
  local w=854 h=480 vbr=1200
  echo "[$(date -u +%FT%TZ)] SOCIAL pipeline: ${w}x${h} ${vbr}kbps"

  if [ "$AUDIO_ONLY" = "1" ]; then
    "$GST" -e \
      "rtmpsrc location=\"${RTMP_URL} live=1\" ! flvdemux name=d \
       d. ! queue ! aacparse ! avdec_aac ! audioconvert ! audioresample \
       ! avenc_aac bitrate=131072 compliance=-2 ! aacparse \
       ! hlssink2 location=\"${HLS_DIR}/stream_%05d.ts\" \
                  playlist-location=\"${HLS_DIR}/stream.m3u8\" \
                  target-duration=1 max-files=10" \
      >> "${LOG_DIR}/gst-${STREAM_KEY}-social-audio.log" 2>&1 &
    PIDS+=("$!")
  else
    local enc
    enc=$(enc_str "$VIDEO_ENC" "$vbr")
    "$GST" -e \
      "rtmpsrc location=\"${RTMP_URL} live=1\" ! flvdemux name=d \
       d. ! queue max-size-time=2000000000 ! h264parse ! avdec_h264 ! videoconvert \
       ! videoscale ! video/x-raw,width=${w},height=${h} \
       ! ${enc} \
       ! mpegtsmux name=mux \
       ! hlssink2 location=\"${HLS_DIR}/stream_%05d.ts\" \
                  playlist-location=\"${HLS_DIR}/stream.m3u8\" \
```

```
                  target-duration=1 max-files=10 \
       d. ! queue ! aacparse ! avdec_aac ! audioconvert ! audioresample \
       ! avenc_aac bitrate=131072 compliance=-2 ! aacparse ! mux." \
      >> "${LOG_DIR}/gst-${STREAM_KEY}-social.log" 2>&1 &
    PIDS+=("$!")
  fi

  cat > "${HLS_DIR}/master.m3u8" << EOF
#EXTM3U
#EXT-X-VERSION:3
$([ "$AUDIO_ONLY" = "1" ] && echo '#EXT-X-STREAM-INF:BANDWIDTH=131000,CODECS="mp4a.40.2"
stream.m3u8' || echo "#EXT-X-STREAM-INF:BANDWIDTH=1350000,RESOLUTION=${w}x${h},CODECS=\"avc1.42e01e,mp
stream.m3u8")
EOF
}

# ── 3b. PRODUCTION mode pipeline  ──────────────────────────────────────────
#  Multi-bitrate tee fan-out, 2s segments
launch_production() {
  echo "[$(date -u +%FT%TZ)] PRODUCTION pipeline: quals=${QUALS} audioOnly=${AUDIO_ONLY}"

  if [ "$AUDIO_ONLY" = "1" ]; then
    # Multi-bitrate audio tee
    "$GST" -e \
      "rtmpsrc location=\"${RTMP_URL} live=1\" ! flvdemux name=d \
       d. ! queue ! aacparse ! avdec_aac ! audioconvert ! audioresample \
       ! audio/x-raw,rate=44100,channels=2 ! tee name=atee \
       atee. ! queue ! avenc_aac bitrate=320000 compliance=-2 ! aacparse \
       ! hlssink2 location=\"${HLS_DIR}/hi_%05d.ts\" \
                  playlist-location=\"${HLS_DIR}/hi.m3u8\" \
                  target-duration=2 max-files=16 \
       atee. ! queue ! avenc_aac bitrate=256000 compliance=-2 ! aacparse \
       ! hlssink2 location=\"${HLS_DIR}/mid_%05d.ts\" \
                  playlist-location=\"${HLS_DIR}/mid.m3u8\" \
                  target-duration=2 max-files=16 \
       atee. ! queue ! avenc_aac bitrate=128000 compliance=-2 ! aacparse \
       ! hlssink2 location=\"${HLS_DIR}/lo_%05d.ts\" \
                  playlist-location=\"${HLS_DIR}/lo.m3u8\" \
                  target-duration=2 max-files=16" \
      >> "${LOG_DIR}/gst-${STREAM_KEY}-prod-audio.log" 2>&1 &
    PIDS+=("$!")

    cat > "${HLS_DIR}/master.m3u8" << 'EOF'
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=320000,CODECS="mp4a.40.2"
hi.m3u8
```

```
#EXT-X-STREAM-INF:BANDWIDTH=256000,CODECS="mp4a.40.2"
mid.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=128000,CODECS="mp4a.40.2"
lo.m3u8
EOF
    return
  fi

  # ── Video tee pipeline ──────────────────────────────────────────────────
  # Build one giant GStreamer pipeline string with all rungs as branches of vtee
  local PIPELINE=""
  PIPELINE+="rtmpsrc location=\"${RTMP_URL} live=1\" ! flvdemux name=d "
  PIPELINE+="d. ! queue max-size-bytes=0 max-size-time=2000000000 "
  PIPELINE+="! h264parse ! avdec_h264 ! videoconvert ! videorate "
  PIPELINE+="! video/x-raw,framerate=30/1 ! tee name=vtee "
  PIPELINE+="d. ! queue max-size-bytes=0 max-size-time=2000000000 "
  PIPELINE+="! aacparse ! avdec_aac ! audioconvert ! audioresample "
  PIPELINE+="! audio/x-raw,rate=44100,channels=2 ! tee name=atee "

  local MASTER_LINES=("#EXTM3U" "#EXT-X-VERSION:3")

  IFS=',' read -ra QUALITY_LIST <<< "$QUALS"
  for q in "${QUALITY_LIST[@]}"; do
    local w h vbr abr bw
    case "$q" in
      1080p) w=1920; h=1080; vbr=4500; abr=192; bw=4700000 ;;
      720p)  w=1280; h=720;  vbr=2800; abr=128; bw=2950000 ;;
      480p)  w=854;  h=480;  vbr=1400; abr=128; bw=1550000 ;;
      360p)  w=640;  h=360;  vbr=700;  abr=96;  bw=810000  ;;
      *) continue ;;
    esac

    local enc
    enc=$(enc_str "$VIDEO_ENC" "$vbr")
    PIPELINE+="vtee. ! queue max-size-bytes=0 max-size-time=0 leaky=downstream "
    PIPELINE+="! videoscale method=bilinear add-borders=true "
    PIPELINE+="! video/x-raw,width=${w},height=${h} "
    PIPELINE+="! ${enc} "
    PIPELINE+="! mpegtsmux name=mux${q} "
    PIPELINE+="! hlssink2 location=\"${HLS_DIR}/${q}_%05d.ts\" "
    PIPELINE+="           playlist-location=\"${HLS_DIR}/${q}.m3u8\" "
    PIPELINE+="           target-duration=2 max-files=16 "
    PIPELINE+="atee. ! queue ! avenc_aac bitrate=$(( abr * 1000 )) compliance=-2 ! aacparse "
    PIPELINE+="! mux${q}. "

    MASTER_LINES+=("#EXT-X-STREAM-INF:BANDWIDTH=${bw},RESOLUTION=${w}x${h},CODECS=\"avc1.42e01e,mp4a.4
    MASTER_LINES+=("${q}.m3u8")
```

```
  done

  echo "[$(date -u +%FT%TZ)] Launching PRODUCTION pipeline (${#QUALITY_LIST[@]} rungs)"
  "$GST" -e "$PIPELINE" \
    >> "${LOG_DIR}/gst-${STREAM_KEY}-prod-video.log" 2>&1 &
  PIDS+=("$!")

  # Write master playlist
  printf '%s\n' "${MASTER_LINES[@]}" > "${HLS_DIR}/master.m3u8"
}

# ── 4. Launch the right pipeline ──────────────────────────────────────────
if [ "$MODE" = "social" ]; then
  launch_social
else
  launch_production
fi

# ── 5. Write PID file ─────────────────────────────────────────────────────
printf '%s\n' "${PIDS[@]}" > "$PID_FILE"
echo "[$(date -u +%FT%TZ)] PIDs: ${PIDS[*]}"

# ── 6. Notify Node.js that pipelines are live ─────────────────────────────
sleep 2    # brief pause for GStreamer to start writing segments
curl -sf -X POST \
  -H "Content-Type: application/json" \
  -d "{\"sessionId\":\"${SESSION_ID}\",\"hlsUrl\":\"/live/${SESSION_ID}/master.m3u8\"}" \
  "${NODE_URL}/api/rtmp-live-ready" 2>/dev/null \
  && echo "[$(date -u +%FT%TZ)] live-ready notified" \
  || echo "[$(date -u +%FT%TZ)] WARNING: live-ready notify failed"

echo "[$(date -u +%FT%TZ)] gst-transcode.sh complete — ${#PIDS[@]} pipeline(s) running"

# ── 7. Wait (keeps nginx-rtmp exec_publish alive until stream ends) ──────
wait
echo "[$(date -u +%FT%TZ)] All pipelines exited"
```

`gst-transcode-done.sh` (1.1 KB)

```bash
#!/usr/bin/env bash
# /var/www/msp/scripts/gst-transcode-done.sh
set -euo pipefail
STREAM_KEY="${1:?}"
NODE_URL="http://127.0.0.1:3001"
HLS_ROOT="${HLS_ROOT:-/var/www/msp/live}"
```

```
LOG_DIR="/var/log/msp"

exec >> "${LOG_DIR}/gst-${STREAM_KEY}.log" 2>&1
echo "[$(date -u +%FT%TZ)] gst-transcode-done.sh key=${STREAM_KEY}"

RESPONSE=$(curl -sf -X POST -H "Content-Type: application/json" \
  -d "{\"streamKey\":\"${STREAM_KEY}\"}" "${NODE_URL}/api/rtmp-done" 2>/dev/null || echo '{}')
SESSION_ID=$(echo "$RESPONSE" | python3 -c "import sys,json;d=json.load(sys.stdin);print(d.get('sessio

if [ -n "$SESSION_ID" ]; then
  PID_FILE="${HLS_ROOT}/${SESSION_ID}/.gst.pid"
  if [ -f "$PID_FILE" ]; then
    while IFS= read -r pid; do
      kill -0 "$pid" 2>/dev/null && kill -SIGINT "$pid" 2>/dev/null \
        && echo "[$(date -u +%FT%TZ)] SIGINT → PID ${pid}" || true
    done < "$PID_FILE"
    sleep 8
    while IFS= read -r pid; do
      kill -0 "$pid" 2>/dev/null && kill -SIGKILL "$pid" 2>/dev/null || true
    done < "$PID_FILE"
    rm -f "$PID_FILE"
  fi
fi
echo "[$(date -u +%FT%TZ)] done"
```

`demucs_script.py` (0.0 KB)

```python
# Demucs stem separation script â€
```